目录

目录.....1
1 概述.....21
2 主要特性.....22
3 封装.....24
3.1 QFN32 封装.....24
3.2 QFN40 封装.....25
3.3 TSSOP28 封装.....26
4 功能框图.....28
5 地址空间.....29
5.1 总线结构.....29
5.2 存储器映射.....29
5.2.1 AHB 地址.....30
5.2.2 APB 地址.....30
6 矢量中断.....32
6.1 中断控制器.....32
6.2 中断列表.....32
6.3 中断控制寄存器.....34
6.3.1 ISER 寄存器.....34
6.3.2 ICER 寄存器.....38
6.3.3 ISPR 寄存器.....41
7 系统控制.....45
7.1 概述.....45
7.2 状态查询.....45
7.3 低功耗模式.....46
7.3.1 低功耗架构.....46
7.3.2 PMU 控制.....47
7.4 寄存器列表.....48



7.5 寄存器描述.....49
7.5.1 UUID 寄存器.....49
7.5.2 USER 寄存器.....49
7.5.3 TAG 寄存器.....50
7.5.4 FSIZE 寄存器.....51
7.5.5 RSIZE 寄存器.....51
7.5.6 GPIOE0 寄存器.....51
7.5.7 GPIOE1 寄存器.....51
7.5.8 WAKEIR 寄存器.....52
7.5.9 WAKECR 寄存器.....52
7.5.10 LPTMRTH 寄存器.....53
7.5.11 INTERVAL 寄存器.....53
7.5.12 VERID 寄存器.....54
8 时钟复位.....55
8.1 概述.....55
8.1.1 简介.....55
8.1.2 MCU 主要时钟源包括：.....55
8.1.3 模块框图.....56
8.1.4 时钟切换.....57
8.1.5 复位信号源.....57
8.1.6 上电时序.....57
8.2 寄存器列表.....58
8.3 时钟复位寄存器描述.....59
8.3.1 SYSRSTF 寄存器 系统复位标志寄存器.....59
8.3.2 SYSRSTE 寄存器 系统复位使能寄存器.....59
8.3.3 RSTKEY 寄存器 复位密钥寄存器.....60
8.3.4 AHBRST 寄存器 AHB 总线外设复位寄存器.....60
8.3.5 ANARST 寄存器 模拟外设复位寄存器.....60
8.3.6 DIGRST 寄存器 数字外设复位寄存器.....61

8.3.7 COMRST 寄存器 通信外设复位寄存器 61
8.3.8 TIMRST 寄存器 数字定时器复位寄存器 62
8.3.9 HSICR 寄存器 内部高速 RC 时钟控制寄存器 62
8.3.10 LSICR 寄存器 内部低速 RC 时钟控制寄存器 62
8.3.11 HSECR 寄存器 晶振时钟控制寄存器 63
8.3.12 PLLCR0 寄存器 PLL 时钟控制寄存器 63
8.3.13 PLLCR1 寄存器 PLL 时钟倍频率系数寄存器 63
8.3.14 SYSCKCR 寄存器 系统时钟控制寄存器 64
8.3.15 MCOCR 寄存器 MCO 时钟输出控制寄存器 65
8.3.16 AHBCKE 寄存器 AHB 总线外设时钟使能寄存器 66
8.3.17 ANACKE 寄存器 模拟外设时钟使能寄存器 66
8.3.18 DIGCKE 寄存器 数字外设时钟使能寄存器 66
8.3.19 COMCKE 寄存器 通信外设时钟使能寄存器 67
8.3.20 TIMCKE 寄存器 数字定时器时钟使能寄存器 67
8.3.21 PERCKS0 寄存器 外设时钟源选择寄存器 68
8.4 应用指南 68
8.4.1 功能说明 68
8.5 快速开始 71
9 DMA 74
9.1 概述 74
9.2 特性 74
9.3 DMA 通道配置 75
9.4 DMA 通道仲裁 75
9.5 DMA 通道传输 75
9.5.1 通道配置 75
9.5.2 数据大小 76
9.5.3 地址模式 76
9.5.4 自动重载 77
9.5.5 中断状态管理 77

9.6 寄存器列表.....77
9.7 寄存器描述.....79
9.7.1 数据源基地址寄存器 DMACHn_SRCB（n=0,1,2,3,4,5）.....79
9.7.2 数据目的基地址 DMACHn_DSTB（n=0,1,2,3,4,5）.....79
9.7.3 数据最大长度配置寄存器 DMACHn_MAX（n=0,1,2,3,4,5）.....79
9.7.4 控制寄存器 DMACHn_CTL（n=0,1,2,3,4,5）.....80
9.7.5 当前源地址寄存器 DMACHn_SCUR（n=0,1,2,3,4,5）.....82
9.7.6 当前目的地址寄存器 DMACHn_DCUR（n=0,1,2,3,4,5）.....82
9.7.7 终止传输计数器 DMACHn_TCNT（n=0,1,2,3,4,5）.....82
9.7.8 中断使能寄存器 DMA_MASK.....82
9.7.9 中断清除寄存器 DMA_CLR.....84
9.7.10 中断状态寄存器 DMA_STAT.....86
9.7.11 通道关联外设选择寄存器 1 DMA_CHCFG1.....88
9.7.12 通道关联外设选择寄存器 2 DMA_CHCFG2.....89
10 MCPWM.....90
10.1 概述.....90
10.2 Base Counter 模块.....91
10.3 FAIL 信号处理.....93
10.4 MCPWM 特殊输出状态.....94
10.5 MCPWM 输出模式.....94
10.5.1 中心对齐模式.....95
10.5.2 中心对齐推挽模式.....96
10.5.3 边沿对齐模式.....96
10.5.4 边沿对齐推挽模式.....97
10.5.5 死区控制.....98
10.5.6 极性设置.....98
10.5.7 自动保护.....99
10.6 ADC Trigger 模块.....99
10.7 中断.....100

10.8 寄存器列表.....100
10.8.1 MCPWM0_TH00 比较门限寄存器.....103
10.8.2 MCPWM0_TH01 MCPWM CH0N 比较门限寄存器.....103
10.8.3 MCPWM0_TH10 MCPWM CH1P 比较门限寄存器.....103
10.8.4 MCPWM0_TH11 MCPWM CH1N 比较门限寄存器.....104
10.8.5 MCPWM0_TH20 MCPWM CH2P 比较门限寄存器.....104
10.8.6 MCPWM0_TH21 MCPWM CH2N 比较门限寄存器.....104
10.8.7 MCPWM0_TH30 MCPWM CH3P 比较门限寄存器.....105
10.8.8 MCPWM0_TH31 MCPWM CH3N 比较门限寄存器.....105
10.8.9 MCPWM0_TH40 MCPWM CH4P 比较门限寄存器.....105
10.8.10 MCPWM0_TH41 MCPWM CH4N 比较门限寄存器.....106
10.8.11 MCPWM0_TH50 MCPWM CH5P 比较门限寄存器.....106
10.8.12 MCPWM0_TH51 MCPWM CH5N 比较门限寄存器.....106
10.8.13 MCPWM0_TMR0 ADC 定时器门限 1 寄存器.....107
10.8.14 MCPWM0_TMR1 ADC 定时器门限 1 寄存器.....107
10.8.15 MCPWM0_TMR2 ADC 定时器门限 2 寄存器.....107
10.8.16 MCPWM0_TMR3 ADC 定时器门限 3 寄存器.....108
10.8.17 MCPWM0_TH0 MCPWM 时基 0 门限寄存器.....108
10.8.18 MCPWM0_TH1 MCPWM 时基 1 门限寄存器.....108
10.8.19 MCPWM0_CNT0 MCPWM 时基 0 计数值寄存器.....109
10.8.20 MCPWM0_CNT1 MCPWM 时基 1 计数值寄存器.....109
10.8.21 MCPWM0_UPDATE MCPWM 手动更新寄存器.....110
10.8.22 MCPWM0_FCNT MCPWM 刹车计数值寄存器.....112
10.8.23 MCPWM0_EVT0 MCPWM 时基 0 外部触发寄存器.....112
10.8.24 MCPWM0_EVT1 MCPWM 时基 1 外部触发寄存器.....113
10.8.25 MCPWM0_DTH00 MCPWM CH0/1/2N 死区控制寄存器.....113
10.8.26 MCPWM0_DTH01 MCPWM CH0/1/2P 死区控制寄存器.....114
10.8.27 MCPWM0_DTH10 MCPWM CH3/4/5N 死区控制寄存器.....114
10.8.28 MCPWM0_DTH11 MCPWM CH3/4/5P 死区控制寄存器.....115

10.8.29 MCPWM0_FLT MCPWM 滤波时钟分频寄存器 115
10.8.30 MCPWM0_SDCFG MCPWM 配置寄存器 115
10.8.31 MCPWM0_AUEN MCPWM 自动更新使能寄存器 117
10.8.32 MCPWM0_TCLK MCPWM 时钟配置寄存器 119
10.8.33 MCPWM0_IE0 MCPWM 时基 0 中断控制寄存器 121
10.8.34 MCPWM0_IF0 MCPWM 时基 0 中断标志寄存器 122
10.8.35 MCPWM0_IE1 MCPWM 时基 1 中断控制寄存器 124
10.8.36 MCPWM0_IF1 MCPWM 时基 1 中断标志寄存器 126
10.8.37 MCPWM0_EIE MCPWM FAIL 中断控制寄存器 128
10.8.38 MCPWM0_EIF MCPWM FAIL 中断标志寄存器 129
10.8.39 MCPWM0_EIF MCPWM DMA 配置寄存器 130
10.8.40 MCPWM0_PP MCPWM 推挽配置寄存器 132
10.8.41 MCPWM0_IO01 MCPWM 通道 01 配置寄存器 133
10.8.42 CPWM0_IO23 MCPWM 通道 23 配置寄存器 134
10.8.43 MCPWM0_IO45 MCPWM 通道 45 配置寄存器 136
10.8.44 MCPWM0_IOCFG MCPWM 通道配置寄存器 138
10.8.45 MCPWM0_FAIL012 MCPWM FAIL012 配置寄存器 140
10.8.46 MCPWM0_FAIL345 MCPWM FAIL345 配置寄存器 141
10.8.47 MCPWM0_CH_DEF MCPWM 短路保护通道配置寄存器 143
10.8.48 MCPWM0_CH_MSK MCPWM 通道屏蔽位寄存器 144
10.8.49 MCPWM0_PRT MCPWM 解保护寄存器 145
11 TIMER 146
11.1 概述 146
11.2 功能框图 146
11.2.1 寄存器模块 147
11.2.2 时钟分频 147
11.2.3 滤波模块 147
11.2.4 通用定时器模块 148
11.3 工作模式 148

11.3.1 基本模式 148
11.3.2 捕获模式 149
11.3.3 比较模式 150
11.3.4 单次触发 150
11.3.5 中心模式（互补 PWM） 151
11.3.6 外部事件 152
11.3.7 外部启动信号 153
11.3.8 外部时钟信号 154
11.3.9 外部重置信号 154
11.3.10 外部门控信号 155
11.4 ADC 触发 155
11.5 寄存器列表 156
11.6 TIMERx 寄存器通用描述 157
11.6.1 TIMERx_CFG Timer0 配置寄存器 158
11.6.2 TIMERx_TH 门限寄存器 163
11.6.3 TIMERx_CNT 计数寄存器 163
11.6.4 TIMERx_CMP0 通道 0 比较捕获寄存器 163
11.6.5 TIMERx_CMP1 通道 1 比较捕获寄存器 164
11.6.6 TIMER0_EVT 外部事件选择寄存器 164
11.6.7 TIMERx_FLT 滤波控制寄存器 167
11.6.8 TIMERx_IE 中断使能寄存器 167
11.6.9 TIMERx_IF 中断标志寄存器 168
11.6.10 TIMERx_IO IO 控制寄存器 169
11.7 应用指南 170
11.7.1 功能说明 170
11.8 快速开始 173
12 QEP 178
12.1 概述 178
12.2 实现说明 178

12.2.1 正交编码信号.....178
12.2.2 符号加脉冲信号.....179
12.2.3 CCW/CW 双脉冲信号.....180
12.3 寄存器列表.....181
12.4 寄存器描述.....182
12.4.1 QEP_CFG QEP 模块配置寄存器.....182
12.4.2 QEP_TH QEP 门限值寄存器.....183
12.4.3 QEP_CNT QEP 计数值寄存器.....183
12.4.4 QEP_IE QEP 中断使能寄存器.....183
12.4.5 QEP_IF QEP 中断标志寄存器.....184
13 HALL.....185
13.1.1 概述.....185
13.2 实现说明.....185
13.2.1 信号来源.....185
13.2.2 工作时钟.....185
13.2.3 信号滤波.....185
13.2.4 捕获.....186
13.2.5 中断.....187
13.2.6 数据流程.....187
13.3 寄存器列表.....188
13.4 寄存器描述.....188
13.4.1 HALL_IR HALL 中断标志寄存器.....188
13.4.2 HALL_IR HALL 中断使能寄存器.....188
13.4.3 HALL_CFG HALL 模块配置寄存器.....189
13.4.4 HALL_TH HALL 门限值寄存器.....190
13.4.5 HALL_CNT HALL 计数值寄存器.....190
13.4.6 HALL_CCR HALL 捕获值寄存器.....190
13.4.7 HALL_INFO HALL 模块信息寄存器.....190
13.4.8 HALL_LOCK HALL 模块解锁寄存器.....191

14 GPIO 192
14.1 概述 192
14.2 寄存器列表 193
14.3 寄存器描述 196
14.3.1 Px_DR 数据寄存器(x=A,B,C) 196
14.3.2 Px_DRS 数据置位寄存器(x=A,B,C) 197
14.3.3 Px_DRC 数据清除寄存器(x=A,B,C) 198
14.3.4 Px_DRT 数据翻转寄存器(x=A,B,C) 198
14.3.5 Px_OESR 输出使能置位寄存器(x=A,B,C) 199
14.3.6 Px_OECR 输出使能清除寄存器(x=A,B,C) 200
14.3.7 Px_ITESR 中断使能置位寄存器(x=A,B,C) 201
14.3.8 Px_ITECR 中断使能清除寄存器(x=A,B,C) 201
14.3.9 Px_ITTR1 中断类型寄存器 1(x=A,B,C) 202
14.3.10 Px_ITTR2 中断类型寄存器 2(x=A,B,C) 203
14.3.11 Px_ITTCR 中断触发类型清除寄存器(x=A,B,C) 204
14.3.12 Px_ITSR 中断状态寄存器(x=A,B,C) 205
14.3.13 Px_AFSR1 复用功能置位寄存器 1(x=A,B,C) 206
14.3.14 Px_AFSR2 复用功能置位寄存器 2(x=A,B,C) 207
14.3.15 Px_AFCR 复用功能清除寄存器(x=A,B,C) 207
14.3.16 Px_ANASR 模拟功能置位寄存器(x=A,B,C) 208
14.3.17 Px_ANACR 模拟功能清除寄存器(x=A,B,C) 209
14.3.18 Px_PUSR 上拉使能置位寄存器(x=A,B,C) 209
14.3.19 Px_PUCR 上拉使能清除寄存器(x=A,B,C) 210
14.3.20 Px_PDSR 下拉使能置位寄存器(x=A,B,C) 211
14.3.21 Px_PDCR 下拉使能清除寄存器(x=A,B,C) 212
14.3.22 Px_DRVSR1 驱动强度置位寄存器 1(x=A,B,C) 213
14.3.23 Px_DRVSR2 驱动强度置位寄存器 2(x=A,B,C) 214
14.3.24 Px_DRVCR 驱动强度清除寄存器(x=A,B,C) 214
14.3.25 Px_STSR 施密特触发置位寄存器(x=A,B,C) 215

14.3.26 Px_STCR 施密特触发清除寄存器(x=A,B,C) 216
14.3.27 Px_IESR 输入使能置位寄存器(x=A,B,C) 217
14.3.28 Px_IECR 输入使能清除寄存器(x=A,B,C) 218
15 UART 219
15.1 概述 219
15.1.1 简介 219
15.1.2 功能框图 219
15.1.3 功能特点 220
15.2 功能描述 221
15.2.1 UART 帧格式 221
15.2.2 波特率配置 221
15.2.3 UART 发送器 221
15.2.4 UART 接收器 221
15.2.5 DMA 功能 222
15.2.6 LIN 功能 222
15.3 UART 中断 224
15.4 寄存器列表 224
15.5 UART0 寄存器描述 225
15.5.1 UART0_SR UART0 状态寄存器 225
15.5.2 UART0_IR UART0 中断标志寄存器 226
15.5.3 UART0_IER UART0 中断使能寄存器 228
15.5.4 UART0_BRR UART0 波特率设置寄存器 230
15.5.5 UART0_DR UART0 数据寄存器 230
15.5.6 UART0_CR UART0 控制寄存器 231
15.5.7 UART0_WATER UART0 水位线设置寄存器 233
15.6 UART1 寄存器描述 234
15.6.1 UART1_SR UART1 状态寄存器 234
15.6.2 UART1_IR UART1 中断标志寄存器 235
15.6.3 UART1_IER UART1 中断使能寄存器 236

15.6.4 UART1_BRR UART1 波特率设置寄存器 238
15.6.5 UART1_DR UART1 数据寄存器 239
15.6.6 UART1_CR UART1 控制寄存器 239
15.6.7 UART1_WATER UART1 水位线设置寄存器 242
16 SPI 243
16.1 概述 243
16.1.1 简介 243
16.1.2 功能框图 243
16.1.3 功能特点 244
16.2 功能描述 244
16.2.1 主机模式 244
16.2.2 从机模式 244
16.2.3 时钟与时序配置 245
16.2.4 DMA 功能 245
16.2.5 SPI 中断 245
16.3 寄存器列表 246
16.4 SPI0 寄存器描述 247
16.4.1 SPI0_IDREV SPI0 版本寄存器 247
16.4.2 SPI0_TFMT SPI0 传输格式寄存器 247
16.4.3 SPI0_TCTRL SPI0 传输控制寄存器 248
16.4.4 SPI0_CMD SPI0 命令寄存器 251
16.4.5 SPI0_ADDR SPI0 地址寄存器 251
16.4.6 SPI0_DATA SPI0 数据寄存器 251
16.4.7 SPI0_CTRL SPI0 控制寄存器 252
16.4.8 SPI0_STATUS SPI0 状态寄存器 253
16.4.9 SPI0_IE SPI0 中断使能寄存器 254
16.4.10 SPI0_IR SPI0 中断状态寄存器 255
16.4.11 SPI0_TIMING SPI0 接口时序寄存器 256
16.4.12 SPI0_MEMCTRL SPI0 内存访问控制寄存器 256



16.4.13 SPI0_SLVST SPI0 从机状态寄存器 257
16.4.14 SPI0_SLVDATACNT SPI0 从机数据计数寄存器 257
16.5 SPI1 寄存器描述 258
16.5.1 SPI1_IDREV SPI1 版本寄存器 258
16.5.2 SPI1_TFMT SPI1 传输格式寄存器 258
16.5.3 SPI1_TCTRL SPI1 传输控制寄存器 259
16.5.4 SPI1_CMD SPI1 命令寄存器 261
16.5.5 SPI1_ADDR SPI1 地址寄存器 262
16.5.6 SPI1_DATA SPI1 数据寄存器 262
16.5.7 SPI1_CTRL SPI1 控制寄存器 262
16.5.8 SPI1_STATUS SPI1 状态寄存器 264
16.5.9 SPI1_IE SPI1 中断使能寄存器 265
16.5.10 SPI1_IR SPI1 中断状态寄存器 265
16.5.11 SPI1_TIMING SPI1 接口时序寄存器 266
16.5.12 SPI1_MEMCTRL SPI1 内存访问控制寄存器 267
16.5.13 SPI1_SLVST SPI1 从机状态寄存器 267
16.5.14 SPI1_SLVDATACNT SPI1 从机数据计数寄存器 268
16.6 应用指南 268
16.6.1 功能说明 268
16.7 快速开始 272
17 I2C 277
17.1 概述 277
17.2 特性 277
17.3 I2C 专业术语 278
17.4 I2C 通讯流程 279
17.4.1 开始与停止 279
17.4.2 组合格式 280
17.5 I2C 协议 280
17.5.1 启动与停止条件 280

17.5.2 从机地址协议.....280
17.5.3 收发协议.....281
17.5.4 起始字节传输协议.....282
17.5.5 TX FIFO.....283
17.5.6 多主机仲裁.....284
17.5.7 时钟同步.....285
17.5.8 从模式操作.....286
17.5.9 主模式操作.....289
17.5.10 I2C 使能关闭.....290
17.5.11 I2C 传输中止.....291
17.5.12 尖峰抑制滤波器.....291
17.5.13 快速模式增强操作.....291
17.5.14 IC_CLK 频率配置.....292
17.5.15 SCL 计算公式.....292
17.6 DMA 功能.....293
17.6.1 DMA 使能.....293
17.6.2 DMA 水位线设置.....293
17.7 寄存器列表.....293
17.8 寄存器描述.....297
17.8.1 I2Cx_CON（x=0,1） I2C 控制寄存器.....297
17.8.2 I2Cx_TAR（x=0,1） I2C 目标地址寄存器.....300
17.8.3 I2Cx_SAR（x=0,1） I2C 从机地址寄存器.....301
17.8.4 I2Cx_DATA_CMD（x=0,1） I2C 数据命令寄存器.....301
17.8.5 I2Cx_SS_SCL_HCNT（x=0,1） I2C 标准模式 SCL 高计数寄存器.....302
17.8.6 I2Cx_SS_SCL_LCNT（x=0,1） I2C 标准模式 SCL 低计数寄存器.....303
17.8.7 I2Cx_FS_SCL_HCNT（x=0,1） I2C 快速模式 SCL 高计数寄存器.....303
17.8.8 I2Cx_FS_SCL_LCNT（x=0,1） I2C 快速模式 SCL 低计数寄存器.....304
17.8.9 I2Cx_INTR_STAT（x=0,1） I2C 中断状态寄存器.....304
17.8.10 I2Cx_INTR_MASK（x=0,1） I2C 中断屏蔽寄存器.....305

17.8.11 I2Cx_RAW_INTR_STAT（x=0,1） I2C 中断原始状态寄存器.....306
17.8.12 I2Cx_RX_TL（x=0,1） I2C 接收 FIFO 阈值寄存器.....310
17.8.13 I2Cx_TX_TL（x=0,1） I2C 发送 FIFO 阈值寄存器.....311
17.8.14 I2Cx_CLR_INTR（x=0,1） I2C 组合与独立中断清除寄存器.....311
17.8.15 I2Cx_CLR_RX_UNDER（x=0,1） I2C 接收下溢清除寄存器.....312
17.8.16 I2Cx_CLR_RX_OVER（x=0,1） I2C 接收上溢清除寄存器.....312
17.8.17 I2Cx_CLR_TX_OVER（x=0,1） I2C 发送上溢清除寄存器.....312
17.8.18 I2Cx_CLR_RD_REQ（x=0,1） I2C 读请求清除寄存器.....312
17.8.19 I2Cx_CLR_TX_ABRT（x=0,1） I2C 发送停止清除寄存器.....312
17.8.20 I2Cx_CLR_RX_DONE（x=0,1） I2C 接收完成清除寄存器.....313
17.8.21 I2Cx_CLR_ACTIVITY（x=0,1） I2C 接收完成清除寄存器.....313
17.8.22 I2Cx_CLR_STOP_DET（x=0,1） I2C 停止检测清除寄存器.....313
17.8.23 I2Cx_CLR_START_DET（x=0,1） I2C 开始检测清除寄存器.....314
17.8.24 I2Cx_CLR_GEN_CALL（x=0,1） I2C 广播清除寄存器.....314
17.8.25 I2Cx_ENABLE（x=0,1） I2C 使能寄存器.....314
17.8.26 I2Cx_STATUS（x=0,1） I2C 状态寄存器.....315
17.8.27 I2Cx_TXFLR（x=0,1） I2C TXFIFO 级数寄存器.....316
17.8.28 I2Cx_RXFLR（x=0,1） I2C RXFIFO 级数寄存器.....317
17.8.29 I2Cx_SDA_HOLD（x=0,1） I2C SDA 保持时间寄存器.....317
17.8.30 I2Cx_TX_ABRT_SOURCE（x=0,1） I2C 发送停止状态寄存器.....317
17.8.31 I2Cx_SLV_DATA_NACK_ONLY（x=0,1） I2C 从机无应答寄存器.....319
17.8.32 I2Cx_DMA_CR（x=0,1） I2C DMA 控制寄存器.....320
17.8.33 I2Cx_DMA_TDLR（x=0,1） I2C DMA 发送水位寄存器.....320
17.8.34 I2Cx_DMA_RDLR（x=0,1） I2C DMA 接收水位寄存器.....321
17.8.35 I2Cx_SDA_SETUP（x=0,1） I2C SDA 建立时间寄存器.....321
17.8.36 I2Cx_ACK_GENERAL_CALL（x=0,1） I2C 广播应答寄存器.....321
17.8.37 I2Cx_ENABLE_STATUS（x=0,1） I2C 使能状态寄存器.....321
17.8.38 I2Cx_FS_SPKLEN（x=0,1） I2C 快速模式尖峰抑制长度寄存器.....323
17.8.39 I2Cx_CLR_RESTART_DET（x=0,1） I2C 清除重启监测寄存器.....324



17.8.40 I2Cx_COMP_PARAM_1（x=0,1） I2C 组件参数寄存器 1 324
17.8.41 I2Cx_COMP_VERSION（x=0,1） I2C 组件版本寄存器 326
17.8.42 I2Cx_COMP_TYPE（x=0,1） I2C 组件类型寄存器 326
17.9 应用指南 326
17.9.1 功能说明 326
17.10 快速开始 329
18 CAN 334
18.1 概述 334
18.1.1 简介 334
18.1.2 功能框图 334
18.2 功能描述 335
18.2.1 工作模式 335
18.2.2 传输 335
18.2.3 中断处理 335
18.2.4 波特率设置 337
18.2.5 ID 过滤器 339
18.2.6 CAN 帧的接收 340
18.2.7 CAN 帧的发送 343
18.2.8 错误管理 345
18.2.9 错误计数 346
18.2.10 主动错误状态 347
18.2.11 被动错误状态 347
18.2.12 关闭状态与关闭恢复 347
18.2.13 仲裁 348
18.3 寄存器列表 348
18.4 CAN 寄存器描述 349
18.4.1 CAN_RBUF CAN 接收缓冲区 349
18.4.2 CAN_TBUF CAN 发送缓冲区 349
18.4.3 CAN_TTS CAN 发送时间戳 349

18.4.4 CAN_STCMDCR CAN 状态命令控制寄存器.....350
18.4.5 CAN_INTLMT CAN 中断阈值寄存器.....354
18.4.6 CAN_LSEQ CAN 低速位时序寄存器.....356
18.4.7 CAN_HSEQ CAN 高速位时序寄存器.....357
18.4.8 CAN_ERRABR CAN 错误与仲裁寄存器.....358
18.4.9 CAN_ACFTIM CAN 滤波器控制与时间戳寄存器.....358
18.4.10 CAN_ACF CAN 验收码寄存器.....360
18.4.11 CAN_VERTTCR CAN 版本信息 TTCAN 控制寄存器.....360
18.4.12 CAN_TTCANMSG CAN TTCAN 参考报文寄存器.....362
18.4.13 CAN_TTCANCFG0 CAN TTCAN 触发配置寄存器 0.....363
18.4.14 CAN_TTCANCFG1 CAN TTCANCFG 触发配置寄存器 1.....364
19 ADC.....366
19.1 概述.....366
19.1.1 简介.....366
19.1.2 功能特点.....366
19.1.3 模块框图.....367
19.2 功能描述.....367
19.2.1 ADC 转换序列.....367
19.2.2 ADC 触发方式.....368
19.2.3 ADC 通道选择.....368
19.2.4 模拟看门狗.....370
19.2.5 ADC 校准.....370
19.2.6 ADC 过采样.....371
19.2.7 ADC 中断.....371
19.2.8 DMA 功能.....372
19.2.9 ADC 时序设置.....372
19.3 ADC 寄存器描述.....374
19.3.1 ADC_DATA0 ADC 数据寄存器 0.....374
19.3.2 ADC_DATA1 ADC 数据寄存器 1.....374

19.3.3 ADC_DATA2 ADC 数据寄存器 2.....374
19.3.4 ADC_DATA3 ADC 数据寄存器 3.....374
19.3.5 ADC_DATA4 ADC 数据寄存器 4.....375
19.3.6 ADC_DATA5 ADC 数据寄存器 5.....375
19.3.7 ADC_DATA6 ADC 数据寄存器 6.....375
19.3.8 ADC_DATA7 ADC 数据寄存器 7.....375
19.3.9 ADC_DATA8 ADC 数据寄存器 8.....376
19.3.10 ADC_DATA9 ADC 数据寄存器 9.....376
19.3.11 ADC_DATA10 ADC 数据寄存器 10.....376
19.3.12 ADC_DATA11 ADC 数据寄存器 11.....377
19.3.13 ADC_DATA12 ADC 数据寄存器 12.....377
19.3.14 ADC_DATA13 ADC 数据寄存器 13.....377
19.3.15 ADC_DATA14 ADC 数据寄存器 14.....377
19.3.16 ADC_DATA15 ADC 数据寄存器 15.....378
19.3.17 ADC_AVGDATA ADC 求平均结果寄存器.....378
19.3.18 ADC_TGSR ADC 触发源选择寄存器.....378
19.3.19 ADC_TGNR IADC 序列数量配置寄存器.....380
19.3.20 ADC_SEQR0 ADC 通道选择寄存器 0.....381
19.3.21 ADC_SEQR1 ADC 通道选择寄存器 1.....382
19.3.22 ADC_SEQR2 ADC 通道选择寄存器 2.....382
19.3.23 ADC_SEQR3 ADC 通道选择寄存器 3.....383
19.3.24 ADC_CR ADC 控制寄存器 383
19.3.25 ADC_OTR ADC 单次触发寄存器.....385
19.3.26 ADC_TCFG ADC 时序配置寄存器.....385
19.3.27 ADC_AWTH ADC 模拟看门狗阈值寄存器 386
19.3.28 ADC_AWE ADC 模拟看门狗使能寄存器 386
19.3.29 ADC_IR ADC 中断标志寄存器.....388
19.3.30 ADC_IE ADC 中断使能寄存器 390
19.3.31 ADC_CALIB ADC 校准参数寄存器.....391

19.3.32 ADC_CALIBDAT ADC 校准结果寄存器 391
19.3.33 ADC_ORIDAT ADC 原始数据寄存器 392
19.4 应用指南 392
19.4.1 功能说明 392
19.4.2 快速开始 397
20 CSA 400
20.1 概述 400
20.1.1 简介 400
20.1.2 主要功能 400
20.1.3 CSA 框图 401
20.2 寄存器列表 401
20.3 CSA 寄存器描述 402
20.3.1 MISC_IR MISC 中断寄存器 402
20.3.2 MISC_CSACR0 CSA 控制寄存器 0 403
20.3.3 MISC_CSACR1 CSA 控制寄存器 1 405
20.3.4 MISC_TEMPCR MISC 温度传感器控制寄存器 408
20.4 应用指南 410
20.4.1 功能说明 410
21 MATH 414
21.1 概述 414
21.1.1 简介 414
21.1.2 功能特点 414
21.2 功能描述 414
21.2.1 开方运算 414
21.2.2 除法运算 415
21.2.3 三角函数运算 415
21.3 寄存器列表 415
21.4 MATH 寄存器描述 416
21.4.1 MATH 寄存器描述 416

21.4.2 MATH_SQRTRI 开方运算输入寄存器 416
21.4.3 MATH_SQRTRO 开方运算结果输出寄存器 417
21.4.4 MATH_DIVIDE 除法被除数输入寄存器 417
21.4.5 MATH_DIVIDS 除法除数输入寄存器 417
21.4.6 MATH_QUOTI 商输出寄存器 417
21.4.7 MATH_REMAIN 余数输出寄存器 417
21.4.8 MATH_THETA 角度输入寄存器 418
21.4.9 MATH_SINCOS 正余弦结果寄存器 418
22 FLC 419
22.1 概述 419
22.1.1 简介 419
22.1.2 功能特点 419
22.1.3 地址分配 419
22.2 功能描述 419
22.2.1 读操作 419
22.2.2 解锁与保护机制 420
22.2.3 编程操作 420
22.2.4 中断 420
22.3 寄存器列表 421
22.4 FLC 寄存器描述 421
22.4.1 FLC_ACR FLC 访问控制寄存器 421
22.4.2 FLC_KEYR FLC 主区域解锁寄存器 421
22.4.3 FLC_INFO_KEYR FLC INFO 区域解锁寄存器 422
22.4.4 FLC_SR FLC 状态寄存器 422
22.4.5 FLC_IER FLC 中断使能寄存器 423
22.4.6 FLC_CR FLC 控制寄存器 424
22.4.7 FLC_DR FLC 数据寄存器 424
23 IWDT 425
23.1 概述 425

23.1.1 简介.....425
23.1.2 功能特点.....425
23.2 功能描述.....425
23.2.1 使用说明.....425
23.3 寄存器列表.....426
23.4 IWDT 寄存器描述.....426
23.4.1 IWDT_RLR IWDT 重装载寄存器.....426
23.4.2 IWDT_CNT IWDT 当前计数值寄存器.....427
23.4.3 IWDT_CR IWDT 控制寄存器.....427
23.4.4 IWDT_CLR IWDT 中断清除寄存器.....427
23.4.5 IWDT_SR IWDT 状态寄存器.....427
23.4.6 IWDT_LOCK IWDT 软件锁寄存器.....428
23.5 应用指南.....428
23.5.1 功能说明.....428
23.6 快速开始.....429



1 概述

低功耗 ODS507x/5x/3x 采用高性能的 ARM® Cortex®-M0+的 32 位微控制器，最高可工作于 96 MHz，采用高速的嵌入式闪存（SRAM 最大 16KB，程序/数据闪存最大 128KB，读取速度为 48M/Word）。本产品集成 I2C、SPI、UART 多种标准接口。集成 12bit A/D 转换器、8-Bit 数模转换器 DAC、可编程增益放大器 PGA、模拟比较器 ACMP、温度传感器。其中 12bit A/D 转换器可以于采集外部传感器信号，降低系统设计成本。芯片内集成的温度传感器则可实现对外部环境温度实时监控。集成看门狗定时器、32 位通用定时器 Timer13/14/15/16，可以满足不同条件下的时钟需求。此外，还集成了除法与开方单元，MCPWM 模块，增强型 DMA，HALL 信号处理模块，QEP 正交编码器模块。

ODS507x/5x/3x 还具有出色的低功耗性能，支持睡眠和深度睡眠两种低功耗模式。在部分掉电的深度睡眠模式下功耗约 8uA，深度睡眠模式支持定时唤醒，引脚唤醒。

这些特点使得 ODS507x/5x/3x 微控制器系列可广泛适用三相直流无刷电机驱动控制、单相直流无刷电机驱动控制。

芯片命名规则：

ODS 503x :入门级 MCU

ODS 505x :性价比 MCU

ODS 507x :高性能 MCU

注：用户手册为上述型号通用

Revision 1.0 – First release. This is the first version of the user manual. Specs might change anytime after more feature analysis without notice. Always check the latest version when designing.

版本 1.0（初版发布）。本手册为用户手册首发版本，规格可能随后续特性验证发生变更，最终设计请以最新修订版为准。

2 主要特性

- 超低功耗工作环境
  - 电源电压范围： 2.2V 到 3.6V
  - 温度范围：-40°C 到 105°C
  - 低功耗模式：睡眠模式，深度睡眠模式
  - 深度睡眠模式下功耗：~8uA
- 内核资源
  - ARM 32bit-Cortex-M0+内核
  - 工作频率: 32KHz~96MHz
- 计算单元
  - 支持硬件除法/开方/正余弦等
  - 存储
  - 128kB 片内 FLASH，带 ECC 保护功能
  - 16KB SRAM
- 复位系统
  - 支持上电复位 POR，掉电复位 BOR
  - 支持低电压监测 PVD
  - 支持看门狗复位
  - 支持软件复位
- GPIO
  - 最多可提供 40 个 IO
  - 均可提供中断
  - 支持上下拉功能
- DMA
  - 6 通道独立 DMA
  - 多个请求间的优先级可通过软件设置
  - 支持循环的缓冲器管理
- ADC
  - 12 位 SAR 结构 ADC
  - 采样速率可到 1.5Msps
  - 支持参考电压可选:
  - VREF 档位：1.5V/2V/2.5V/3V/VDD
  - 支持内部定时器定时触发
  - 支持通道可编程轮询
- PGA
  - 支持电机单/双/三电阻电流采样
  - 支持 1x,2x,4x,8x,12x,16x,32x,64x 倍增益
  - 支持内部比较器过流
- TEMP
  - 支持内部节温检测 TJ
  - 支持外部温度检测 TS
- CAN 通信
  - 支持 CAN2.0
- UART 通信
  - 支持波特率可编程
  - 支持 LIN 协议
- IIC
  - 支持标准速率（100Kb/s），快速速率（1Mb/s）
- SPI
  - 支持主/从模式
  - 最快传输速度为系统时钟 1/4
- 通用定时器

- 4 个 32bit 通用定时器
- 支持捕获模式
- 支持比较模式

- MCPWM 定时器
  - 支持 6 路互补或 12 路独立不交叠 PWM 输出
  - 支持掩码及掩码预设
  - 支持边沿/中心对齐
  - 支持移相
  - 支持死区可配
  - 可用于触发 ADC
- 其他
  - 支持 HALL 滤波和捕获
  - 支持 QEP 正交编码(CW/CCW 双脉冲)
  - 支持 SWD 调试接口
- 安全功能
  - 128 位唯一 UID
  - 128 位用户 UID
  - 支持 Flash 加密，配置后在 Debug 模式下 Flash 不可读

3 封装

- 支持 QFN32(ODS5070-KBQ8)
- 支持 QFN40(ODS5070-FBQ8、ODS5071N200-FBQ8)
- 支持 TSSOP-28(ODS5070-YBP8、ODS5071P40-YBP8)

3.1 QFN32 封装

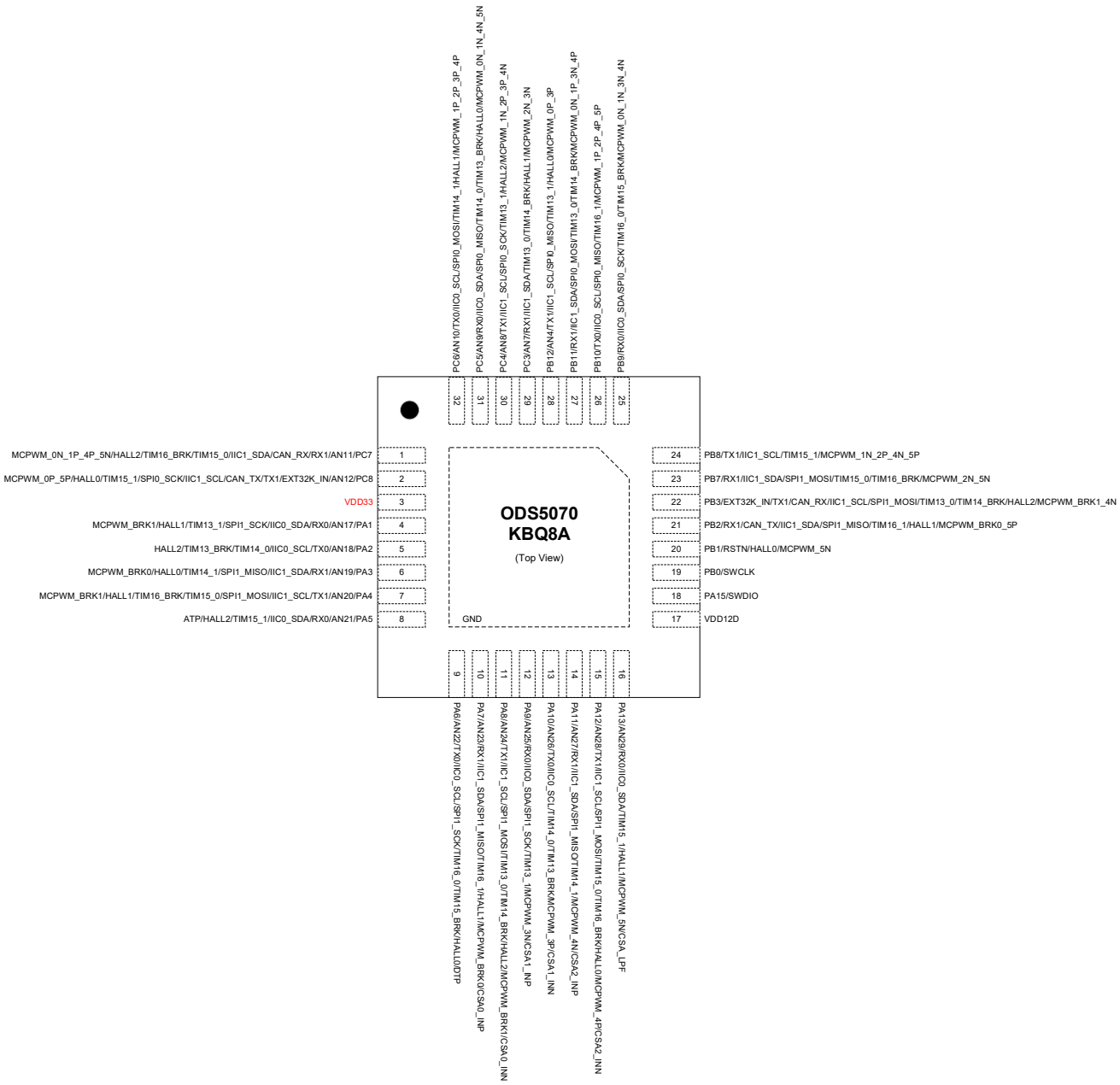


图 3-1 ODS5070-KBQ8 封装示意图

3.2 QFN40 封装

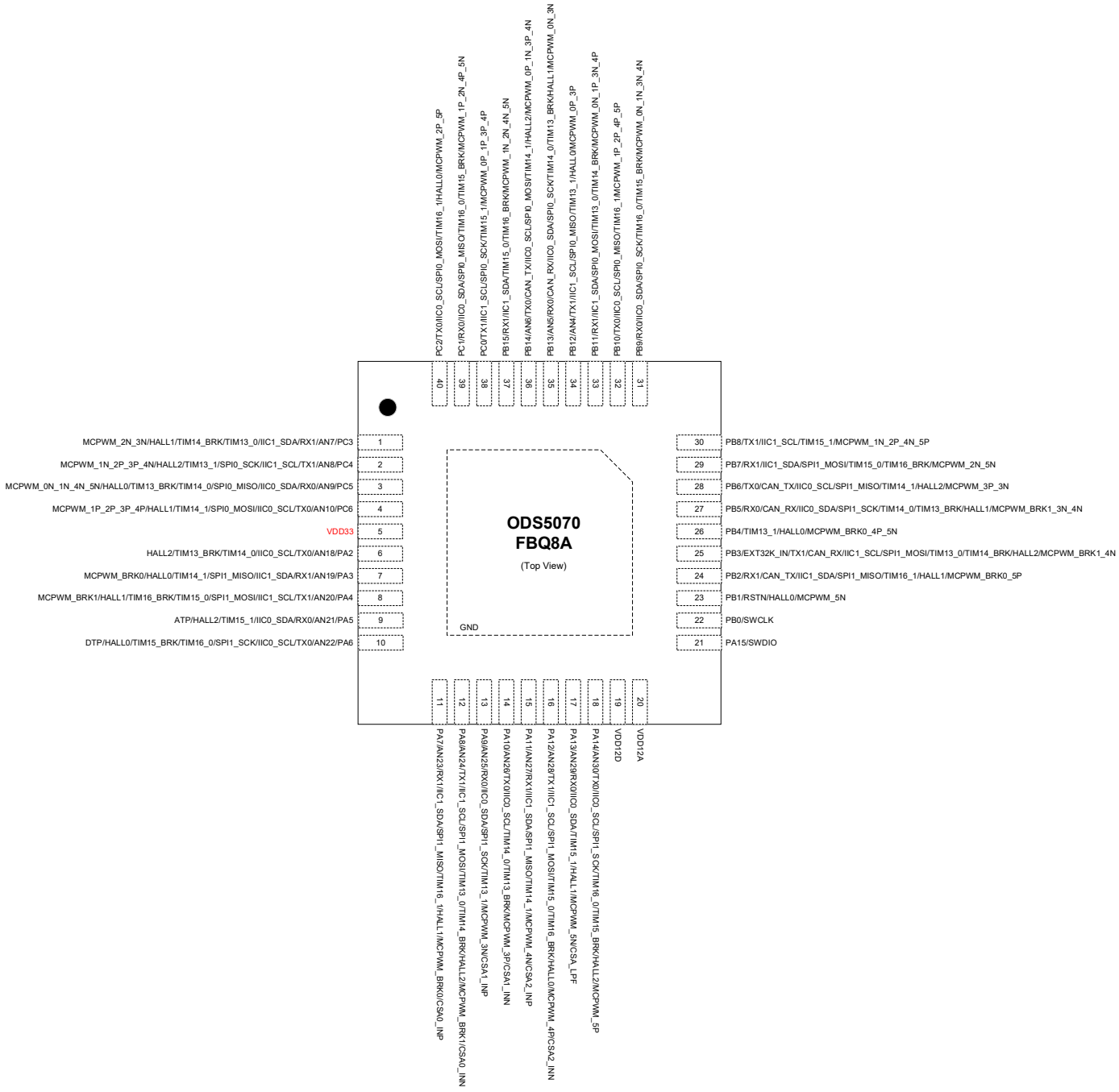


图 3-2 ODS5070-FBQ8 封装示意图

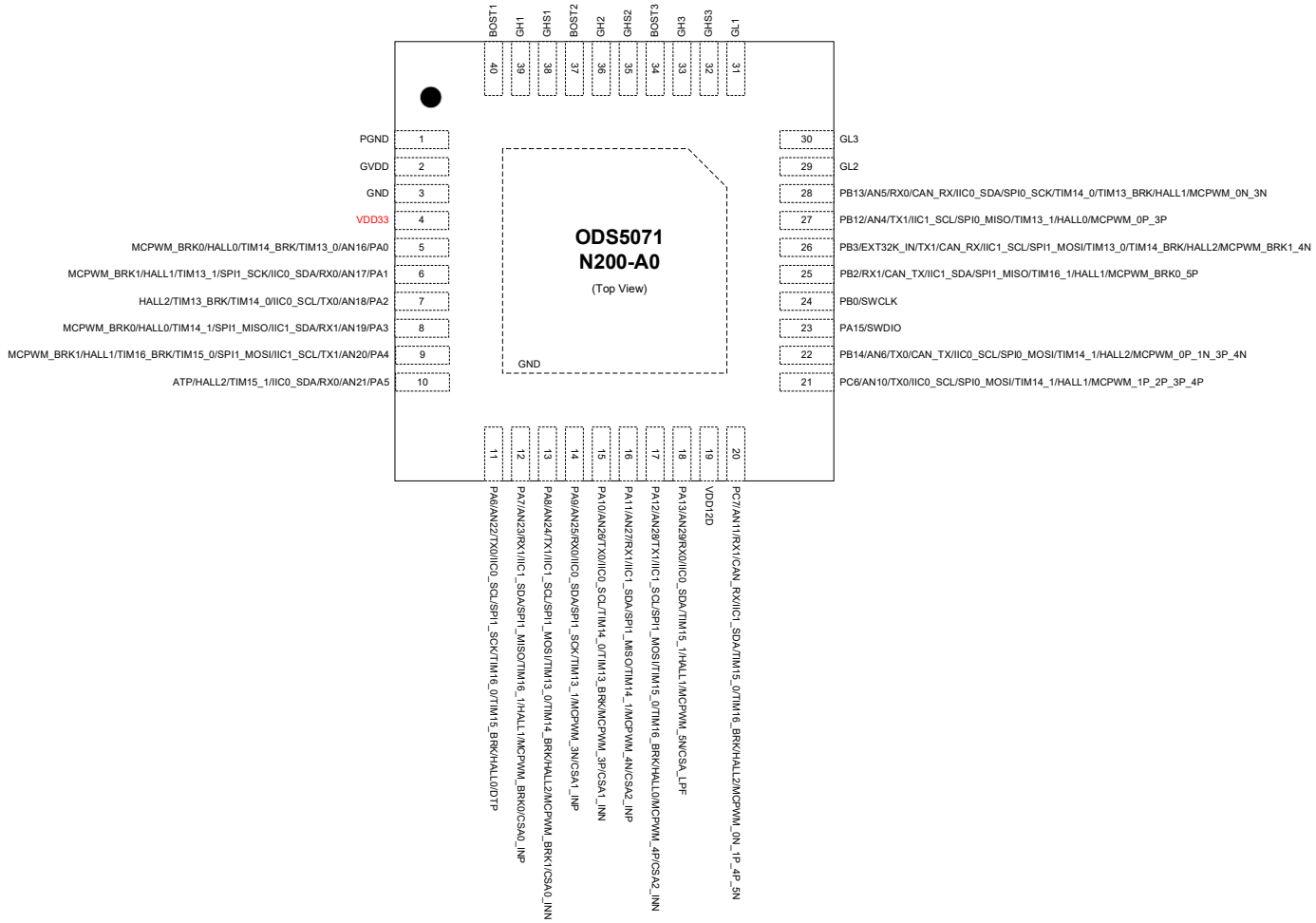


图 3-3 ODS5071N200-FBQ8 封装示意图

3.3 TSSOP28 封装

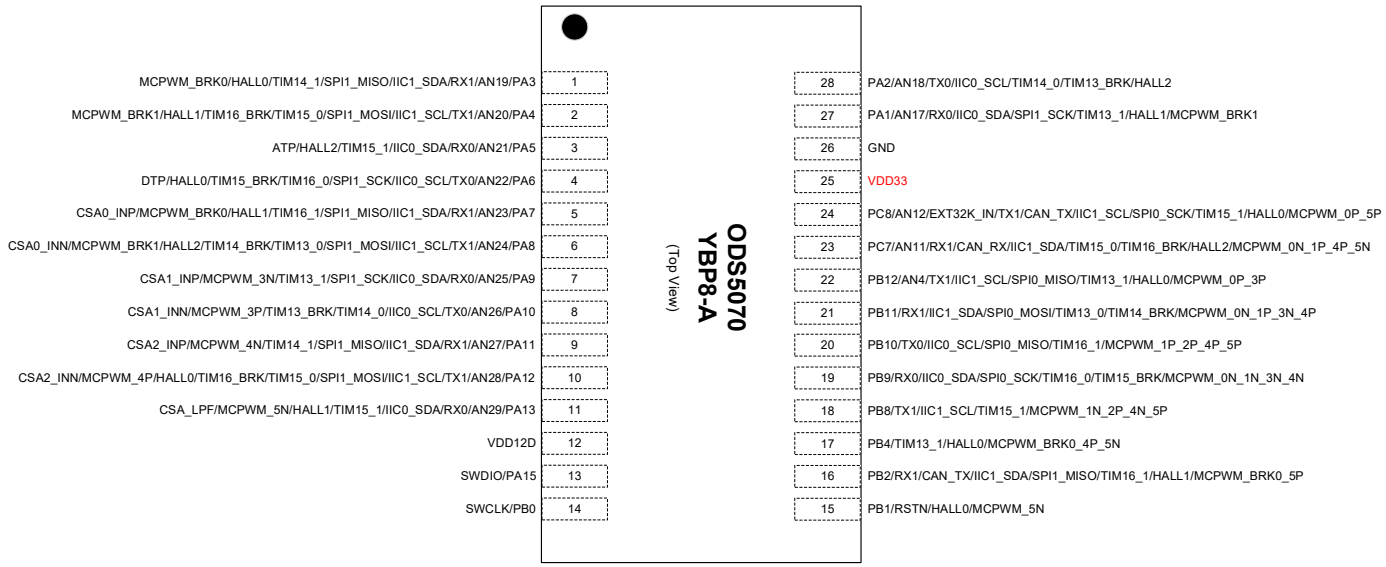


图 3-4 ODS5070-YBP8 封装示意图

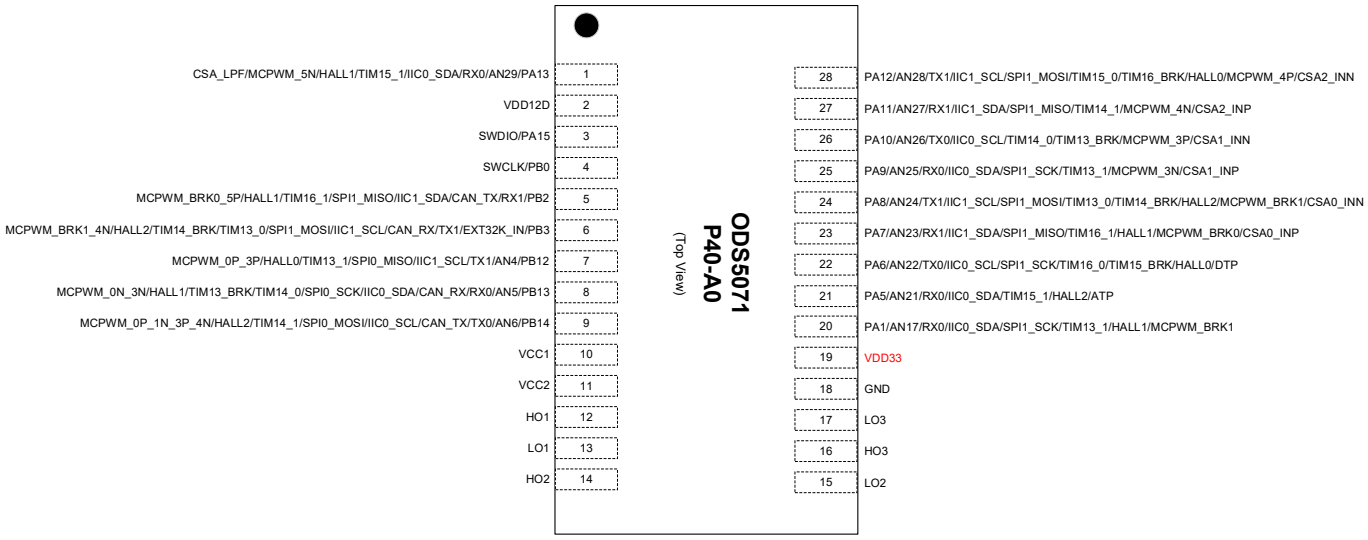


图 3-5 ODS5071P40-YBP8 封装示意图

4 功能框图

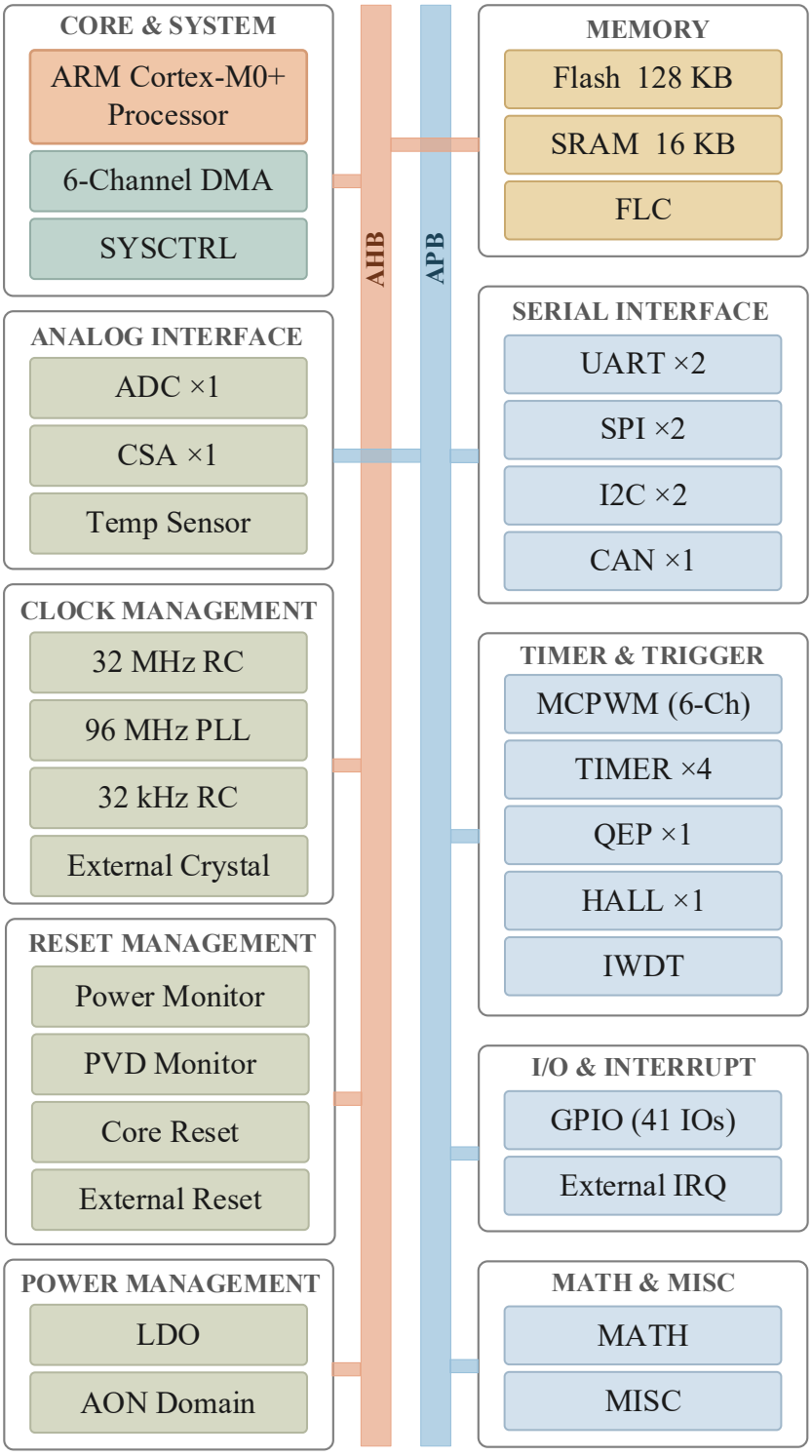


图 4-1 ODS507x 功能框图

5 地址空间

5.1 总线结构

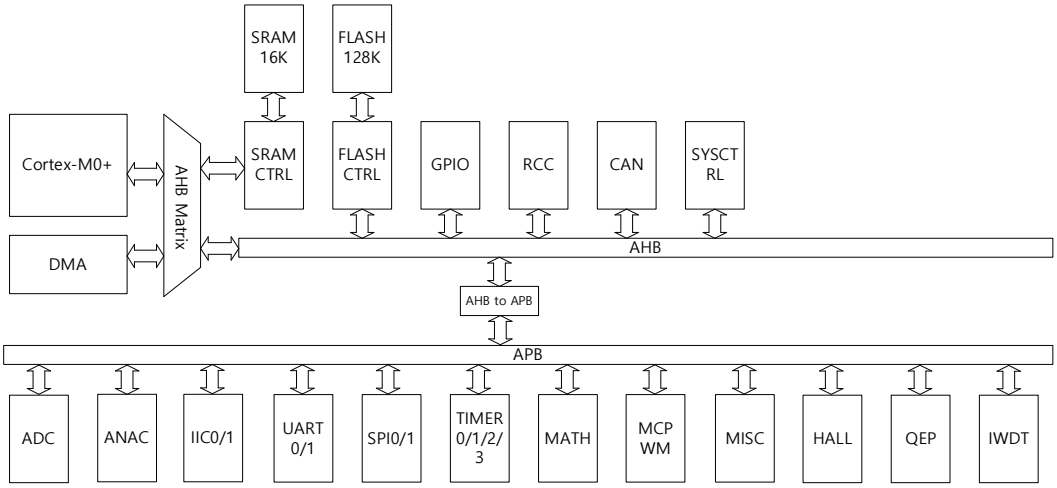


图 5-1 总线结构示意图

5.2 存储器映射

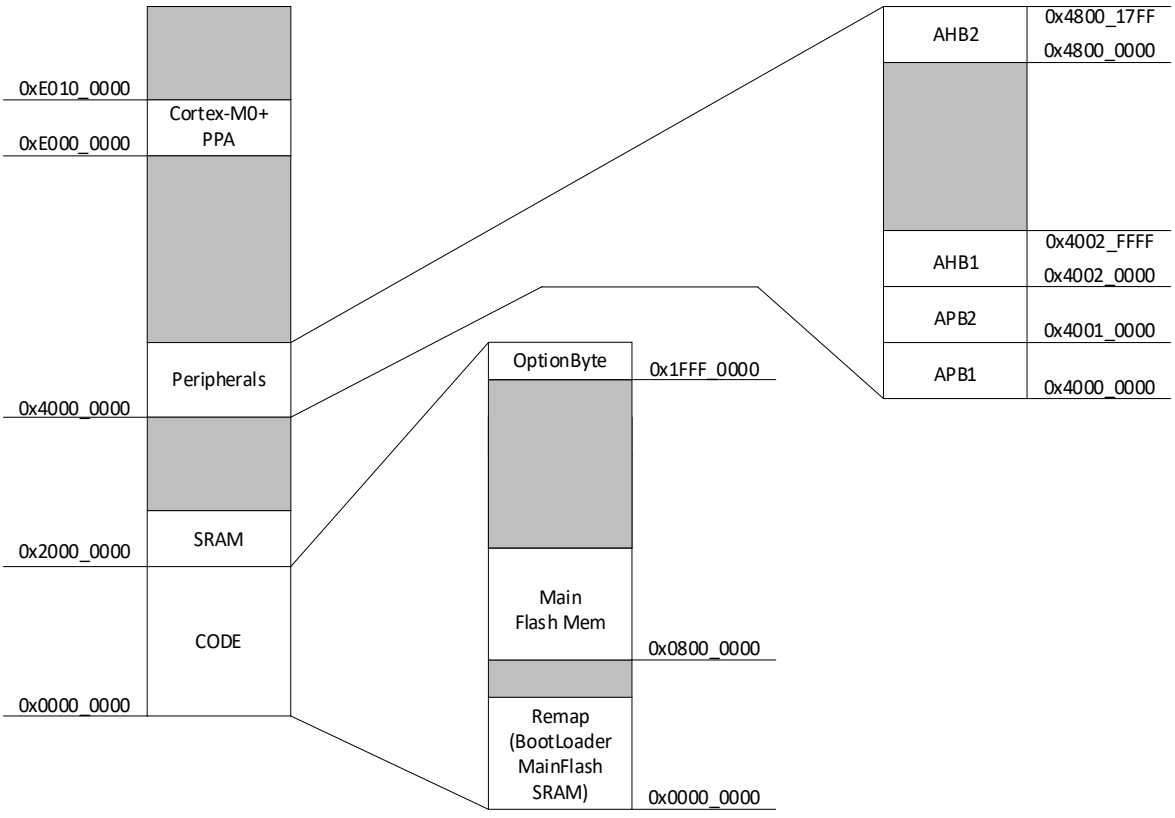


图 5-2 地址映射示意图

5.2.1 AHB 地址

AHB Memory Address	Size	Description
0x0000 0000 – 0x0001 FFFF	128KB	主代码空间
0x2000 0000 – 0x2000 3FFF	16KB	SRAM 数据区
0x4000 0000 – 0x4001 FFFF	128KB	APB 总线区(见 APB 地址分配）
0x4002 0000 – 0x4002 03FF	1KB	RCC 时钟复位
0x4002 0400 – 0x4002 07FF	1KB	SYSCFG 系统配置
0x4002 0800 – 0x4002 0FFF	2KB	保留
0x4002 1000 – 0x4002 13FF	1KB	CAN
0x4002 1400 – 0x4002 1FFF	3KB	保留
0x4002 2000 – 0x4002 23FF	1KB	FLASH
0x4002 2400 – 0x47FF FFFF		保留
0x4800 0000 – 0x4800 03FF	1KB	GPIOA
0x4800 0400 – 0x4800 07FF	1KB	GPIOB
0x4800 0800 – 0x4800 0BFF	1KB	GPIOC

5.2.2 APB 地址

APB Memory Address	**Size**	**Description**
0x4000 0000 – 0x4000 03FF	1KB	DMA
0x4000 0400 – 0x4000 1BFF	6KB	保留
0x4000 1C00 – 0x4000 1FFF	1KB	TIM13
0x4000 2000 – 0x4000 23FF	1KB	TIM14
0x4000 2400 – 0x4000 2FFF	3KB	保留
0x4000 3000 – 0x4000 33FF	1KB	IWDT
0x4000 3400 – 0x4000 37FF	1KB	保留
0x4000 3800 – 0x4000 3BFF	1KB	SPI0
0x4000 3C00 – 0x4000 3FFF	1KB	保留
0x4000 4000 – 0x4000 43FF	1KB	UART0

APB Memory Address	Size	Description
0x4000 4400 – 0x4000 47FF	1KB	UART1
0x4000 4800 – 0x4000 53FF	3KB	保留
0x4000 5400 – 0x4000 57FF	1KB	I2C0
0x4000 5800 – 0x4000 5BFF	1KB	I2C1
0x4000 5C00 – 0x4000 5FFF	1KB	保留
0x4000 6000 – 0x4000 63FF	1KB	MATH
0x4000 6400 – 0x4000 67FF	1KB	POSIF
0x4000 6800 – 0x4000 77FF	4KB	保留
0x4000 7800 – 0x4000 7BFF	1KB	MISC
0x4000 7C00 – 0x4000 7FFF	1KB	ANAC
0x4001 0000 – 0x4001 0BFF		
0x4001 0C00 – 0x4001 0FFF	1KB	MCPWM
0x4001 2400 – 0x4001 27FF	1KB	ADC
0x4001 3800 – 0x4001 3FFF	1KB	SPI1
0x4001 4000 – 0x4001 43FF	1KB	TIM15
0x4001 4400 – 0x4001 47FF	1KB	TIM16
0x4001 5800 – 0x4001 5BFF	1KB	DBG

6 矢量中断

6.1 中断控制器

NVIC 用于中断的高效处理。矢量中断控制器最大可支持 32 个中断源。每个中断源拥有独立软件可编程的中断优先级。矢量中断控制器收集来自不同中断源的中断请求，依据中断优先级对中断请求进行仲裁。最高优先级的中断将获得控制权并请求处理器。

矢量中断控制器支持中断嵌套。当处理器正在处理一个中断请求的同时，来了一个更高优先级的中断请求，处理器将中断当前中断服务程序的处理，响应该更高优先级的中断请求。在更高优先级的中断请求处理结束时，CPU 返回被打断的中断服务程序继续执行。矢量中断控制器允许高优先级的中断请求抢占低优先级的中断请求，但不允许同级别或者低优先级的中断抢占，保证了中断响应的实时性。

6.2 中断列表

Exception Number	IRQ Number	Priority	Exception or Interrupt	Vector
			Reserved	0x00
1	-	-3	Reset	0x04
2	-14	-2	NMI(IWDG & WWDT)	0x08
3	-13	-1	HardFault	0x0C
4~10	-	-	Reserved	-
11	-5	configurable	SVCall	0x2C
12 ~ 13	-	-	Reserved	-
14	-2	configurable	PendSV	0x38
15	-1	configurable	SysTick	0x3C
16	0	configurable	AON	0x40
17	1	configurable	FLASH	0x44
18	2	configurable	UART0	0x48

Exception Number	IRQ Number	Priority	Exception or Interrupt	Vector
19	3	configurable	UART1	0x4C
20	4	configurable	SPI0	0x50
21	5	configurable	SPI1	0x54
22	6	configurable	MCPWM(IRQ0)	0x58
23	7	configurable	MCPWM(IRQ1)	0x5C
24	8	configurable	I2C0	0x60
25	9	configurable	I2C1	0x64
26	10	configurable	GPIOA	0x64
27	11	configurable	GPIOB	0x64
28	12	configurable	GPIOC	0x70
29	13	configurable	TIM13	0x74
30	14	configurable	TIM14	0x78
31	15	configurable	TIM15	0x7C
32	16	configurable	TIM16	0x80
33	17	configurable	ADC	0x84
34	18	configurable	HALL	0x88
35	19	configurable	QEP	0x8C
36	20	configurable	DMA	0x90
37	21	configurable	BOR	0x94
38	22	configurable	PVD	0x98
39	23	configurable	CSA	0x9C
40	24	configurable	TJ_OTS	0xA0
41	25	configurable	TJ_OTW	0xA4
42	26	configurable	PLL_ERR	0xA8
43	27	configurable	ANAC0	0xAC
44	28	configurable	ANAC1	0xB0

Exception Number	IRQ Number	Priority	Exception or Interrupt	Vector
45	29	configurable	--	0xB4
46	30	configurable	--	0xB8
47	31	configurable	--	0xBC

6.3 中断控制寄存器

地址	寄存器	复位值	描述	备注
0xE000_E100	ISER	0x0	系统中断使能寄存器	--
0xE000_E180	ICER	0x0	系统中断禁止寄存器	--
0xE000_E200	ISPR	0x0	系统中断挂起使能寄存器	--
0xE000_E280	ICPR	0x0	系统中断挂起禁止寄存器	--
0xE000_ED10	SCR	0x0	系统控制寄存器	--

6.3.1 ISER 寄存器

地址：0xE000E100

位域	类型	复位	名称	描述
[31:29]	--	--	--	--
[28]	R/W1	0	ANAC1	模拟中断 1 使能 读：0：关闭 ；1：使能 写：0：无效；1：使能
[27]	R/W1	0	ANAC0	模拟中断 0 使能 读：0：关闭 ；1：使能 写：0：无效；1：使能
[26]	R/W1	0	PLL_ERR	PLL 异常中断使能 读：0：关闭 ；1：使能 写：0：无效；1：使能

位域	类型	复位	名称	描述
[25]	R/W1	0	TJ_OTW	温度检测 OTW 中断使能 读：0：关闭 ；1：使能 写：0：无效；1：使能
[24]	R/W1	0	TJ_OTS	温度检测 OTS 中断使能 读：0：关闭 ；1：使能 写：0：无效；1：使能
[23]	R/W1	0	CSA	过流检测中断使能 读：0：关闭 ；1：使能 写：0：无效；1：使能
[22]	R/W1	0	PVD	低电压检测中断使能 读：0：关闭 ；1：使能 写：0：无效；1：使能
[21]	R/W1	0	BOR	上电复位中断使能 读：0：关闭 ；1：使能 写：0：无效；1：使能
[20]	R/W1	0	DMA	DMA 中断使能 读：0：关闭 ；1：使能 写：0：无效；1：使能
[19]	R/W1	0	QEP	QEP 编码器中断使能 读：0：关闭 ；1：使能 写：0：无效；1：使能
[18]	R/W1	0	HALL	HALL 编码器中断使能 读：0：关闭 ；1：使能 写：0：无效；1：使能
[17]	R/W1	0	ADC	ADC 中断使能 读：0：关闭 ；1：使能 写：0：无效；1：使能

位域	类型	复位	名称	描述
[16]	R/W1	0	TIM16	TIM16 中断使能 读：0：关闭 ；1：使能 写：0：无效；1：使能
[15]	R/W1	0	TIM15	TIM15 中断使能 读：0：关闭；1：使能 写：0：无效；1：使能
[14]	R/W1	0	TIM14	TIM14 中断使能 读：0：关闭；1：使能 写：0：无效；1：使能
[13]	R/W1	0	TIM13	TIM13 中断使能 读：0：关闭；1：使能 写：0：无效；1：使能
[12]	R/W1	0	GPIOC	GPIOC 中断使能 读：0：关闭 ；1：使能 写：0：无效；1：使能
[11]	R/W1	0	GPIOB	GPIOB 中断使能 读：0：关闭 ；1：使能 写：0：无效；1：使能
[10]	R/W1	0	GPIOA	GPIOA 中断使能 读：0：关闭 ；1：使能 写：0：无效；1：使能
[9]	R/W1	0	I2C1	I2C1 中断使能 读：0：关闭 ；1：使能 写：0：无效；1：使能
[8]	R/W1	0	I2C0	I2C0 中断使能 读读：0：关闭 ；1：使能 写：0：无效；1：使能

位域	类型	复位	名称	描述
[7]	R/W1	0	MCPWM(IRQ1)	MCPWM 中断 IRQ1 使能 读：0：关闭 ；1：使能 写：0：无效；1：使能
[6]	R/W1	0	MCPWM(IRQ0)	MCPWM 中断 IRQ0 使能 读：0：关闭；1：使能 写：0：无效；1：使能
[5]	R/W1	0	SPI1	SPI1 中断使能 读：0：关闭；1：使能 写：0：无效；1：使能
[4]	R/W1	0	SPI0	SPI0 中断使能 读：0：关闭；1：使能 写：0：无效；1：使能
[3]	R/W1	0	UART1	UART1 中断使能 读：0：关闭；1：使能 写：0：无效；1：使能
[2]	R/W1	0	UART0	UART0 中断使能 读：0：关闭 ；1：使能 写：0：无效；1：使能
[1]	R/W1	0	FLASH	FLASH 中断使能 读：0：关闭 ；1：使能 写：0：无效；1：使能
[0]	R/W1	0	AON	AON 低功耗唤醒中断使能 读：0：关闭 ；1：使能 写：0：无效；1：使能

6.3.2 ICER 寄存器

地址：0xE000E180

位域	类型	复位	名称	描述
[31:29]	--	--	--	--
[28]	W1	0	ANAC1	模拟中断 1 禁止 读：0 写：0：无效；1：禁止
[27]	W1	0	ANAC0	模拟中断 0 禁止 读：0 写：0：无效；1：禁止
[26]	W1	0	PLL_ERR	PLL 异常中断禁止 读：0 写：0：无效；1：禁止
[25]	W1	0	TJ_OTW	温度检测 OTW 中断禁止 读：0 写：0：无效；1：禁止
[24]	W1	0	TJ_OTS	温度检测 OTS 中断禁止 读：0 写：0：无效；1：禁止
[23]	W1	0	CSA	过流检测中断禁止 读：0 写：0：无效；1：禁止
[22]	W1	0	PVD	低电压检测中断禁止 读：0 写：0：无效；1：禁止
[21]	W1	0	BOR	上电复位中断禁止 读：0：关闭 ；1：使能 写：0：无效；1：使能

位域	类型	复位	名称	描述
[20]	W1	0	DMA	DMA 中断禁止 读：0 写：0：无效；1：禁止
[19]	W1	0	QEP	QEP 编码器中断禁止 读：0 写：0：无效；1：禁止
[18]	W1	0	HALL	HALL 编码器中断禁止 读：0 写：0：无效；1：禁止
[17]	W1	0	ADC	ADC 中断禁止 读：0 写：0：无效；1：禁止
[16]	W1	0	TIM16	TIM16 中断禁止 读：0 写：0：无效；1：禁止
[15]	W1	0	TIM15	TIM15 中断禁止 读：0 写：0：无效；1：禁止
[14]	W1	0	TIM14	TIM14 中断禁止 读：0 写：0：无效；1：禁止
[13]	W1	0	TIM13	TIM13 中断禁止 读：0 写：0：无效；1：禁止
[12]	W1	0	GPIOC	GPIOC 中断禁止 读：0 写：0：无效；1：禁止

位域	类型	复位	名称	描述
[11]	W1	0	GPIOB	GPIOB 中断禁止 读：0 写：0：无效；1：禁止
[10]	W1	0	GPIOA	GPIOA 中断禁止 读：0 写：0：无效；1：禁止
[9]	W1	0	I2C1	I2C1 中断禁止 读：0 写：0：无效；1：禁止
[8]	W1	0	I2C0	I2C0 中断禁止 读：0 写：0：无效；1：禁止
[7]	W1	0	MCPWM(IRQ1)	MCPWM 中断 IRQ1 禁止 读：0 写：0：无效；1：禁止
[6]	W1	0	MCPWM(IRQ0)	MCPWM 中断 IRQ0 禁止 读：0：关闭；1：使能 写：0：无效；1：使能
[5]	W1	0	SPI1	SPI1 中断禁止 读：0 写：0：无效；1：禁止
[4]	W1	0	SPI0	SPI0 中断禁止 读：0 写：0：无效；1：禁止
[3]	W1	0	UART1	UART1 中断禁止 读：0 写：0：无效；1：禁止

位域	类型	复位	名称	描述
[2]	W1	0	UART0	UART0 中断禁止 读：0 写：0：无效；1：禁止
[1]	W1	0	FLASH	FLASH 中断禁止 读：0 写：0：无效；1：禁止
[0]	W1	0	AON	AON 低功耗唤醒中断禁止 读：0 写：0：无效；1：禁止

6.3.3 ISPR 寄存器

地址：0xE000E200

位域	类型	复位	名称	描述
[31:29]	--	--	--	--
[28]	W1	0	ANAC1	模拟中断 1 挂起禁止 读：0 写：0：无效；1：挂起禁止
[27]	W1	0	ANAC0	模拟中断 0 挂起禁止 读：0 写：0：无效；1：挂起禁止
[26]	W1	0	PLL_ERR	PLL 异常中断挂起禁止 读：0 写：0：无效；1：挂起禁止
[25]	W1	0	TJ_OTW	温度检测 OTW 中断挂起禁止 读：0 写：0：无效；1：挂起禁止

位域	类型	复位	名称	描述
[24]	W1	0	TJ_OTS	温度检测 OTS 中断挂起禁止 读：0 写：0：无效；1：挂起禁止
[23]	W1	0	CSA	过流检测中断挂起禁止 读：0 写：0：无效；1：挂起禁止
[22]	W1	0	PVD	低电压检测中断禁止 读：0 写：0：无效；1：挂起禁止
[21]	W1	0	BOR	上电复位中断挂起禁止 读：0 写：0：无效；1：挂起禁止
[20]	W1	0	DMA	DMA 中断挂起禁止 读：0 写：0：无效；1：挂起禁止
[19]	W1	0	QEP	QEP 编码器中断挂起禁止 读：0 写：0：无效；1：挂起禁止
[18]	W1	0	HALL	HALL 编码器中断挂起禁止 读：0 写：0：无效；1：挂起禁止
[17]	W1	0	ADC	ADC 中断挂起禁止 读：0 写：0：无效；1：挂起禁止
[16]	W1	0	TIM16	TIM16 中断挂起禁止 读：0 写：0：无效；1：挂起禁止

位域	类型	复位	名称	描述
[15]	W1	0	TIM15	TIM15 中断挂起禁止 读：0 写：0：无效；1：挂起禁止
[14]	W1	0	TIM14	TIM14 中断挂起禁止 读：0 写：0：无效；1：挂起禁止
[13]	W1	0	TIM13	TIM13 中断挂起禁止 读：0 写：0：无效；1：挂起禁止
[12]	W1	0	GPIOC	GPIOC 中断挂起禁止 读：0 写：0：无效；1：挂起禁止
[11]	W1	0	GPIOB	GPIOB 中断挂起禁止 读：0 写：0：无效；1：挂起禁止
[10]	W1	0	GPIOA	GPIOA 中断挂起禁止 读：0 写：0：无效；1：挂起禁止
[9]	W1	0	I2C1	I2C1 中断挂起禁止 读：0 写：0：无效；1：挂起禁止
[8]	W1	0	I2C0	I2C0 中断挂起禁止 读：0 写：0：无效；1：挂起禁止
[7]	W1	0	MCPWM(IRQ1)	MCPWM 中断 IRQ1 挂起禁止 读：0 写：0：无效；1：挂起禁止

位域	类型	复位	名称	描述
[6]	W1	0	MCPWM(IRQ0)	MCPWM 中断 IRQ0 挂起禁止 读：0 写：0：无效；1：挂起禁止
[5]	W1	0	SPI1	SPI1 中断挂起禁止 读：0 写：0：无效；1：挂起禁止
[4]	W1	0	SPI0	SPI0 中断挂起禁止 读：0 写：0：无效；1：挂起禁止
[3]	W1	0	UART1	UART1 中断挂起禁止 读：0 写：0：无效；1：挂起禁止
[2]	W1	0	UART0	UART0 中断挂起禁止 读：0 写：0：无效；1：挂起禁止
[1]	W1	0	FLASH	FLASH 中断挂起禁止 读：0 写：0：无效；1：挂起禁止
[0]	W1	0	AON	AON 低功耗唤醒中断挂起禁止 读：0 写：0：无效；1：挂起禁止

7 系统控制

7.1 概述

系统控制主要包括：

- 系统硬件状态查询
- UUID 信息查询
- 低功耗模式配置
- 版本查询

7.2 状态查询

为了方便产品追踪和管理，芯片在制造过程中，会将特有信息写入到 FLASH 的指定区域。芯片在上电的过程中，会将将该区域内的数据读出并映射到系统控制中相就的寄存器组中，随时可读查询。

- 查询封装信息，（GPIO 的使用）
- 查询唯一标识码 UUID(128 位）
- 查询硬件规格(FLASH 大小，SRAM 大小)
- 查询芯片保护状态
- 查询芯片加工状态
- 查询芯片功能状态
- 查询芯片版本

7.3 低功耗模式

芯片工作模式主要有：正常模式、睡眠模式、深度睡眠模式

使用 ARM 的 WFI 指令使芯片进入到睡眠模式和深度睡眠模式，由系统控制寄存器 SCR 中的 SLEEPDEEP 位来区分两种睡眠模式。可参考系统控制寄存器(SCR)说明

进入睡眠模式后，只支持 GPIO 唤醒和 LPTMR 唤醒

注：两种睡眠模式的区别，深度睡眠采用了大部分模块掉电形式来降低功耗

7.3.1 低功耗架构

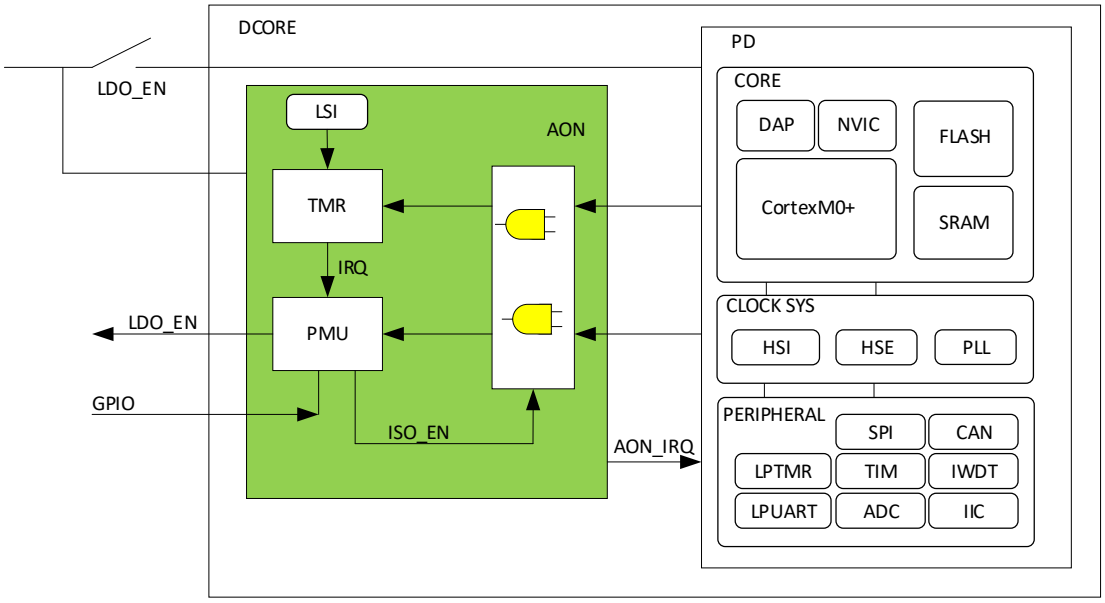


图 7-1 低功耗架构示意图

执行 WFI 后，CortexM0+内核输出 SLEEPING 和 DEEPSLEEP 两个指示信号，控制部分 PMU 当检测到两个信号状态，控制 LDO 及 ISO 状态，由于 D 深度睡眠模式有掉电部分，芯片内部有掉电部分的信号输出到非掉电部分的电路中时，必须要有隔离单元 ISO，进入深度睡眠后，先使能隔离单元 ISO，之后 PMU 关掉 LDO 供电，只有 AON 部分存在供电，当 GPIO 输入有效唤醒信号或 LPTMR 达到配置中的阈值后，即可唤醒，输出 AON_IRQQ 中断，软件可根据中断信号 AON_IRQ，来判定唤醒过程

7.3.2 PMU 控制

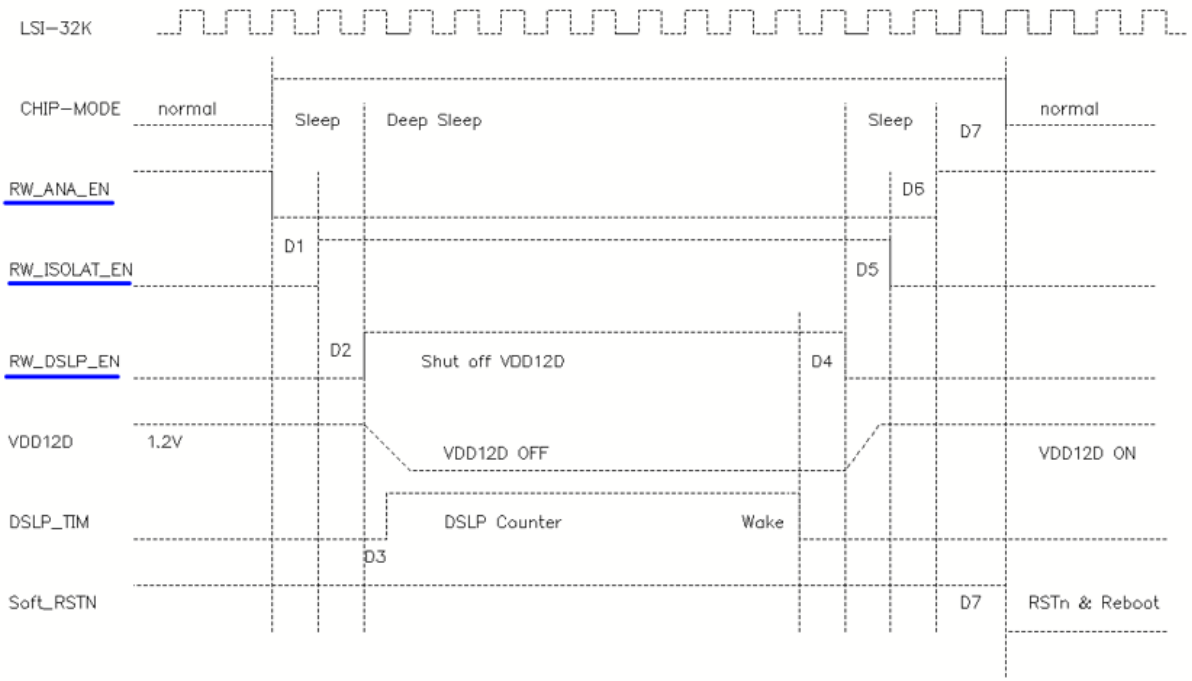


图 7-2PMU 控制示意图

低功耗模式及唤醒机制主要控制部分即是 PMU，即电源管理单元，其主要部分是由状态机来控制模拟相应时序，控制功耗机制中的上电，唤醒，工作，隔离，掉电过程的相关控制

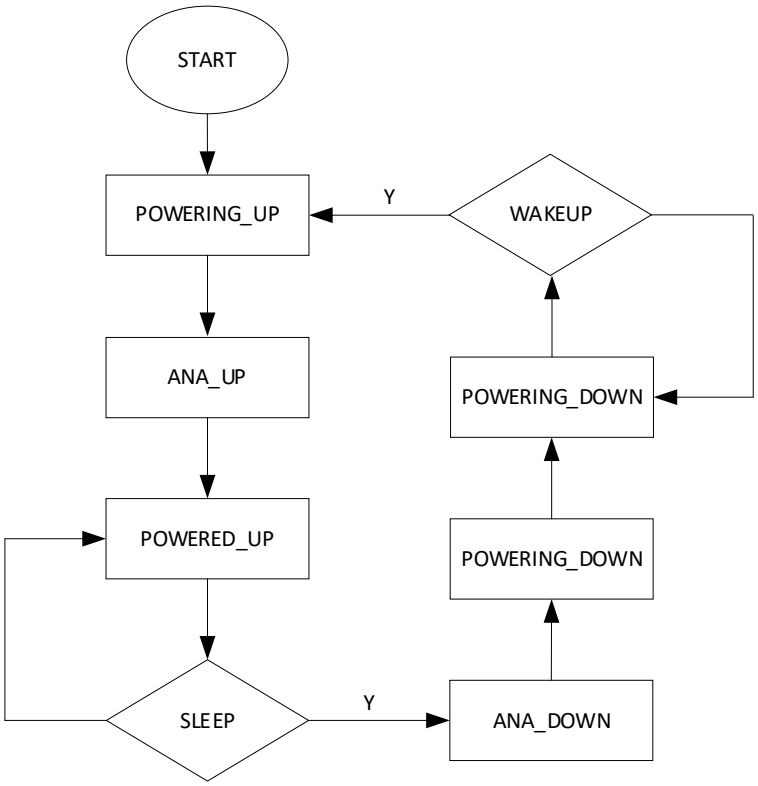


图 7-3 低功耗模式流程图

7.4 寄存器列表

地址	寄存器	复位值	描述	备注
0x4002 0400	UUID0	0x0	UUID[31:0]	
0x4002 0404	UUID1	0x0	UUID[63:32]	
0x4002 0408	UUID2	0x0	UUID[95:64]	
0x4002 040C	UUID3	0x0	UUID[127:96]	
0x4002 0410	USER0	0x0	USER0	
0x4002 0414	USER1	0x0	USER1	
0x4002 0418	USER2	0x0	USER2	
0x4002 041C	USER3	0x0	USER3	
0x4002 0420	TAG	0x0	生产过程记录状态寄存器	
0x4002 0424	FSIZE	0x0	FLASH 规格查询寄存器	
0x4002 0428	RSIZE	0x0	SRAM 规格查询寄存器	
0x4002 042C	GPIOE0	0x0	GPIOA/B 组管脚状态查询	
0x4002 0430	GPIOE1	0x0	GPIOC 组管脚状态查询	
0x4002 0440	USER4	0x0	USER4	
0x4002 0444	USER5	0x0	USER5	
0x4002 0500	WAKEIR	0x0	低功耗唤醒中断状态寄存器	
0x4002 0504	WAKECR	0x0	低功耗控制寄存器	
0x4002 0508	LPTMRTH	0x400	低功耗唤醒 TMR 阈值设置寄存器	
0x4002 050C	INTERVAL	0x0	低功耗唤醒控制信号时序控制	
0x4002 07F8	VERID	0x0	芯片版本状态查询	

7.5 寄存器描述

7.5.1 UUID 寄存器

地址：0x40020400

位域	类型	复位	名称	描述
[31:0]	R	0	UUID[31:0]	UUID

地址：0x40020404

位域	类型	复位	名称	描述
[31:0]	R	0	UUID[63:32]	UUID

地址：0x40020408

位域	类型	复位	名称	描述
[31:0]	R	0	UUID[95:64]	UUID

地址：0x4002040C

位域	类型	复位	名称	描述
[31:0]	R	0	UUID[127:96]	UUID

7.5.2 USER 寄存器

地址：0x40020410

位域	类型	复位	名称	描述
[31:0]	R	0	USER0	--

地址：0x40020414

位域	类型	复位	名称	描述
[31:0]	R	0	USER1	--

地址：0x40020418

位域	类型	复位	名称	描述
[31:0]	R	0	USER2	--

地址：0x4002041C

位域	类型	复位	名称	描述
[31:0]	R	0	USER3	--

地址：0x40020440

位域	类型	复位	名称	描述
[31:0]	R	0	USER4	--

地址：0x40020444

位域	类型	复位	名称	描述
[31:0]	R	0	USER5	--

7.5.3 TAG 寄存器

地址：0x40020420

位域	类型	复位	名称	描述
[31:11]	R	0	--	--
[10]	R	0	DISRSTN	GPIOB1 禁用 RSTN 功能复用
[9]	R	0	PROTECT2	芯片 LEVEL2 加密级别
[8]	R	0	PROTECT1	芯片 LEVEL1 加密级别
[7:6]	R	0	--	--
[5]	R	0	RRAM_CPLOCK	芯片 RRAM 存储器的 CP 测试接口禁用
[4]	R	0	CHIPINFO_TAG	芯片已加载芯片规格信息标志位
[3]	R	0	TRIM_TAG3	芯片已加载第 4 组模拟 TRIM 值用于校准
[2]	R	0	TRIM_TAG2	芯片已加载第 3 组模拟 TRIM 值用于校准
[1]	R	0	TRIM_TAG1	芯片已加载第 2 组模拟 TRIM 值用于校准
[0]	R	0	TRIM_TAG0	芯片已加载第 1 组模拟 TRIM 值用于校准

7.5.4 FSIZE 寄存器

地址：0x40020424

位域	类型	复位	名称	描述
[31:26]	R	0	--	--
[25:0]	R	0	FLASH_SIZE	指示 FLASH 容量规格，单位 1KB 注：最大可支持 128KB-32Byte

7.5.5 RSIZE 寄存器

地址：0x40020428

位域	类型	复位	名称	描述
[31:16]	R	0	--	--
[15:0]	R	0	SRAM_SIZE	指示 FLASH 容量规格，单位 1KB 注：最大可支持 16KB

7.5.6 GPIOE0 寄存器

地址：0x4002042C

位域	类型	复位	名称	描述
[31:16]	R	0	GPIOBE	指示 GPIOA 组的使能状态
[15:0]	R	0	GPIOAE	指示 GPIOB 组的使能状态

7.5.7 GPIOE1 寄存器

地址：0x40020430

位域	类型	复位	名称	描述
[31:16]	R	0	DISCODE	查询 DISRSTN 密钥 当 DISCODE 为 5AA5 时，DISRSTN 状态位会置 1
[15:0]	R	0	GPIOCE	指示 GPIOC 组的使能状态

7.5.8 WAKEIR 寄存器

地址：0x40020500

位域	类型	复位	名称	描述
[31:19]	R	0	--	--
[18]	R/W1C	0	RSTN_WAKEUP	RSTN 复位低功耗唤醒标志位
[17]	R/W1C	0	GPIO_WAKEUP	GPIO 低功耗唤醒标志位
[16]	R/W1C	0	LPTMR_WAKEUP	LPTRM 低功耗唤醒标志位
[15:3]	R	0	--	--
[2]	R/W	0	RSTN_IE	RSTN 低功耗唤醒中断使能
[1]	R/W	0	GPIO_IE	GPIO 低功耗唤醒中断使能
[0]	R/W	0	LPTMR_IE	LPTMR 低功耗唤醒中断使能

7.5.9 WAKECR 寄存器

地址：0x40020504

位域	类型	复位	名称	描述
[31:28]	R	0	--	--
[27]	R/W	0	GPIO_IN11_POL	PC4 用于低功耗唤醒的极性配置
[26]	R/W	0	GPIO_IN10_POL	PC3 用于低功耗唤醒的极性配置
[25]	R/W	0	GPIO_IN9_POL	PB13 用于低功耗唤醒的极性配置
[24]	R/W	0	GPIO_IN8_POL	PB12 用于低功耗唤醒的极性配置
[23]	R/W	0	GPIO_IN7_POL	PB5 用于低功耗唤醒的极性配置
[22]	R/W	0	GPIO_IN6_POL	PB4 用于低功耗唤醒的极性配置
[21]	R/W	0	GPIO_IN5_POL	PB3 用于低功耗唤醒的极性配置
[20]	R/W	0	GPIO_IN4_POL	PB2 用于低功耗唤醒的极性配置
[19]	R/W	0	GPIO_IN3_POL	PA14 用于低功耗唤醒的极性配置
[18]	R/W	0	GPIO_IN2_POL	PA13 用于低功耗唤醒的极性配置
[17]	R/W	0	GPIO_IN1_POL	PA4 用于低功耗唤醒的极性配置
[16]	R/W	0	GPIO_IN0_POL	PA3 用于低功耗唤醒的极性配置

位域	类型	复位	名称	描述
[15:13]	R/W	0	--	--
[12]	R/W	0	LPTMR_EN	LPTMR 用于低功耗唤醒计数器使能
[11]	R/W	0	GPIO_IN11_EN	PC4 用于低功耗唤醒的使能
[10]	R/W	0	GPIO_IN10_EN	PC3 用于低功耗唤醒的使能
[9]	R/W	0	GPIO_IN9_EN	PB13 用于低功耗唤醒的使能
[8]	R/W	0	GPIO_IN8_EN	PB12 用于低功耗唤醒的使能
[7]	R/W	0	GPIO_IN7_EN	PB5 用于低功耗唤醒的使能
[6]	R/W	0	GPIO_IN6_EN	PB4 用于低功耗唤醒的使能
[5]	R/W	0	GPIO_IN5_EN	PB3 用于低功耗唤醒的使能
[4]	R/W	0	GPIO_IN4_EN	PB2 用于低功耗唤醒的使能
[3]	R/W	0	GPIO_IN3_EN	PA14 用于低功耗唤醒的使能
[2]	R/W	0	GPIO_IN2_EN	PA13 用于低功耗唤醒的使能
[1]	R/W	0	GPIO_IN1_EN	PA4 用于低功耗唤醒的使能
[0]	R/W	0	GPIO_IN0_EN	PA3 用于低功耗唤醒的使能

7.5.10 LPTMRTH 寄存器

地址：0x40020508

位域	类型	复位	名称	描述
[31:0]	R/W	0	LPTMRTH	低功耗计数器阈值配置

7.5.11 INTERVAL 寄存器

地址：0x4002050C

位域	类型	复位	名称	描述
[31:6]	R	0	--	--
[5:0]	R	0x14	INTERVAL	配置低功耗控制信号等待时间

7.5.12 VERID 寄存器

地址：0x400207F8

位域	类型	复位	名称	描述
[31:4]	R	0xBD01	--	--
[5:0]	R	0	RO_VER_CODE	芯片版本号

8 时钟复位

8.1 概述

8.1.1 简介

MCU 内集成了时钟复位控制（RCC）模块，用于对芯片内部各类时钟源的使能、倍频、分频、选择及输出进行统一管理，并对各功能模块的时钟使能和复位进行集中控制。模块支持 HSI（内部高速 RC 振荡时钟）、HSE（外部高速晶振时钟）、LSI（内部低速 RC 振荡时钟）以及 PLL 倍频时钟共 4 类时钟源，各时钟源可独立使能/禁止，并通过对应的就绪标志判断其是否稳定可用。系统时钟支持在 HSI、HSE、PLLCLK 等多个时钟源间无缝切换，不影响程序运行，同时提供 MCO 时钟输出功能，便于通过外部引脚观测芯片内部时钟状态。此外，模块还统一管理各外设的时钟使能与复位控制，以及系统各类复位源的使能配置与复位状态查询。

8.1.2 MCU 主要时钟源包括：

- 高速晶振时钟，频率 16Mhz
- 内部高速 RC 时钟，频率 32MHz
- PLL 高速时钟，最大频率 96MHz
- 内部低速时钟，频率 32KHz

8.1.3 模块框图

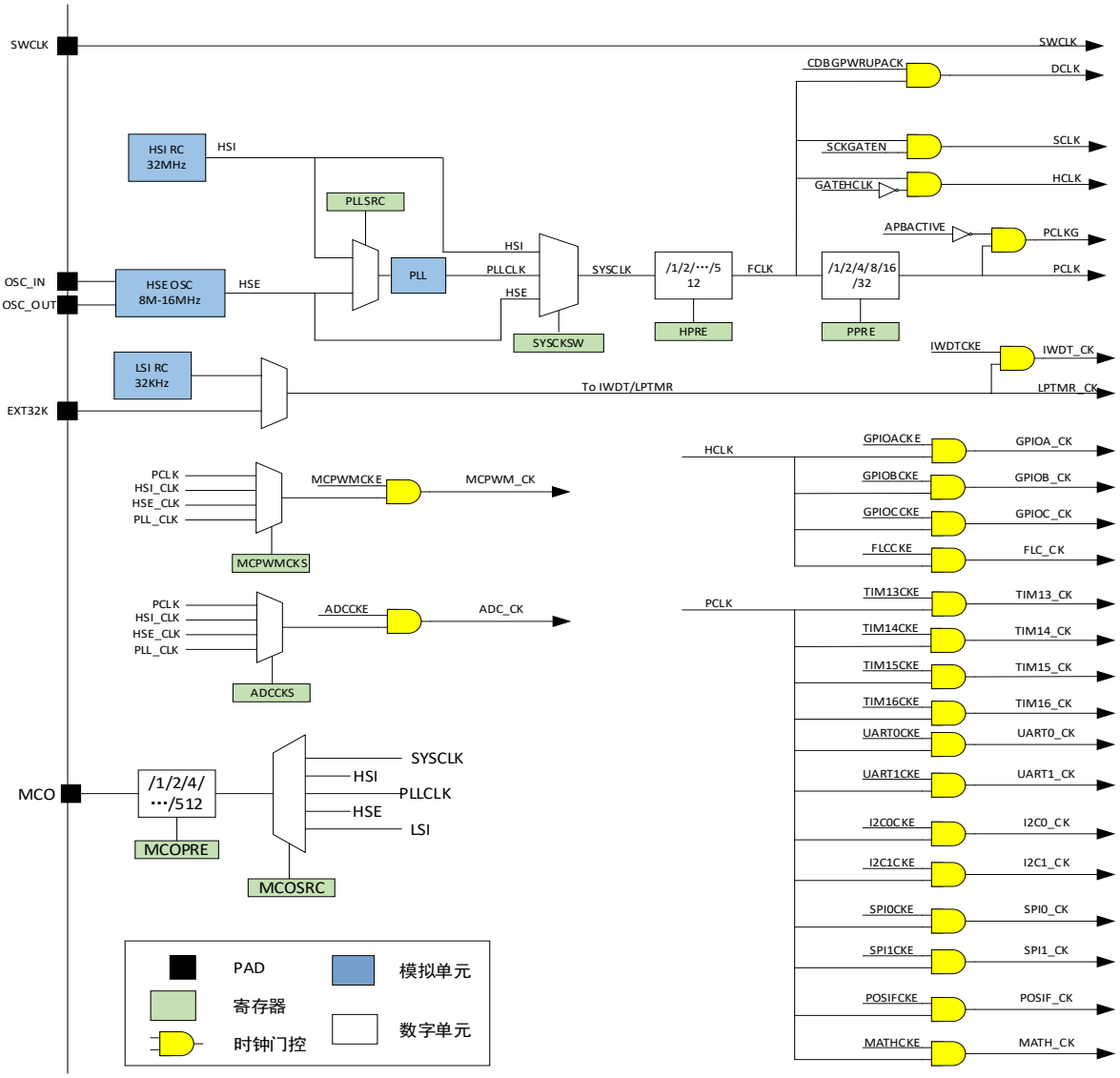


图 8-1 时钟配置框图

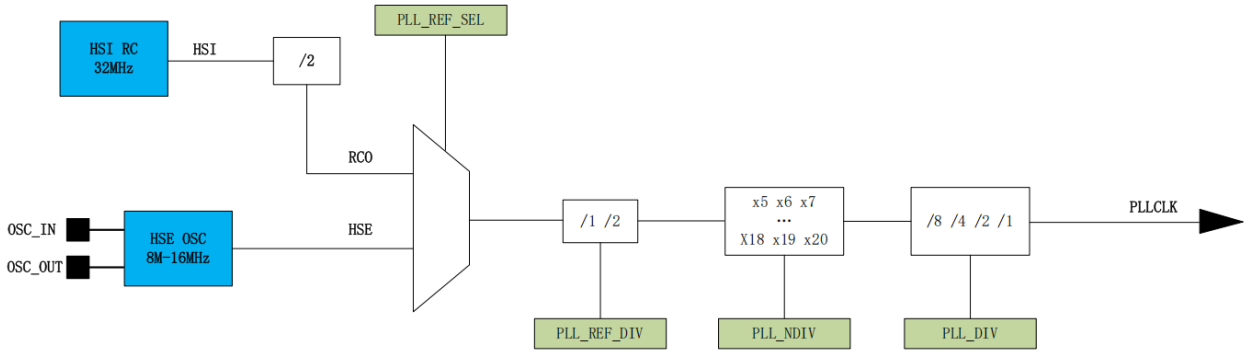


图 8-2 PLL 配置框图功能描述

8.1.4 时钟切换

芯片提供高速晶体振荡时钟，内部 RC 振荡时钟，内部倍频时钟系统主时钟。在程序运行过程中，3 个时钟源可以无缝切换，不影响程序运行。

为保证系统能正常运行，配置时钟切换必须满足以下条件：

- 切换时钟必须使能
- 切换时钟必须 Ready
- 用户必须选择切换时钟 SYSCLK_SEL

8.1.5 复位信号源

复位系统有多个复位源，主要分为几个部分：

- 上电复位: A2D_VDD_RSTN
- 内核复位：SYSRESETREQ/LOCKUP/PMURESETREQ
- 安全机制复位：IWDTRESETREQ
- 外部复位：RSTN

8.1.6 上电时序

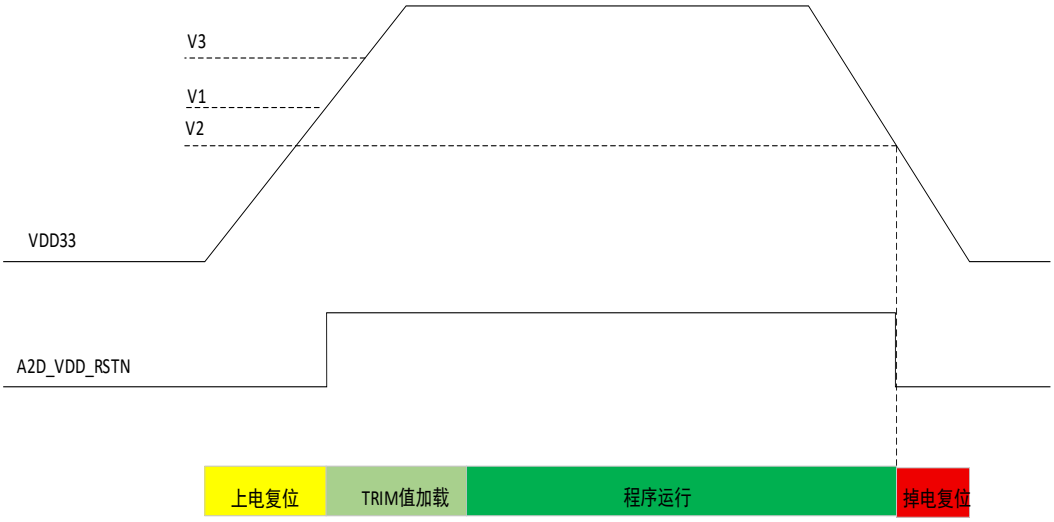


图 8-3 时钟上电时序

- **注：**
- **V1 为 POR 电压阈值**
- **V2 为 BOR 电压阈值**
- **V3 为 PVD 电压阈值，可配置**

8.2 寄存器列表

时钟在芯片中的基地址是 0x4002_0000

表 8-1 时钟复位寄存器地址分配

名称	偏移地址	描述
VERID	0x00	
SYSRSTF	0x04	系统复位标志位
SYSRSTE	0x08	系统复位使能
RSTKEY	0x0C	复位密钥寄存器
AHBRST	0x10	AHB 总线外设复位寄存器
ANARST	0x14	模拟外设复位寄存器
DIGRST	0x18	数字外设复位寄存器
COMRST	0x1C	通信外设复位寄存器
TIMRST	0x20	数字定时器复位寄存器
HSICR	0x30	内部高速 RC 时钟控制寄存器
LSICR	0x34	内部低速 RC 时钟控制寄存器
HSECR	0x38	晶振时钟控制寄存器
PLLCR0	0x40	PLL 时钟控制寄存器
PLLCR1	0x44	PLL 时钟倍频率系数寄存器
SYSCKCR	0x50	系统时钟控制寄存器
MCOCR	0x54	MCO 时钟输出控制寄存器
AHBCKE	0x58	AHB 总线外设时钟使能寄存器
ANACKE	0x5C	模拟外设时钟使能寄存器
DIGCKE	0x60	数字外设时钟使能寄存器
COMCKE	0x64	通信外设时钟使能寄存器
TIMCKE	0x68	数字定时器时钟使能寄存器
PERCKS0	0x70	外设时钟源选择寄存器

8.3 时钟复位寄存器描述

8.3.1 SYSRSTF 寄存器　　系统复位标志寄存器

地址：0x4002_0004

复位值:0x0

位域	类型	复位	名称	描述
[31:25]	R	0	--	--
[24]	R	0	RETRIM	由 RETRIM 引起的系统复位
[23:17]	R	0	--	--
[16]	R	0	IWDT	由 WDT 引起的系统复位
[15:11]	R	0	--	--
[10]	R	0	LOCKUP	由内核锁后引起的系统复位
[9]	R	0	SFRST	由内核 SYSRESET 指令引起的系统复位
[8]	R	0	PINRST	由外部 RSTN 引起的系统复位
[7:3]	R	0	--	--
[2]	R	0	PVD	由低电压监测 PVD 引起的系统复位
[1]	R	0	BOR	由掉电监测引起的系统复位
[0]	R	0	POR	由上电监测引起的系统复位

8.3.2 SYSRSTE 寄存器　　系统复位使能寄存器

地址：0x4002_0008

复位值:0x0

位域	类型	复位	名称	描述
[31:17]	R/W	0	--	--
[16]	R/W	0	IWDTRSTE	使能 IWDT 饿狗对系统复位
[15:11]	R/W	0	--	--
[10]	R/W	0	LOCKUPRSTE	使能内核死锁后对系统复位
[9:3]	R/W	0	--	--

[2]	R/W	0	PVDRSTE	使能低电压监测 PVD 对系统复位
[1:0]	R/W	0	--	--

8.3.3 RSTKEY 寄存器　复位密钥寄存器

地址：0x4002_000C

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	R/W	0	--	--
[15:0]	W	0	RETRIM	写入 0xAB56 可产生一次 RETRIM 系统复位

8.3.4 AHBRST 寄存器　AHB 总线外设复位寄存器

地址：0x4002_0010

复位值:0x0

位域	类型	复位	名称	描述
[31:19]	W1	0	--	--
[18]	W1	0	GPIOCRST	复位 GPIOC 控制
[17]	W1	0	GPIOBRST	复位 GPIOB 控制
[16]	W1	0	GPIOARST	复位 GPIOA 控制
[15:1]	W1	0	--	--
[0]	W1	0	FLCRST	复位 FLASH 控制器

8.3.5 ANARST 寄存器　模拟外设复位寄存器

地址：0x4002_0014

复位值:0x0

位域	类型	复位	名称	描述
[31:1]	W1	0	--	--
[0]	W1	0	ADCRST	复位 ADC 控制器

8.3.6 DIGRST 寄存器　数字外设复位寄存器

地址：0x4002_0018

复位值:0x0

位域	类型	复位	名称	描述
[31:17]	W1	0	--	--
[16]	W1	0	IWDTRST	复位 IWDT
[15:3]	R	0	--	--
[2]	W1	0	QEPRST	复位 QEP 编码器
[1]	W1	0	HALLRST	复位 HALL 编码器
[0]	W1	0	MATHRST	复位 MATH 计算模块

8.3.7 COMRST 寄存器　通信外设复位寄存器

地址：0x4002_001C

复位值:0x0

位域	类型	复位	名称	描述
[31:26]	--	--	--	--
[25]	W1	0	I2C1RST	复位 I2C1 外设
[24]	W1	0	I2C0RST	复位 I2C0 外设
[23:18]	--	--	--	--
[17]	W1	0	SPI1RST	复位 SPI1 外设
[16]	W1	0	SPI0RST	复位 SPI0 外设
[15:10]	--	--	--	--
[9]	W1	0	UART1RST	复位 UART1 外设
[8]	W1	0	UART0RST	复位 UART0 外设
[7:1]	--	--	--	--
[0]	W1	0	CAN0RST	复位 CAN 外设

8.3.8 TIMRST 寄存器　数字定时器复位寄存器

地址：0x4002_0020

复位值:0x0

位域	类型	复位	名称	描述
[31:25]	--	--	--	--
[24]	W1	0	MCPWMRST	复位 MCPWM 外设
[23:17]	--	--	--	--
[16]	W1	0	TIM16RST	复位 TIM16 外设
[15]	W1	0	TIM15RST	复位 TIM15 外设
[14]	W1	0	TIM14RST	复位 TIM14 外设
[13]	W1	0	TIM13RST	复位 TIM13 外设
[12:0]	--	--	--	--

8.3.9 HSICR 寄存器　内部高速 RC 时钟控制寄存器

地址：0x4002_0030

复位值:0x1

位域	类型	复位	名称	描述
[31]	R	0	HSI_RDY	HSI 输出稳定指示信号
[30:1]	--	--	--	--
[0]	R/W	1	HSI_EN	高速 RC 时钟 HSI 使能

8.3.10 LSICR 寄存器　内部低速 RC 时钟控制寄存器

地址：0x4002_0034

复位值:0x1

位域	类型	复位	名称	描述
[31]	R	0	LSI_RDY	LSI 输出稳定指示信号
[30:1]	--	--	--	--
[1]	R/W	0	LSIEXE	低速时钟外部输出选择

[0]	R/W	1	LSI_EN	低速 RC 时钟 LSI 使能

8.3.11 HSECR 寄存器　晶振时钟控制寄存器

地址: 0x4002_0038

复位值:0x0

位域	类型	复位	名称	描述
[31]	R	0	HSE_RDY	HSE 输出稳定指示信号
[30:2]	--	--	--	--
[2]	R/W	0	HSE_XI_OFF	晶振 XI 管脚禁用
[1]	R/W	0	HSE_XO_OFF	晶振 XO 管脚禁用
[0]	R/W	0	HSE_EN	晶振时钟使能

8.3.12 PLLCR0 寄存器　PLL 时钟控制寄存器

地址: 0x4002_0040

复位值:0x0

位域	类型	复位	名称	描述
[31:18]	--	--	--	--
[17]	R	0	RO_PLL_RDY	PLL 输出 RDY 只读信号
[16]	R/W1C	0	PLL_ERR	PLL 失锁指示信号
[15]	R	0	PLL_RDY	PLL 时钟稳定指示信号
[14:1]	--	--	--	--
[0]	R/W	0	PLL_EN	PLL 倍频时钟使能

8.3.13 PLLCR1 寄存器　PLL 时钟倍频率系数寄存器

地址: 0x4002_0044

复位值:0x403

位域	类型	复位	名称	描述
[31:12]	--	--	--	--

[11:8]	R/W	0x4	PLL_MULT	PLL 输出倍频系数 0000：5 0100：9 1000：13 1111：20 step：1
[7:2]	--	--	--	--
[1:0]	R/W	0x3	PLL_DIV	PLL 输出分频系数 00:1/8 01:1/4 10:1/2 11:1

8.3.14 SYSCKCR 寄存器　　系统时钟控制寄存器

地址：0x4002_0050

复位值:0x0

位域	类型	复位	名称	描述
[31:29]	--	--	--	--
[28:24]	R/W	0	PPRE	PCLK 分频系数
[23:21]	--	--	--	--
[20:16]	R/W	0	HPRE	HCLK 分频系数
[7:2]	--	--	--	--
[1:0]	R/W	0	SYSCKSEL	系统时钟源选择： 00：HSI 01：HSE 10：PLL

8.3.15 MCOCR 寄存器　MCO 时钟输出控制寄存器

地址：0x4002_0054

复位值:0x0

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11:8]	R/W	0	MCOPRE	MCO 时钟分频输出 0x0：不分频 0x1：2 分频 0x2：4 分频 0x3：8 分频 0x4：16 分频 0x5：32 分频 0x6：64 分频 0x7：128 分频 其它：256 分频
[7:4]	--	--	--	--
[3:0]	R/W	0x3	MCOSRC	MCO 时钟输出选择 0x0：SYSCLK 0x1：HSI 0x2：PLL 0x3：HSE 0x4：LSI

8.3.16 AHBCKE 寄存器　AHB 总线外设时钟使能寄存器

地址：0x4002_0058

复位值:0x0

位域	类型	复位	名称	描述
[31:19]	--	--	--	--
[18]	R/W	0	GPIOCCKE	GPIOC 时钟使能
[17]	R/W	0	GPIOBCKE	GPIOB 时钟使能
[16]	R/W	0	GPIOACKE	GPIOA 时钟使能
[15:1]	R/W	0	--	--
[0]	R/W	0	FLCCKE	FLASH 控制器时钟使能

8.3.17 ANACKE 寄存器　模拟外设时钟使能寄存器

地址：0x4002_005C

复位值:0x0

位域	类型	复位	名称	描述
[31:1]	--	--	--	--
[0]	R/W	0	ADCCKE	ADC 控制器时钟使能

8.3.18 DIGCKE 寄存器　数字外设时钟使能寄存器

地址：0x4002_0060

复位值:0x0

位域	类型	复位	名称	描述
[31:17]	--	--	--	--
[16]	R/W	0	IWDTCKE	IWDT 时钟使能
[15:3]	--	--	--	--
[2]	R/W	0	QEPCKE	QEP 编码器时钟使能
[1]	R/W	0	HALLCKE	HAL 编码器时钟使能
[0]	R/W	0	MATHCKE	MATH 算术单元时钟使能

8.3.19 COMCKE 寄存器　通信外设时钟使能寄存器

地址：0x4002_0064

复位值:0x0

位域	类型	复位	名称	描述
[31:26]	--	--	--	--
[25]	R/W	0	I2C1CKE	通信外设 I2C1 时钟使能
[24]	R/W	0	I2C0CKE	通信外设 I2C0 时钟使能
[23:18]	--	--	--	--
[17]	R/W	0	SPI1CKE	通信外设 SPI1 时钟使能
[16]	R/W	0	SPI0CKE	通信外设 SPI0 时钟使能
[15:10]	--	--	--	--
[9]	R/W	0	UART1CKE	通信外设 UART1 时钟使能
[8]	R/W	0	UART0CKE	通信外设 UART0 时钟使能
[7:1]	--	--	--	--
[0]	R/W	0	CAN0CKE	通信外设 CAN 时钟使能

8.3.20 TIMCKE 寄存器　数字定时器时钟使能寄存器

地址：0x4002_0068

复位值:0x0

位域	类型	复位	名称	描述
[31:25]	--	--	--	--
[24]	R/W	0	MCPWMCKE	外设 MCPWM 时钟使能
[23:17]	--	--	--	--
[16]	R/W	0	TIM16CKE	外设 TIM16 时钟使能
[15]	R/W	0	TIM15CKE	外设 TIM15 时钟使能
[14]	R/W	0	TIM14CKE	外设 TIM14 时钟使能
[13]	R/W	0	TIM13CKE	外设 TIM13 时钟使能
[12:0]	R/W	--	--	--

8.3.21 PERCKS0 寄存器　外设时钟源选择寄存器

地址: 0x4002_0070

复位值:0x0

位域	类型	复位	名称	描述
[31:6]	--	--	--	--
[5:4]	R/W	0	MCPWMCKS	MCPWM 工作时钟选择： 0x0：PCLK 0x1：HSI 0x2：HSE 0x3：PLL
[3:2]	--	--	--	--
[1:0]	R/W	0	ADC0CKS	ADC 工作时钟选择： 0x0：PCLK 0x1：HSI 0x2：HSE 0x3：PLL

8.4 应用指南

8.4.1 功能说明

1. 时钟源

MCU 支持 HSI（内部 32MHz RC）、HSE（外部晶振/外部时钟输入）、LSI（内部 32KHz RC）以及 PLL（倍频后最高可达 96MHz）共 4 类时钟源，均可独立使能，并通过就绪标志（HSIRDY/HSERDY/LSIRDY/RO_PLL_RDY）判断是否稳定可用。

SDK 接口：

clk_erc_e rcc_hal_osc_enable(clock_osc_e osc, uint32_t timeout_us); /*使能指定时钟源并等待就绪，超时返回 ERC_CLK_OSC_NOT_READY*/

void rcc_hal_osc_enable_all(void); /*一次性使能 HSI/PLL/LSI/HSE*/

注意：HSE/LSE 为外部晶振，起振需要时间，使用前必须等待对应 RDY 标志置 1；HSI 为内部 RC，上电后很快即可就绪。

2. 系统时钟切换

系统时钟 SYSCLK 由 SYSCKCR 寄存器的 SYSCKSW[1:0]选择，可在 HSI/HSE/PLLCLK1/PLLCLK2 之间无缝切换，不影响程序运行；切换前必须确保目标时钟源已使能且 Ready。

SDK 接口：

void rcc_hal_sys_select_clock(clock_sys_select sel); /*直接写 SYSCKSW，不做就绪检查*/

clk_erc_e rcc_hal_sys_switch_clock(clock_sys_select sel); /*切换前自动检查 HSE/PLL 是否 Ready，未就绪返回 ERC_CLK_OSC_NOT_READY*/

clock_sys_select rcc_hal_sys_clock_status(void); /*查询当前系统时钟源*/

**推荐使用 rcc_hal_sys_switch_clock()而非 rcc_hal_sys_select_clock()，避免切换到未就绪的时钟源导致系统时钟丢失。

3. FCLK/PCLK 分频

FCLK 由 SYSCLK 经 SYSCKCR.HPRE 分频得到（FCLK = SYSCLK / (HPRE + 1)），PCLK 由 FCLK 经 PPRE 分频得到（PCLK = FCLK / (PPRE + 1)）。

SDK 接口：

void rcc_hal_sys_div_fclk(clock_sys_fclk_div div); /*CLOCK_SYS_FCLK_DIV1~DIV16*/

void rcc_hal_sys_div_pclk(clock_sys_pclk_div div); /*CLOCK_SYS_PCLK_DIV1~DIV16*/

uint32_t rcc_hal_get_fclk_freq(void); /*返回当前 FCLK 频率(Hz)*/

uint32_t rcc_hal_get_pclk_freq(void); /*返回当前 PCLK 频率(Hz)*/

4. PLL 配置

PLL 输出频率由 PLLCR1 的 PLLMULT（倍频，寄存器编码=真实倍频-5，范围 5~20 倍）与 PLLDIV（分频，1/2/4/8）共同决定：PLL_CLK = PLL_CLK_SOURCE × PLLMULT / PLLDIV，PLL_CLK_SOURCE 可选内部 16MHz RC 或外部 HSE。

配置流程：关闭 PLL → 配置倍频/分频 → 使能 PLL → 等待 PLLCR0.RO_PLL_RDY 锁定 → 再切换系统时钟源到 PLLCLK1/2。

SDK 接口：

void rcc_hal_pll_enable(void);

void rcc_hal_pll_disable(void);

clk_erc_e rcc_hal_pll_config(uint8_t mult, uint8_t div); /*mult:PLL_MUTL_5~PLL_MUTL_20, div:PLL_DIV_1/2/4/8*/

uint8_t rcc_hal_pll_ready(void); /*1=PLL 已锁定*/

clk_erc_e rcc_hal_pll_wait_ready(uint32_t timeout_us); /*阻塞等待 PLL 锁定，超时返回 ERC_CLK_OSC_NOT_READY*/

uint32_t rcc_hal_pll_freq(void); /*返回当前 PLL 输出频率(Hz)*/

**一站式接口 rcc_hal_sys_clock_full_config()可一次完成 PLL 配置、使能、等待锁定、切换系统时钟源、配置 FCLK/PCLK 分频，推荐优先使用。

clk_erc_e rcc_hal_sys_clock_full_config(clock_sys_select sel, uint8_t pll_mult, uint8_t pll_div, clock_sys_fclk_div fclk_div, clock_sys_pclk_div pclk_div);

5. MCO 时钟输出

MCO 可将系统时钟、HSI、PLLCLK、HSE、LSI 中的任意一路输出到芯片引脚（复用功能，需先配置对应引脚为 MCO 复用模式），并支持 1~512 分频，便于用示波器观测内部时钟状态。

SDK 接口：

void rcc_hal_mco_clock_select(clock_mco_select sel); /*CLOCK_MCO_SYS/HSI/PLLCLK/HSE/LSI*/

void rcc_hal_mco_div(clock_mco_div div); /*CLOCK_MCO_DIV1~DIV256*/

uint32_t rcc_hal_get_mco_freq(void); /*返回 MCO 输出频率(Hz) = 源频率 / 2^div*/

6. 外设时钟使能与复位

每个外设时钟由 AHB/ANA/DIG/COM/TIM 时钟使能寄存器中的对应位控制，复位由 AHBRST/ANARST/DIGRST/COMRST/TIMRST 寄存器中的对应位控制；ADC0/MCPWM0/CAN0 还可通过 PCKS0 寄存器选择功能时钟源（HSI/EOSC/PLL）。

SDK 接口（统一枚举接口，rcc_periph_e 覆盖 FLC/GPIOA~C/ADC0/MATH/HALL/QEP/IWDT/UART0~1/SPI0~1/IIC0~1/TIM13~16/MCPWM0）：

void rcc_drv_periph_clk_enable(RCC_TypeDef *rcc_ptr, rcc_periph_e periph);

void rcc_drv_periph_clk_disable(RCC_TypeDef *rcc_ptr, rcc_periph_e periph);

void rcc_drv_periph_reset(RCC_TypeDef *rcc_ptr, rcc_periph_e periph); /*置位复位*/

void rcc_drv_periph_reset_release(RCC_TypeDef *rcc_ptr, rcc_periph_e periph); /*释放复位*/

clk_erc_e rcc_drv_periph_clk_src_select(RCC_TypeDef *rcc_ptr, rcc_clksrc_periph_e periph, periph_clk_src_e src);

同时也保留了针对每个外设的直写寄存器薄封装函数，方便按外设直接调用，例如：

void flash_clk_enable(void); void gpio_clk_enable(void);

void ADC_Clk_Enable(void); void ADC_Clk_Disable(void);

void uart_clk_enable(void); void spi_clk_enable(void); void i2c_clk_enable(void);

void mcpwm_clk_enable(void); void TIM13_Clk_Enable(void); /*TIM14~16 同理*/

7. 系统复位管理

SYSRSTF 记录复位来源（POR/BOR/LVD/NRST 引脚/SYSRESETREQ/Lockup/IWDT/HSE 失效/PLL 失锁/Re-trim），写 1 清零；SYSRSTE 使能对应事件触发系统复位，该寄存器带写保护，写入时须同步写入 KEYCODE 解锁。

SDK 接口：

uint32_t rcc_drv_get_reset_status(RCC_TypeDef *rcc_ptr); /*读取 SYSRSTSR，用 CLOCK_RST_FLAG_*位掩码判断*/

void rcc_drv_clear_reset_flag(RCC_TypeDef *rcc_ptr, uint32_t flags);

clk_erc_e rcc_drv_reset_src_enable(RCC_TypeDef *rcc_ptr, clock_rst_src_e src, uint8_t en); /*内部自动携带 KEYCODE*/

void rcc_drv_sys_reset(void); /*软件触发系统复位(NVIC_SystemReset)*/

void rcc_drv_boot_reset_retrim(RCC_TypeDef *rcc_ptr); /*写 RSTKEY=0xAB56，触发 boot 复位并重新校准*/

8.5 快速开始

最小可用的系统时钟配置（HSI 直接作为系统时钟）示例：

流程：使能外设时钟 → 使能 HSI 并等待就绪 → 切换系统时钟到 HSI → 配置 FCLK/PCLK 分频

flash_clk_enable();

```
gpio_clk_enable();
uart_clk_enable();
rcc_hal_osc_enable(CLOCK_OSC_HSI, 1000);          /*使能 HSI 并等待就绪，超时 1000 次轮
询*/
rcc_hal_sys_switch_clock(CLOCK_SYS_HSI);          /*安全切换系统时钟到 HSI*/
rcc_hal_sys_div_fclk(CLOCK_SYS_FCLK_DIV1);        /*FCLK = SYSCLK / 1*/
rcc_hal_sys_div_pclk(CLOCK_SYS_PCLK_DIV1);        /*PCLK = FCLK / 1*/
```

**HSI 为内部 RC 振荡器，上电后无需外部晶振即可使用，是最简单的系统时钟配置方式。

使用 PLL 获得更高系统时钟

若需要更高主频（最高 96MHz），需先配置并锁定 PLL，再切换系统时钟源。推荐使用一站式接口 rcc_hal_sys_clock_full_config()，内部已包含 PLL 配置、使能、等待锁定、切源、分频的完整流程：

```
clk_erc_e erc = rcc_hal_sys_clock_full_config(
                    CLOCK_SYS_PLLCLK1,    /*系统时钟源：PLLCLK1*/
                    PLL_MUTL_6,           /*倍频系数：x6*/
                    PLL_DIV_2,            /*输出分频：/2*/
                    CLOCK_SYS_FCLK_DIV1, /*FCLK 不分频*/
                    CLOCK_SYS_PCLK_DIV1   /*PCLK 不分频*/
                );
if (erc != ERC_CLK_NO_ERROR)
{
    /*PLL_CLK_SOURCE(16MHz) x 6 / 2 = 48MHz 未锁定或参数非法，需检查 PLL_MUTL
_x/PLL_DIV_x 取值*/
}
```

PLL 计算公式：PLL_CLK = PLL_CLK_SOURCE × mult / div，PLL_CLK_SOURCE 由芯片内部 16MHz RC 或外部 HSE 经 ANAC_CLK 寄存器选择（见 rcc_hal_pll_config()内部实现），mult 对应 PLL_MUTL_5~PLL_MUTL_20（5~20 倍），div 对应 PLL_DIV_1/2/4/8。

SDK 内部 RCC_Init()已提供按 sys_config.h 中 SYS_CLK 目标频率自动反推 mult/div 的算法（尝

试 div=1/2/4/8，找到能整除且 mult∈[5,20]的精确解），系统上电流程中直接调用即可：

```
flash_clk_enable();
gpio_clk_enable();
uart_clk_enable();
RCC_Init();                    /*根据 sys_config.h 的 SYS_CLK 自动选择 HSI/HSE/PLL 并计算 PLL 参数*/
rcc_hal_osc_enable_all();     /*使能 HSI/PLL/LSI/HSE，为后续时钟切换、MCO、IWDT 等做准备*/
```

MCO 时钟输出观测（可选）

将内部时钟源输出到 MCO 引脚，便于用示波器验证时钟切换/分频是否生效：

```
rcc_hal_mco_clock_select(CLOCK_MCO_SYS);     /*MCO 输出系统时钟*/
rcc_hal_mco_div(CLOCK_MCO_DIV1);              /*不分频*/
```

**FCLK/PCLK 分频无法通过 MCO 引脚直接观测，只能通过延时函数或外设时序间接验证。

查询当前时钟频率

```
uint32_t mcu_clk = 0U;
clock_freq_get(SCG_SYS, &mcu_clk);     /*读取当前系统时钟频率*/
INFO("MCU_CLK = %dMHz
", mcu_clk / 1000000U);
```

外设时钟使能与复位（以 UART0 为例）

```
rcc_drv_periph_clk_enable(rcc_ptr, RCC_PERIPH_UART0);     /*使能 UART0 外设时钟*/
rcc_drv_periph_reset(rcc_ptr, RCC_PERIPH_UART0);          /*复位 UART0*/
rcc_drv_periph_reset_release(rcc_ptr, RCC_PERIPH_UART0);/*释放 UART0 复位*/
```

**切换系统时钟源前，若外设（如 UART）正在收发数据，应先阻塞等待其收发完成，切换后按新时钟频率重新初始化该外设的波特率等参数（可参考 rcc_hal_sys_change_clock()中对 UART0 的处理方式）。

9 DMA

9.1 概述

DMA 模块用于实现外设和内存间或内存和内存间或外设和外设间的数据传输，而不需要 CPU 干预。DMA 模块支持 6 条通道用于外设和存储器之间的数据传输。

DMA 通道传输时共用一个数据缓冲区，基于 Round-Robin 轮询且通道优先级可配的机制进行传输仲裁，每个通道都可进行四种优先级的配置。该缓冲区内含一个 FIFO，大小为 16 x 32-bit。

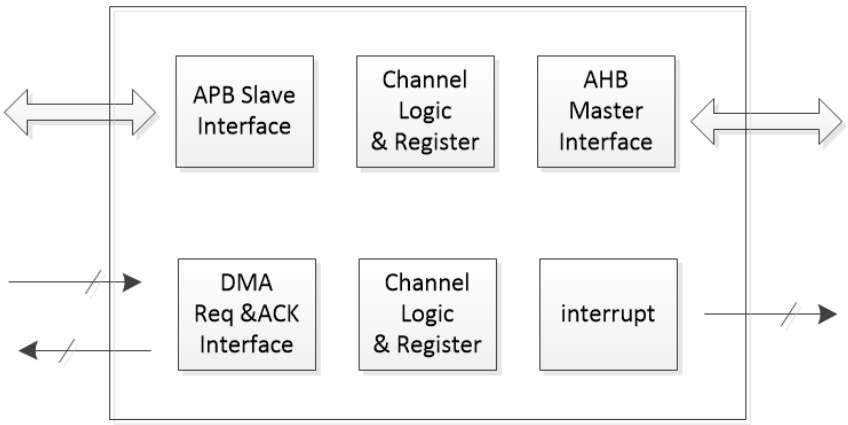


图 9-1 DMA 框图

9.2 特性

DMA 具有以下主要特征：

- 提供 6 个独立的可配置的硬件 DMA 请求传输通道，实现外设和内存间的直接数据传输，每个通道同样支持软件触发。
- 每个通道之间的仲裁使用通道优先级可配置的“Round-Robin”机制
- 所有通道均可配置内存、外设作为访问的源和目标的直接数据传输
- 8-bit、16-bit、和 32-bit 宽度的数据传输
- 按固定优先级自动实现通道传输仲裁
- DMA 通过系统 AHB 总线完成传输数据
- DMA 模块寄存器通过外设总线 APB 进行配置
- 每个通道都有 3 个状态标志位（DMA 传输完成和 DMA 当前传输通道），还有一个通道配置错误状态标志位。

- 所有通道均支持自动重复传输

9.3 DMA 通道配置

有 6 个 DMA 通道用于外设和内存之间的数据传输，通道是相互独立的。每个通道可支持多个外设，即一个通道同一时间只能处理一个外设请求。

当需要 DMA 模块在外设和内存间传输数据时，需要将指定 DMA 通道关联至某一特定外设（详见通道关联外设选择寄存器 DMA_CHCFG），例如 UART_RX、SPI_TX 等；当需要 DMA 模块在外设和外设间传输数据时，需要将指定 DMA 通道关联至源外设，同时使能对应外设的 DMA 请求控制位；当需要 DMA 模块在内存和内存之间传输数据时，需要将指定 DMA 通道的内存和内存之间的数据传输使能（控制寄存器 DMA_CTRL 的 M2M 置 1）。

9.4 DMA 通道仲裁

DMA 最高可支持 6 个通道同时传输，通道之间的仲裁使用通道优先级可配置的“Round-Robin”机制，用户可通过配置 DMA 控制寄存器（DMA_CTRL）的 PL 位域来配置不同通道的传输优先级。优先级可分为四个等级：低，中等，高，很高。

通道之间的仲裁整体上基于“Round-Robin”机制，即当各个通道优先级一样的情况下，这些通道的数据传输基于固定的优先级并发的轮询传输，通道基于“Round-Robin”固定轮询顺序如下：

通道 5 -> 通道 4 -> 通道 3 -> 通道 2 -> 通道 1 -> 通道 0 -> 通道 5 -> …

当通道优先级不一样的情况下，高优先级的通道可抢占当前的传输通道，而低优先级的通道必须等到全部高优先级通道完成当前传输后才会启动或继续传输。当任意几个通道的优先级一样时（如都为中等优先级），则这些通道在没有其他更高优先级通道的情况下，会同时以“Round-Robin”机制进行数据传输，在这些通道全部完成数据传输后，较低优先级的通道开始工作，以此类推。

原则上改变通道的优先级需要在通道空闲状态下进行，不建议用户在通道工作的情况下改变通道优先级。

9.5 DMA 通道传输

9.5.1 通道配置

DMA 请求可以来自多个外设，在发生一个事件后，外设发送一个请求信号到 DMA。DMA 根

据通道优先级仲裁处理请求。当 DMA 开始响应外设的请求时，DMA 会发送一个 ACK 信号给外设，外设收到 ACK 信号后，释放它的请求，等待下一个请求的发生。

数据传输由一系列的数据包实现。数据包共有两种：一是从数据源至 DMA 控制器；另一种则是从 DMA 控制器至数据目的地。此 DMA 也支持“外设至外设”访问，此时需要把源地址使能 DMA 模式，而目的地址所对应的外设要当作内存来访问并禁止该外设的 DMA 模式。

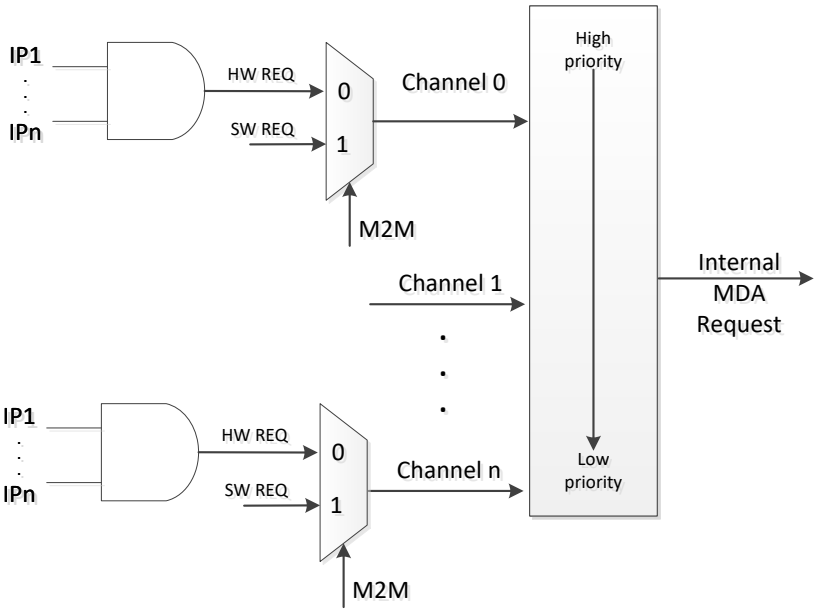


图 9-2 DMA 通道请求图

9.5.2 数据大小

DMA 传输的数据位宽可配置为 8-bit、16-bit 或 32bit，配置控制寄存器 DMACHn_CTL 的 SSIZE/DSIZE。DMA 传输最大数据量是可配置的（配置数据最大长度配置寄存器 DMACHn_MAX），最大可达到 65535，DMA 完成一次请求的数据传输，数据量进行一次递减。

DMA 通道总传输大小=传输数据量 × 数据位宽

9.5.3 地址模式

DMA 的源地址和目的地址分别可以配置为固定地址或者地址递增，具体配置见控制寄存器 DMACHn_CTL 的 SINC 和 DINC。

地址固定表示，每次数据传输后，当前地址不变；地址递增表示，每次数据传输后当前地址以 1、2 或 4 为间隔递增。

9.5.4 自动重载

当 DMA 控制寄存器 DMACHn_CTL 中的自动重载控制位 RT 被置位时，当 DMA 通道数据传输完全完成时，通道 n 当前地址和通道 n 当前传输大小将分别自动载入对应的初始值。通道 n 仍然有效且下一个相关的 DMA 请求在不需要软件重新配置。

9.5.5 中断状态管理

6 个通道的任意两个通道配置了相同的关联外设设备，配置错误标志位将会立起，如果使能中断，会产生中断。当 DMA 所有数据传输完成，相应通道的传输完成标志位将会立起（IFDn），可以通过配置中断使能寄存器 DMA_MASK 打开对应通道的中断。通过读取状态寄存器 DMA_STAT 的 ACTn 知道当前正在传输数据的通道。

9.6 寄存器列表

地址	寄存器	描述	备注
0x4000_0000	DMACH0_SRCB	DMA 通道 0 数据源基地址	
0x4000_0004	DMACH0_DSTB	DMA 通道 0 数据目的基地址	
0x4000_0008	DMACH0_MAX	DMA 通道 0 数据块最大长度	
0x4000_000C	DMACH0_CTL	DMA 通道 0 控制寄存器	
0x4000_0010	DMACH0_SCUR	DMA 通道 0 当前数据源地址	
0x4000_0014	DMACH0_DCUR	DMA 通道 0 当前数据目的地址	
0x4000_0018	DMACH0_TCNT	DMA 通道 0 终止传输计数器	
0x4000_0020	DMACH1_SRCB	DMA 通道 1 数据源基地址	
0x4000_0024	DMACH1_DSTB	DMA 通道 1 数据目的基地址	
0x4000_0028	DMACH1_MAX	DMA 通道 1 数据块最大长度	
0x4000_002C	DMACH1_CTL	DMA 通道 1 控制寄存器	
0x4000_0030	DMACH1_SCUR	DMA 通道 1 当前数据源地址	
0x4000_0034	DMACH1_DCUR	DMA 通道 1 当前数据目的地址	
0x4000_0038	DMACH1_TCNT	DMA 通道 1 终止传输计数器	
0x4000_0040	DMACH2_SRCB	DMA 通道 2 数据源基地址	

地址	寄存器	描述	备注
0x4000_0044	DMACH2_DSTB	DMA 通道 2 数据目的基地址	
0x4000_0048	DMACH2_MAX	DMA 通道 2 数据块最大长度	
0x4000_004C	DMACH2_CTL	DMA 通道 2 控制寄存器	
0x4000_0050	DMACH2_SCUR	DMA 通道 2 当前数据源地址	
0x4000_0054	DMACH2_DCUR	DMA 通道 2 当前数据目的地址	
0x4000_0058	DMACH2_TCNT	DMA 通道 2 终止传输计数器	
0x4000_0060	DMACH3_SRCB	DMA 通道 3 数据源基地址	
0x4000_0064	DMACH3_DSTB	DMA 通道 3 数据目的基地址	
0x4000_0068	DMACH3_MAX	DMA 通道 3 数据块最大长度	
0x4000_006C	DMACH3_CTL	DMA 通道 3 控制寄存器	
0x4000_0070	DMACH3_SCUR	DMA 通道 3 当前数据源地址	
0x4000_0074	DMACH3_DCUR	DMA 通道 3 当前数据目的地址	
0x4000_0078	DMACH3_TCNT	DMA 通道 3 终止传输计数器	
0x4000_0080	DMACH4_SRCB	DMA 通道 4 数据源基地址	
0x4000_0084	DMACH4_DSTB	DMA 通道 4 数据目的基地址	
0x4000_0088	DMACH4_MAX	DMA 通道 4 数据块最大长度	
0x4000_008C	DMACH4_CTL	DMA 通道 4 控制寄存器	
0x4000_0090	DMACH4_SCUR	DMA 通道 4 当前数据源地址	
0x4000_0094	DMACH4_DCUR	DMA 通道 4 当前数据目的地址	
0x4000_0098	DMACH4_TCNT	DMA 通道 4 终止传输计数器	
0x4000_00A0	DMACH5_SRCB	DMA 通道 5 数据源基地址	
0x4000_00A4	DMACH5_DSTB	DMA 通道 5 数据目的基地址	
0x4000_00A8	DMACH5_MAX	DMA 通道 5 数据块最大长度	
0x4000_00AC	DMACH5_CTL	DMA 通道 5 控制寄存器	
0x4000_00B0	DMACH5_SCUR	DMA 通道 5 当前数据源地址	
0x4000_00B4	DMACH5_DCUR	DMA 通道 5 当前数据目的地址	
0x4000_00B8	DMACH5_TCNT	DMA 通道 5 终止传输计数器	

地址	寄存器	描述	备注
0x4000_00C0	DMA_MASK	DMA 中断使能寄存器	
0x4000_00C4	DMA_CLR	DMA 中断清除寄存器	
0x4000_00C8	DMA_STAT	DMA 状态寄存器	
0x4000_00CC	DMA_CHCFG1	DMA 通道关联外设选择寄存器 1	
0x4000_00D0	DMA_CHCFG2	DMA 通道关联外设选择寄存器 2	

9.7 寄存器描述

9.7.1 数据源基地址寄存器 DMACHn_SRCB（n=0,1,2,3,4,5）

位域	类型	复位	名称	描述
[31:0]	R/W	0x0	SOURCE	DMA 通道 n 数据源基地址：当 DMA 控制器启动一次 DMA 传输时，SOURCE 会被送入当前数据源地址寄存器 DMACHn_SCUR 中（n = 0,1,2,3,4,5）。

9.7.2 数据目的基地址 DMACHn_DSTB（n=0,1,2,3,4,5）

位域	类型	复位	名称	描述
[31:0]	R/W	0x0	DEST	DMA 通道 n 数据目的基地址：当 DMA 控制器启动一次 DMA 传输时，DEST 会被送入当前数据目的地址寄存器 DMACHn_DCUR 中（n = 0,1,2,3,4,5）。

9.7.3 数据最大长度配置寄存器 DMACHn_MAX（n=0,1,2,3,4,5）

位域	类型	复位	名称	描述
[31:16]	--	--	--	
[15:0]	R/W	0	MAX_CNT	DMA 通道 n 数据块最大长度：设定下次 DMA 传输时数据块的最大长度，只有低 16 位有效。数据长度的单位为指定 DMA 传输的数据宽度— 字节（8 位），半字（16 位），或字(32 位)。当 DMA 控制器启动一次 DMA 传输时，该寄存器的值将被送入终止传输计数寄存器

				DMA_CHn_TCNT（n = 0,1,2,3,4,5）。

9.7.4 控制寄存器 DMACHn_CTL（n=0,1,2,3,4,5）

位域	类型	复位	名称	描述
[31:15]	--	--	--	
[14]	R/W	0	RT	控制 DMA 自动重复数据传输： 0b：设定数量的数据传输完成后 DMA 终止 1b：设定数量的数据传输完成后 DMA 自动重复工作
[13]	R/W	0	DIR	控制 DMA 传输的方向： 0b：外设至内存直接数据传输，外设为数据源 1b：内存至外设直接数据传输，外设为数据目的地
[12]	--	--	--	--
[11]	R/W	0	M2M	设定内存不同区域间的直接快速数据传输 0b：外设和内存间的直接数据传输 1b：内存和内存间的直接数据传输
[10:9]	R/W	0	PL	配置 DMA 控制器通道 n 的传输优先级 00b：低 01b：中等 10b：高 11b：很高
[8:7]	R/W	0	DSIZE	设定 DMA 控制器至目的地的数据传输宽度 00b：每次传 1 个字节（8 位） 01b：每次传 1 个半字（16 位） 10b：每次传 1 个字（32 位） 11b：保留
[6:5]	R/W	0	SBURST	SBURST 定义了 DMA 突发传输的数据长度。如果外设为数据源，则其设定了在往内存写入数据前 DMA 控制器从外设处获得并存入自身 FIFO 的数据字长度；如果外设

位域	类型	复位	名称	描述
				为数据传输的目的地，则 DMA 控制器会自动按此突发传输的长度从内存中获取适量的数据；如果通道 n 被设成内存至内存的直接传输，则该值为数据源的突发传输长度。 00b：每次传 1 个单位 01b：保留 10b：保留 11b：保留 由此可知，一次 DMA 传输的数据包总长度由 SBURST 和 SSIZE 值共同决定。
[4:3]	R/W	0	SSIZE	设定数据源至 DMA 控制器的数据传输宽度 00b：每次传 1 个字节（8 位） 01b：每次传 1 个半字（16 位） 10b：每次传 1 个字（32 位） 11b：保留
[2]	R/W	0	DINC	控制每次 DMA 传输后目的地址自动递增 0b：每次传输后目的地址不变 1b：每次传输后目的地址自动递增
[1]	R/W	0	SINC	控制每次 DMA 传输后源地址自动递增 0b：每次传输后源地址不变 1b：每次传输后源地址自动递增
[0]	R/W	0	EN	使能 DMA 通道进行数据传输 0b：禁止 DMA 通道数据传输 1b：打开 DMA 通道数据传输，在该位被置 1 前必须先设定好数据源寄存器，数据目的寄存器和数据长度寄存器。如果在数据传输过程中该位被软件清零，则此 DMA 传输通道将被复位。

9.7.5 当前源地址寄存器 DMACHn_SCUR（n=0,1,2,3,4,5）

位域	类型	复位	名称	描述
[31:0]	R	0	SCUR	DMA 通道 n 当前源地址：该寄存器只读，指示当前 DMA 传输时的数据源地址。如果 SINC 为 1，则每次传输后该寄存器的值会随着源地址的递增而同步改变；如果 SINC 为 0，则其内容保持不变。

9.7.6 当前目的地址寄存器 DMACHn_DCUR（n=0,1,2,3,4,5）

位域	类型	复位	名称	描述
[31:0]	R	0	DCUR	DMA 通道 n 当前目的地址：该寄存器只读，指示当前 DMA 传输时的数据目的地址。如果 DINC 为 1，则每次传输后该寄存器的值会随着目的地址的递增而同步改变；如果 DINC 为 0，则其内容保持不变。

9.7.7 终止传输计数器 DMACHn_TCNT（n=0,1,2,3,4,5）

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R	0	TCNT	DMA 通道 n 传输计数：此寄存器只有低 16 位有效且只读，表示一次 DMA 传输过程中还有多少数据单元尚未完成传输。DMA 控制器在传输了一个数据单元后该寄存器就递减，直到归零，并结束本次 DMA 传输过程。

9.7.8 中断使能寄存器 DMA_MASK

位域	类型	复位	名称	描述
[31]	R/W	0	MME	配置错误中断使能 0b： 禁止 DMA 通道配置错误中断 1b： 使能 DMA 通道配置错误中断
[30:14]	--	--	--	--

[13]	R/W	0	ME5	通道 5 数据错误中断使能 0b： 禁止 DMA 通道 5 数据错误中断 1b： 使能 DMA 通道 5 数据错误中断
[12]	R/W	0	ME4	通道 4 数据错误中断使能 0b： 禁止 DMA 通道 4 数据错误中断 1b： 使能 DMA 通道 4 数据错误中断
[11]	R/W	0	ME3	通道 3 数据错误中断使能 0b： 禁止 DMA 通道 3 数据错误中断 1b： 使能 DMA 通道 3 数据错误中断
[10]	R/W	0	ME2	通道 2 数据错误中断使能 0b： 禁止 DMA 通道 2 数据错误中断 1b： 使能 DMA 通道 2 数据错误中断
[9]	R/W	0	ME1	通道 1 数据错误中断使能 0b： 禁止 DMA 通道 1 数据错误中断 1b： 使能 DMA 通道 1 数据错误中断
[8]	R/W	0	ME0	通道 0 数据错误中断使能 0b： 禁止 DMA 通道 0 数据错误中断 1b： 使能 DMA 通道 0 数据错误中断
[7:6]	--	--	--	--
[5]	R/W	0	MD5	通道 5 数据传输中断使能 0b： 禁止 DMA 通道 5 数据传输中断 1b： 使能 DMA 通道 5 数据传输中断
[4]	R/W	0	MD4	通道 4 数据传输中断使能 0b： 禁止 DMA 通道 4 数据传输中断 1b： 使能 DMA 通道 4 数据传输中断
[3]	R/W	0	MD3	通道 3 数据传输中断使能 0b： 禁止 DMA 通道 3 数据传输中断 1b： 使能 DMA 通道 3 数据传输中断

[2]	R/W	0	MD2	通道 2 数据传输中断使能 0b： 禁止 DMA 通道 2 数据传输中断 1b： 使能 DMA 通道 2 数据传输中断
[1]	R/W	0	MD1	通道 1 数据传输中断使能 0b： 禁止 DMA 通道 1 数据传输中断 1b： 使能 DMA 通道 1 数据传输中断
[0]	R/W	0	MD0	通道 0 数据传输中断使能 0b： 禁止 DMA 通道 0 数据传输中断 1b： 使能 DMA 通道 0 数据传输中断

9.7.9 中断清除寄存器 DMA_CLR

位域	类型	复位	名称	描述
[31]	W	0	CME	配置错误中断清除 0b： 无作用 1b： 清除 DMA 通道配置错误中断
[30:14]	--	--	--	--
[13]	W	0	CE5	通道 5 数据错误中断清除 0b： 无作用 1b： 清除 DMA 通道 5 数据错误中断
[12]	W	0	CE4	通道 4 数据错误中断清除 0b： 无作用 1b： 清除 DMA 通道 4 数据错误中断
[11]	W	0	CE3	通道 3 数据错误中断清除 0b： 无作用 1b： 清除 DMA 通道 3 数据错误中断
[10]	W	0	CE2	通道 2 数据错误中断清除 0b： 无作用 1b： 清除 DMA 通道 2 数据错误中断

[9]	W	0	CE1	通道 1 数据错误中断清除 0b： 无作用 1b： 清除 DMA 通道 1 数据错误中断
[8]	W	0	CE0	通道 0 数据错误中断清除 0b： 无作用 1b： 清除 DMA 通道 0 数据错误中断
[7:6]	--	--	--	--
[5]	W	0	CD5	通道 5 数据传输中断清除 0b： 无作用 1b： 清除 DMA 通道 5 数据传输中断
[4]	W	0	CD4	通道 4 数据传输中断清除 0b： 无作用 1b： 清除 DMA 通道 4 数据传输中断
[3]	W	0	CD3	通道 3 数据传输中断清除 0b： 无作用 1b： 清除 DMA 通道 3 数据传输中断
[2]	W	0	CD2	通道 2 数据传输中断清除 0b： 无作用 1b： 清除 DMA 通道 2 数据传输中断
[1]	W	0	CD1	通道 1 数据传输中断清除 0b： 无作用 1b： 清除 DMA 通道 1 数据传输中断
[0]	W	0	CD0	通道 0 数据传输中断清除 0b： 无作用 1b： 清除 DMA 通道 0 数据传输中断

9.7.10 中断状态寄存器 DMA_STAT

位域	类型	复位	名称	描述
[31]	R	0	IFME	配置错误中断标志 0b： DMA 通道配置正确 1b： DMA 通道配置错误
[30:22]	--	--	--	--
[21]	R	0	ACT5	通道 5 数据传输标志 0b： DMA 通道 5 空闲 1b： DMA 通道 5 数据数据传输中
[20]	R	0	ACT4	通道 4 数据传输标志 0b： DMA 通道 4 空闲 1b： DMA 通道 4 数据数据传输中
[19]	R	0	ACT3	通道 3 数据传输标志 0b： DMA 通道 3 空闲 1b： DMA 通道 3 数据数据传输中
[18]	R	0	ACT2	通道 2 数据传输标志 0b： DMA 通道 2 空闲 1b： DMA 通道 2 数据数据传输中
[17]	R	0	ACT1	通道 1 数据传输标志 0b： DMA 通道 1 空闲 1b： DMA 通道 1 数据数据传输中
[16]	R	0	ACT0	通道 0 数据传输标志 0b： DMA 通道 0 空闲 1b： DMA 通道 0 数据数据传输中
[15:14]	--	--	--	--
[13]	R	0	IFE5	通道 5 数据错误中断标志 0b： 无中断请求 1b： DMA 通道 5 数据错误中断请求

位域	类型	复位	名称	描述
[12]	R	0	IFE4	通道 4 数据错误中断标志 0b： 无中断请求 1b： DMA 通道 4 数据错误中断请求
[11]	R	0	IFE3	通道 3 数据错误中断标志 0b： 无中断请求 1b： DMA 通道 3 数据错误中断请求
[10]	R	0	IFE2	通道 2 数据错误中断标志 0b： 无中断请求 1b： DMA 通道 2 数据错误中断请求
[9]	R	0	IFE1	通道 1 数据错误中断标志 0b： 无中断请求 1b： DMA 通道 1 数据错误中断请求
[8]	R	0	IFE0	通道 0 数据错误中断标志 0b： 无中断请求 1b： DMA 通道 0 数据错误中断请求
[7:6]	--	--	--	--
[5]	R	0	IFD5	通道 5 数据传输中断标志 0b： 无中断请求 1b： DMA 通道 5 数据传输中断请求
[4]	R	0	IFD4	通道 4 数据传输中断标志 0b： 无中断请求 1b： DMA 通道 4 数据传输中断请求
[3]	R	0	IFD3	通道 3 数据传输中断标志 0b： 无中断请求 1b： DMA 通道 3 数据传输中断请求
[2]	R	0	IFD2	通道 2 数据传输中断标志 0b： 无中断请求

位域	类型	复位	名称	描述
				1b： DMA 通道 2 数据传输中断请求
[1]	R	0	IFD1	通道 1 数据传输中断标志 0b： 无中断请求 1b： DMA 通道 1 数据传输中断请求
[0]	R	0	IFD0	通道 0 数据传输中断标志 0b： 无中断请求 1b： DMA 通道 0 数据传输中断请求

注：

IFD0-IFD5 位为 DMA 数据传输中断标志．当 DMA 通道完成了一个完整的数据包传输后，对应的中断标志位被置 1．

IFE0-IFE5 位为错误标志位．因任何原因导致某一错误标志被置 1 后，对应 DMA 通道的当前数据传输被立即终止并禁止任何后续的数据传输，直到软件重新使能 DMACHn_CTL.EN 位．

ACT0-ACT5 位为对应 DMA 通道的工作标志位．如果 DMA 通道正忙于进行数据传输，则对应位为 1，否则为 0．

IFME 位 DMA 模块各通道和外设功能关联错误标志位．如果任一 DMA 通道和外设功能的关联出现错误（例如外设功能被重复关联，或关联非法外设功能），则 IFME 位为 1，否则为 0．

9.7.11 通道关联外设选择寄存器 1 DMA_CHCFG1

位域	类型	复位	名称	描述
[31：30]	--	--	--	--
[29:24]	R/W	0x3	CH3SEL	DMA 通道 3 源设备选择：选择 DMA 通道 3 关联的外设功能
[23：22]	--	--	--	--
[21:16]	R/W	0x2	CH2SEL	DMA 通道 2 源设备选择：选择 DMA 通道 2 关联的外设功能
[15:14]	--	--	--	--
[13:8]	R/W	0x1	CH1SEL	DMA 通道 1 源设备选择：选择 DMA 通道 1 关联的外设功能
[7:6]	--	--	--	--
[5:0]	R/W	0x0	CH0SEL	DMA 通道 0 源设备选择：选择 DMA 通道 0 关联的外设功能

9.7.12 通道关联外设选择寄存器 2 DMA_CHCFG2

位域	类型	复位	名称	描述
[31:14]	--	--	--	--
[13:8]	R/W	0x5	CH5SEL	DMA 通道 5 源设备选择：选择 DMA 通道 5 关联的外设功能
[7:6]	--	--	--	--
[5:0]	R/W	0x4	CH4SEL	DMA 通道 4 源设备选择：选择 DMA 通道 4 关联的外设功能

表 9-1 DMA 通道关联外设

CHnSEL	关联外设
00000b	SPI0 TX
00001b	SPI0 RX
00010b	SPI1 TX
00011b	SPI1 RX
00100b	I2C0 TX
00101b	I2C0 RX
00110b	I2C1 TX
00111b	I2C1 RX
01000b	MCPWM_REQ0
01001b	MCPWM_REQ1
01010b	TIMER13
01011b	TIMER14
01100b	TIMER15
01101b	TIMER16
01110b	UART0_TX
01111b	UART0_RX
10000b	UART1_TX
10001b	UART1_RX
10010b	ADC
10011b ~ 11111b	保留

10 MCPWM

10.1 概述

MCPWM 模块，是一个精确控制电机驱动波形输出的模块。

包含两个 16 位递增计数器，用于提供两个计数周期（以下称为两个时基）。计数器的时钟分频有 1/2/4/8 四种选项，产生的分频时钟频率分别为 96MHz/48MHz/24MHz/12MHz。通道 0/1/2 固定使用时基 0，通道 3/4/5 固定使用时基 1。

MCPWM 模块共包含 6 个 PWM 生成子模块。

可以产生 6 对互补或 12 路独立不交叠的 PWM 信号

支持边沿对齐 PWM

中心对齐 PWM

移相 PWM

同时可以产生 4 路与 MCPWM 同时基的定时信息，用于触发 ADC 模块同步采样，进行与 MCPWM 的联动。

包含一组急停保护模块，用于不依赖 CPU 软件的处理快速关断 MCPWM 模块输出。MCPWM 模块可输入 4 路急停信号，其中 2 路来自芯片 IO，2 路来自片内比较器的输出。当急停事件发生时（支持有效电平极性选择），根据 MCPWM0_FAIL012/MCPWM0_FAIL345/MCPWM0_CH_MSK 的设置关闭相应的 PWM 通道，以避免短路发生。

对急停信号有独立滤波模块。

MCPWM 的每个输出 IO 支持两种控制模式:PWM 硬件控制或者软件直接控制（用于 EABS 软刹车，或 BLDC 方波换相控制）。

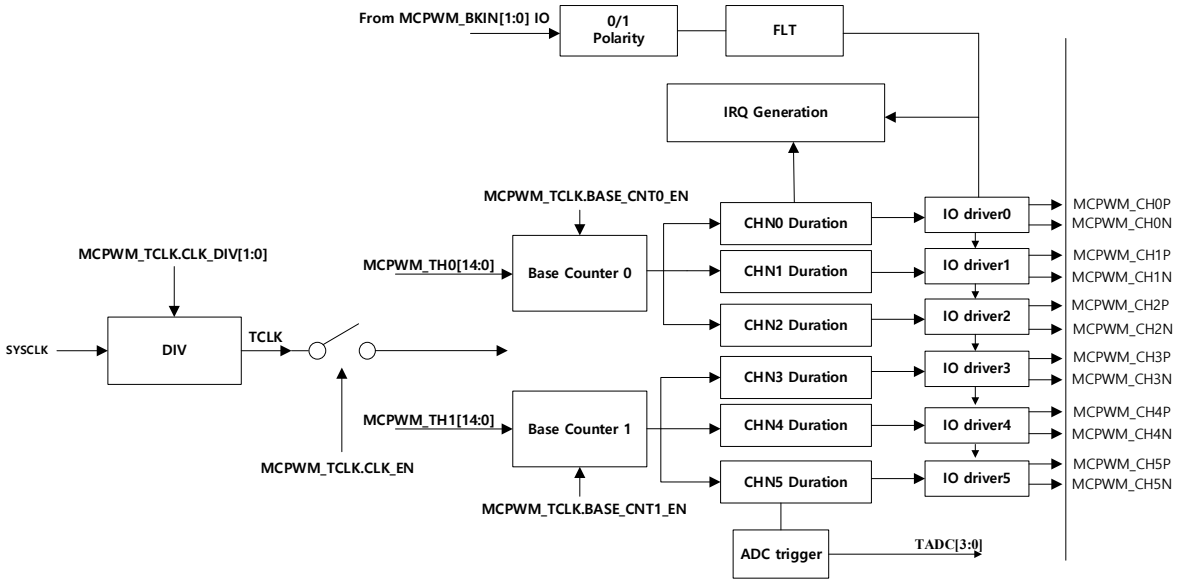


图 10-1 MCPWM 模块框图

为了保证定时精度，通常采用 96MHz 的时钟作为 MCPWM 模块工作频率。

10.2 Base Counter 模块

该模块主要是由两个递增计数器组成，其计数门限值为 TH0/TH1，计数器从 t0 时刻开始从-TH 递增计数，在 t1 时刻过 0 点，在 t2 时刻计数到 TH 完成一次计数循环，回到-TH，重新开始计数。计数周期为（TH×2+1）×计数时钟周期。

在 t0/t1（本次 t0 即上一次 t2）可产生定时事件中断，MCPWM0_IFx.T0_IF 和 MCPWM0_IFx.T1_IF 将被置位。

可通过寄存器配置 MCPWM0_TCLK.BASE_CNT0_EN/MCPWM0_TCLK.

BASE_CNT1_EN/MCPWM0_TCLK.BASE_CNT1_EN 控制 Base Counter 0/1 的启动和停止。控制 Base Counter 0/1 的启动和停止。

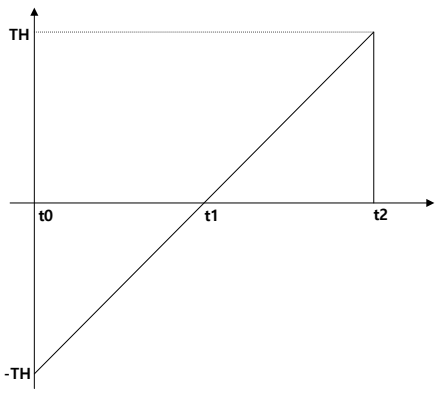


图 10-2 Base Counter t0/t1 时序

在运行 MCPWM 模块前，用户一般需要将门限值 MCPWM0_TH00~MCPWM0_TH51，死区寄存器 MCPWM0_DTH00/MCPWM0_DTH01/MCPWM0_DTH10/MCPWM0_DTH11 配置好。在实际运行过程中，也可动态改变比较门限值和死区寄存器。这些配置好的寄存器，可通过写 MCPWM0_UPDATE 寄存器实现手动更新，也可以通过配置 MCPWM0_SDCFG.T1_UPDATE_EN 及 MCPWM0_SDCFG.T0_UPDATE_EN 进行硬件自动更新。硬件更新，仅在 t0 及 t1 时刻（可配置 t0 或 t1 更新和 t0 t1 时刻都更新）才能产生更新事件，硬件把加载寄存器的值载入到实际运行的寄存器中。而更新事件的发生频率可以配置，即每间隔 N 个 t0 及 t1 时刻才发生更新。无论是否发生更新，t0/t1 时刻均可产生相应的中断。若硬件把加载寄存器的值到载入实际运行的寄存器后，产生装载中断。

通过配置 MCPWM0_SDCFG 寄存器选择更新发生在 t0 或者 t1 或者 t0/t1 都更新，配置更新间隔数，间隔数为 1～16。最频繁的更新配置为更新发生在 t0 和 t1，连续发生。最低速的更新配置为更新发生在 t1，每 16 个 t1 更新一次。

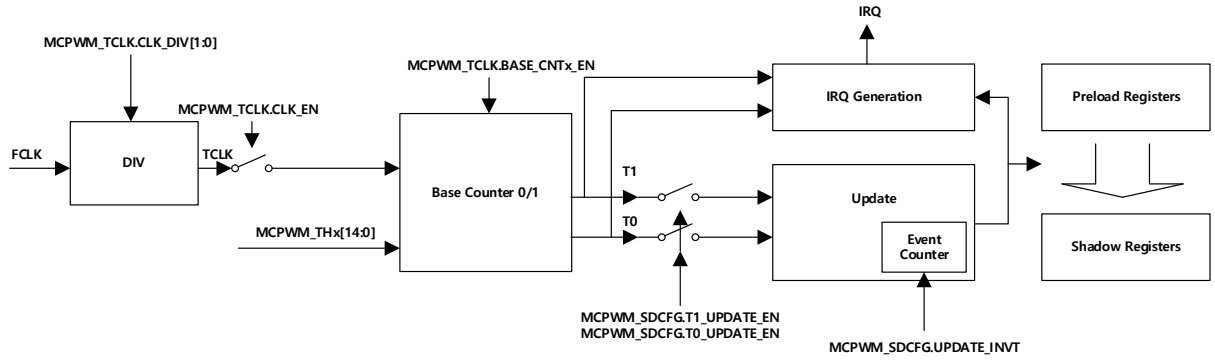


图 10-3 MCPWM 更新机制

时基 0 和时基 1 对应的 TH 寄存器 MCPWM0_THx 和计数寄存器 MCPWM0_CNTx（x=0,1）均存在影子寄存器，且均支持手动更新（通过软件向 MCPWM0_UPDATE 写入对应位进行更新）和自动更新（发生特定硬件事件触发更新）。影子寄存器可以在不开启 MCPWM TCLK 时钟（即 MCPWM0_TCLK.TCLK_EN=0）的情况下进行更新。MCPWM0_TCLK.BASE_CNT0_EN 和 MCPWM0_TCLK.BASE_CNT1_EN 分别控制时基 0 和时基 1 的计数使能。 当 MCPWM0_TCLK.BASE_CNT0_EN=0 时，时基 0 的 CNT 在完成更新后保持值不变，时基 1 的 CNT 同理，实现完全相同。

此外，为了精确控制第一个 MCPWM 周期的计数，建议将 MCPWM0_THx 和 MCPWM0_CNTx 同时更新。当然也支持 MCPWM0_THx 和 MCPWM0_CNTx 分别更新。然后再通过软件写入或外部事件触发计数开始，从而将 MCWPM_TCLk.BASE_CNT0_EN/MCWPM_TCLk.BASE_CNT1_EN 置位，分别使能时基 0 和时基 1 的 CNT 计数。

10.3 FAIL 信号处理

FAIL 信号即急停信号，主要用于在出现异常时迅速关断功率管，以免造成不可逆的硬件损坏。 该信号处理模块主要是根据实际情况设置急停事件，实现快速关断 PWM 的输出。有 2 路 fail 信号输入 MCPWM，即 FAIL0~FAIL1，分别可以来自芯片 I0MCPWM_BKIN[1:0]或芯片内部比较器的输出 CMP[1:0]。

其中 MCPWM 的 P/N 通道 0/1/2 可以选择使用 CMP[0]或 MCPWM_BKIN[0],MCPWM 的 P/N 通道 3 可以选择使用 CMP[1]或 MCPWM_BKIN[1]。

表 10-1 FAIL 信号分布表

	MCPWM CH0/1/2		MCPWM CH3/4/5	
	Fail0	Fail1	Fail2	Fail3
MCPWM_BKIN0	√		√	
MCPWM_BKIN1		√		√

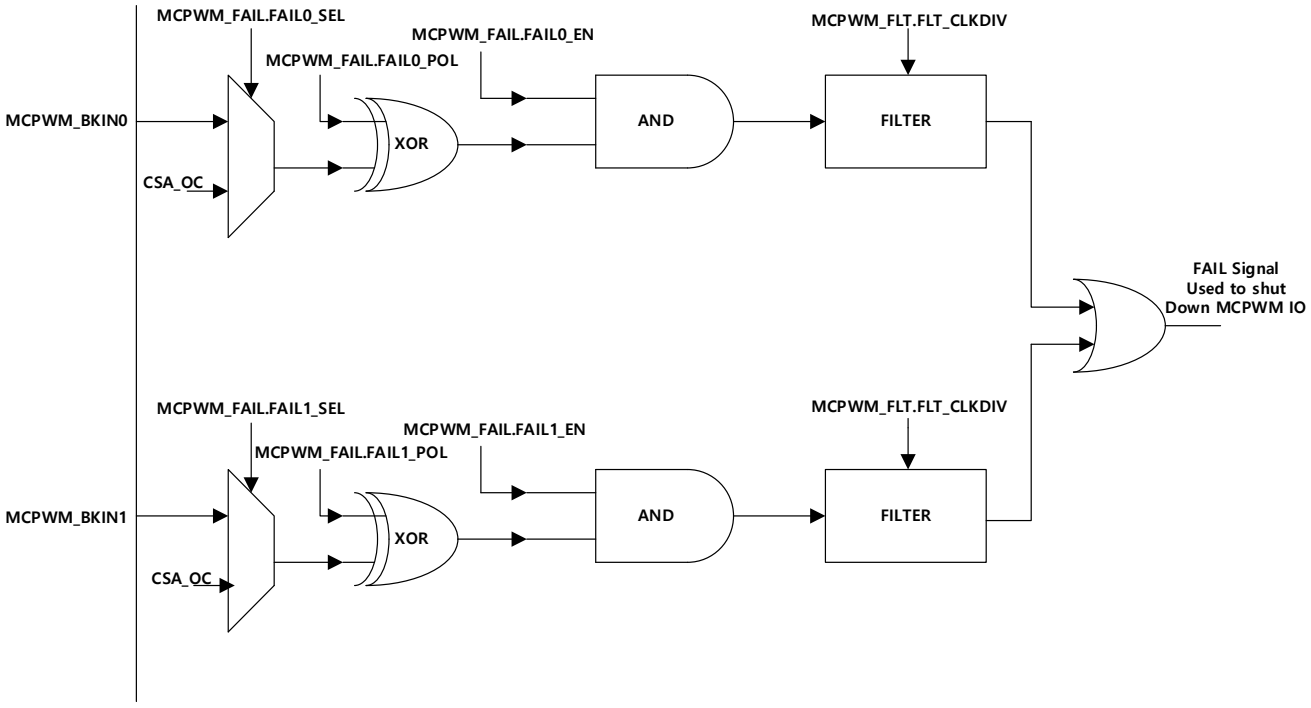


图 10-4 MCPWM FAIL 逻辑示意图

Filter 滤波模块的时钟，来自系统主时钟 MCLK 的门控时钟 FCLK，

见 SYS_CLK_FEN，并经过可选的两级分频，第一级分频由 MCPWM0_TCLK.CLK_DIV 控制，进行 1/2/4/8 倍分频。第二级分频可实现 1～256 倍的分频，使用 MCPWM0_FLT.FLT_CLKDIV

[7:0]作为第二级的分频系数；如图 10-5 所示。

MCPWM 模块使用分频后的时钟进行 Fail 信号滤波，滤波宽度固定为 16 个周期，即输入信号必须保持至少 16 个分频时钟周期（两级分频后的时钟）稳定后，硬件才判定其为有效输入信号。滤波时间常数的公式为，其中 TMCLK 为 MCLK/FCLK[4]的时钟周期，96MHz 对应 10.42ns。

T = TMCLK×(MCPWM0_TCLK.CLK_DIV) ×(FLT_CLKDIV+1) ×16

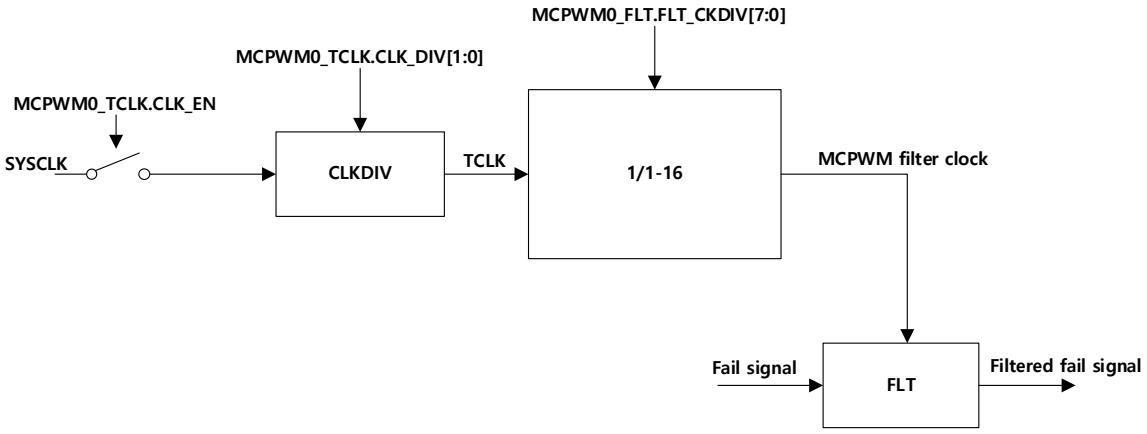


图 10-5 MCPWM Fail 信号滤波时钟生成逻辑

一旦发生 FAIL 事件，硬件将 IO 输出强制变为 MCPWM0_FAILx.

CHxN_DEFAULT 和 MCPWM0_FAILx.CHxP_DEFAULT 寄存器所指定的故障缺省值，此时 MCPWM0_FAILx.CHxN_DEFAUL

T 和 MCPWM0_FAILx.CHxP_DEFAULT 的值直接输出到 IO 口，不再受到 MCPWM0_FAILx.FAIL_POL 等极性控制的影响。其中 MCPWM0_FAIL012 用于控制通道 0/1/2 的故障缺省设置，MCPWM0_FAIL345 用于控制通道 3/4/5 的故障缺省设置。

FAIL 信号进入 MCPWM 模块后，可以通过设置 MCPWM_TCLK 进行滤波。

10.4 MCPWM 特殊输出状态

电机控制中经常会用到全 0 和全 1 输出状态，以下互补模式设置可以得到期望的输出。

1. 如果 THn0≥THn1，芯片处于恒 0 状态（CHP 关闭，CHN 开启），无死区

2. 如果 THn0=-TH，THn1=TH，芯片处于恒 1 状态（CHP 开启，CHN 关闭），无死区

10.5 MCPWM 输出模式

该模块根据实际 MCPWM 的寄存器配置情况，将 IO 设置到相应电平。IO Driver 模块的整体数据流程图如下：

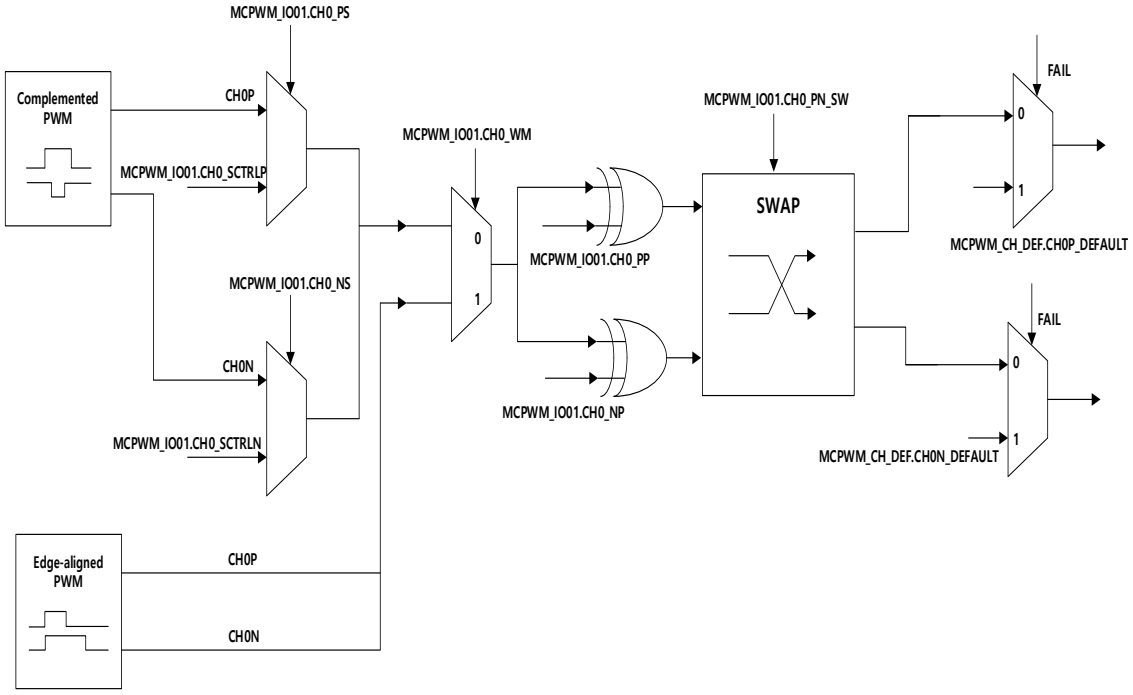


图 10-6 IO Driver 模块数据流程图

10.5.1 中心对齐模式

4 个 MCPWM IO Driver 采用独立的控制门限，独立死区宽度（每一对互补 IO 的死区需要独立配置，即 4 个死区配置寄存器），共享数据更新事件。

采用 TH0 和 TH1 控制第个 MCPWM IO 的启动、关闭动作，n 为 0/1/2/3/4/5。

当计数器 CNT 值向上计数达到 TH0（即 t3 时刻），关闭 CHN，经过死区延时 DTH0，打开 CHP。

当计数器 CNT 值向上计数达到 TH1（即 t4 时刻），关闭 CHP，经过死区延 DTH1，打开 CHN。

采用独立的启动和关闭时间控制，可以提供相位控制的能力。

死区延时保证 CHP/CHN 不会同时打开，避免短路发生。

t3/t4 时刻均会产生相应中断。

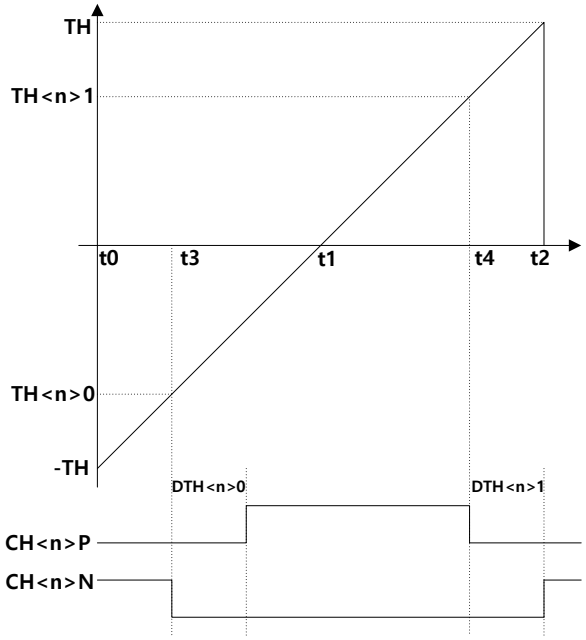


图 10-7 MCPWM 时序 TH<n>0 和 TH<n>1 中心对齐模式

10.5.2 中心对齐推挽模式

中心对齐推挽模式。第一个周期内，在 t3 时刻 CHP 打开，在 t4 时刻，CHP 关闭。第二个周期内，在 t3 时刻 CHN 打开，在 t4 时刻，CHN 关闭。

t3/t4 时刻均会产生相应中断。

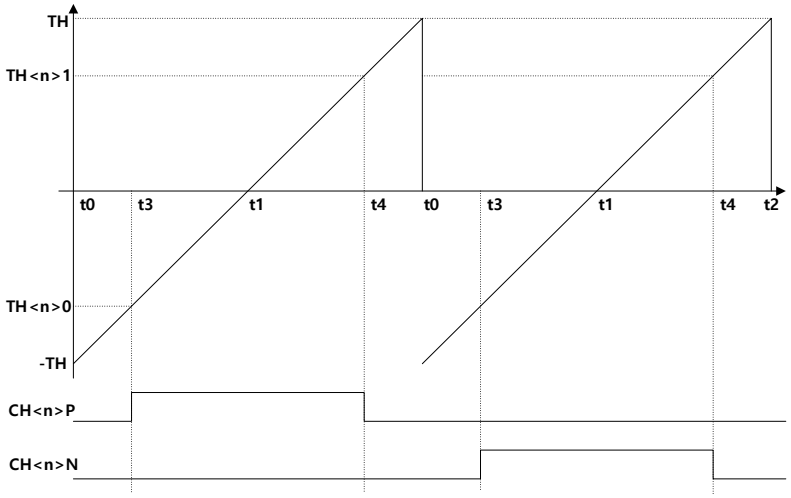


图 10-8 MCPWM 时序 TH<n>0 和 TH<n>1 中心对齐推挽模式

10.5.3 边沿对齐模式

边沿对齐模式。在 t0 时刻 CHP/CHN 同时打开，在 t3 时刻，CHP 关闭；在 t4 时刻，CHN 关闭。

t3/t4 时刻均会产生相应中断。

边沿对齐模式下，CHP/CHN 无需死区保护。

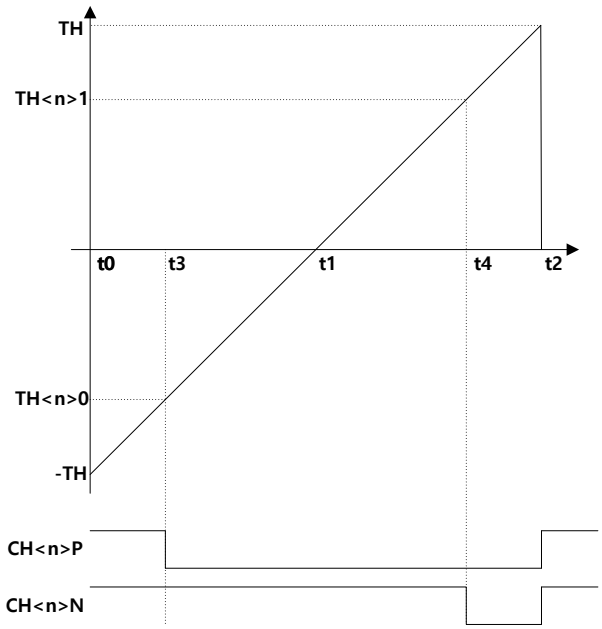


图 10-9 MCPWM 时序 TH<n>0 和 TH<n>1 边沿对齐模式

10.5.4 边沿对齐推挽模式

边沿对齐推挽模式。第一个周期内，在 t0 时刻 CHP 打开，在 t3 时刻，CHP 关闭。第二个周期内，在 t0 时刻 CHN 打开，在 t3 时刻，CHN 关闭。

t0/t3 时刻均会产生相应中断。

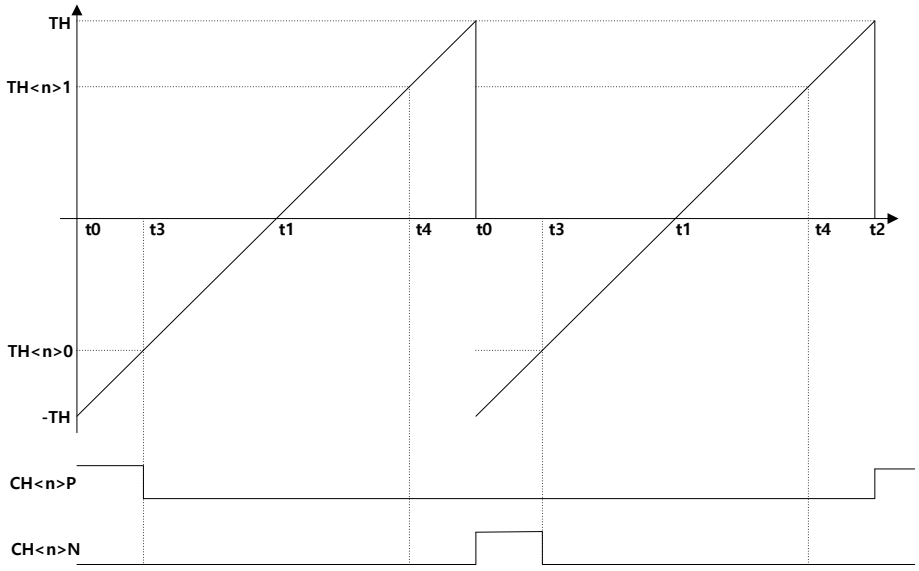


图 10-10 MCPWM 时序 TH<n>0 和 TH<n>1 边沿对齐模式

10.5.5 死区控制

MCPWM IO 是一对互斥控制信号 CHP/CHN，控制如下图所示的电路：

当 CHP 为高/CHN 为低时，Vout 输出高（VDD）；

当 CHP 为低/CHN 为高时，Vout 输出低（VSS）；

当 CHP 为高/CHN 为高时，Vout 输出不确定，但是会产生 VDD 到 VSS 的短路；

当 CHP 为低/CHN 为低时，Vout 输出不确定。

必须避免 CHP/CHN 同时打开的情况。死区的引入，可以有效避免 VDD 到 VSS 的短路。

四组 MCPWM IO 的死区宽度共享同一个死区宽度设置 MCPWM0_DTH0/1。

对于互补模式 MCPWM IO 自动插入死区。

对于边沿对齐模式，MCPWM IO 无死区。

在 IO Driver 模块中增加 CHP/CHN 冲突检测，发生冲突时，自动将 IO 拉低，同时给出 错误中断（错误中断标志位可软件清除或者硬件自动清除）。

MCPWM IO 也可通过软件配置的方式输出，此时，死区控制通过软件实现，如果 MCPWM 模 块配置为中心对齐模式，硬件短路保护机制仍有效，保证 P 和 N 不同时打开。

CHP/CHN，在 IO 上可以互换。

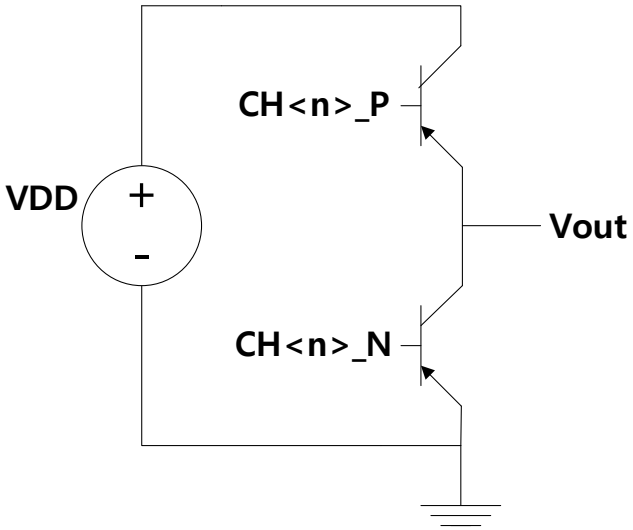


图 10-11 MCPWM IO 控制示意图

10.5.6 极性设置

CHP/CHN 的有效电平可以配置为高有效/低有效，每个 IO 的有效电平单独可配。 CHP/CHN 输出到 IO 的位置通过软件配置可以互换。

10.5.7 自动保护

当发生急停事件（Fail 事件），应立刻将 CHP/CHN 自动切换到默认安全电平状态。需要注意默认电平配置（MCPWM0_CH_DEF.CHxN_DEFAULT 和 MCPWM0_CH_DEF.CHxP_DEFAULT 控制认电平）。

芯片正常工作后，IO 默认输出的电平是寄存器 MCPWM0_CH_DEF.CHxN_DEFAULT 和 MCPWM0_CH_DEF.CHxP_DEFAULT 指定值，当用户配置完毕，MCPWM 正常工作后，配置 MCPWM0_FAILx.MCPWM0_OE（即 MOE）为 1，IO 输出电平受到 MCPWM IO 模块控制。

当发生 FAIL 短路状况时，硬件立即切换到 IO 默认输出电平。

当芯片调试中，CPU 被上位机软件暂停运行时，可暂停 MCPWM 正常输出，转而输出默认电平值。

10.6 ADC Trigger 模块

MCPWM 可以提供 ADC 采样控制。当计数器计数到 MCPWM0_TMR0/MCPWM0_TMR1/MCPWM0_TMR2/MCPWM0_TMR3 时，可产生定时事件，触发 ADC 采样。该触发信号可同时输出到 GPIO，便于调试之用。输出的具体 GPIO，参见对应器件的 datasheet。其中 MCPWM0_TMR0/MCPWM0_TMR1 固定使用时基 0，MCPWM0_TMR2/MCPWM0_TMR3 可以选择使用时基 0/1，见 MCPWM0_TCLK。

表 10-1 MCPWM 计数阈值与事件对应表

MCPWMx 时基 0 t0 事件	MCPWMx_CNT0==-MCPWMx_TH0
MCPWMx 时基 0 t1 事件	MCPWMx_CNT0==0
MCPWMx 时基 1 t0 事件	MCPWMx_CNT1==-MCPWMx_TH1
MCPWMx 时基 1 t1 事件	MCPWMx_CNT1==0
MCPWMx TIO0[0]事件	MCPWMx_CNT0==MCPWMx_TH00
MCPWMx TIO0[1]事件	MCPWMx_CNT0==MCPWMx_TH01
MCPWMx TIO1[0]事件	MCPWMx_CNT0==MCPWMx_TH10
MCPWMx TIO1[1]事件	MCPWMx_CNT0==MCPWMx_TH11
MCPWMx TIO2[0]事件	MCPWMx_CNT0==MCPWMx_TH20
MCPWMx TIO2[1]事件	MCPWMx_CNT0==MCPWMx_TH21

MCPWMx TIO3[0]事件	MCPWMx_CNT1==MCPWMx_TH30
MCPWMx TIO3[1]事件	MCPWMx_CNT1==MCPWMx_TH31
MCPWMx TIO4[0]事件	MCPWMx_CNT1==MCPWMx_TH40
MCPWMx TIO4[1]事件	MCPWMx_CNT1==MCPWMx_TH41
MCPWMx TIO5[0]事件	MCPWMx_CNT1==MCPWMx_TH50
MCPWMx TIO5[1]事件	MCPWMx_CNT1==MCPWMx_TH51
MCPWMx TADC[0]事件	TMR0
MCPWMx TADC[1]事件	TMR1
MCPWMx TADC[2]事件	TMR2
MCPWMx TADC[3]事件	TMR3

10.7 中断

MCPMW_IF0 和 MCPWM0_EIF[5:4]、MCPWM0_EIF[8]标志置位且使能的情况下，产生 MCPWM0_IRQ0 中断；

MCPMW_IF1 和 MCPWM0_EIF[7:6]、MCPWM0_EIF[9]标志置位且使能的情况下，产生 MCPWM0_IRQ1 中断。

10.8 寄存器列表

地址	寄存器	描述	备注
0x4001 0C00	MCPWM0_TH00	CH0_P 比较门限值寄存器	存在影子寄存器
0x4001 0C04	MCPWM0_TH01	CH0_N 比较门限值寄存器	存在影子寄存器
0x4001 0C08	MCPWM0_TH10	CH1_P 比较门限值寄存器	存在影子寄存器
0x4001 0C0C	MCPWM0_TH11	CH1_N 比较门限值寄存器	存在影子寄存器
0x4001 0C10	MCPWM0_TH20	CH2_P 比较门限值寄存器	存在影子寄存器
0x4001 0C14	MCPWM0_TH21	CH2_N 比较门限值寄存器	存在影子寄存器
0x4001 0C18	MCPWM0_TH30	CH3_P 比较门限值寄存器	存在影子寄存器
0x4001 0C1C	MCPWM0_TH31	CH3_N 比较门限值寄存器	存在影子寄存器
0x4001 0C20	MCPWM0_TH40	CH4_P 比较门限值寄存器	存在影子寄存器

地址	寄存器	描述	备注
0x4001 0C24	MCPWM0_TH41	CH4_N 比较门限值寄存器	存在影子寄存器
0x4001 0C28	MCPWM0_TH50	CH5_P 比较门限值寄存器	存在影子寄存器
0x4001 0C2C	MCPWM0_TH51	CH5_N 比较门限值寄存器	存在影子寄存器
0x4001 0C30	MCPWM0_TMR0	ADC 采样定时器比较门限 0 寄存器	存在影子寄存器
0x4001 0C34	MCPWM0_TMR1	ADC 采样定时器比较门限 1 寄存器	存在影子寄存器
0x4001 0C38	MCPWM0_TMR2	ADC 采样定时器比较门限 2 寄存器	存在影子寄存器
0x4001 0C3C	MCPWM0_TMR2	ADC 采样定时器比较门限 3 寄存器	存在影子寄存器
0x4001 0C40	MCPWM0_TH0	时基 0 门限值寄存器	写保护，存在影子寄存器
0x4001 0C44	MCPWM0_TH1	时基 1 门限值寄存器	写保护，存在影子寄存器
0x4001 0C48	MCPWM0_CNT0	时基 0 计数器寄存器	存在影子寄存器
0x4001 0C4C	MCPWM0_CNT1	时基 1 计数器寄存器	存在影子寄存器
0x4001 0C50	MCPWM0_UPDATE	加载控制寄存器	
0x4001 0C54	MCPWM0_FCNT	FAIL 时刻 CNT 值	
0x4001 0C58	MCPWM0_EVT0	时基 0 外部触发	
0x4001 0C5C	MCPWM0_EVT1	时基 1 外部触发	
0x4001 0C60	MCPWM0_DTH00	CH0/1/2 N 通道死区宽度控制	写保护
0x4001 0C64	MCPWM0_DTH01	CH0/1/2 P 通道死区宽度控制	写保护
0x4001 0C68	MCPWM0_DTH10	CH3/4/5 N 通道死区宽度控制	写保护
0x4001 0C6C	MCPWM0_DTH11	CH3/4/5 P 通道死区宽度控制	写保护
0x4001 0C70	MCPWM0_FLT	滤波时钟分频寄存器	写保护
0x4001 0C74	MCPWM0_SDCFG	MCPWM 加载配置寄存器	写保护
0x4001 0C78	MCPWM0_AUEN	自动更新使能寄存器	写保护
0x4001 0C7C	MCPWM0_TCLK	时钟分频控制寄存器	写保护

地址	寄存器	描述	备注
0x4001 0C80	MCPWM0_IE0	时基 0 中断控制寄存器	写保护
0x4001 0C84	MCPWM0_IF0	时基 0 中断标志位寄存器	
0x4001 0C88	MCPWM0_IE1	中断控制寄存器	写保护
0x4001 0C8C	MCPWM0_IF1	中断标志位寄存器	
0x4001 0C90	MCPWM0_EIE	异常中断控制寄存器	写保护
0x4001 0C94	MCPWM0_EIF	异常中断标志位寄存器	
0x4001 0C98	MCPWM0_RE	DMA 请求控制寄存器	写保护
0x4001 0C9C	MCPWM0_PP	推挽模式使能寄存器	写保护
0x4001 0CA0	MCPWM0_IO01	CH0 CH1 IO 控制寄存器	写保护
0x4001 0CA4	MCPWM0_IO23	CH2 CH3 IO 控制寄存器	写保护
0x4001 0CA8	MCPWM0_IO45	CH4 CH5 IO 控制寄存器	写保护
0x4001 0CAC	MCPWM0_IOCFG	IO 控制寄存器	写保护
0x4001 0CB0	MCPWM0_FAIL012	CH0 CH1 CH2 短路控制寄存器	写保护
0x4001 0CB4	MCPWM0_FAIL345	CH3 CH4 CH5 短路控制寄存器	写保护
0x4001 0CB8	MCPWM0_CH_DEF	短路保护通道输出默认值	写保护
0x4001 0CBC	MCPWM0_CH_MSK	通道屏蔽寄存器	写保护
0x4001 0CCC	MCPWM0_PRT	保护寄存器	

对于所有存在影子寄存器的 MCPWM 配置寄存器，写入的是预装载寄存器，只有更新事件发生时才会把预装载寄存器值写入影子寄存器，读出时读出的是影子寄存器的值。而且实际起到配置作用的是影子寄存器。

具有写保护的寄存器，需要先解除写保护之后，才能写入，否则写入无效。

10.8.1 MCPWM0_TH00 比较门限寄存器

地址:0x4001_0C00，无写保护，存在影子寄存器

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	TH00	MCPWM CH0_P 比较门限值，为预装寄存器，16 位有符号数；发生更新事件时，此寄存器加载至到 MCPWM 实际运行系统中。写入的是预装载寄存器，更新事件发生时才会把预装载寄存器值写入影子寄存器，读出时读出的是影子寄存器的值。

10.8.2 MCPWM0_TH01 MCPWM CH0N 比较门限寄存器

地址:0x4001_0C04

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	TH01	MCPWM CH0_N 比较门限值，16 位有符号数；发生更新事件时，本寄存器加载到 MCPWM 实际运行系统中。

10.8.3 MCPWM0_TH10 MCPWM CH1P 比较门限寄存器

地址:0x4001_0C08

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	TH10	MCPWM CH1_P 比较门限值，16 位有符号数；发生更新事件时，本寄存器加载到 MCPWM 实际运行系统中。

10.8.4 MCPWM0_TH11 MCPWM CH1N 比较门限寄存器

地址:0x4001_0C0C

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	TH11	MCPWM CH1_N 比较门限值，16 位有符号数；发生更新事件时，本寄存器加载到 MCPWM 实际运行系统中。

10.8.5 MCPWM0_TH20 MCPWM CH2P 比较门限寄存器

地址:0x4001_0C10

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	TH20	MCPWM CH2_P 比较门限值，16 位有符号数；发生更新事件时，本寄存器加载到 MCPWM 实际运行系统中。

10.8.6 MCPWM0_TH21 MCPWM CH2N 比较门限寄存器

地址:0x4001_0C14

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	TH21	MCPWM CH2_N 比较门限值，16 位有符号数；发生更新事件时，本寄存器加载到 MCPWM 实际运行系统中。

10.8.7 MCPWM0_TH30 MCPWM CH3P 比较门限寄存器

地址:0x4001_0C18

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	TH30	MCPWM CH3_P 比较门限值，16 位有符号数；发生更新事件时，本寄存器加载到 MCPWM 实际运行系统中。

10.8.8 MCPWM0_TH31 MCPWM CH3N 比较门限寄存器

地址:0x4001_0C1C

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	TH31	MCPWM CH3_N 比较门限值，16 位有符号数；发生更新事件时，本寄存器加载到 MCPWM 实际运行系统中。

10.8.9 MCPWM0_TH40 MCPWM CH4P 比较门限寄存器

地址:0x4001_0C20

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	TH40	MCPWM CH4_P 比较门限值，16 位有符号数；发生更新事件时，本寄存器加载到 MCPWM 实际运行系统中。

10.8.10 MCPWM0_TH41 MCPWM CH4N 比较门限寄存器

地址:0x4001_0C24

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	TH41	MCPWM CH4_N 比较门限值，16 位有符号数；发生更新事件时，本寄存器加载到 MCPWM 实际运行系统中。

10.8.11 MCPWM0_TH50 MCPWM CH5P 比较门限寄存器

地址:0x4001_0C28

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	TH50	MCPWM CH5_P 比较门限值，16 位有符号数；发生更新事件时，本寄存器加载到 MCPWM 实际运行系统中。

10.8.12 MCPWM0_TH51 MCPWM CH5N 比较门限寄存器

地址:0x4001_0C2C

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	TH51	MCPWM CH5_N 比较门限值，16 位有符号数；发生更新事件时，本寄存器加载到 MCPWM 实际运行系统中。

10.8.13 MCPWM0_TMR0 ADC 定时器门限 1 寄存器

地址:0x4001_0C30

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	TMR0	ADC 采样定时器比较门限 0 寄存器，16 位有符号数；当 MCPWM0_CNT0=TMR0 时产生 TADC[0]事件触发 ADC 进行采样。 MCPWM 发生更新事件后，本寄存器加载到 MCPWM 实际运行系统中。

10.8.14 MCPWM0_TMR1 ADC 定时器门限 1 寄存器

地址:0x4001_0C34

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	TMR1	ADC 采样定时器比较门限 0 寄存器，16 位有符号数；当 MCPWM0_CNT0=TMR1 时产生 TADC[1]事件触发 ADC 进行采样。 MCPWM 发生更新事件后，本寄存器加载到 MCPWM 实际运行系统中。

10.8.15 MCPWM0_TMR2 ADC 定时器门限 2 寄存器

地址:0x4001_0C38

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--

[15:0]	R/W	0	TMR2	ADC 采样定时器比较门限 0 寄存器，16 位有符号数；当 MCPWM0_CNT0=TMR2 时产生 TADC[2]事件触发 ADC 进行采样。 MCPWM 发生更新事件后，本寄存器加载到 MCPWM 实际运行系统中。

10.8.16 MCPWM0_TMR3 ADC 定时器门限 3 寄存器

地址:0x4001_0C3C

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	TMR3	ADC 采样定时器比较门限 0 寄存器，16 位有符号数；当 MCPWM0_CNT0=TMR3 时产生 TADC[3]事件触发 ADC 进行采样。 MCPWM 发生更新事件后，本寄存器加载到 MCPWM 实际运行系统中。

10.8.17 MCPWM0_TH0 MCPWM 时基 0 门限寄存器

地址:0x4001_0C40

写保护的寄存器

复位值:0x7FFF

位域	类型	复位	名称	描述
[31:15]	--	--	--	--
[14:0]	R/W	0	TH0	MCPWM 时基 0 计数器门限值，15 位无符号数，MCPWM 实际运行系统中的时基 0 计数器从-TH 计数到 TH；发生更新事件后，本寄存器加载到 MCPWM 实际运行系统中。

10.8.18 MCPWM0_TH1 MCPWM 时基 1 门限寄存器

地址:0x4001_0C44

写保护的寄存器

复位值:0x7FFF

位域	类型	复位	名称	描述
[31:15]	--	--	--	--
[14:0]	R/W	0	TH1	MCPWM 时基 1 计数器门限值，15 位无符号数，MCPWM 实际运行系统中的时基 1 计数器从-TH 计数到 TH；发生更新事件后，本寄存器加载到 MCPWM 实际运行系统中。

10.8.19 MCPWM0_CNT0 MCPWM 时基 0 计数值寄存器

地址:0x4001_0C48

无写保护的寄存器

复位值:0x7FFF

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	CNT0	向此寄存器写入，更改时基 0 计数器的设定值，发生更新事件后，本寄存器加载到 MCPWM 实际运行系统的时基 0 CNT 中。读出的数据为 MCPWM 实际运行系统中时基 0 计数器的值。实际读出的计数范围为-TH～+TH

10.8.20 MCPWM0_CNT1 MCPWM 时基 1 计数值寄存器

地址:0x4001_0C4C

无写保护的寄存器

复位值:0x7FFF

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	CNT1	向此寄存器写入，更改时基 1 计数器的设定值，发生更新事件后，本寄存器加载到 MCPWM 实际运行系统的时基 1 CNT 中。 读出的数据为 MCPWM 实际运行系统中时基 1 计数器的值。实际读出的计数范围为-TH～+TH

10.8.21 MCPWM0_UPDATE　　MCPWM 手动更新寄存器

地址:0x4001_0C50

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:28]	--	--	--	--
[27]	RO	0	CNT1_UPDATE	写 1 产生软件（手动）触发，将 MCPWM0_CNT1 由预装载寄存器加载到 MCPWM 运行所使用的影子寄存器
[26]	RO	0	TH1_UPDATE	写 1 产生软件（手动）触发，将 MCPWM0_TH1 由预装载寄存器加载到 MCPWM 运行所使用的影子寄存器
[25]	RO	0	TMR3_UPDATE	写 1 产生软件（手动）触发，将 MCPWM0_TMR3 由预装载寄存器加载到 MCPWM 运行所使用的影子寄存器
[24]	RO	0	TMR2_UPDATE	写 1 产生软件（手动）触发，将 MCPWM0_TMR2 由预装载寄存器加载到 MCPWM 运行所使用的影子寄存器
[23:22]	--	--	--	--
[21]	RO	0	TH51_UPDATE	写 1 产生软件（手动）触发，将 MCPWM0_TH51 由预装载寄存器加载到 MCPWM 运行所使用的影子寄存器
[20]	RO	0	TH50_UPDATE	写 1 产生软件（手动）触发，将 MCPWM0_TH50 由预装载寄存器加载到 MCPWM 运行所使用的影子寄存器
[19]	RO	0	TH41_UPDATE	写 1 产生软件（手动）触发，将 MCPWM0_TH41 由预装载寄存器加载到 MCPWM 运行所使用的影子寄存器
[18]	RO	0	TH40_UPDATE	写 1 产生软件（手动）触发，将 MCPWM0_TH40 由预装载寄存器加载到 MCPWM 运行所使用的影子寄存器
[17]	RO	0	TH31_UPDATE	写 1 产生软件（手动）触发，将 MCPWM0_TH31 由预装载寄存器加载到 MCPWM 运行所使用的影子寄存器
[16]	RO	0	TH30_UPDATE	写 1 产生软件（手动）触发，将 MCPWM0_TH30 由预

				装载寄存器加载到 MCPWM 运行所使用的影子寄存器
[15:12]	--	--	--	--
[11]	RO	0	CNT0_UPDATE	写 1 产生软件（手动）触发，将 MCPWM0_CNT0 由预装载寄存器加载到 MCPWM 运行所使用的影子寄存器
[10]	RO	0	TH0_UPDATE	写 1 产生软件（手动）触发，将 MCPWM0_TH0 由预装载寄存器加载到 MCPWM 运行所使用的影子寄存器
[9]	RO	0	TMR1_UPDATE	写 1 产生软件（手动）触发，将 MCPWM0_TMR1 由预装载寄存器加载到 MCPWM 运行所使用的影子寄存器
[8]	RO	0	TMR0_UPDATE	写 1 产生软件（手动）触发，将 MCPWM0_TMR0 由预装载寄存器加载到 MCPWM 运行所使用的影子寄存器
[7:6]	--	--	--	--
[5]	RO	0	TH21_UPDATE	写 1 产生软件（手动）触发，将 MCPWM0_TH21 由预装载寄存器加载到 MCPWM 运行所使用的影子寄存器
[4]	RO	0	TH20_UPDATE	写 1 产生软件（手动）触发，将 MCPWM0_TH20 由预装载寄存器加载到 MCPWM 运行所使用的影子寄存器
[3]	RO	0	TH11_UPDATE	写 1 产生软件（手动）触发，将 MCPWM0_TH11 由预装载寄存器加载到 MCPWM 运行所使用的影子寄存器
[2]	RO	0	TH10_UPDATE	写 1 产生软件（手动）触发，将 MCPWM0_TH10 由预装载寄存器加载到 MCPWM 运行所使用的影子寄存器
[1]	RO	0	TH01_UPDATE	写 1 产生软件（手动）触发，将 MCPWM0_TH01 由预装载寄存器加载到 MCPWM 运行所使用的影子寄存器
[0]	RO	0	TH00_UPDATE	写 1 产生软件（手动）触发，将 MCPWM0_TH00 由预装载寄存器加载到 MCPWM 运行所使用的影子寄存器

向 MCPWM0_UPDATE 对应位写 1 可以触发寄存器值从预装载寄存器写入影子寄存器，MCPWM0_UPDATE 写入后自动清零，无需软件清零。每写 1 一次，进行一次软件/手动触发。对 MCPWM0_UPDATE[15:14]置位不会对 MCPWM0_CNT0/1 造成影响。

向 MCPWM0_UPDATE 写 0 不作用，不推荐写入 0。

10.8.22 MCPWM0_FCNT　MCPWM 刹车计数值寄存器

地址:0x4001_0C54

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	FCNT	若 MCPWM0_FAIL012[15]=1，当发生 fail0/1 事件时，记录 MCPWM0_CNT0 值，存入 MCPWM0_FCNT；若 MCPWM0_FAIL345[15]=1，当发生 fail2/3 事件时，记录 MCPWM0_CNT1 值，存入 MCPWM0_FCNT

10.8.23 MCPWM0_EVT0　MCPWM 时基 0 外部触发寄存器

地址:0x4001_0C58

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	R/W	0	TIM3_CMP1	TIMER3 CMP1 事件触发时基 0 开始计数
[14]	R/W	0	TIM3_CMP0	TIMER3 CMP0 事件触发时基 0 开始计数
[13]	R/W	0	TIM2_CMP1	TIMER2 CMP1 事件触发时基 0 开始计数
[12]	R/W	0	TIM2_CMP0	TIMER2 CMP0 事件触发时基 0 开始计数
[11]	R/W	0	TIM1_CMP1	TIMER1 CMP1 事件触发时基 0 开始计数
[10]	R/W	0	TIM1_CMP0	TIMER1 CMP0 事件触发时基 0 开始计数
[9]	R/W	0	TIM0_CMP1	TIMER0 CMP1 事件触发时基 0 开始计数
[8]	R/W	0	TIM0_CMP0	TIMER0 CMP0 事件触发时基 0 开始计数
[7:4]	--	--	--	--

[3]	R/W	0	PWM0_TMR3	MCPWM0 TMR3 事件触发时基 0 开始计数
[2]	R/W	0	PWM0_TMR2	MCPWM0 TMR2 事件触发时基 0 开始计数
[1]	R/W	0	PWM0_TMR1	MCPWM0 TMR1 事件触发时基 0 开始计数
[0]	R/W	0	PWM0_TMR0	MCPWM0 TMR0 事件触发时基 0 开始计数

10.8.24 MCPWM0_EVT1 MCPWM 时基 1 外部触发寄存器

地址:0x4001_0C5C

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	R/W	0	TIM3_CMP1	TIMER3 CMP1 事件触发时基 1 开始计数
[14]	R/W	0	TIM3_CMP0	TIMER3 CMP0 事件触发时基 1 开始计数
[13]	R/W	0	TIM2_CMP1	TIMER2 CMP1 事件触发时基 1 开始计数
[12]	R/W	0	TIM2_CMP0	TIMER2 CMP0 事件触发时基 1 开始计数
[11]	R/W	0	TIM1_CMP1	TIMER1 CMP1 事件触发时基 1 开始计数
[10]	R/W	0	TIM1_CMP0	TIMER1 CMP0 事件触发时基 1 开始计数
[9]	R/W	0	TIM0_CMP1	TIMER0 CMP1 事件触发时基 1 开始计数
[8]	R/W	0	TIM0_CMP0	TIMER0 CMP0 事件触发时基 1 开始计数
[7:4]	--	--	--	--
[3]	R/W	0	PWM0_TMR3	MCPWM0 TMR3 事件触发时基 1 开始计数
[2]	R/W	0	PWM0_TMR2	MCPWM0 TMR2 事件触发时基 1 开始计数
[1]	R/W	0	PWM0_TMR1	MCPWM0 TMR1 事件触发时基 1 开始计数
[0]	R/W	0	PWM0_TMR0	MCPWM0 TMR0 事件触发时基 1 开始计数

10.8.25 MCPWM0_DTH00 MCPWM CH0/1/2N 死区控制寄存器

地址:0x4001_0C60

写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:10]	--	--	--	--
[9:0]	R/W	0	DTH00	MCPWM CH0/1/2 N 通道死区宽度控制寄存器，10bit 无符号数

10.8.26 MCPWM0_DTH01 MCPWM CH0/1/2P 死区控制寄存器

地址:0x4001_0C64

写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:10]	--	--	--	--
[9:0]	R/W	0	DTH01	MCPWM CH0/1/2 P 通道死区宽度控制寄存器，10bit 无符号数

10.8.27 MCPWM0_DTH10 MCPWM CH3/4/5N 死区控制寄存器

地址:0x4001_0C68

写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:10]	--	--	--	--
[9:0]	R/W	0	DTH10	MCPWM CH3/4/5 N 通道死区宽度控制寄存器，10bit 无符号数

10.8.28 MCPWM0_DTH11　　MCPWM CH3/4/5P 死区控制寄存器

地址:0x4001_0C6C

写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:10]	--	--	--	--
[9:0]	R/W	0	DTH11	MCPWM CH3/4/5 P 通道死区宽度控制寄存器，10bit 无符号数

10.8.29 MCPWM0_FLT　　MCPWM 滤波时钟分频寄存器

地址:0x4001_0C70

写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:8]	--	--	--	--
[7:0]	R/W	0	FLT_CLKDIV	FAIL 信号输入的滤波时钟分频寄存器，基于系统时钟分频，影响 MCPWM0_FAIL[1:0]。计算公式如下：系统时钟/（FLT_CLKDIV + 1）。分频范围是 1-256。

MCPWM 使用分频后的时钟对 FAIL 信号固定进行 16 周期的滤波。当使用 256 分频时，滤波宽度为 4096 个 TCLK 时钟宽度。

10.8.30 MCPWM0_SDCFG　　MCPWM 配置寄存器

地址:0x4001_0C74

写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:15]	--	--	--	--
[14]	R/W	0	TR1_AEC	更新事件是否自动清除 MCPWM0_EIF[7:6]并置位

				MCPWM0_FAIL345.MOE，恢复 MCPWM 通道 3/4/5 信号输出。 1:使能自动故障清除功能； 0:关闭自动故障清除功能。
[13]	R/W	0	TR1_T1_UEN	时基 1 t1（过零）事件更新使能。 1:使能； 0，关闭。
[12]	R/W	0	TR1_T0_UEN	时基 1 t1（过零）事件更新使能。 1:使能； 0，关闭。
[11:8]	R/W	0	TR1_UP_INTV	时基 1 更新间隔。一旦 t0 和 t1 事件发生次数同 TR1_UP_INTV 相等，MCPWM 系统自动触发 MCPWM0_TH1，TH30~TH51 和 MCPWM0_TMR 寄存器加载到 MCPWM 运行系统的操作。若 TR1_T1_UEN 和 TR1_T0_UEN 均关闭，将不会触发此类型加载，只能手动触发加载。
[7]	--	--	--	--
[6]	R/W	0	TR0_AEC	更新事件是否自动清除 MCPWM0_EIF[5:4]并置位 MCPWM0_FAIL012.MOE，恢复 MCPWM 通道 0/1/2 信号输出。 1:使能自动故障清除功能； 0:关闭自动故障清除功能。
[5]	R/W	0	TR0_T1_UEN	时基 0 t1（过零）事件更新使能。 1:使能； 0，关闭。
[4]	R/W	0	TR0_T0_UEN	时基 0 t0（起点）事件更新使能。 1:使能； 0，关闭。

[3:0]	R/W	0	TR0_UP_INTV	时基 0 更新间隔。一旦 t0 和 t1 事件发生次数同 TR0_UP_INTV+1 相等，MCPWM 系统自动触发 MCPWM0_TH0，MCPWM0_TH00~TH21 和 MCPWM0_TMR 寄存器加载到 MCPWM 运行系统的操作。若 TR0_T1_UEN 和 TR0_T0_UEN 均关闭，将不会触发此类型加载，只能手动触发加载。

10.8.31 MCPWM0_AUEN MCPWM 自动更新使能寄存器

地址:0x4001_0C78

写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:28]	--	--	--	--
[27]	R/W	0	CNT1_AUPDATE	MCPWM0_CNT1 自动加载使能。 1:加载； 0:不加载。
[26]	R/W	0	TH1_AUPDATE	MCPWM0_TH1 自动加载使能。 1:加载； 0:不加载。
[25]	R/W	0	TMR3_AUPDATE	MCPWM0_TMR3 自动加载使能。 1:加载； 0:不加载。
[24]	R/W	0	TMR2_AUPDATE	MCPWM0_TMR2 自动加载使能。 1:加载； 0:不加载。
[23:22]	--	--	--	--
[21]	R/W	0	TH51_AUPDATE	MCPWM0_TH51 自动加载使能。 1:加载；

				0:不加载。
[20]	R/W	0	TH50_AUPDATE	MCPWM0_TH50 自动加载使能。 1:加载； 0:不加载。
[19]	R/W	0	TH41_AUPDATE	MCPWM0_TH41 自动加载使能。 1:加载； 0:不加载。
[18]	R/W	0	TH40_AUPDATE	MCPWM0_TH40 自动加载使能。 1:加载； 0:不加载。
[17]	R/W	0	TH31_AUPDATE	MCPWM0_TH31 自动加载使能。 1:加载； 0:不加载。
[16]	R/W	0	TH30_AUPDATE	MCPWM0_TH30 自动加载使能。 1:加载； 0:不加载。
[15:12]	--	--	--	--
[11]	R/W	0	CNT0_AUPDATE	MCPWM0_CNT0 自动加载使能。 1:加载； 0:不加载。
[10]	R/W	0	TH0_AUPDATE	MCPWM0_TH0 自动加载使能。 1:加载； 0:不加载。
[9]	R/W	0	TMR1_AUPDATE	MCPWM0_TMR1 自动加载使能。 1:加载； 0:不加载。
[8]	R/W	0	TMR0_AUPDATE	MCPWM0_TMR0 自动加载使能。 1:加载；

				0:不加载。
[7:6]	--	--	--	--
[5]	R/W	0	TH21_AUPDATE	MCPWM0_TH21 自动加载使能。 1:加载； 0:不加载。
[4]	R/W	0	TH20_AUPDATE	MCPWM0_TH20 自动加载使能。 1:加载； 0:不加载。
[3]	R/W	0	TH11_AUPDATE	MCPWM0_TH11 自动加载使能。 1:加载； 0:不加载。
[2]	R/W	0	TH10_AUPDATE	MCPWM0_TH10 自动加载使能。 1:加载； 0:不加载。
[1]	R/W	0	TH01_AUPDATE	MCPWM0_TH01 自动加载使能。 1:加载； 0:不加载。
[0]	R/W	0	TH00_AUPDATE	MCPWM0_TH00 自动加载使能。 1:加载； 0:不加载。

10.8.32 MCPWM0_TCLK MCPWM 时钟配置寄存器

地址:0x4001_0C7C

写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11]	R/W	0	CMP_CTRL_CNT1	比较器控制 MCPWM0_CNT1 计数

[10]	R/W	0	CMP_CTRL_CNT0	比较器控制 MCPWM0_CNT0 计数
[9]	R/W	0	EVT_CNT1_EN	时基 1 外部触发使能
[8]	R/W	0	EVT_CNT0_EN	时基 0 外部触发使能
[7]	R/W	0	BASE_CNT1_EN	MCPWM 时基 1 计数器使能开关。 1:使能； 0:关闭。--
[6]	R/W	0	TH51_AUPDATE	MCPWM 时基 0 计数器使能开关。 1:使能； 0:关闭。
[5]	R/W	0	TMR3_TB	TMR3 时基选择 0:时基 0， 1:时基 1
[4]	R/W	0	TMR2_TB	TMR2 时基选择 0:时基 0， 1:时基 1
[3]	--	--	--	--
[2]	R/W	0	CLK_EN	MCPWM 工作时钟使能。 1:使能； 0:关闭。
[1:0]	R/W	0	CLK_DIV	MCPWM 工作时钟分频寄存器。 0:系统时钟 1:系统时钟/2 2:系统时钟/4 3:系统时钟/8

只有使能 MCPWM0_TCLK.CLK_EN，才能完成影子寄存器更新。配置完成影子寄存器之后，同 时开启 MCPWM0_TCLK.BASE_CNT0/1_EN 可以使得时基 0 和时基 1 同步开始计数。如果 TH0 和 TH1 设置为相同值，则时基 0 和时基 1 完全同频。 当使用外部触发 MCPWM 开始计数时，需要配置 BASE_CNTx_EN 为 0，EVT_CNTx_EN 为 1，同时设置 MCPWM0_EVTx 选择合适的外部

触发源。待触发事件发生后，BASE_CNTx_EN 会由硬件置 1，MCPWM 对应计数器开始计数。

10.8.33 MCPWM0_IE0 MCPWM 时基 0 中断控制寄存器

地址:0x4001_0C80

写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:15]	--	--	--	--
[14]	R/W	0	UP_IE	时基 0 更新事件中断源使能。 1，使能； 0，关闭。
[13]	R/W	0	TMR3_IE	MCPWM0_CNT0 等于 MCPWM0_TMR3 中断源使能。 2，使能； 0，关闭。
[12]	R/W	0	TMR2_IE	MCPWM0_CNT0 等于 MCPWM0_TMR2 中断源使能。 3，使能； 0，关闭。
[11]	R/W	0	TMR1_IE	MCPWM0_CNT0 等于 MCPWM0_TMR1 中断源使能。 1，使能； 0，关闭。
[10]	R/W	0	TMR0_IE	MCPWM0_CNT0 等于 MCPWM0_TMR0 中断源使能。 4，使能； 0，关闭。
[9:8]	--	--	--	--
[7]	R/W	0	TH21_IE	MCPWM0_CNT0 等于 MCPWM0_TH21 中断源使能。 5，使能； 0，关闭。
[6]	R/W	0	TH20_IE	MCPWM0_CNT0 等于 MCPWM0_TH21 中断源使能。 6，使能； 0，关闭。

[5]	R/W	0	TH11_IE	MCPWM0_CNT0 等于 MCPWM0_TH11 中断源使能。 7，使能； 0，关闭。
[4]	R/W	0	TH10_IE	MCPWM0_CNT0 等于 MCPWM0_TH10 中断源使能。 8，使能； 0，关闭。
[3]	R/W	0	TH01_IE	MCPWM0_CNT0 等于 MCPWM0_TH01 中断源使能。 9，使能； 0，关闭。
[2]	R/W	0	TH00_IE	MCPWM0_CNT0 等于 MCPWM0_TH00 中断源使能。 10， 使能； 0，关闭。
[1]	R/W	0	T1_IE	时基 0 T1 事件，MCPWM0_CNT0 的计数值回 0 中断源使能。 1，使能； 0，关闭。
[0]	R/W	0	T0_IE	时基 0 T0 事件，MCPWM0_CNT0 的计数值等于 MCPWM0_TH0 中断源使能。 1，使能； 0，关闭。

10.8.34 MCPWM0_IF0 MCPWM 时基 0 中断标志寄存器

地址:0x4001_0C84

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:15]	--	--	--	--
[14]	RW1C	0	UP_IF	时基 0 更新事件： MCPWM0_TH0/MCPWM0_TH00~MCPWM0_TH21/MCPWM0_TMR 等寄存器更新到 MCPWM 实际运行系统的中断事

				件。 1，发生； 0，没发生。 写 1 清零。
[13]	RW1C	0	TMR3_IF	MCPWM0_CNT0 等于 MCPWM0_TMR3 中断事件。 1，发生； 0，没发生。 写 1 清零。
[12]	RW1C	0	TMR2_IF	MCPWM0_CNT0 等于 MCPWM0_TMR2 中断事件。 1，发生； 0，没发生。 写 1 清零。
[11]	RW1C	0	TMR1_IF	MCPWM0_CNT0 等于 MCPWM0_TMR1 中断事件。 1，发生； 0，没发生。 写 1 清零。
[10]	RW1C	0	TMR0_IF	MCPWM0_CNT0 等于 MCPWM0_TMR0 中断事件。 1，发生； 0，没发生。 写 1 清零。
[9:8]	--	--	--	--
[7]	RW1C	0	TH21_IF	MCPWM0_CNT0 等于 MCPWM0_TH21 中断事件。 1，发生； 0，没发生。 写 1 清零。
[6]	RW1C	0	TH20_IF	MCPWM0_CNT0 等于 MCPWM0_TH20 中断事件。 1，发生； 0，没发生。

				写 1 清零。
[5]	RW1C	0	TH11_IF	MCPWM0_CNT0 等于 MCPWM0_TH11 中断事件。 1，发生； 0，没发生。 写 1 清零。
[4]	RW1C	0	TH10_IF	MCPWM0_CNT0 等于 MCPWM0_TH10 中断事件。 1，发生； 0，没发生。 写 1 清零。
[3]	RW1C	0	TH01_IF	MCPWM0_CNT0 等于 MCPWM0_TH01 中断事件。 1，发生； 0，没发生。 写 1 清零。
[2]	RW1C	0	TH00_IF	MCPWM0_CNT0 等于 MCPWM0_TH00 中断事件。 1，发生； 0，没发生。 写 1 清零。
[1]	RW1C	0	T1_IF	t1 事件，MCPWM0_CNT0 等于 0 中断事件。 1，发生； 0，没发生。 写 1 清零。
[0]	RW1C	0	T0_IF	t0 事件，MCPWM0_CNT0 等于 MCPWM0_TH 中断事件。 1，发生； 0，没发生。 写 1 清零。

10.8.35 MCPWM0_IE1　　MCPWM 时基 1 中断控制寄存器

地址:0x4001_0C88

写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:15]	--	--	--	--
[14]	R/W	0	UP_IE	时基 1 更新事件中断源使能。 1，使能； 0，关闭。
[13]	R/W	0	TMR3_IE	MCPWM0_CNT1 等于 MCPWM0_TMR3 中断源使能。 1，使能； 0，关闭。
[12]	R/W	0	TMR2_IE	MCPWM0_CNT1 等于 MCPWM0_TMR2 中断源使能。 1，使能； 0，关闭。
[11:8]	--	--	--	--
[7]	R/W	0	TH51_IE	MCPWM0_CNT1 等于 MCPWM0_TH21 中断源使能。 1，使能； 0，关闭。
[6]	R/W	0	TH50_IE	MCPWM0_CNT1 等于 MCPWM0_TH21 中断源使能。 1，使能； 0，关闭。
[5]	R/W	0	TH41_IE	MCPWM0_CNT1 等于 MCPWM0_TH11 中断源使能。 1，使能； 0，关闭。
[4]	R/W	0	TH40_IE	MCPWM0_CNT1 等于 MCPWM0_TH10 中断源使能。 1，使能； 0，关闭。
[3]	R/W	0	TH31_IE	MCPWM0_CNT1 等于 MCPWM0_TH01 中断源使能。 1，使能； 0，关闭。

[2]	R/W	0	TH30_IE	MCPWM0_CNT1 等于 MCPWM0_TH00 中断源使能。 1，使能； 0，关闭。
[1]	R/W	0	T1_IE	时基 1 T1 事件，MCPWM0_CNT1 的计数值回 0 中断源使能。 1，使能； 0，关闭。
[0]	R/W	0	T0_IE	时基 1 T0 事件，MCPWM0_CNT1 的计数值等于 MCPWM0_TH1 中断源使能。 1，使能； 0，关闭。

10.8.36 MCPWM0_IF1 MCPWM 时基 1 中断标志寄存器

地址:0x4001_0C8C

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:15]	--	--	--	--
[14]	RW1C	0	UP_IF	时基 1 更新事件： MCPWM0_TH1/MCPWM0_TH30~MCPWM0_TH51/MCPWM0_TMR 等寄存器更新到 MCPWM 实际运行系统的中断事件。 1，发生； 0，没发生。 写 1 清零。
[13]	RW1C	0	TMR3_IF	MCPWM0_CNT1 等于 MCPWM0_TMR3 中断事件。 1，发生； 0，没发生。 写 1 清零。

[12]	RW1C	0	TMR2_IF	MCPWM0_CNT1 等于 MCPWM0_TMR2 中断事件。 1，发生； 0，没发生。 写 1 清零。
[11:8]	--	--	--	--
[7]	RW1C	0	TH51_IF	MCPWM0_CNT1 等于 MCPWM0_TH21 中断事件。 1，发生； 0，没发生。 写 1 清零。
[6]	RW1C	0	TH50_IF	MCPWM0_CNT1 等于 MCPWM0_TH20 中断事件。 1，发生； 0，没发生。 写 1 清零。
[5]	RW1C	0	TH41_IF	MCPWM0_CNT1 等于 MCPWM0_TH11 中断事件。 1，发生； 0，没发生。 写 1 清零。
[4]	RW1C	0	TH40_IF	MCPWM0_CNT1 等于 MCPWM0_TH10 中断事件。 1，发生； 0，没发生。 写 1 清零。
[3]	RW1C	0	TH31_IF	MCPWM0_CNT1 等于 MCPWM0_TH01 中断事件。 1，发生； 0，没发生。 写 1 清零。
[2]	RW1C	0	TH30_IF	MCPWM0_CNT1 等于 MCPWM0_TH00 中断事件。 1，发生； 0，没发生。

				写 1 清零。
[1]	RW1C	0	T1_IF	t1 事件，MCPWM0_CNT1 等于 0 中断事件。 1，发生； 0，没发生。 写 1 清零。
[0]	RW1C	0	T0_IF	t0 事件，MCPWM0_CNT1 等于 MCPWM0_TH 中断事件。 1，发生； 0，没发生。 写 1 清零。

10.8.37 MCPWM0_EIE MCPWM FAIL 中断控制寄存器

地址:0x4001_0C90

写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:8]	--	--	--	--
[7]	R/W	0	FAIL3_IE	FAIL3 中断源使能。 1，使能； 0，关闭。
[6]	R/W	0	FAIL2_IE	FAIL2 中断源使能。 1，使能； 0，关闭。
[5]	R/W	0	FAIL1_IE	FAIL1 中断源使能。 1，使能； 0，关闭。
[4]	R/W	0	FAIL0_IE	FAIL0 中断源使能。 1，使能； 0，关闭。

[3:0]	--	--	--	--

10.8.38 MCPWM0_EIF MCPWM FAIL 中断标志寄存器

地址:0x4001_0C94

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:8]	--	--	--	--
[7]	RW1C	0	FAIL3_IF	FAIL3 中断源事件。 1 ， 发生； 0，没发生。 写 1 清零。
[6]	RW1C	0	FAIL2_IF	FAIL2 中断源事件。 1 ， 发生； 0 ， 没发生。 写 1 清零。
[5]	RW1C	0	FAIL1_IF	FAIL1 中断源事件。 1 ， 发生； 0 ， 没发生。 写 1 清零。
[4]	RW1C	0	FAIL0_IF	FAIL0 中断源事件。 1 ， 发生； 0 ， 没发生。 写 1 清零。
[3:0]	--	--	--	--

10.8.39 MCPWM0_EIF MCPWM DMA 配置寄存器

地址:0x4001_0C98

写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	R/W	0	TR1_T1_RE1	时基 1 T1 事件 DMA 请求 1 使能。 1:使能； 0:关闭。
[14]	R/W	0	TR1_T0_RE1	时基 1 T0 事件 DMA 请求 1 使能。 1:使能； 0:关闭。
[13]	R/W	0	TR0_T1_RE1	时基 0 T1 事件 DMA 请求 1 使能。 1:使能； 0:关闭。
[12]	R/W	0	TR0_T0_RE1	时基 0 T0 事件 DMA 请求 1 使能。 1:使能； 0:关闭。
[11]	R/W	0	TMR3_RE1	MCPWM 计数器命中 TMR3，DMA 请求 1 使能。 1:使能； 0:关闭。
[10]	R/W	0	TMR2_RE1	MCPWM 计数器命中 TMR2，DMA 请求 1 使能。 1:使能； 0:关闭。
[9]	R/W	0	TMR1_RE1	MCPWM 计数器命中 TMR1，DMA 请求 1 使能。 1:使能； 0:关闭。
[8]	R/W	0	TMR0_RE1	MCPWM 计数器命中 TMR0，DMA 请求 1 使能。

				1:使能； 0:关闭。
[7]	R/W	0	TR1_T1_RE0	时基 1 T1 事件 DMA 请求 0 使能。 1:使能； 0:关闭。
[6]	R/W	0	TR1_T0_RE0	时基 1 T0 事件 DMA 请求 0 使能。 1:使能； 0:关闭。
[5]	R/W	0	TR1_T0_RE0	时基 0 T1 事件 DMA 请求 0 使能。 1:使能； 0:关闭。
[4]	R/W	0	TR0_T0_RE0	时基 0 T0 事件 DMA 请求 0 使能。 1:使能； 0:关闭。
[3]	R/W	0	TMR3_RE0	MCPWM 计数器命中 TMR3，DMA 请求 0 使能。 1:使能； 0:关闭。
[2]	R/W	0	TMR2_RE0	MCPWM 计数器命中 TMR2，DMA 请求 0 使能。 1:使能； 0:关闭。
[1]	R/W	0	TMR1_RE0	MCPWM 计数器命中 TMR1，DMA 请求 0 使能。 1:使能； 0:关闭。
[0]	R/W	0	TMR0_RE0	MCPWM 计数器命中 TMR0，DMA 请求 0 使能。 1:使能； 0:关闭。

MCPWM 可以产生两个 DMA 请求信号。如果使能 MCPWM0_RE 寄存器中的 RE0，且对应的事件标志置位，会产生 DMA 请求信号 0；如果使能 MCPWM0_RE 寄存器中的 RE1，且对应的事件

标志置位，会产生 DMA 请求信号 1。两个请求信号可以分别触发两个 DMA 通道进行搬运。

10.8.40 MCPWM0_PP　MCPWM 推挽配置寄存器

地址:0x4001_0C9C

写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:6]	--	--	--	--
[5]	R/W	0	IO5_PPE	IO5 推挽模式使能信号。 写 1，使能； 写 0，关闭。
[4]	R/W	0	IO4_PPE	IO4 推挽模式使能信号。 写 1，使能； 写 0，关闭。
[3]	R/W	0	IO3_PPE	IO3 推挽模式使能信号。 写 1，使能； 写 0，关闭。
[2]	R/W	0	IO2_PPE	IO2 推挽模式使能信号。 写 1，使能； 写 0，关闭。
[1]	R/W	0	IO1_PPE	IO1 推挽模式使能信号。 写 1，使能； 写 0，关闭。
[0]	R/W	0	IO0_PPE	IO0 推挽模式使能信号。 写 1，使能； 写 0，关闭。

推挽模式使能信号。根据工作模式不同而不同。边沿模式，开启边沿模式的推挽模式；中心对齐，开启中心对齐的推挽模式。

10.8.41 MCPWM0_IO01　MCPWM 通道 01 配置寄存器

地址:0x4001_0CA0

写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	R/W	0	CH1_WM	CH1 工作模式选择。 1:边沿模式； 0:互补模式。
[14]	R/W	0	CH1_PN_SW	CH1 的 P 和 N 通道输出互换选择。即 P 通道信号最后从 N 通道输出，N 通道的信号最后从 P 通道输出。 1:互换； 0:不互换。
[13]	R/W	0	CH1_SCTRLP	当 CH1_PS=1 时，输出到 CH1 P 通道的值。
[12]	R/W	0	CH1_SCTRLN	当 CH1_NS=1 时，输出到 CH1 N 通道的值。
[11]	R/W	0	CH1_PS	CH1 P 来源。 1:来自 CH1_SCTRLP； 0:MCPWM 内部计数器产生。
[10]	R/W	0	CH1_NS	CH1 N 来源。1:来自 CH1_SCTRLN； 0:MCPWM 内部计数器产生。
[9]	R/W	0	CH1_PP	CH1 P 极性选择。 1:CH1 P 信号取反输出； 0:CH1 P 信号正常输出。
[8]	R/W	0	CH1_NP	CH1 N 极性选择。 1:CH1 N 信号取反输出； 0:CH1 N 信号正常输出。
[7]	R/W	0	CH0_WM	CH0 工作模式选择。 1:边沿模式；

				0:互补模式。
[6]	R/W	0	CH0_PN_SW	CH0 的 P 和 N 通道输出互换选择。即 P 通道信号最后从 N 通道输出，N 通道的信号最后从 P 通道输出。 1:互换； 0:不互换。
[5]	R/W	0	CH0_SCTRLP	当 CH0_PS =1 时，输出到 CH0 P 通道的值。
[4]	R/W	0	CH0_SCTRLN	当 CH0_NS =1 时，输出到 CH0 N 通道的值。
[3]	R/W	0	CH0_PS	CH0 P 来源。 1:来自 CH0_SCTRLP； 0:MCPWM 实际运行系统中计数器产生。
[2]	R/W	0	CH0_NS	CH0 N 来源。 1:来自 CH0_SCTRLN； 0:MCPWM 实际运行系统中计数器产生。
[1]	R/W	0	CH0_PP	CH0 P 极性选择。 1:CH0 P 信号取反输出； 0:CH0 P 信号正常输出。
[0]	R/W	0	CH0_NP	CH0 N 极性选择。 1:CH0 N 信号取反输出； 0:CH0 N 信号正常输出。 极性选择跟随通道交换，例如 CH0 N 选择取反输出，同时选择了通道交换，则交换后的 CH0 N 仍是取反输出。

10.8.42 CPWM0_IO23 MCPWM 通道 23 配置寄存器

地址:0x4001_0CA4

写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--

[15]	R/W	0	CH3_WM	CH3 工作模式选择。 1:边沿模式； 0:互补模式。
[14]	R/W	0	CH3_PN_SW	CH3 的 P 和 N 通道输出互换选择。即 P 通道信号最后从 N 通道输出，N 通道的信号最后从 P 通道输出。 1:互换； 0:不互换。
[13]	R/W	0	CH3_SCTRLP	当 CH3_PS=1 时，输出到 CH3 P 通道的值。
[12]	R/W	0	CH3_SCTRLN	当 CH3_NS=1 时，输出到 CH3 N 通道的值。
[11]	R/W	0	CH3_PS	CH3 P 来源。 1:来自 CH3_SCTRLP； 0:MCPWM 内部计数器产生。
[10]	R/W	0	CH3_NS	CH3 N 来源。 1:来自 CH3_SCTRLN； 0:MCPWM 内部计数器产生。
[9]	R/W	0	CH3_PP	CH3 P 极性选择。 1:CH3 P 信号取反输出； 0:CH3 P 信号正常输出。
[8]	R/W	0	CH3_NP	CH3 N 极性选择。 1:CH3 N 信号取反输出； 0:CH3 N 信号正常输出。
[7]	R/W	0	CH2_WM	CH2 工作模式选择。 1:边沿模式； 0:互补模式。
[6]	R/W	0	CH2_PN_SW	CH2 的 P 和 N 通道输出互换选择。即 P 通道信号最后从 N 通道输出，N 通道的信号最后从 P 通道输出。 1:互换 ；0:不互换。

[5]	R/W	0	CH2_SCTRLP	当 CH2_PS =1 时，输出到 CH2 P 通道的值。
[4]	R/W	0	CH2_SCTRLN	当 CH2_NS =1 时，输出到 CH2 N 通道的值。
[3]	R/W	0	CH2_PS	CH2 P 来源。 1:来自 CH2_SCTRLP； 0:MCPWM 实际运行系统中计数器产生。
[2]	R/W	0	CH2_NS	CH2 N 来源。 1:来自 CH2_SCTRLN； 0:MCPWM 实际运行系统中计数器产生。
[1]	R/W	0	CH2_PP	CH2 P 极性选择。 1:CH2 P 信号取反输出； 0:CH2 P 信号正常输出。
[0]	R/W	0	CH2_NP	CH2 N 极性选择。 1:CH2 N 信号取反输出； 0:CH2 N 信号正常输出。 极性选择跟随通道交换，例如 CH2 N 选择取反输出，同时选择了通道交换，则交换后的 CH2 N 仍是取反输出。

10.8.43 MCPWM0_IO45 MCPWM 通道 45 配置寄存器

地址:0x4001_0CA8

写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	R/W	0	CH5_WM	CH5 工作模式选择。 1:边沿模式； 0:互补模式。
[14]	R/W	0	CH5_PN_SW	CH5 的 P 和 N 通道输出互换选择。即 P 通道信号最后从 N 通道输出，N 通道的信号最后从 P 通道输出。

				1:互换； 0:不互换。
[13]	R/W	0	CH5_SCTRLP	当 CH5_PS=1 时，输出到 CH5 P 通道的值。
[12]	R/W	0	CH5_SCTRLN	当 CH5_NS=1 时，输出到 CH5 N 通道的值。
[11]	R/W	0	CH5_PS	CH5 P 来源。 1:来自 CH5_SCTRLP； 0:MCPWM 内部计数器产生。
[10]	R/W	0	CH5_NS	CH5 N 来源。 1:来自 CH5_SCTRLN； 0:MCPWM 内部计数器产生。
[9]	R/W	0	CH5_PP	CH5 P 极性选择。 1:CH5 P 信号取反输出； 0:CH5 P 信号正常输出。
[8]	R/W	0	CH5_NP	CH5 N 极性选择。 1:CH5 N 信号取反输出； 0:CH5 N 信号正常输出。
[7]	R/W	0	CH4_WM	CH4 工作模式选择。 1:边沿模式； 0:互补模式。
[6]	R/W	0	CH4_PN_SW	CH4 的 P 和 N 通道输出互换选择。即 P 通道信号最后从 N 通道输出，N 通道的信号最后从 P 通道输出。 1:互换； 0:不互换。
[5]	R/W	0	CH4_SCTRLP	当 CH4_PS =1 时，输出到 CH4 P 通道的值。
[4]	R/W	0	CH4_SCTRLN	当 CH4_NS =1 时，输出到 CH4 N 通道的值。
[3]	R/W	0	CH4_PS	CH4 P 来源。 1:来自 CH4_SCTRLP； 0:MCPWM 实际运行系统中计数器产生。

[2]	R/W	0	CH4_NS	CH4 N 来源。 1:来自 CH4_SCTRLN； 0:MCPWM 实际运行系统中计数器产生。
[1]	R/W	0	CH4_PP	CH4 P 极性选择。 1:CH4 P 信号取反输出； 0:CH4 P 信号正常输出。
[0]	R/W	0	CH4_NP	CH4 N 极性选择。 1:CH4 N 信号取反输出； 0:CH4 N 信号正常输出。 极性选择跟随通道交换，例如 CH4 N 选择取反输出，同时选择了通道交换，则交换后的 CH4 N 仍是取反输出。

10.8.44 MCPWM0_IOCFG MCPWM 通道配置寄存器

地址:0x4001_0CAC

写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[13]	R/W	0	CH5_SCTRLP	当 CH5_PS =1 时，输出到 CH5 P 通道的值。
[12]	R/W	0	CH5_SCTRLN	当 CH5_NS =1 时，输出到 CH5 N 通道的值。
[11]	R/W	0	CH4_SCTRLP	当 CH4_PS =1 时，输出到 CH4 P 通道的值。
[10]	R/W	0	CH4_SCTRLN	当 CH4_NS =1 时，输出到 CH4 N 通道的值。
[9]	R/W	0	CH3_SCTRLP	当 CH3_PS =1 时，输出到 CH3 P 通道的值。
[8]	R/W	0	CH3_SCTRLN	当 CH3_NS =1 时，输出到 CH3 N 通道的值。
[23:22]	--	--	--	--
[21]	R/W	0	CH5_PS	CH5 P 来源。 1:来自 CH5_SCTRLP； 0:MCPWM 实际运行系统中计数器产生。

[20]	R/W	0	CH5_NS	CH5 N 来源。 1:来自 CH5_SCTRLN； 0:MCPWM 实际运行系统中计数器产生。
[19]	R/W	0	CH4_PS	CH4 P 来源。 1:来自 CH4_SCTRLP； 0:MCPWM 实际运行系统中计数器产生。
[18]	R/W	0	CH4_NS	CH4 N 来源。 1:来自 CH4_SCTRLN； 0:MCPWM 实际运行系统中计数器产生。
[17]	R/W	0	CH3_PS	CH3 P 来源。 1:来自 CH3_SCTRLP； 0:MCPWM 实际运行系统中计数器产生。
[16]	R/W	0	CH3_NS	CH3 N 来源。 1:来自 CH3_SCTRLN； 0:MCPWM 实际运行系统中计数器产生。
[15:14]	--	--	--	--
[13]	R/W	0	CH2_SCTRLP	当 CH2_PS =1 时，输出到 CH2 P 通道的值。
[12]	R/W	0	CH2_SCTRLN	当 CH2_NS =1 时，输出到 CH2 N 通道的值。
[11]	R/W	0	CH1_SCTRLP	当 CH1_PS =1 时，输出到 CH1 P 通道的值。
[10]	R/W	0	CH1_SCTRLN	当 CH1_NS =1 时，输出到 CH1 N 通道的值。
[9]	R/W	0	CH0_SCTRLP	当 CH0_PS =1 时，输出到 CH0 P 通道的值。
[8]	R/W	0	CH0_SCTRLN	当 CH0_NS =1 时，输出到 CH0 N 通道的值。
[7:6]	--	--	--	--
[5]	R/W	0	CH2_PS	CH2 P 来源。 1:来自 CH2_SCTRLP； 0:MCPWM 实际运行系统中计数器产生。
[4]	R/W	0	CH2_NS	CH2 N 来源。 1:来自 CH2_SCTRLN；

				0:MCPWM 实际运行系统中计数器产生。
[3]	R/W	0	CH1_PS	CH1 P 来源。 1:来自 CH1_SCTRLP； 0:MCPWM 实际运行系统中计数器产生。
[2]	R/W	0	CH1_NS	CH1 N 来源。 1:来自 CH1_SCTRLN； 0:MCPWM 实际运行系统中计数器产生。
[1]	R/W	0	CH0_PS	CH0 P 来源。 1:来自 CH0_SCTRLP； 0:MCPWM 实际运行系统中计数器产生。
[0]	R/W	0	CH0_NS	CH0 N 来源。 1:来自 CH0_SCTRLN； 0:MCPWM 实际运行系统中计数器产生。

10.8.45 MCPWM0_FAIL012 MCPWM FAIL012 配置寄存器

地址:0x4001_0CB0

写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	R/W	0	FAIL_0CAP	当发生 fail0/1 事件时，将 MCPWM0_CNT0 值存入 MCPWM0_FCNT
[14:12]	--	--	--	--
[11:10]	R/W	0	FAIL1_SEL	FAIL1 来源选择 0: IO MCPWM0_BKIN1 1: CSA OC 输出
[9:8]	R/W	0	FAIL0_SEL	FAIL0 来源选择 0: IO MCPWM0_BKIN0 1: CSA OC 输出
[7]	R/W	0	HALT_PRT	MCU 进入 HALT 状态，MCPWM 输出值选择。

				1:正常输出； 0:强制 MCPWM 输出保护值。
[6]	R/W	0	MOE	MOE 控制 MCPWM CH0/CH1/CH2 通道 P 和 N 输出值。 1:输出 MCPWM 产生的正常信号 0:输出 MCPWM0_CH_DEF 设置的 CHxN_DEFAULT 和 CHxP_DEFAULT 默认值，此默认值不受极性/通道选择等控制。 MCPWM0_EIF.FAIL1_IF 和 MCPWM0_EIF.FAIL0_IF 任意一位变 1 将触发 MOE 变成 0，输出默认值。
[5]	R/W	0	FAIL1_EN	FAIL1 输入使能。 1:使能； 0:关闭。
[4]	R/W	0	FAIL0_EN	FAIL0 输入使能。 1:使能； 0:关闭。
[3]	R/W	0	FAIL1_POL	FAIL1 极性选择。 1:信号极性取反输入，信号输入低为有效电平； 0:信号极性正常输入，信号输入高为有效电平。
[2]	R/W	0	FAIL0_POL	FAIL0 极性选择。 1:信号极性取反输入，信号输入低为有效电平； 0:信号极性正常输入，信号输入高为有效电平。
[1:0]	--	--	--	--

FAIL0/1 信号用于工作于时基 0 的 MCPWM 通道 0/1/2。当 FAIL0/1 有效时，将暂停 MCPWM 时基 0 对应的计数器 MCPWM0_CNT0，并产生对应的错误中断标志 MCPWM0_EIF。

10.8.46 MCPWM0_FAIL345　MCPWM FAIL345 配置寄存器

地址:0x4001_0CB4

写保护的寄存器

复位值:0x

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	R/W	0	FAIL_1CAP	当发生 fail2/3 事件时，将 MCPWM0_CNT1 值存入 MCPWM0_FCNT
[14:12]	--	--	--	--
[11:10]	R/W	0	FAIL3_SEL	FAIL3 来源选择 0: IO MCPWM0_BKIN1 1: CSA OC 输出
[9:8]	R/W	0	FAIL2_SEL	FAIL2 来源选择 0: IO MCPWM0_BKIN0 1: CSA OC 输出
[7]	R/W	0	HALT_PRT	MCU 进入 HALT 状态，MCPWM 输出值选择。 1:正常输出； 0:强制 MCPWM 输出保护值。
[6]	R/W	0	MOE	MOE 控制 MCPWM CH3/CH4/CH5 通道 P 和 N 输出值。 1:输出 MCPWM 产生的正常信号 0:输出 MCPWM0_CH_DEF 设置的 CHxN_DEFAULT 和 CHxP_DEFAULT 默认值，此默认值不受极性/通道选择等控制。 MCPWM0_EIF.FAIL2_IF 和 MCPWM0_EIF.FAIL3_IF 任意一位变 1 将触发 MOE 变成 0，输出默认值。
[5]	R/W	0	FAIL3_EN	FAIL3 输入使能。 1:使能； 0:关闭。
[4]	R/W	0	FAIL2_EN	FAIL2 输入使能。 1:使能； 0:关闭。
[3]	R/W	0	FAIL3_POL	FAIL3 极性选择。

				1:信号极性取反输入，信号输入低为有效电平； 0:信号极性正常输入，信号输入高为有效电平。
[2]	R/W	0	FAIL2_POL	FAIL2 极性选择。 1:信号极性取反输入，信号输入低为有效电平； 0:信号极性正常输入，信号输入高为有效电平。
[1:0]	--	--	--	--

FAIL2/3 信号用于工作于时基 0 的 MCPWM 通道 3/4/5。当 FAIL2/3 有效时，将暂停 MCPWM 时基 0 对应的计数器 MCPWM0_CNT1，并产生对应的错误中断标志 MCPWM0_EIF。

MCPWM0_FAIL 可以用来设置紧急停车事件，封锁 MCPWM 的信号输出。通道 0/1/2 的急停事件有两个 FAIL0 和 FAIL1，通道 3/4/5 的急停事件有两个 FAIL2 和 FAIL3。 每个 FAIL 信号共有 3 个信号来源可选，比较器输出，MCPWM0_BKIN 和 CLUOUT。

FAIL 的输入信号可以使用数字滤波，滤波时钟的第一级分频由 MCPWM0_TCLK.CLK_DIV 寄存器设置，第二级分频由 MCPWM0_FLT.FLT_CLKDIV[7:0]寄存器设置。

最后滤波电路会对 FAIL 信号进行 16 个滤波时钟的滤波，即只有信号稳定时间超过 16 个滤波周期才能通过滤波器。即滤波宽度=滤波时钟周期*16。

更多信息可以参考 FAIL 信号处理。

10.8.47 MCPWM0_CH_DEF MCPWM 短路保护通道配置寄存器

地址:0x4001_0CB8

写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11]	R/W	0	CH5P_DEFAULT	CH5_P 通道默认值
[10]	R/W	0	CH5N_DEFAULT	CH5_N 通道默认值
[9]	R/W	0	CH4P_DEFAULT	CH4_P 通道默认值
[8]	R/W	0	CH4N_DEFAULT	CH4_N 通道默认值
[7]	R/W	0	CH3P_DEFAULT	CH3 P 通道默认值

[6]	R/W	0	CH3N_DEFAULT	CH3 N 通道默认值。当发生 FAIL 事件或 MOE 为 0 时，相应通道输出默认电平。默认电平输出不受 MCPWM0_IO23 的 BIT0、BIT1、BIT8、BIT9、BIT6、BIT14 通道交换和极性控制的影响，直接控制通道输出。
[5]	R/W	0	CH2P_DEFAULT	CH2_P 通道默认值
[4]	R/W	0	CH2N_DEFAULT	CH2_N 通道默认值
[3]	R/W	0	CH1P_DEFAULT	CH1_P 通道默认值
[2]	R/W	0	CH1N_DEFAULT	CH1_N 通道默认值
[1]	R/W	0	CH0P_DEFAULT	CH0_P 通道默认值
[0]	R/W	0	CH0N_DEFAULT	CH0_N 通道默认值。当发生 FAIL 事件或 MOE 为 0 时，相应通道输出默认电平。默认电平输出不受 MCPWM0_IO01 和 MCPWM0_IO23 的 BIT0、BIT1、BIT8、BIT9、BIT6、BIT14 通道交换和极性控制的影响，直接控制通道输出。

10.8.48 MCPWM0_CH_MSK　MCPWM 通道屏蔽位寄存器

地址:0x4001_0CBC

写保护的寄存器

复位值:0xFFF

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11]	R/W	0	CH5P_FAIL_EN	CH5_P 通道 FAIL 事件使能，高有效，默认开启
[10]	R/W	0	CH5N_FAIL_EN	CH5_N 通道 FAIL 事件使能，高有效，默认开启
[9]	R/W	0	CH4P_FAIL_EN	CH4_P 通道 FAIL 事件使能，高有效，默认开启
[8]	R/W	0	CH4P_FAIL_EN	CH4_N 通道 FAIL 事件使能，高有效，默认开启
[7]	R/W	0	CH3P_FAIL_EN	CH3_P 通道 FAIL 事件使能，高有效，默认开启
[6]	R/W	0	CH3N_FAIL_EN	CH3_N 通道 FAIL 事件使能，高有效，默认开启

[5]	R/W	0	CH2P_FAIL_EN	CH3_N 通道 FAIL 事件使能，高有效，默认开启
[4]	R/W	0	CH2N_FAIL_EN	CH2_N 通道 FAIL 事件使能，高有效，默认开启
[3]	R/W	0	CH1P_FAIL_EN	CH1_P 通道 FAIL 事件使能，高有效，默认开启
[2]	R/W	0	CH1N_FAIL_EN	CH1_N 通道 FAIL 事件使能，高有效，默认开启
[1]	R/W	0	CH0P_FAIL_EN	CH0_P 通道 FAIL 事件使能，高有效，默认开启
[0]	R/W	0	CH0N_FAIL_EN	CH0_N 通道 FAIL 事件使能， 1：发生 FAIL 事件或 MCPWM0_FAIL012.MOE=0 时，CH0_N 通道电平输出为默认值， 0：发生 FAIL 事件或 MCPWM0_FAIL012.MOE=0 时，CH0_N 通道电平不受影响，仍由 MCPWM 内部硬件控制。默认开启 FAIL 使能

10.8.49 MCPWM0_PRT MCPWM 解保护寄存器

地址:0x4001_0CC0

无写保护的寄存器

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	PRT	写入 0xDEAD，解除 MCPWM 寄存器写保护；写入其它值，MCPWM 寄存器进入写保护。此寄存器读出恒为 0。

11 TIMER

11.1 概述

定时器模块有以下特点：

- 独立工作，可工作在不同频率下
- Timer 为 32bit 通用定时器
- 每个通用定时器处理 2 个外部输入信号（捕获模式）
- 每个通用定时器可产生 2 个输出信号（比较模式）
- 输入信号可以进行最大 2040 个时钟的滤波

11.2 功能框图

如图 11-2 所示，通用定时器 TIMER 主要包括 4 个独立的 Timer。分别可以独立配置运行计数时钟和滤波常数。每个 Timer 可以用于输出特定周期占空比的波形，也可以捕获外部波形进行周期占空比的检测。

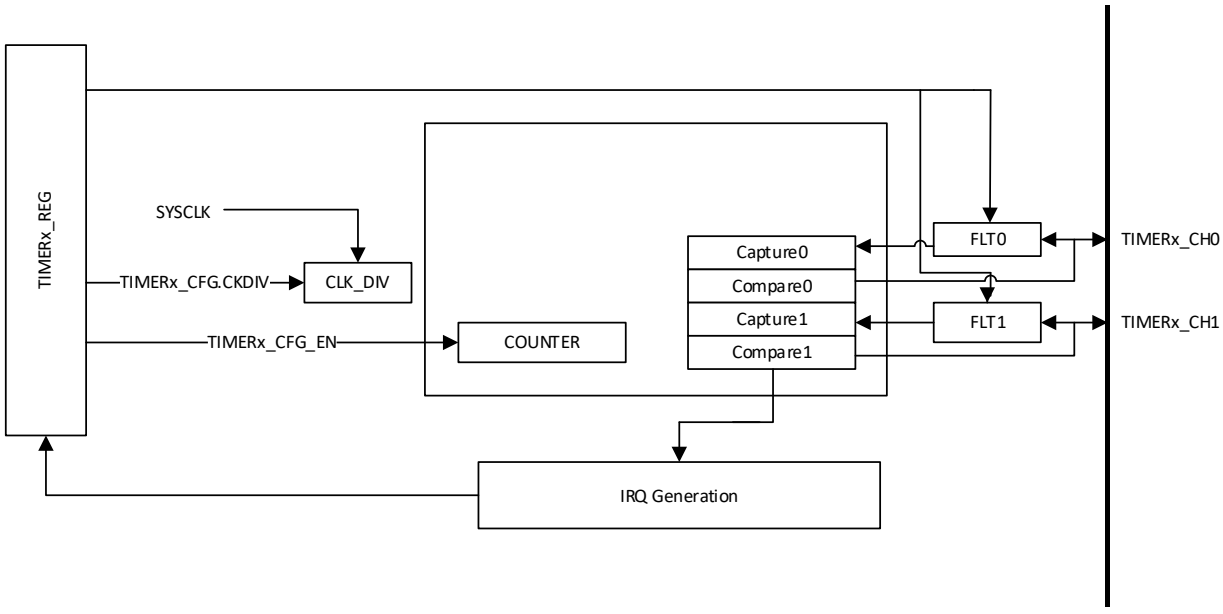


图 11-1 TIMER 模块功能框图

11.2.1 寄存器模块

对各个子模块控制寄存器的读写，各个子模块状态、结果寄存器的访问。以及各个子模块中断信号的处理和中断产生。

11.2.2 时钟分频

Timer 使用 TIMERx_CFG.CLK_DIV 进行分频，可以降低计数器的计数频率，分频系数可以设置为 1/2/4/8/16/32/64/128.

11.2.3 滤波模块

滤波模块对来自芯片外部的输入信号进行滤波，降低毛刺对定时器功能的影响。

每个定时器模块有两个通道，在工作于输入捕获模式时，两个通道共享一个滤波系数。 通过配置滤波寄存器 TIMERx_FLT，可以调整滤波宽度为 0~2040 个 Timer 计数时钟宽度。 输入信号滤波使用 Timer 计数的分频时钟，TIMERx_CFG.CLK_DIV 对 Timer 计数时钟的分频对滤波时钟有影响，即 Timer 分频后滤波宽度随之变大。 如图 11-2-3 所示，原始输入信号在 t1~t6 几个时刻发生了翻转，滤波器宽度配置成 T。可以看到只有 t3 和 t6 时刻发生的翻转维持了大于 T 的时间，因此从滤波器的输出看，信号仅发生了两次翻转。

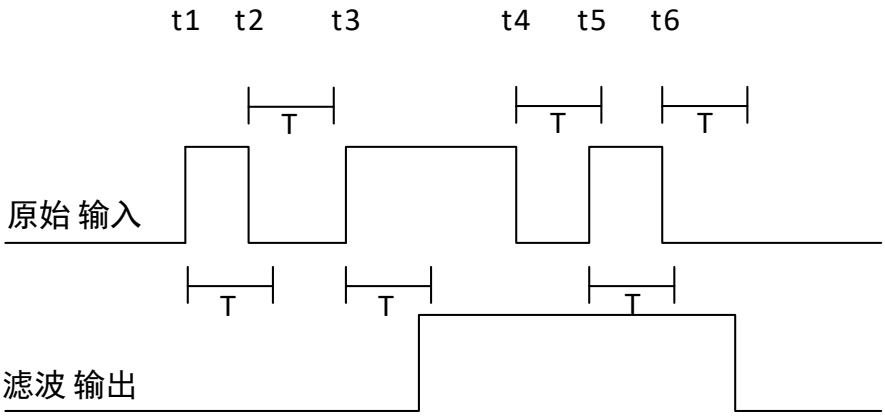


图 11-2 TIMER 滤波示意图

11.2.4 通用定时器模块

通用定时器模块实现了通用的定时器功能，包括比较和捕获工作模式。每个定时器包含两个通道，可以处理两个外部输入信号或者产生两个脉冲信号送到芯片外部。 通用定时器模块，支持外部事件触发开始计数。外部事件源头可配。当外部事件触发后，定时器开始自增。支持使用外部信号作为 timer 时钟进行计数。

11.3 工作模式

11.3.1 基本模式

Timer 中的计数器可以选择递增或递减方向计数，默认为递增计数。

若为递增计数，则计数器从 0 计数到 TH 值，再回到 0 重新开始计数，计数器回到 0 时，产生回零中断。若为递减计数，则计数器从 TH 计数到 0 值，再回到 TH 重新开始计数，计数器回到 0 时，产生回零中断。实际计数周期为 clk_freq*（TH+1）。

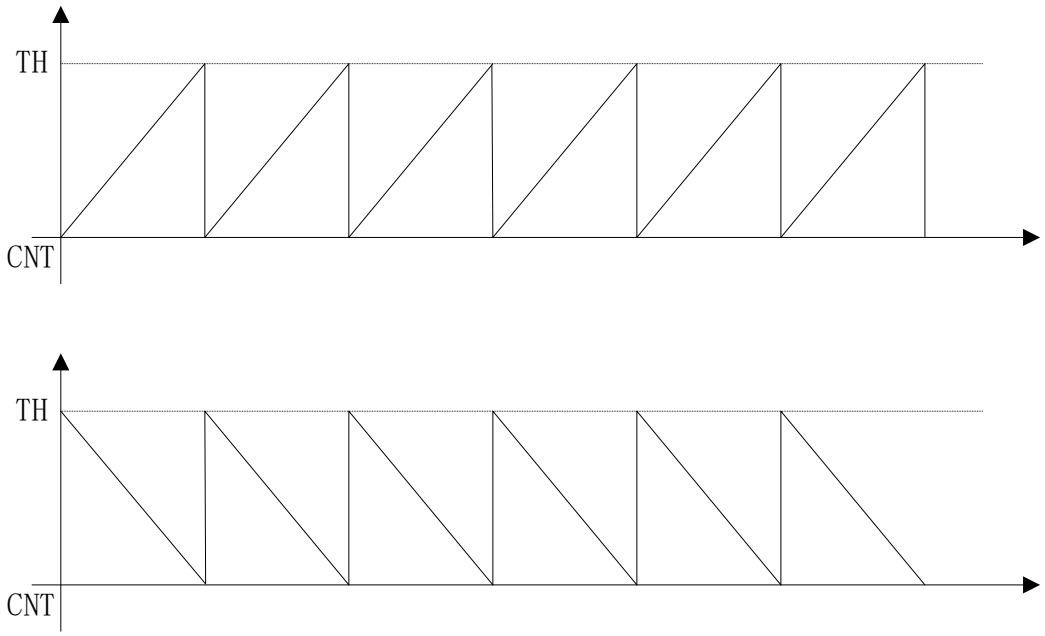
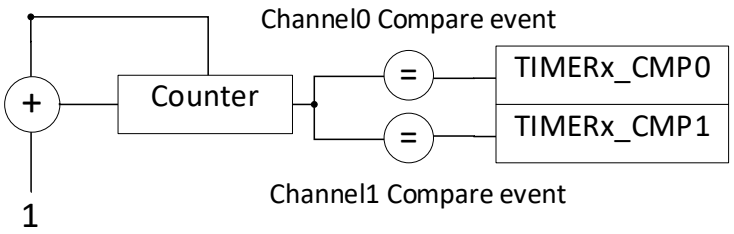


图 11-3 TIMER 计数示意图

11.3.2 捕获模式

捕获模式下，可以使用 Timer 来检测输入信号的上升/下降或者双沿，发生捕获事件（即输入信号电平变化）时，定时器计数值存入 TIMERx_CMP 寄存器，并产生捕获中断。计数器回零时，仍然会产生回零中断。



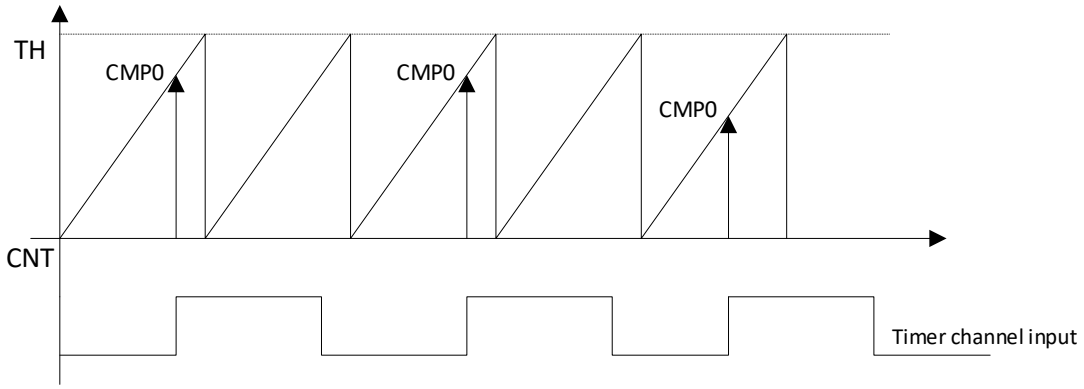


图 11-4 捕获模式

如图 11-3-2 所示，定时器设置为上升沿捕获。在 CAP0/CAP1/CAP2 三个时刻点，捕获到输入信号发生上升沿变化，对应时刻点的定时器计数值将存入 TIMERx_CMP 寄存器中。 捕获模式下，通道 0/1 捕获的信号来源可以通过 TIMERx_CFG.SRC0 和 TIMERx_SRC1 进行设置，信号源可以设置为来自 IO 的通道信号，以及来自 IO 的两个通道信号的异或。 可以通过设置 TIMERx_CFG.CAP0_CLR_EN 或 TIMERx_CFG.CAP1_CLR_EN 使能 Timer 自动清零，即当通道 0/1 发生捕获事件时，TIMER_CNT 自动回零重新开始计数。这一功能便于 Timer 计算捕获信号的周期和占空比。 例如，如果设置了 Timer0 的通道 0/1 同时捕获同一个信号，通道 0 捕获上升沿，通道 1 捕获下降沿。且使能 CAP0_CLR_EN，即发生通道 0 捕获事件时 TIMER_CNT 自动回零。则当发生通道 0 捕获事件（CAP0）时，TIMERx_CMP0 记录的值为信号上一个周波的周期，TIMERx_CMP1 记录的值为信号上一个周波的高电平占空比。 同理，如果设置了 Timer0 的通道 0/1 同时捕获同一个信号，通道 0 捕获上升沿，通道 1 捕获下降沿。且使能 CAP1_CLR_EN，即发生通道 1 捕获事件时 TIMER_CNT

自动回零。则当发生通道 1 捕获事件（CAP1）时，TIMERx_CMP1 记录的值为信号上一个周波的周期，TIMERx_CMP0 记录的值为信号上一个周波的低电平占空比。

11.3.3 比较模式

比较模式下，计数器计数到 TIMERx_CMP 值时，产生比较中断。比较模式可以驱动一个比较脉冲发生，在回零时，向 IO 口输出一个电平（极性可配置），在比较事件发生时，电平翻转，向 IO 口输出另一个电平。计数器回零时，仍然会产生回零中断。设置 TIMERx_CMP0=0，可使得 Timer X 通道 0 为恒 1，设置 TIMERx_CMP0=TIMERx_TH+1，可使得 Timer 通道 0 为恒 0。

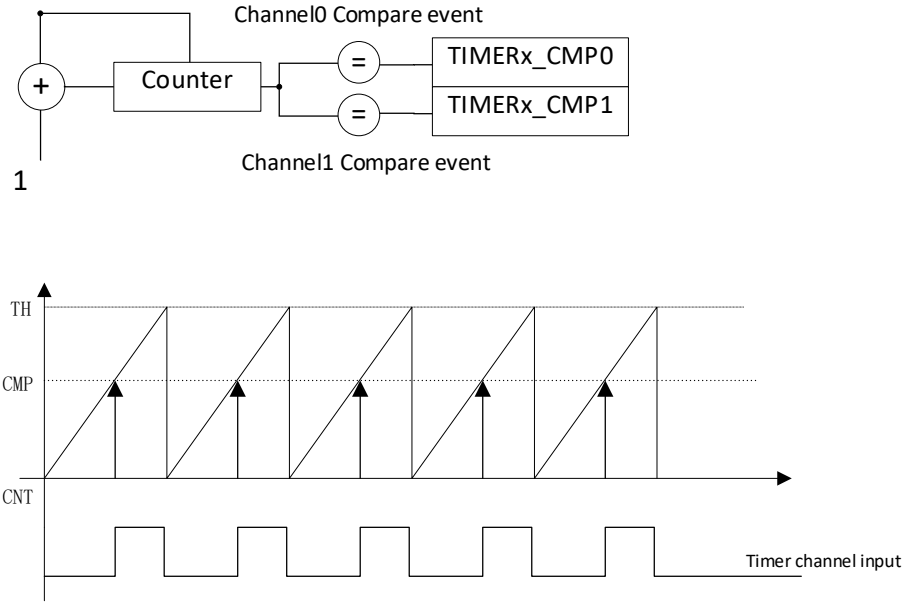


图 11-5 TIMER 比较模式

11.3.4 单次触发

在比较模式下，当 TIMERx_CFG.EN=0，即 Timer 没有开始计数时，通过向 TIMERx_CFG.ONE_TRIG 写入 1，可以触发 Timer 发出一个周期特定占空比的波形。结束后 Timer 回到空闲状态，通道不再动作。如果 TIMER 单次触发仍在计数周期内再次发生单次触发，TIMER 不会响应，仍继续完成本次周期计数。即在单次触发计数过程中发生的过于频繁的软件单次触发不被 TIMER 接受。如图 11-3-4，第 3 次向 ONE_TRIG 写 1 无效，不会触发 TIMER 重新开始计数。

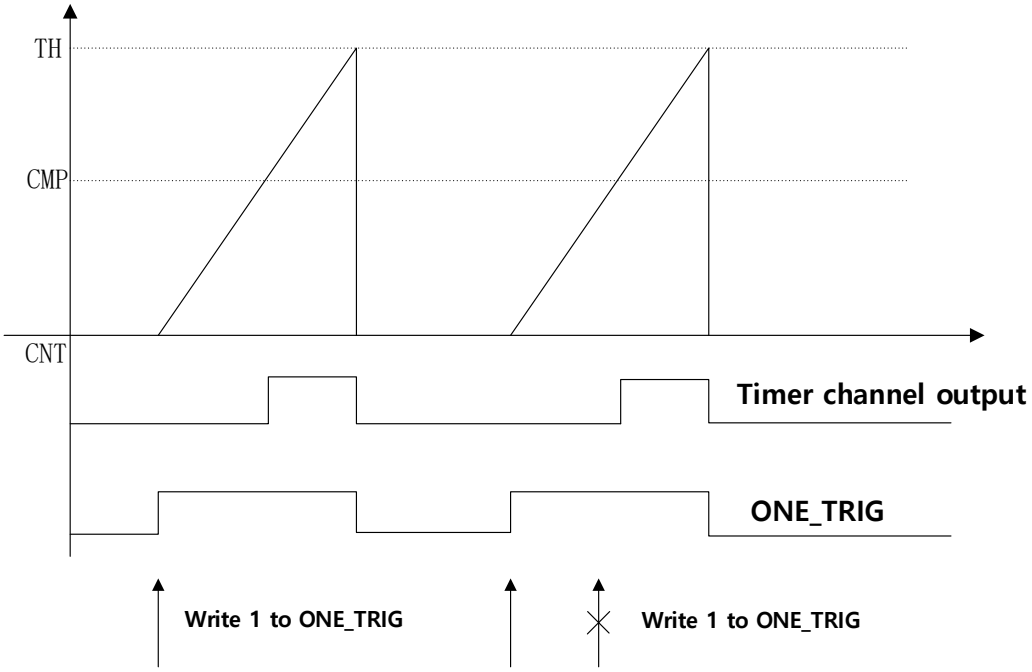


图 11-6 TIMER 单次触发

11.3.5 中心模式（互补 PWM）

除递增计数和递减外，计数器还可以设置为中心计数模式，即 TIMERx_CFG.CENTER=1。此时，计数器从 0 递增计数至 TH 后再递减计数回 0，开始下一个周期的中心计数。实际计数周期为 clk_freq*（2*TH+1）。中心计数模式可以用于输出互补 PWM 波形，通过合理设置 TIMERx_CMP0/1 可以产生上下管导通死区，死区时间=TIMERx_CMP1-TIMERx_CMP0。此外，需要合理设置通道的初始值以产生互补波形。Timer 在递增计数和递减计数期间都会有 TIMERx_CNT=TIMER_CMP0/1 的时刻，但只有递增计数阶段会产生中断事件。

此外，Timer0 支持 FAIL 事件及时关断 PWM 输出。发生 FAIL 事件时，Timer 通道 0/1 输出值由 TIMER0_IO.CH0_DEFAULT 和 TIMER0_IO.CH1_DEFAULT 指定，同时 TIMER0_IO.MOE 被 FAIL 事件清零，软件将 MOE 置 1 后，Timer 通过恢复输出 PWM 波形。

Timer0 同时配备了影子寄存器 TIMER0_STH/TIMER0_SCMP0/TIMER0_SCMP1。当 TIMER0_CFG.SHADOW=1 时，表示使用影子寄存器作为计数器的动作指示，TIMER0_TH/TIMER0_CMP0/TIMER0_CMP1 仅仅作为预装载寄存器，向向 TIMER0_TH TIMER0_CMP0/TIMER0_CMP1 写入仅仅修改预装载值，不改变影子寄存器值。TIMER0 的影子寄存器可以通过向 TIMER0_CFG.UPDATE 写 1 进行软件更新；或者可以等待 TIMER 过零事件自动更新。

当 TIMER0_CFG.SHADOW=0 时，与普通 Timer 相同，仍使用 TIMER0_TH/TIMER0_CMP0/TIMER0_CMP1 作为计数器的动作指示。

读取 TIMER0_TH/TIMER0_CMP0/TIMER0_CMP1 时，总是返回影子寄存器值 TIMER0_STH/TIMER0_SCMP0/TIMER0_SCMP1。

表 11-1 TIMER0 影子寄存器对应关系

TIMER0_CFG.SHADOW 设定值	0	1
TIMER0_CNT 命中 TH	TIMER0_TH	TIMER0_STH
TIMER0_CNT 命中 CMP0	TIMER0_CMP0	TIMER0_SCMP0
TIMER0_CNT 命中 CMP1	TIMER0_CMP1	TIMER0_SCMP1

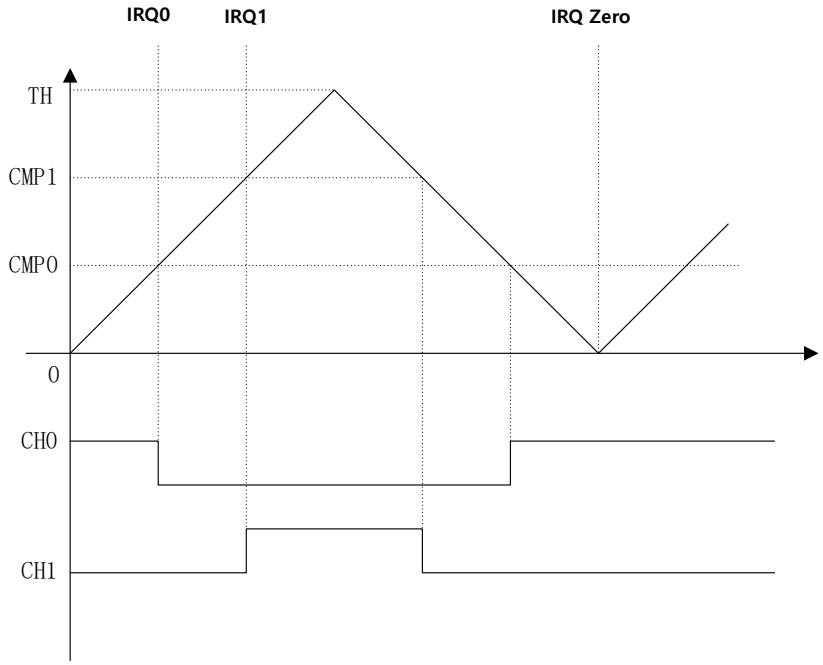


图 11-7 TIMER 中心计数模式

11.3.6 外部事件

Timer 支持与其他 Timer 通道、MCPWM 的 ADC 触发事件以及 CLU 输出进行联动。外部信号可以作为 Timer 的外部启动信号、外部时钟信号、外部重置信号、外部门控信号。

外部事件在 Timer 内部只有一种解释，因此只能使用一种外部事件功能，以上 4 种功能不可以同时使用，亦即 TIMERx_CFG.ETON，TIMERx_CFG.XCLK_EN，TIMERx_CFG.RL_EN， TIMERx_CFG.GATE_EN 4 个设置位只能有一个为 1。

以下说明均以 TIMER 正向递增计数为例，递减计数同理。 外部事件通过 TIMERx_EVT 进行选择。

11.3.7 外部启动信号

当 TIMERx_CFG.EN=0 且 TIMERx_CFG.ETON=1 时，TIMERx 在发生外部事件前 CNT 停留在 0，在生外部事件时启动计数。外部启动信号被处理为一个脉冲信号后进行 TIMER 的启动触发。

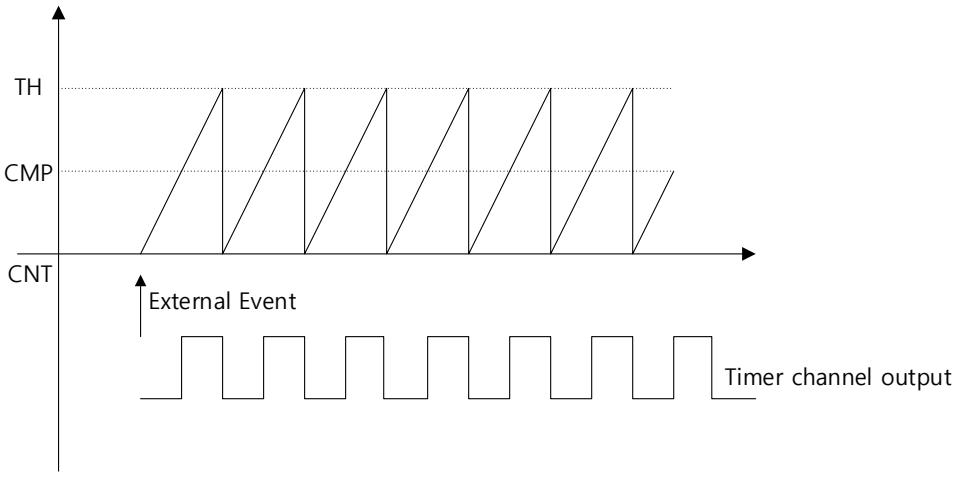


图 11-8 TIMER 外部事件触发启动

外部启动也可以与单次触发进行联合使用，此时软件需要先配置 ETON=1，然后配置 ONE_TRIG=1，等待外部事件触发后，TIMER 进行一个周期的计数，周期结束后 ONE_TRIG 被硬件清零。

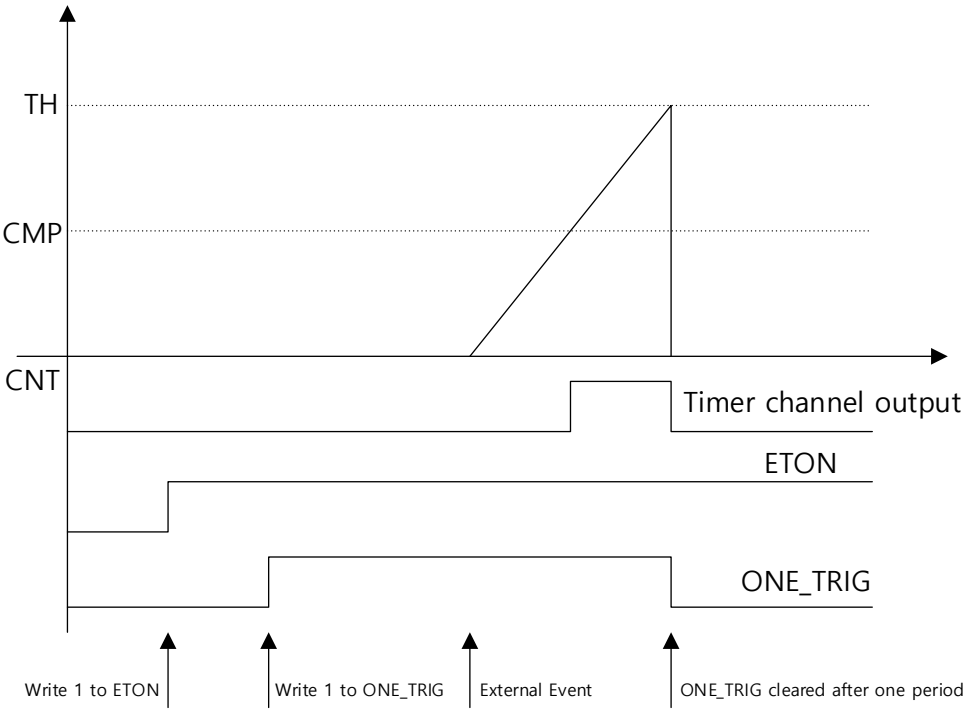


图 11-9 TIMER 外部事件触发单次（单周期）计数

11.3.8 外部时钟信号

通过设置 TIMERx_CFG.XCLK_EN=1，TIMER 可以使用外部信号作为计数时钟。外部时钟的选择通过 TIMERx_EVT 进行设置。

当外部信号出现上升沿时，TIMERx_CNT 递增（或递减）。特别地，如果使用 TIMERm 的通道输入作为 TIMERx 的外部时钟，则 TIMERm 的通道输入为受到 TIMERm 滤波后的信号。

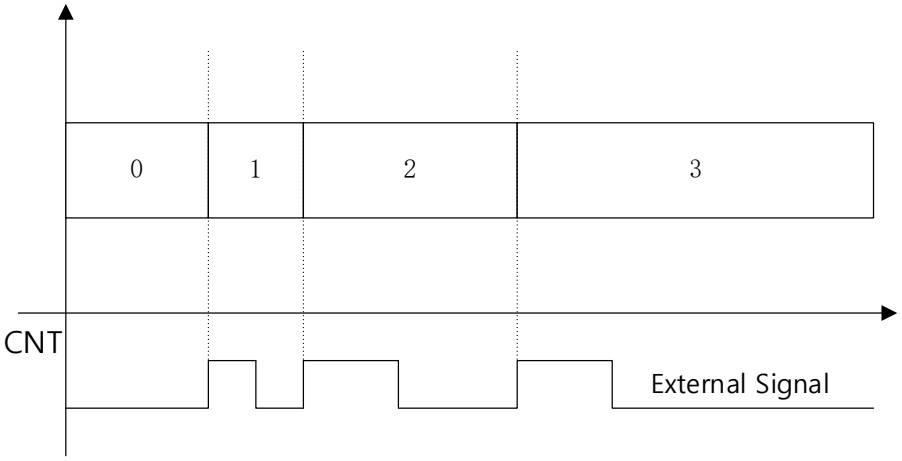


图 11-10 TIMER 使用外部时钟计数

11.3.9 外部重置信号

通过设置 TIMERx_CFG.RL_EN=1，TIMER 可以使用外部信号作为重置信号。外部重置信号的选择通过 TIMERx_EVT 进行设置。如果 TIMER 设置为递增计数或中心计数，发生外部信号重置时，TIMERx_CNT 被重置为 0；如果 TIMER 设置为递减计数，发生外部信号重置时，TIMERx_CNT 被重置为 TIMERX_TH。外部重置信号被处理为一个脉冲信号后对 TIMER 进行重置。

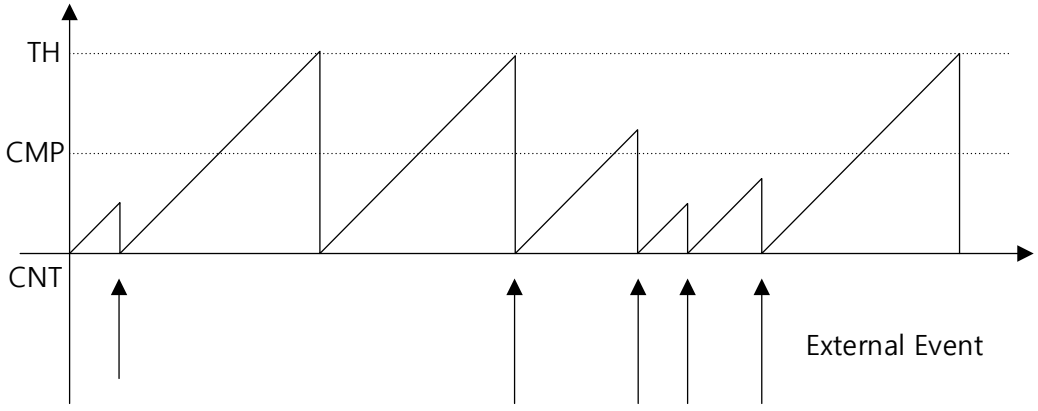


图 11-11 TIMER 外部信号重置功能

11.3.10 外部门控信号

通过设置 TIMERx_CFG.GATE_EN=1，TIMER 可以使用外部信号作为门控信号。外部门控信号的选择通过 TIMERx_EVT 进行设置。门控信号电平为高时，TIMER 进行计数，电平为低时，TIMER 停止计数，TIMERx_CNT 保持不变。由于门控信号需要为电平信号，因此 MCPWM 的 ADC 触发信号无法作为外部门控信号使用。特别地，如果使用 TIMERm 的通道输入作为 TIMERx 计数的门控信号，通道输入信号受到 TIMERm 的滤波影响。

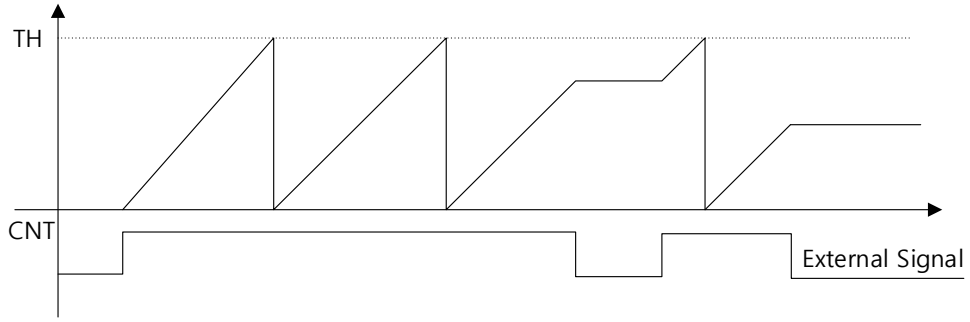


图 11-12 TIMER 外部信号门控功能

以 TIMER2 通道 1 作为 TIMER3 的外部事件信号为例，需要通过通过 TIMER2_CFGRC1 设置 TIMER2 的通道 1 信号来自 IO TIM2_CH1，通过 TIMER2_CFG.CH1_MODE 设置 TIMER2 通道 1 为捕获模式。此外，需要注意 TIMER2_FLT 设置，对 TIMER2 通道 1 输入信号仍有滤波作用。GPIO 需要做相应设置以允许信号通过 GPIO 进入 TIMER2 通道 1。最后通过 TIMER3_EVT 设置 TIMER2 通道 1 作为 TIMER3 外部信号的作用，通常只设置 TIMER3_CFG.ETON，TIMER3_CFG.XCLK_EN，TIMER3_CFG.RL_EN，TIMER3_CFG.GATE_EN 4 个设置位只能有一个为 1。

11.4 ADC 触发

Timer0/1 的比较事件（CMP0/1）可作为 ADC 采样触发事件。

11.5 寄存器列表

Timer0 在芯片中的基地址是 0x4000_1C00

表 11-2 Timer0 寄存器地址分配

名称	偏移地址	描述
TIMER0_CFG	0x00	Timer0 配置寄存器
TIMER0_TH	0x04	Timer0 计数门限寄存器
TIMER0_CNT	0x08	Timer0 计数值寄存器
TIMER0_CMP0	0x0C	Timer0 比较/捕获寄存器 0
TIMER0_CMP1	0x10	Timer0 比较/捕获寄存器 1
TIMER0_EVT	0x14	Timer0 外部事件选择寄存器
TIMER0_FLT	0x18	Timer0 滤波控制寄存器
TIMER0_IE	0x1C	Timer0 中断使能寄存器
TIMER0_IF	0x20	Timer0 中断标志寄存器
TIMER0_IO	0x24	Timer0 IO 控制寄存器

Timer1 在芯片中的基地址是 0x4000_2000

表 11-3 Timer1 寄存器地址分配

名称	偏移地址	描述
TIMER1_CFG	0x00	Timer1 配置寄存器
TIMER1_TH	0x04	Timer1 计数门限寄存器
TIMER1_CNT	0x08	Timer1 计数值寄存器
TIMER1_CMP0	0x0C	Timer1 比较/捕获寄存器 0
TIMER1_CMP1	0x10	Timer1 比较/捕获寄存器 1
TIMER1_EVT	0x14	Timer1 外部事件选择寄存器
TIMER1_FLT	0x18	Timer1 滤波控制寄存器
TIMER1_IE	0x1C	Timer1 中断使能寄存器
TIMER1_IF	0x20	Timer1 中断标志寄存器
TIMER1_IO	0x24	Timer1 IO 控制寄存器

Timer2 在芯片中的基地址是 0x4001_4000

表 11-4 Timer2 寄存器地址分配

名称	偏移地址	描述
TIMER2_CFG	0x00	Timer2 配置寄存器
TIMER2_TH	0x04	Timer2 计数门限寄存器
TIMER2_CNT	0x08	Timer2 计数值寄存器
TIMER2_CMP0	0x0C	Timer2 比较/捕获寄存器 0
TIMER2_CMP1	0x10	Timer2 比较/捕获寄存器 1
TIMER2_EVT	0x14	Timer2 外部事件选择寄存器
TIMER2_FLT	0x18	Timer2 滤波控制寄存器
TIMER2_IE	0x1C	Timer2 中断使能寄存器
TIMER2_IF	0x20	Timer2 中断标志寄存器
TIMER2_IO	0x24	Timer2 IO 控制寄存器

Timer3 在芯片中的基地址是 0x4001_4400

表 11-5 Timer3 寄存器地址分配

名称	偏移地址	描述
TIMER3_CFG	0x00	Timer3 配置寄存器
TIMER3_TH	0x04	Timer3 计数门限寄存器
TIMER3_CNT	0x08	Timer3 计数值寄存器
TIMER3_CMP0	0x0C	Timer3 比较/捕获寄存器 0
TIMER3_CMP1	0x10	Timer3 比较/捕获寄存器 1
TIMER3_EVT	0x14	Timer3 外部事件选择寄存器
TIMER3_FLT	0x18	Timer3 滤波控制寄存器
TIMER3_IE	0x1C	Timer3 中断使能寄存器
TIMER3_IF	0x20	Timer3 中断标志寄存器
TIMER3_IO	0x24	Timer3 IO 控制寄存器

11.6 TIMERx 寄存器通用描述

基地址汇总

外设	基地址
TIMER0	0x4000_1C00
TIMER1	0x4000_2000
TIMER2	0x4001_4000
TIMER3	0x4001_4400

11.6.1 TIMERx_CFG Timer0 配置寄存器

偏移:0x00

复位值:0x1000

位域	类型	复位	名称	描述
[31]	RW	0	EN	Timer 模块整体使能，高有效
[30]	--	--	--	--
[29]	R/W	0	CAP1_CLR_EN	当发生 CAP1 捕获事件时，清零 Timer 计数器，高有效
[28]	R/W	0	CAP0_CLR_EN	当发生 CAP0 捕获事件时，清零 Timer 计数器，高有效
[27]	R/W	0	UPDATE	软件更新启用影子寄存器时，向 UPDATE 写 1 将 TH/CMP0/CMP1 的预装载值加载到影子寄存器 STH/SCMP0/SCMP1 之中。自动清零，读回恒为 0。写 0 无作用。 推荐使用 TIMER0_CFG|=BIT27;的方式进行手动更新影子寄存器的操作，防止手动更新时误将其他配置清零。
[26]	R/W	0	SHADOW	0:不启用影子寄存器，软件写入 TH/CMP0/CMP1 时直接更新对应的影子寄存器 1:启用影子寄存器，软件写入 TH/CMP0/CMP1 时进仅仅更新预装载值，当 Timer 发生过零事件时才将预装载值写入影子寄存器

[25]	R/W	0	ONE_TRIG	单次发送模式，此位需要在 Timer 比较模式下使用，且 EN 需设置为 0 ETON 为 0 时，软件向 ONE_TRIG 写 1 触发 Timer 发送一个周期的特定占空比的脉冲，Timer 计数一个周期后停止。此位在 Timer 计数的当前周期内读为 1，Timer 停止后，自动清零。 ETON 为 1 时，即使软件配置 ONE_TRIG=1，也需要等待外部事件触发，Timer 计数一个周期后停止。ONE_TRIG 位在外部触发 Timer 计数一个周期停止后被硬件清零。
[24]	R/W	0	CENTER	中心计数模式使能 0: Timer 向上从 0 计数至 TH，然后回 0，或 Timer 向下从 TH 计数至 0，然后回到 TH 1: Timer 向上从 0 计数至 TH，然后向下计数至 0
[23]	R/W	0	DIR	0:0->TH 递增计数，1：TH->递减计数 此位在 CENTER=1 时只读，用于指示当前计数方向
[22:20]	R/W	0	CLK_DIV	Timer 计数器频率配置，计数器计数频率是系统主时钟频率的 2CLK_DIV 分频。默认值为 0，不分频。 0:1 分频 1:2 分频 2:4 分频 3:8 分频 4:16 分频 5:32 分频 6:64 分频 7:128 分频

[19]	R/W	0	ETON	Timer 计数器外部启动使能 0:自动运行 1:等待外部事件触发计数，外部触发信号根据 TIMER0_EVT.EVT_SRC 进行选择需要注意的是，使用外部触发也需要设置 TIMERx_CFG.EN=1，除非是外部信号进行的单次触发，即 TIMERx_CFG.ONE_TRIG=1。
[18]	R/W	0	GATE_EN	Timer 暂停使能 0：不暂停 1：当外部信号为低时，Timer 暂停计数，外部信号根据 TIMER0_EVT.EVT_SRC 进行选择
[17]	R/W	0	RL_EN	Timer 重装使能 0：禁用外部事件重装，Timer 向上计数至 TH 回 0，或向下计数值 0 回到 TH 1：使能外部事件重装，重装信号根据 TIMERx_EVT.EVT_SRC 进行选择，当向上计数时，发生外部事件，Timer 重装为 0；当向下计数时，发生外部事件，Timer 重装为 TH。
[16]	R/W	0	XCLK_EN	Timer 时钟源 0:芯片内部时钟 1:外部时钟，时钟源根据 TIMERx_EVT.EVT_SRC 进行选择，当使用外部时钟时，CLK_DIV 不再起作用，即时钟不再进行分频
[15:12]	R/W	1	SRC1	Timer 捕获模式通道 1 信号来源。默认为 3’h1。 0: Timer 通道 0，来自芯片 GPIO（参见 Datasheet 及应用配置） 1: Timer 通道 1，来自芯片 GPIO（参见 Datasheet 及应用配置）

				9: Timer 通道 0 和 1 的异或
[11]	R/W	0	CH1_POL	Timer 通道 1 在比较模式下的输出极性控制，当计数器 CNT
[10]	R/W	0	CH1_MODE	Timer 通道 1 的工作模式选择，默认值为 0。 0:比较模式。输出方波，在 Timer 通道 1 计数器计数值等于 0 或等于 Timer 比较捕获寄存器值时，IO 发生翻转。 1:捕获模式。 当 Timer 通道 1 输入信号发生捕获事件时，将计数器计数值存入 Timer 通道 1 比较捕获寄存器。
[9]	R/W	0	CH1_FE_CAP_EN	Timer 通道 1 下降沿捕获事件使能。 1:使能； 0:关闭。 Timer 通道 1 输入信号发生 1→0 跳变被视为捕获事件。下降沿事件使能可以与上升沿事件使能并存。
[8]	R/W	0	CH1_RE_CAP_EN	Timer 通道 1 上升沿捕获事件使能。 1:使能； 0:关闭。 Timer 通道 1 输入信号发生 0→1 跳变被视为捕获事件。上升沿事件使能可以与下降沿事件使能并存。
[7:4]	R/W	0	SRC0	Timer 捕获模式通道 0 信号来源。默认为 3'h0。 0: Timer 通道 0，来自芯片 GPIO（参见 Datasheet 及应用配置） 1: Timer 通道 1，来自芯片 GPIO（参见 Datasheet 及应用配置） 9: Timer 通道 0 和 1 的异或

[3]	R/W	0	CH0_POL	Timer 通道 0 在比较模式下的输出极性控制：当计数器 CNT
[2]	R/W	0	CH0_MODE	Timer 通道 0 的工作模式选择，默认值为 0。 0:比较模式。输出方波，在 Timer 通道 0 计数器计数值等于 0 或等于 Timer 比较捕获寄存器值时，IO 发生翻转。 1:捕获模式。 当 Timer 通道 0 输入信号发生捕获事件时，将计数器计数值存入 Timer 通道 0 比较捕获寄存器。
[1]	R/W	0	CH0_FE_CAP_EN	Timer 通道 0 下降沿捕获事件使能。 1:使能； 0:关闭。 Timer 通道 0 输入信号发生 1→0 跳变被视为捕获事件。下降沿事件使能可以与上升沿事件使能并存。
[0]	R/W	0	CH0_RE_CAP_EN	Timer 通道 0 上升沿捕获事件使能。 1:使能； 0:关闭。 Timer 通道 0 输入信号发生 0→1 跳变被视为捕获事件。上升沿事件使能可以与下降沿事件使能并存。

当使用外部时钟计数时，也需要设置 TIMERx_CFG.TON=1，且需要开启时钟模块 TIMCKE 对于 Timer 时钟的使能。使用外部时钟计数时，也需要设置 TIMERx_TH。

通常外部事件仅仅作为外部启动、外部门控、外部时钟和外部清零的一个用途使用，即通常 TIMERx_CFG[19:16]中仅设置 1bit 为 1。

如果设置 SRC0 为 4'h9 捕获两通道异或值，（通常为捕获编码器正交的两个通道），需要同时设置 CMP0_CLR_EN=1，CH0_FE_CAP_EN=1，CH0_RE_CAP_EN=1，CH0_MODE=1，即捕获上升沿、下降沿，且每次有信号边沿都清零计数器。CMP0 即为捕获的两次信号边沿之间的计数值。

11.6.2 TIMERx_TH 门限寄存器

偏移:0x04

复位值:0xFFFFFFFF

位域	类型	复位	名称	描述
[31:0]	R/W	0xFFFFFFFF	TH	Timer 计数器计数门限。向上计数从 0 计数到 TH 值后回 0，或向下计数从 TH 计数到 0 后回到 TH

11.6.3 TIMERx_CNT 计数寄存器

偏移:0x08

复位值:0x0

位域	类型	复位	名称	描述
[31:0]	R/W	0	CNT	Timerx 计数器当前计数值。写操作可以写入新的计数值。

11.6.4 TIMERx_CMP0 通道 0 比较捕获寄存器

地址:0x0C

复位值:0x0

位域	类型	复位	名称	描述
[31:0]	R/W	0	CMP0	Timerx 通道 0 工作在比较模式时，当计数器计数值等于 CMP0 时，发生比较事件。 Timer 通道 0 工作在捕获模式时，发生捕获事件时的计数器计数值存入 CMP0 寄存器。

11.6.5 TIMERx_CMP1 通道 1 比较捕获寄存器

偏移:0x10

复位值:0x0

位域	类型	复位	名称	描述
[31:0]	R/W	0	CMP1	Timerx 通道 1 工作在比较模式时，当计数器计数值等于 CMP1 时，发生比较事件。 Timerx 通道 1 工作在捕获模式时，发生捕获事件时的计数器计数值存入 CMP1 寄存器。

11.6.6 TIMER0_EVT 外部事件选择寄存器

偏移 0x14

复位值:0x0

位域	类型	复位	名称	描述
[31:4]	--	--	--	--
[3:0]	R/W	0	EVT_SRC	Timer 外部事件选择寄存器，本寄存器需要配合 TIMERx_CFG.ETON，TIMERx_CFG.GATE_EN，TIMERx_CFG.RL_EN，TIMERx_CFG.XCLK_EN 使用。通常 4 个设置位不同时使用。 当设置 ETON=1 时，根据本寄存器选择触发 Timer 计数的事件。 需要注意的是，Timer 自身的比较事件无法触发 Timer 进行计数。当 ETON=1 时，触发信号的上升沿触发 Timer 开始计数，用作触发源的 Timer 需要工作在比较模式，无须从外部输入信号至 Timer 通道 0:不可用 1:不可用 2:TIMER1 通道 0 比较/捕获事件 3:TIMER1 通道 1 比较/捕获事件

				4:TIMER2 通道 0 比较/捕获事件 5:TIMER2 通道 1 比较/捕获事件 6:TIMER3 通道 0 比较/捕获事件 7:TIMER3 通道 1 比较/捕获事件 12:MCPWM TADC[0]比较事件 13:MCPWM TADC[1]比较事件 14:MCPWM TADC[2]比较事件 15:MCPWM TADC[3]比较事件 当 GATE_EN=1，且外部信号为高时 Timer 计数，外部信号为低时， Timer 暂停，当 GATE_EN=1 时，Timer 使用对应外部信号的电平信号作为计数门控。GATE_EN 仅支持配合 0～11 的外部事件使用，当 EVT_SRC 选择为 0~7 时，外部信号为 Timer 通道输入信号经过对应 Timer 滤波后的信号，作为门控信号的 Timer 通道需要配置为输入使能。 0:TIMER0 通道 0 滤波后输入信号 1:TIMER0 通道 1 滤波后输入信号 2:TIMER1 通道 0 滤波后输入信号 3:TIMER1 通道 1 滤波后输入信号 4:TIMER2 通道 0 滤波后输入信号 5:TIMER2 通道 1 滤波后输入信号 6:TIMER3 通道 0 滤波后输入信号 7:TIMER3 通道 1 滤波后输入信号 当 RL_EN 为高时，Timer 使用外部信号作为重装载信号作为重装载信号的 Timer 通道需要配置为输入使能 0:TIMER0 通道 0 比较/捕获事件

				1:TIMER0 通道 1 比较/捕获事件 2:TIMER1 通道 0 比较/捕获事件 3:TIMER1 通道 1 比较/捕获事件 4:TIMER2 通道 0 比较/捕获事件 5:TIMER2 通道 1 比较/捕获事件 6:TIMER3 通道 0 比较/捕获事件 7:TIMER3 通道 1 比较/捕获事件 12:MCPWM TADC[0]比较事件 13:MCPWM TADC[1]比较事件 14:MCPWM TADC[2]比较事件 15:MCPWM TADC[3]比较事件 当 XCLK_EN 为高时，Timer 使用外部信号作为 Timer 外部时钟， Timer 在时钟上升沿进行计数动作。如果使用 Timer 通道作为外部时钟，通道信号受到对应 Timer 滤波设置影响。 0:TIMER0 通道 0 输入信号 1:TIMER0 通道 1 输入信号 2:TIMER1 通道 0 输入信号 3:TIMER1 通道 1 输入信号 4:TIMER2 通道 0 输入信号 5:TIMER2 通道 1 输入信号 6:TIMER3 通道 0 输入信号 7:TIMER3 通道 1 输入信号 12:MCPWM TADC[0]比较事件 13:MCPWM TADC[1]比较事件 14:MCPWM TADC[2]比较事件 15:MCPWM TADC[3]比较事件

11.6.7 TIMERx_FLT 滤波控制寄存器

偏移:0x18

复位值:0x0

位域	类型	复位	名称	描述
[31:8]	--	--	--	--
[7:0]	R/W	0	FLT	通道 0/1 信号滤波宽度选择。取值范围 0～255。 FLT 为 0 时，不对通道进行滤波。 FLT 不为 0 时，对通道信号进行滤波：滤波宽度为 FLT×8。当通道信号电平稳定超过 FLT×8 个系统时钟周期宽度时，滤波器输出更新；否则，滤波器保持当前的输出不变。

注意，以上滤波器的工作时钟与对应的 Timer 的分频后的工作时钟为相同时钟，受 TIMERx_CFG.CLK_DIV 的分频系数控制。

假设 TIMERx_FLT = 0x6；TIMERx_CFG.CLK_DIV = 0x2;则 Timer 的运行时钟相对系统时钟进行了 4 分频。通道输入信号需要经过 8×6 倍 Timer 的运行时钟的滤波，亦即 8×6×4 倍系统时钟的滤波。

11.6.8 TIMERx_IE 中断使能寄存器

偏移:0x1C

复位值:0x0

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11]	R/W	0	FAIL_RE	Timer FAIL 事件 DMA 请求使能，高电平有效。
[10]	R/W	0	ZC_RE	Timer 计数器过 0 DMA 请求使能，高电平有效。
[9]	R/W	0	CH1_RE	Timer 通道 1 比较/捕获 DMA 请求使能，高电平有效。
[8]	R/W	0	CH0_RE	Timer 通道 0 比较/捕获 DMA 请求使能，高电平有效。
[7:4]	--	--	--	--
[3]	R/W	0	FAIL_IE	Timer FAIL 事件中断使能，高电平有效。
[2]	R/W	0	ZC_IE	Timer 计数器过 0 中断使能，高电平有效。

[1]	R/W	0	CH1_IE	Timer 通道 1 比较/捕获中断使能，高电平有效。
[0]	R/W	0	CH0_IE	Timer 通道 0 比较/捕获中断使能，高电平有效。

DMA 请求事件，与中断为相同事件标志，DMA 请求被 DMA 受理后，中断标志会被 DMA 硬件自动清除，无需 CPU 软件干预。需要注意的是，通常开启某个事件的 DMA 请求则关闭对应的中断使能。

11.6.9 TIMERx_IF 中断标志寄存器

偏移:0x20

复位值:0x0

位域	类型	复位	名称	描述
[31:4]	--	--	--	--
[3]	RW1C	0	FAIL_IF	Timer Fail 事件中断标志。高电平有效，写 1 清 0
[2]	RW1C	0	ZC_IF	Timer 计数器过 0 中断标志。高电平有效，写 1 清 0
[1]	RW1C	0	CH1_IF	Timer 通道 1 比较/捕获中断标志。高电平有效，写 1 清 0
[0]	RW1C	0	CH0_IF	Timer 通道 0 比较/捕获中断标志。高电平有效，写 1 清 0

在比较模式下，当 Timer 设置为中心计数（0->TH->0），即 TIMERx_CFG.CENTER=1 时。 CH0_IF 和 CH1_IF 只有在 Timer 从 0 递增计数至 TH 的过程中，当 TIMERx_CNT=TIMERx_CMP0/1 时位；在 Timer 从 TH 递减计数到 0 的过程中不会置位。

当 TIMERx_CFG.DIR=0

中断标志写 1 清零，一般不建议用如下|=方式清零，因为|=是先读取中断标志，将对应位改为 1 再写入清零，如果同时有其他中断标志置位，会被一起清零，而这通常不是软件所期望的。例如，如下写法本意是清零 ZC_IF，但如果同时 CH0_IF 在写入执行前置 1 了，则软件先读取回 TIMERx_IF 值为 0x24，然后执行或操作 0x4|0x1=0x5，然后写入，同时对 CH0_IF 和 ZC_IF 进行了清零，可能导致 Timer 少进入一次因捕获而产生的中断。 TIMERx_IF|=0x4;

如果希望清零 ZC_IF 标志位，应以如下方式，直接对 BIT2 写 1. TIMERx_IF=0x4;

11.6.10 TIMERx_IO IO 控制寄存器

偏移:0x24

复位值:0x0

位域	类型	复位	名称	描述
[31:10]	--	--	--	--
[9]	R/W	0	CH1_DEFAULT	发生 Fail 事件 MOE 清零后，CH1 通道输出值
[8]	R/W	0	CH0_DEFAULT	发生 Fail 事件 MOE 清零后，CH0 通道输出值
[7]	R/W	0	HALT_PRT	MCU 进入 HALT 状态，Timer 输出值选择。 1:输出 CH1_DEFAULT 和和 CH0_DEFAULT 0:正常输出
[6]	R/W	0	MOE	当 TIMER0_CFG.CENTER=1 时，Timer 通道 0/1 输出能 1:输出正常信号 0:输出 CH0_DEFAULT 和 CH1_DEFAULT 默认值 TIMER0_IF.FAIL_IF 变 1 将触发 MOE 变成 0，通道输出默认值。 若要恢复输出正常信号，需要先将 FAIL_IF 清零，然后软件设置 MOE=1
[5:3]	--	--	--	--
[2]	R/W	0	FAIL_SEL	FAIL 信号选择 0：TIMER0_FAIL，来自 GPIO 1：保留位
[1]	R/W	0	FAIL_POL	FAIL 信号极性 0：高电平 FAIL 1：低电平 FAIL
[0]	R/W	0	FAIL_EN	FAIL 信号使能 0：禁用 FAIL 1：使能 FAIL

11.7 应用指南

11.7.1 功能说明

时钟与分频

SDK 提供 TIM13~TIM16 共 4 个通用 32bit 定时器（TIMER13~TIMER16，寄存器模型同上文 TIMERx），使用前必须先开启对应的定时器时钟（TIMCKE），再由 TIMERx_CFG.CLK_DIV 对计数时钟进行 1/2/4/8/16/32/64/128 分频。

SDK 接口：

void TIM13_Clk_Enable(void); /*TIM14/15/16 同理：TIM14_Clk_Enable()~TIM16_Clk_Enable()，关闭接口为 TIMx_Clk_Disable()*/

void TIM_hal_Counter_set_div(TIMx_TypeDef *timex, uint8_t div); /*div：TIMER_DIV_1~TIMER_DIV_128*/

注意：计数时钟 = 系统时钟 / 分频系数；PWM 周期 = (TH+1) / 计数时钟；输入滤波时钟与计数时钟相同，同样受 CLK_DIV 影响。

统一初始化

SDK 将计数、输入捕获、输出比较等功能的配置统一收敛到配置结构体 TIMx_config_t 中，用户只需填写结构体，再调用 TIM_hal_init()一次完成全部寄存器配置（时钟源/分频/计数方向/计数模式/TH/CMP/通道模式与极性/捕获边沿/滤波/事件源/中断/NVIC），函数末尾自动使能定时器。

SDK 接口：

void TIM_hal_init(TIMx_TypeDef *timex, TIMx_config_t *config); /*timex：TIMER13/TIMER14/TIMER15/TIMER16，其他取值直接返回*/

TIMx_config_t 主要成员如下，取值宏定义见 TIM_Type.h：

tim_div /*时钟分频：TIMER_DIV_1~TIMER_DIV_128*/

tim_clk_src /*时钟源：TIMERx_INNERCLK（内部）/ TIMERx_OUTSIDECLK（外部信号）*/

tim_cnt_dir /*计数方向：TIMERx_CNTR_UP / TIMERx_CNTR_DOWN*/

tim_cnt_mode /*计数模式：TIMERx_EDGE_MODE（边沿）/ TIMERx_SYMMETRIC_MODE（中心对称）*/

tim_reload_mode /*重载模式：TIMERx_AUTO_RELOAD / TIMERx_EXTTRIG_RELOAD*/

tim_shadow /*影子寄存器：TIMERx_SHADOW_REG_DIS / TIMERx_SHADOW_REG_EN*/

tim_gate_en /*外部门控：TIMERx_STOP_DISABLE / TIMERx_STOP_ENABLE*/

tim_ext_start_en /*外部事件启动：TIMERx_EXT_START_DIS / TIMERx_EXT_START_EN*/

tim_chN_mode /*通道 N 模式：TIMERx_MODE_PWM（比较）/ TIMERx_MODE_CAP（捕获），N=0/1*/

tim_chN_pol /*通道 N 输出极性：TIMERx_IOPOL_POSITIVE / TIMERx_IOPOL_NEGAGTIVE*/

tim_chN_cap_src /*通道 N 捕获源：TIMERx_SRC_IO0/IO1/CLU0~CLU4/IO0_XOR_IO1*/

tim_chN_rise_cap / tim_chN_fall_cap /*通道 N 上升沿/下降沿捕获使能*/

tim_chN_clr_en /*通道 N 捕获后计数器清零使能*/

tim_th / tim_cmp0 / tim_cmp1 /*门限值（周期）、通道 0/1 比较值（捕获值）*/

tim_evt_src /*外部事件源（TIMERx_EVT），0 表示不配置*/

tim_filt /*输入滤波系数，0 表示不配置*/

tim_moe /*主输出使能（IO.MOE），PWM 输出时必须置 1*/

tim_irq_en /*中断使能位掩码：TIMERx_IRQ_CH0 | TIMERx_IRQ_CH1 | TIMERx_IRQ_ZERO | TIMERx_IRQ_FAIL，0 表示不使能中断*/

priority /*NVIC 中断优先级，0 最高，仅 tim_irq_en 非 0 时有效*/

注意：①结构体中未赋值的成员（如 tim_reload_mode、tim_shadow）按 0 处理，建议使用指定初始化器；②TIM 引脚的 GPIO 复用需在调用 TIM_hal_init()前自行初始化（demo 中 PB11/PB12 复用功能 10 为 TIM13 的 CH0/CH1），HAL 不负责 GPIO；③TIM_hal_init()不会先复位定时器，重新初始化前请先调用 tim_hal_reset()。

计数模式（定时中断）

通道不使用时，定时器作为普通定时器：计数器从 0（递减时从 TH）开始计数，到达门限值 TH 后回到起点并产生过 0 中断，定时周期 = (TH+1) × 分频后时钟周期。

SDK 接口：

void tim_hal_th_set(TIMx_TypeDef *timex, uint32_t th); /*修改门限值（周期）*/

void tim_hal_cntdir_set(TIMx_TypeDef *timex, uint8_t dir); /*0：递增 1：递减*/

uint32_t TIM_hal_Counter_num(TIMx_TypeDef *timex); /*读取当前计数值 CNT*/

void tim_hal_cnt_set(TIMx_TypeDef *timex, uint32_t cnt_num); /*写计数值 CNT*/

输出比较模式（PWM 输出）

通道模式设为 TIMERx_MODE_PWM、tim_moe=1 时，通道 0/1 可输出 PWM：周期由 TH 决定，占空比由 CMP0/CMP1 与 TH 之比决定（边沿模式周期为 TH+1 个计数时钟；中心模式为 2×TH+1 个计数时钟，可输出中心对齐 PWM）。输出极性可选，POSITIVE 时计数值小于 CMP 输出高电平。

SDK 接口：

void tim_hal_compn_set(TIMx_TypeDef *timex, uint8_t chx, uint32_t duty); /*设置通道 chx（0/1）的比较值 CMP*/

void tim_hal_compn_pol_set(TIMx_TypeDef *timex, uint8_t chx, uint8_t pol); /*设置通道输出极性*/

void tim_hal_cnt_mid_set(TIMx_TypeDef *timex, uint8_t mid); /*切换边沿/中心计数模式*/

void tim_hal_onetrig(TIMx_TypeDef *timex); /*单次触发：输出一个指定占空比的脉冲后自动停止*/

float tim_hal_ch0_duty(TIMx_TypeDef *timex); float tim_hal_ch1_duty(TIMx_TypeDef *timex); /*按 CMP/TH 计算当前占空比（0~1）*/

注意：tim_hal_compn_set()的参数是比较值而不是百分比，应保证 CMP 不大于 TH；单次触发需在定时器未使能（EN=0）时进行，即先调用 tim_hal_disable()再调用 tim_hal_onetrig()。

输入捕获模式

通道模式设为 TIMERx_MODE_CAP 后，可选择上升沿、下降沿或双沿作为捕获事件，发生捕获时当前 CNT 值被锁存到 CMP0/CMP1，并可同时清零计数器，由此可测量输入信号的周期和脉宽。捕获源可选 IO0/IO1/CLU 或 IO0 与 IO1 的异或值（用于编码器正交信号）。

SDK 接口：

uint32_t tim_hal_ch0_capnum(TIMx_TypeDef *timex); /*读取通道 0 捕获值 CMP0*/

uint32_t tim_hal_ch1_capnum(TIMx_TypeDef *timex); /*读取通道 1 捕获值 CMP1*/

void tim_hal_flt(TIMx_TypeDef *timex, uint8_t flt); /*设置输入滤波系数，0~255，滤除毛刺*/

注意：捕获模式下对应的 GPIO 需配置为输入；输入信号周期应使捕获值在 32bit 范围内不溢出，必要时增大分频系数。

中断与状态

SDK 支持通道 0、通道 1、过 0、FAIL 共 4 类中断（TIMERx_IRQ_CH0/CH1/ZERO/FAIL），在 tim_irq_en 中配置后 TIM_hal_init()会自动设置 NVIC 优先级并使能中断。中断服务函数名为 TIM13_IRQHandler~TIM16_IRQHandler。

SDK 接口：

uint8_t tim_hal_irq_status(TIMx_TypeDef *timex); /*读取中断标志 bit0~3，读取的同时会清除全部标志*/

uint8_t tim_hal_irq_en_status(TIMx_TypeDef *timex); /*读取已使能的中断位*/

void tim_hal_irq_disable(TIMx_TypeDef *timex); /*关闭全部中断并清除标志*/

注意：tim_hal_irq_status()读取后标志即被清除，中断服务函数中应只调用一次，并与 tim_hal_irq_en_status()按位与后再判断中断类型。

定时器控制

void tim_hal_enable(TIMx_TypeDef *timex); /*使能（启动）定时器*/

void tim_hal_disable(TIMx_TypeDef *timex); /*关闭定时器*/

void tim_hal_reset(TIMx_TypeDef *timex); /*清除定时器全部配置，重新初始化前调用*/

void tim_hal_para_display(TIMx_TypeDef *timex); /*打印分频、TH、CMP0/1 及占空比，调试用*/

11.8 快速开始

下面以 TIM13 为例（对应 demo\timer13_demo，PB11 为通道 0，PB12 为通道 1），三种模式的公共流程为：使能外设时钟 → 初始化系统时钟 → 配置 GPIO 复用 → 填写 TIMx_config_t → 调用 TIM_hal_init()。

公共配置：时钟与配置结构体

flash_clk_enable();

gpio_clk_enable();

```
TIM13_Clk_Enable();     /*使能 TIM13 时钟*/
RCC_Init();             /*系统时钟配置，默认 HIRC  32MHz，可在 sys_config.h 中修改*/
TIMx_config_t tim_cfg = {
    .tim_div        = TIMER_DIV_32,          /*32MHz/32 = 1MHz 计数时钟*/
    .tim_clk_src    = TIMERx_INNERCLK,       /*内部时钟*/
    .tim_cnt_dir    = TIMERx_CNTR_UP,        /*向上计数*/
    .tim_cnt_mode   = TIMERx_EDGE_MODE,      /*边沿计数*/
    .tim_ch0_mode   = TIMERx_MODE_PWM,
    .tim_ch0_pol    = TIMERx_IOPOL_POSITIVE,
    .tim_ch1_mode   = TIMERx_MODE_PWM,
    .tim_ch1_pol    = TIMERx_IOPOL_POSITIVE,
    .tim_ch0_cap_src = TIMERx_SRC_IO0,
    .tim_ch1_cap_src = TIMERx_SRC_IO1,
    .tim_th         = 1000,                  /*周期：1MHz/1000 = 1kHz*/
    .tim_cmp0       = 500,                   /*CH0 占空比 50%*/
    .tim_cmp1       = 700,                   /*CH1 占空比 70%*/
    .tim_moe        = 1,                     /*主输出使能*/
    .tim_irq_en     = 0,                     /*不使能中断*/
};
```

GPIO 复用配置（PB11/PB12）

```
gpio_settings_config_t setting_cfg;
setting_cfg.gpio_port = GPIO_B;
setting_cfg.gpio_pin  = GPIO_PIN_11;          /*PB11：CH0*/
setting_cfg.gpio_alt  = GPIO_ALT_FUNC_10;     /*复用为 TIM13 通道*/
setting_cfg.gpio_dir  = GPIO_DIR_OUTPUT;      /*捕获模式时改为 GPIO_DIR_INPUT*/
setting_cfg.pull_cfg  = GPIO_PULL_DISABLE;
setting_cfg.gpio_strength = GPIO_STRENGTH_MODE_0;
```

```
setting_cfg.gpio_interrupt_cfg_info.int_handle = NULL;
gpio_init(&setting_cfg);
setting_cfg.gpio_pin  = GPIO_PIN_12;                    /*PB12：CH1*/
gpio_init(&setting_cfg);
```

输出比较：双通道 PWM 输出

```
tim_cfg.tim_ch0_mode = TIMERx_MODE_PWM;
tim_cfg.tim_ch1_mode = TIMERx_MODE_PWM;
tim_cfg.tim_irq_en   = 0;                               /*纯 PWM 输出，不使能中断*/
TIM_hal_init(TIMER13, &tim_cfg);                        /*配置完成并自动启动，PB11 输出 1kHz/50%，PB12 输出 1kHz/70%*/
```

运行中修改占空比和周期：

```
tim_hal_compn_set(TIMER13, TIMERx_CH0, 300);    /*CH0 比较值改为 300，占空比 30%*/
tim_hal_th_set(TIMER13, 2000);                  /*周期改为 2000 个计数时钟，即 500Hz*/
```

注意：修改周期后需同步调整 CMP 值才能保持原占空比。

输入捕获：双通道捕获

```
tim_cfg.tim_ch0_mode     = TIMERx_MODE_CAP;
tim_cfg.tim_ch1_mode     = TIMERx_MODE_CAP;
tim_cfg.tim_ch0_rise_cap = 1;                   /*CH0 上升沿捕获*/
tim_cfg.tim_ch1_fall_cap = 1;                   /*CH1 下降沿捕获*/
tim_cfg.tim_ch0_clr_en   = 1;                   /*捕获后计数器清零*/
tim_cfg.tim_ch1_clr_en   = 1;
tim_cfg.tim_irq_en       = TIMERx_IRQ_CH0 | TIMERx_IRQ_CH1;    /*捕获产生中断*/
TIM_hal_init(TIMER13, &tim_cfg);
```

在中断服务函数中读取标志并取得捕获值：

```
void TIM13_IRQHandler(void)
{
    uint8_t status = tim_hal_irq_status(TIMER13) & tim_hal_irq_en_status(TIMER13);
    if (status & TIMERx_IRQ_CH0)
```

```
    {
        uint32_t cap0 = tim_hal_ch0_capnum(TIMER13);    /*CH0 捕获值，反映输入信号周期
*/
    }
    if (status & TIMERx_IRQ_CH1)
    {
        uint32_t cap1 = tim_hal_ch1_capnum(TIMER13);    /*CH1 捕获值*/
    }
}
```

注意：捕获值单位为分频后的计数时钟周期（本例 1μs）；demo 中断服务函数仅翻转 PC6，用示波器观察 PC6 可确认捕获发生。

计数模式：定时中断

```
tim_cfg.tim_irq_en = TIMERx_IRQ_ZERO;            /*过 0 中断*/
TIM_hal_init(TIMER13, &tim_cfg);                 /*每 1ms（TH=1000，1MHz）产生一次过 0
中断*/
void TIM13_IRQHandler(void)
{
    uint8_t status = tim_hal_irq_status(TIMER13) & tim_hal_irq_en_status(TIMER13);
    if (status & TIMERx_IRQ_ZERO)
    {
        /*定时处理，例如翻转 PC6*/
    }
}
```

单次触发

```
tim_hal_disable(TIMER13);    /*单次触发需在定时器未使能时进行*/
tim_hal_onetrig(TIMER13);    /*输出一个指定占空比的脉冲后定时器自动停止*/
```

重新初始化

```
tim_hal_reset(TIMER13);                  /*清除原有配置*/
```

TIM_hal_init(TIMER13, &tim_cfg); /*按新配置重新初始化*/

注意：①demo 通过修改 main.c 顶部的 tim_mode（COMPARE/CAPTURE/COUNTER）切换三种模式；②TIM14~TIM16 只需将 TIMER13、TIM13_Clk_Enable()、TIM13_IRQHandler 替换为对应通道，GPIO 引脚按芯片复用表选择；③使用 tim_irq_en 时需保证对应 TIMx_IRQHandler 已实现，否则会进入默认中断函数。

12 QEP

12.1 概述

编码器接口支持正交编码信号、符号加脉冲信号、CW/CCW 双脉冲信号三种模式。

QEP 的输入信号 T1/T2/Z 分别来自对应的 GPIO 输入，并经过内部的滤波系数控制。

12.2 实现说明

12.2.1 正交编码信号

正交编码信号多用于计数编码器圈数，输入为 T1/T2 两个信号，支持下表中两个模式。

概括来讲，T1/T2 的跳变沿会导致计数器递增或递减。而计数器计数方向（递增或递减）由跳变信号之外的另一个稳态信号的电平高低决定。

如果 T1 发生了上升沿跳变，则看 T2 是高电平还是低电平，如果是高电平则计数器递减，如果是低电平计数器递增，T1 下降沿计数器变化相反。

如果 T2 发生了上升沿跳变，则看 T1 是高电平还是低电平，如果是高电平则计数器递增，如果是低电平计数器递减，T2 下降沿计数器变化相反。

逻辑关系如下公式所示：

Counter Up = (T1 !=T2) @(T1 triggering edges) | (T1 == T2) @ (T2 triggering edges) Counter Down = (T1 == T2) @ (T1 triggering edges) | (T1 != T2) @ (T2 triggering edges)

表 12-1 编码器正交编码工作模式

计数模式	T1/T2 电平状态（稳态信号）	T1 变化边沿状态		T2 变化边沿状态	
		上升沿	下降沿	上升沿	下降沿
仅 T1 计数	T2 高	递减	递增	不计数	不计数
	T2 低	递增	递减	不计数	不计数
T1/T2 都计数	T2 高	递减	递增	不计数	不计数
	T2 低	递增	递减	不计数	不计数
	T1 高	不计数	不计数	递增	递减
	T1 低	不计数	不计数	递减	递增

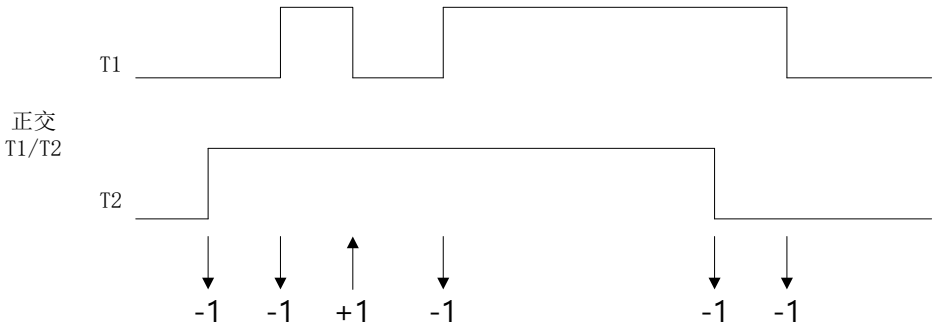


图 12-1 编码器只在 T1 时刻计数的正交编码信号计数情况

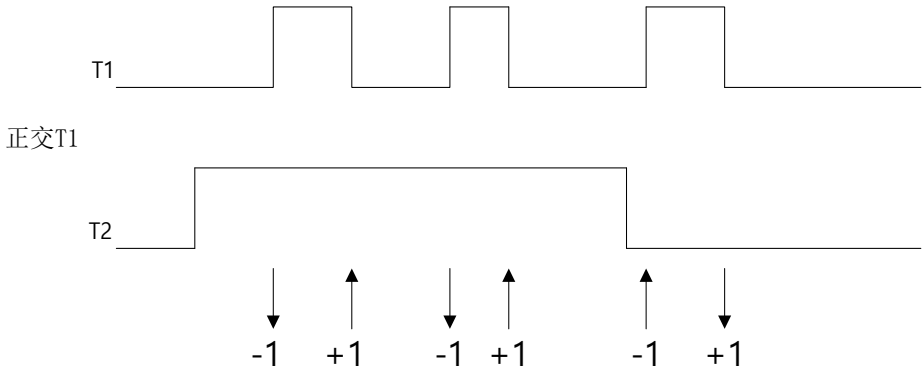


图 12-2 编码器在 T1 或 T2 时刻计数的正交编码信号计数情况

12.2.2 符号加脉冲信号

这种工作模式下，T1 为脉冲信号，T2 为符号信号。T1 的边沿触发计数，T2 电平控制计数方向，高则递增，低则递减。可以配置仅 T1 上升沿计数还是 T1 上升下降沿都计数。

Counter Up = (T2==1) @ (T1 triggering edges)

Counter Down = (T2==0) @ (T1 triggering edges)

表 12-2 编码器符号加脉冲工作模式

计数模式	T2 电平状态（稳态信号）	T1 变化边沿状态	
		上升沿	下降沿
仅 T1 上升沿	高	递增	不计数
	低	递减	不计数
T1 上升下降沿	高	递增	递减
	低	递减	递增

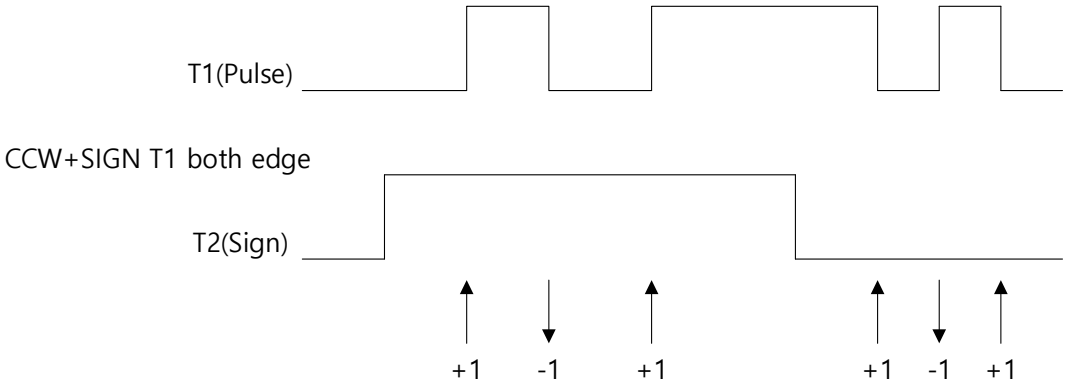


图 12-3 编码器在 T1 上升下降沿都计数的符号加脉冲信号计数情况

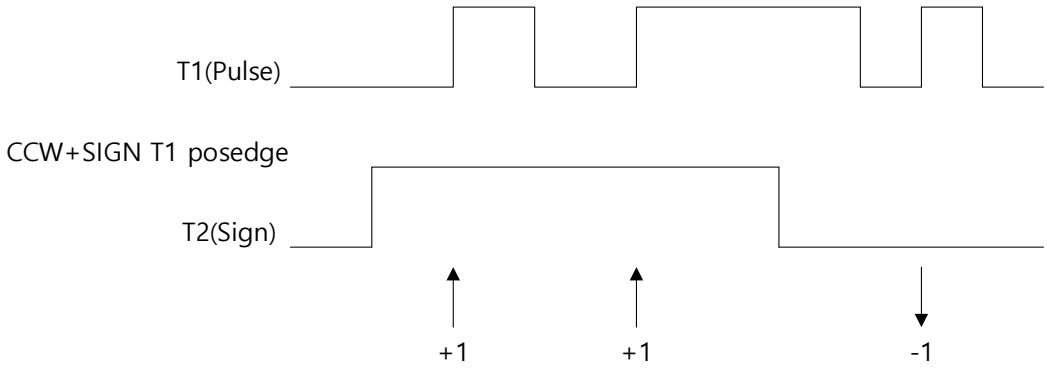


图 12-4 编码器在仅 T1 上升沿计数的符号加脉冲信号计数情况

12.2.3 CCW/CW 双脉冲信号

在 T1 跳变时计数器递增，在 T2 跳变时计数器递减。可以配置计数器仅在上升沿变化或者在上升下降沿都变化。以下式表示

Counter Up = 1 @ (T1 triggering edges)

Counter Down = 1 @ (T2 triggering edges)

表 12-3 编码器 CCW/CW 双脉冲工作模式

计数模式	变化边沿状态			
	T1 上升沿	T1 下降沿	T2 上升沿	T2 下降沿
T1/T2 上升沿	递增	不计数	递减	不计数
T1/T2 上升下降沿	递增	递增	递减	递减

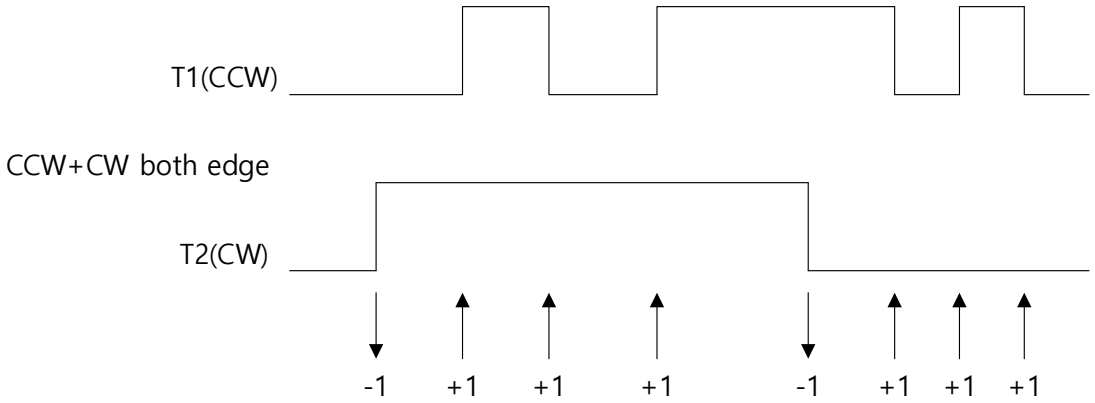


图 12-5 编码器仅在 T1/T2 上升下降沿计数的 CCW/CW 双脉冲信号计数情况

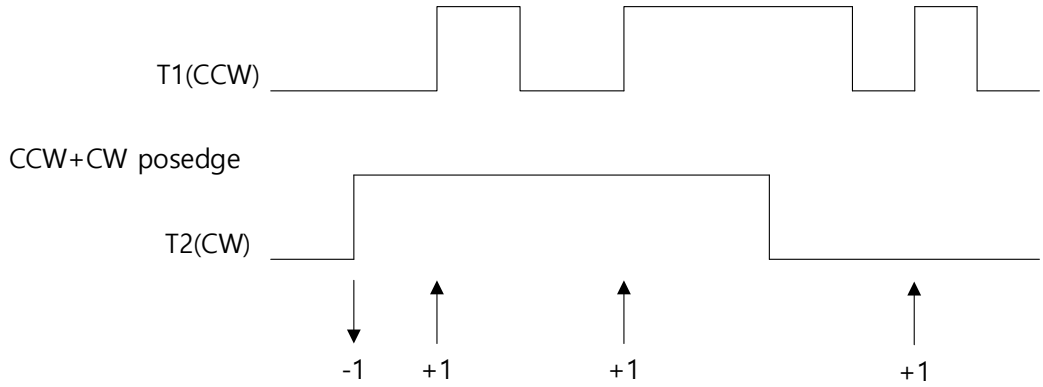


图 12-6 编码器仅在 T1/T2 上升沿计数的 CCW/CW 双脉冲信号计数情况

12.3 寄存器列表

QEP 模块寄存器的基地址是 0x4000_6400，寄存器列表如下:

表 12-4 QEP 模块寄存器地址分配

名称	偏移地址	描述
QEP_CFG	0x30	QEP 模块配置寄存器
QEP_TH	0x34	QEP 门限值寄存器
QEP_CNT	0x38	QEP 计数值寄存器
QEP_IE	0x3C	QEP 中断使能寄存器
QEP_IF	0x40	QEP 中断标志寄存器

12.4 寄存器描述

12.4.1 QEP_CFG　QEP 模块配置寄存器

地址:0x4000_6430

复位值:0x0

位域	类型	复位	名称	描述
[31]	--	--	--	--
[30:20]	R/W	0	QEP_FLT	滤波宽度，低于对应脉冲宽度的信号将被硬件自动过滤掉。滤波宽度的计算公式为[10:0] + 1。
[19]	--	--	--	--
[18:16]	R/W	0	QEP_CLKDIV	QEP 时钟分频系数。默认不分频。 00:不分频 01:2 分频 10:4 分频 11:8 分频
[15]	R/W	0	QEP_EN	QEP 模块整体使能
[14:11]	--	--	--	--
[10]	R/W	0	FE_CNT_EN	CCW+SIGN/CCW+CW 两种模式下，是否在下降沿进行计数（上升沿 总是计数）
[9:8]	R/W	0	MODE	编码器模式选择 00：counting on T1 01：counting on T1 & T2 以上两种模式都为正交编码信号计数模式 10：CCW+SIGN，符号加脉冲信号计数模式 11：CCW+CW，CCW+CW 双脉冲信号计数模式
[7:4]	--	--	--	--
[3]	R/W	0	ZNC	Z 信号清零极性选择：低电平/下降沿清零使能
[2]	R/W	0	ZPC	Z 信号清零极性选择：高电平/上升沿清零使能

[1]	R/W	0	ZLC	Z 信号电平清零 QEP 计数器使能
[0]	R/W	0	ZEC	Z 信号边沿清零 QEP 计数器使能

当 ZEC=1 时，使用 Z 信号的跳变沿进行 QEP 计数的清零，当 ZNC 同时为 1 时，Z 信号的下降沿清 零 QEP 计数器，当 ZPC 同时为 1 时，Z 信号的上升沿清零 QEP 计数器；ZNC 和 ZPC 可以同时为 1，则 Z 信号发生变化则清零 QEP 计数器，清零后立即释放复位，QEP 计数器立即准备好进行后续 计数。 当 ZLC=1 时，使用 Z 信号的有效电平进行 QEP 计数的清零，当 ZNC 同时为 1 时，Z 信号低电平时 清零 QEP 计数器，当 ZPC 同时为 1 时，Z 信号高电平清零 QEP 计数器；通常使用 Z 信号有效电平 清零 QEP 计数器时，ZNC 和 ZPC 不会同时为 1，否则计数器一直处于清零状态。使用有效电平清 零时，只有 Z 信号翻转后，QEP 计数器才会恢复计数。

12.4.2 QEP_TH QEP 门限值寄存器

地址:0x4000_6434

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	TH	计数门限 TH。编码器向上计数（增）到 TH 值后，再次向上计数会 导致计数器回到 0。编码器向下计数（减）到 0 值后，再次向下计数 会导致计数器回到 TH。

12.4.3 QEP_CNT QEP 计数值寄存器

地址:0x4000_6438

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	CNT	QEP 计数值

12.4.4 QEP_IE QEP 中断使能寄存器

地址:0x4000_643C

复位值:0x0

位域	类型	复位	名称	描述
[31:2]	--	--	--	--
[1]	R/W	0	OF_IE	上溢出中断使能，高电平有效。
[0]	R/W	0	UF_IE	下溢出中断使能，高电平有效。

12.4.5 QEP_IF　QEP 中断标志寄存器

地址:0x4000_6440

复位值:0x0

位域	类型	复位	名称	描述
[31:2]	--	--	--	--
[1]	RW1C	0	OF_IF	上溢出中断标志。高电平有效，写 1 清 0。当计数器计数达到计数门 限时，上计数事件触发上溢出中断。
[0]	RW1C	0	UF_IF	下溢出中断标志。高电平有效，写 1 清 0。当计数器计数达到 0 时， 下计数事件触发下溢出中断。

13 HALL

13.1.1 概述

芯片共支持 3 路 HALL 信号输入。

对于输入的 HALL 传感器信号，所进行的处理包括:

- 滤波，消除 HALL 信号毛刺的影响
- 捕获，HALL 输入有变化时，记录当前的定时器值，并输出中断
- 溢出，HALL 信号长时间不发生变化导致计数器溢出，并输出中断

13.2 实现说明

13.2.1 信号来源

HALL 信号来源于 GPIO，对于每一路 HALL 信号，芯片有两个 IO 可以作为该信号的来源。通 过配置 GPIO 寄存器，用户可以选择将其中一个 GPIO 的输入信号做为 HALL 信号使用。

详细管脚位置说明见芯片器件 datasheet。

13.2.2 工作时钟

HALL 模块工作频率可调。通过配置 HALL_CFG.CLK_DIV 寄存器，可以选择系统主时钟的 1/2/4/8 分频作为 HALL 模块工作频率，滤波和计数均采用该频率工作。

13.2.3 信号滤波

滤波模块主要用于去除 HALL 信号上的毛刺。

滤波包括两级滤波器，两级滤波电路可单独开启，也可全部开启:

第一级采用 7 判 5 进行滤波，即连续 7 个采样点中，如果达到或超过 5 个 1 则输出 1，如果 达到或超过 5 个 0 则输出 0，否则输出保持上一次的滤波结果。通过配置 HALL_CFG.FIL_75 可以选择是否使能第一级滤波器。具体如下图所示:

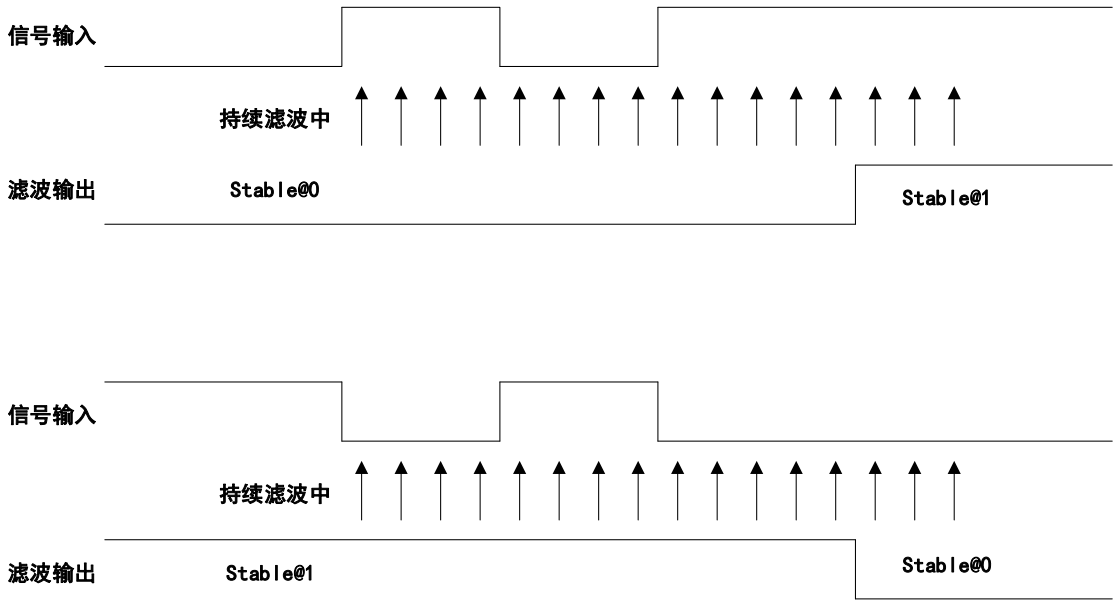


图 13-1 7/5 滤波模块框图

第二级采用连续滤波，在连续 N 个采样点中，如全为 0 则输出 0，如全为 1 则输出 1，否则输出保持上一次的滤波结果。

通过配置 HALL_CFG.FIL_LEN 可以配置第二级滤波器滤波深度，即连续采样个数。连续采样个 数最大为 215，滤波时间常数计算公式如下:

Tfilter = Tclk * (HALL_CFG.FIL_LEN[14:0] + 1)

举例，在 96MHz 工作频率下，周期 Tclk 是 10.42ns，寄存器配置最大为 32767，最长滤波宽度为约 10.42ns×32768≈340us。

通过访问 HALL_INFO.FIL_DATA[2:0]可以捕捉滤波后的 HALL 信号；HALL_INFO.RAW_DATA[2:0] 则是滤波前原始 HALL 输入信号。

13.2.4 捕获

捕获模块用于测量两次 HALL 信号变化之间的时间，其核心为一个 24 位计数器，在 96MHz 工作频率下，如果通过 HALL_CFG.CLK_DIV=3 设置 Hall 时钟 8 分频，则最大可以记录约 2^24*8/96e6=1. 4s 的时间宽度，达到 10.42ns 的时间分辨率。

HALL_CNT 从 0 开始计数，当发生 HALL 信号变化时，将此时刻的 HALL_CNT 值保存到 HALL_WIDTH 寄存器，将此时刻的 HALL 信号保存到 HALL_INFO.FIL_DATA，输出 HALL 信号变化中断，HALL_CNT 重新从 0 开始计数。

当计数器计数值达到 HALL_TH 时，输出 HALL 计数器溢出中断，计数器重新从 0 开始计数。

13.2.5 中断

捕获、溢出事件触发中断，中断使能控制位位于 HALL_CFG.CHG_IE 和 HALL_CFG.OV_IE，中断 标志位位于 HALL_INFO.CHG_IF 和 HALL_INFO.OV_IF。终端标志可以通过对 HALL_INFO.CHG_IF 和 HALL_INFO.OV_IF 写 1 清空。

HALL 中断事件可以作为 DMA 请求，但需要在 HALL_CFG 中使能 CHG_RE，OV_RE 或 SW_RE。 当 DMA 收到传输请求开始数据搬运后，会将中断标志清除，不再需要软件清除。

13.2.6 数据流程

HALL 模块的数据流程如下图所示，FCLK 为受 SYS_CLK_FEN.HALL_CLK_EN 门控信号控制的 HALL 时钟，开通后为 96MHz 的 PLL 时钟。

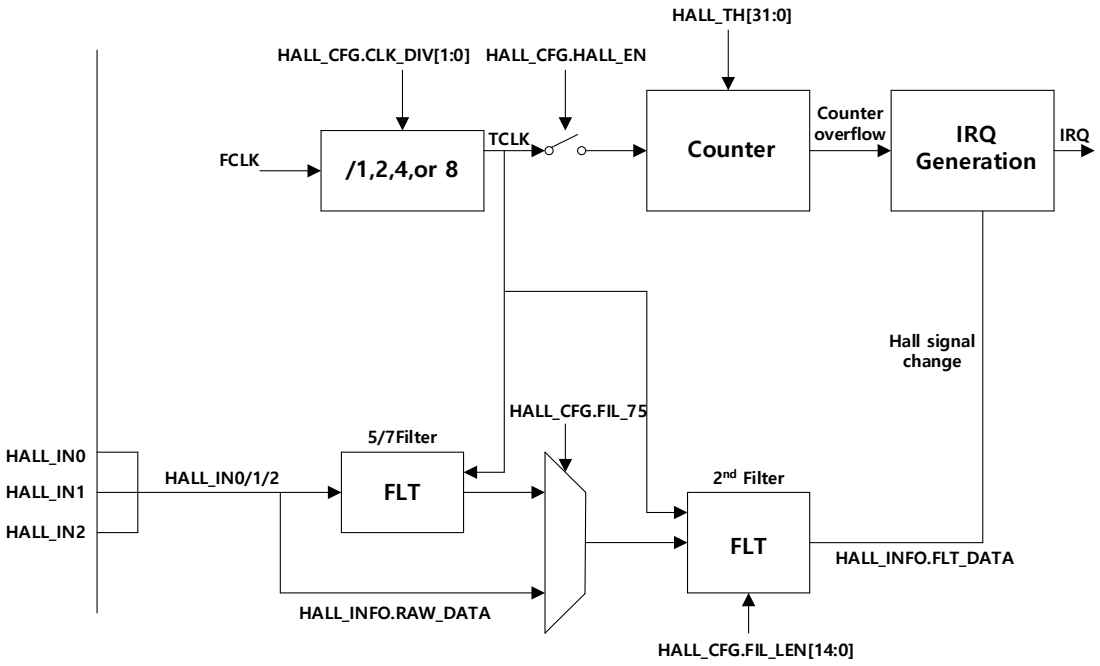


图 13-2 数据流程框图

13.3 寄存器列表

HALL 模块寄存器的基地址是 0x4000_6400，寄存器列表如下:

表 13-1 HALL 模块寄存器地址分配

名称	偏移地址	描述
HALL_IR	0x04	HALL 中断标志寄存器
HALL_IE	0x08	HALL 中断使能寄存器
HALL_CFG	0x0C	HALL 模块配置寄存器
HALL_TH	0x10	HALL 门限值寄存器
HALL_CNT	0x14	HALL 计数寄存器
HALL_CCR	0x18	HALL 捕获值寄存器
HALL_INFO	0x1C	HALL 模块信息寄存器
HALL_LOCK	0x20	HALL 解锁寄存器

13.4 寄存器描述

13.4.1 HALL_IR HALL 中断标志寄存器

地址:0x4000_6404

复位值:0x0

位域	类型	复位	名称	描述
[31:3]	--	--	--	--
[2]	RW1C	0	SW_IF	软件触发 HALL 信号变化中断。HALL_CFG bit[3] 写 1 触发，该位写 1 清零。
[1]	RW1C	0	OV_IF	HALL 计数器溢出事件标志，写 1 清零
[0]	RW1C	0	CHG_IF	HALL 信号变化事件标志，写 1 清零

13.4.2 HALL_IR HALL 中断使能寄存器

地址:0x4000_6408

复位值:0x0

位域	类型	复位	名称	描述
[31:3]	--	--	--	--
[2]	R/W	0	SW_IE	软件触发 HALL 信号变化中断使能，高电平有效。当 SW_IE=1 时，软件向 HALL_CFG.SW_IF 写 1，可以手动产生 HALL 信号变化中断，并 令 HALL_IR.SW_IF 置位。
[1]	R/W	0	OV_IE	HALL 计数器溢出中断使能开关。默认关闭。 0: 关闭 1: 使能
[0]	R/W	0	CHG_IE	HALL 信号变化中断使能开关。默认关闭。 0: 关闭 1: 使能

13.4.3 HALL_CFG HALL 模块配置寄存器

地址:0x4000_640C

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	R/W	0	FIL_LEN	滤波宽度，低于对应脉冲宽度的信号将被硬件自动过滤掉。滤波宽度的计算公式为[15:0] + 1。
[15:13]	--	--	--	--
[12]	R/W	0	FIL_75	7/5 滤波开关（连续采样 7 次，5 次值一致）。默认关闭。 0: 关闭 1: 使能
[11:6]	--	--	--	--
[5:4]	R/W	0	CLK_DIV	HALL 时钟分频系数。默认不分频。 00:不分频 01:2 分频 10:4 分频 11:8 分频

[3:1]	--	--	--	--
[0]	R/W	0	HALL_EN	HALL 模块使能开关。默认关闭。 0: 关闭 1: 使能

13.4.4 HALL_TH HALL 门限值寄存器

地址:0x4000_6410

复位值:0x0

位域	类型	复位	名称	描述
[31:0]	R/W	0	TH	HALL 计数器门限值

13.4.5 HALL_CNT HALL 计数值寄存器

地址:0x4000_6414

复位值:0x0

位域	类型	复位	名称	描述
[31:0]	R/W	0	CNT	HALL 计数值，写入任意值可清零

13.4.6 HALL_CCR HALL 捕获值寄存器

地址:0x4000_6418

复位值:0x0

位域	类型	复位	名称	描述
[31:0]	R/W	0	CCR	HALL 宽度计数器值

13.4.7 HALL_INFO HALL 模块信息寄存器

地址:0x4000_641C

复位值:0x0

位域	类型	复位	名称	描述
[31:19]	--	--	--	--
[18:16]	R/W	0	RAW_DATA	HALL 值，未滤波结果
[15:3]	--	--	--	--
[2:0]	R/W	0	FLT_DATA	HALL 值，滤波结果

13.4.8 HALL_LOCK　HALL 模块解锁寄存器

地址:0x4000_6420

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	WO	0	KEY	写 0x5A5A 解锁
[15:1]	--	--	--	--
[0]	RO	0	HALL_UNLOCK	KEY 写入 0x5A5A 后，HALL_UNLOCK 置 1

14 GPIO

14.1 概述

芯片集成了 3 组 GPIO,PA/PB/PC。PA16 个，PB16 个，PC9 个。部分 GPIO 可以作为系统休眠唤醒源。

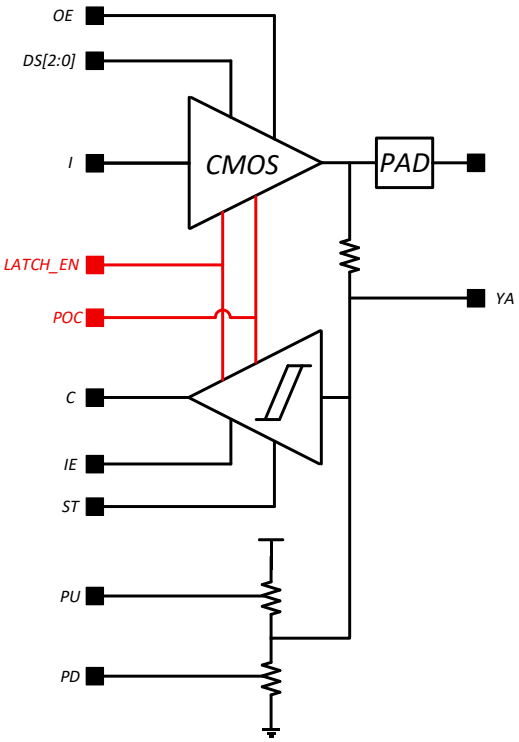


图 14-1 GPIO 功能框图

如上图所示，OE 输出使能（Output Enable），控制输出缓冲器的开关；高有效时，PAD 被驱动为 I 的电平；DS[2:0] 驱动强度控制（Drive Strength），3 位信号可配置输出缓冲的驱动电流（如 4/8/16/24mA 档位），适配不同负载与 EMI 需求；I 输出数据输入（来自 MCU 内部 GPIO 寄存器的输出信号）；LATCH_EN 数据锁存使能，低功耗模式下锁存输出状态，防止时钟停止时 PAD 电平浮动；POC 上电控制（Power-On Control），在上电复位阶段强制 IO 进入稳定状态，避免不确定电平；C 输入信号; IE 输入使能（Input Enable），控制输入缓冲器的开关；高有效时，PAD 上的外部信号会被采样送入内部逻辑； ST 施密特触发使能（Schmitt Trigger），开启后输入缓冲带迟滞，适合噪声大、边沿缓慢的信号； PU/PD 上拉 / 下拉电阻使能，控制 PAD 的默认电平状态（上拉到 VDD / 下拉到 VSS），防止浮空输入；

14.2 寄存器列表

地址	寄存器	描述	备注
0x4800_0000	PA_VERID	PA 版本寄存器	
0x4800_0004	PA_DR	PA 数据寄存器	
0x4800_0008	PA_DRS	PA 数据置位寄存器	
0x4800_000C	PA_DRC	PA 数据清除寄存器	
0x4800_0010	PA_DRT	PA 数据翻转寄存器	
0x4800_0014	PA_OESR	PA 输出使能置位寄存器	
0x4800_0018	PA_OECR	PA 输出使能清除寄存器	
0x4800_001C	PA_ITESR	PA 中断使能置位寄存器	
0x4800_0020	PA_ITECR	PA 中断使能清除寄存器	
0x4800_0024	PA_ITTR1	PA 中断触发类型寄存器 1	
0x4800_0028	PA_ITTR2	PA 中断触发类型寄存器 2	
0x4800_0034	PA_ITTCR	PA 中断触发类型清除寄存器	
0x4800_0038	PA_ITSR	PA 中断状态寄存器	
0x4800_003C	PA_AFSR1	PA 复用功能置位寄存器 1	
0x4800_0040	PA_AFSR2	PA 复用功能置为寄存器 2	
0x4800_004C	PA_AFCR	PA 复用功能清除寄存器	
0x4800_0050	PA_ANASR	PA 模拟功能置位寄存器	
0x4800_0054	PA_ANACR	PA 模拟功能清除寄存器	
0x4800_0058	PA_PUSR	PA 上拉使能置位寄存器	
0x4800_005C	PA_PUCR	PA 上拉使能清除寄存器	
0x4800_0060	PA_PDSR	PA 下拉使能置位寄存器	
0x4800_0064	PA_PDCR	PA 下拉使能清除寄存器	
0x4800_0068	PA_DRVSR1	PA 驱动强度置位寄存器 1	
0x4800_006C	PA_DRVSR2	PA 驱动强度置位寄存器 2	
0x4800_0078	PA_DRVCR	PA 驱动强度清除寄存器	

0x4800_0084	PA_STSR	PA 施密特触发置位寄存器	
0x4800_0088	PA_STCR	PA 施密特触发清除寄存器	
0x4800_008C	PA_IESR	PA 输入使能置位寄存器	
0x4800_0090	PA_IECR	PA 输入使能清除寄存器	

地址	寄存器	描述	备注
0x4800_0400	PB_VERID	PB 版本寄存器	
0x4800_0404	PB_DR	PB 数据寄存器	
0x4800_0408	PB_DRS	PB 数据置位寄存器	
0x4800_040C	PB_DRC	PB 数据清除寄存器	
0x4800_0410	PB_DRT	PB 数据翻转寄存器	
0x4800_0414	PB_OESR	PB 输出使能置位寄存器	
0x4800_0418	PB_OECR	PB 输出使能清除寄存器	
0x4800_041C	PB_ITESR	PB 中断使能置位寄存器	
0x4800_0420	PB_ITECR	PB 中断使能清除寄存器	
0x4800_0424	PB_ITTR1	PB 中断触发类型寄存器 1	
0x4800_0428	PB_ITTR2	PB 中断触发类型寄存器 2	
0x4800_0434	PB_ITTCR	PB 中断触发类型清除寄存器	
0x4800_0438	PB_ITSR	PB 中断状态寄存器	
0x4800_043C	PB_AFSR1	PB 复位功能置位寄存器 1	
0x4800_0440	PB_AFSR2	PB 复位功能置为寄存器 2	
0x4800_044C	PB_AFCR	PB 复用功能清除寄存器	
0x4800_0450	PB_ANASR	PB 模拟功能置位寄存器	
0x4800_0454	PB_ANACR	PB 模拟功能清除寄存器	
0x4800_0458	PB_PUSR	PB 上拉使能置位寄存器	
0x4800_045C	PB_PUCR	PB 上拉使能清除寄存器	
0x4800_0460	PB_PDSR	PB 下拉使能置位寄存器	
0x4800_0464	PB_PDCR	PB 下拉使能清除寄存器	

0x4800_0468	PB_DRVSR1	PB 驱动强度置位寄存器 1	
0x4800_046C	PB_DRVSR2	PB 驱动强度置位寄存器 2	
0x4800_0478	PB_DRVCR	PB 驱动强度清除寄存器	
0x4800_0484	PB_STSR	PB 施密特触发置位寄存器	
0x4800_0488	PB_STCR	PB 施密特触发清除寄存器	
0x4800_048C	PB_IESR	PB 输入使能置位寄存器	
0x4800_0490	PB_IECR	PB 输入使能清除寄存器	

地址	寄存器	描述	备注
0x4800_0800	PC_VERID	PC 版本寄存器	
0x4800_0804	PC_DR	PC 数据寄存器	
0x4800_0808	PC_DRS	PC 数据置位寄存器	
0x4800_080C	PC_DRC	PC 数据清除寄存器	
0x4800_0810	PC_DRT	PC 数据翻转寄存器	
0x4800_0814	PC_OESR	PC 输出使能置位寄存器	
0x4800_0818	PC_OECR	PC 输出使能清除寄存器	
0x4800_081C	PC_ITESR	PC 中断使能置位寄存器	
0x4800_0820	PC_ITECR	PC 中断使能清除寄存器	
0x4800_0824	PC_ITTR1	PC 中断触发类型寄存器 1	
0x4800_0828	PC_ITTR2	PC 中断触发类型寄存器 2	
0x4800_0834	PC_ITTCR	PC 中断触发类型清除寄存器	
0x4800_0838	PC_ITSR	PC 中断状态寄存器	
0x4800_083C	PC_AFSR1	PC 复位功能置位寄存器 1	
0x4800_0840	PC_AFSR2	PC 复位功能置为寄存器 2	
0x4800_084C	PC_AFCR	PC 复用功能清除寄存器	
0x4800_0850	PC_ANASR	PC 模拟功能置位寄存器	
0x4800_0854	PC_ANACR	PC 模拟功能清除寄存器	
0x4800_0858	PC_PUSR	PC 上拉使能置位寄存器	

0x4800_085C	PC_PUCR	PC 上拉使能清除寄存器	
0x4800_0860	PC_PDSR	PC 下拉使能置位寄存器	
0x4800_0864	PC_PDCR	PC 下拉使能清除寄存器	
0x4800_0868	PC_DRVSR1	PC 驱动强度置位寄存器 1	
0x4800_086C	PC_DRVSR2	PC 驱动强度置位寄存器 2	
0x4800_0878	PC_DRVCR	PC 驱动强度清除寄存器	
0x4800_0884	PC_STSR	PC 施密特触发置位寄存器	
0x4800_0888	PC_STCR	PC 施密特触发清除寄存器	
0x4800_088C	PC_IESR	PC 输入使能置位寄存器	
0x4800_0890	PC_IECR	PC 输入使能清除寄存器	

14.3 寄存器描述

14.3.1 Px_DR 数据寄存器(x=A,B,C)

地址:0x4800_0004/0x4800_0404/0x4800_0804

复位值: PA 0x0; PB 0x0; PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	R/W	0	DR15	PIN15 数据读写：读返回 PAD 当前电平，写设置输出锁存值
[14]	R/W	0	DR14	PIN14 数据读写：读返回 PAD 当前电平，写设置输出锁存值
[13]	R/W	0	DR13	PIN13 数据读写：读返回 PAD 当前电平，写设置输出锁存值
[12]	R/W	0	DR12	PIN12 数据读写：读返回 PAD 当前电平，写设置输出锁存值
[11]	R/W	0	DR11	PIN11 数据读写：读返回 PAD 当前电平，写设置输出锁存值
[10]	R/W	0	DR10	PIN10 数据读写：读返回 PAD 当前电平，写设置输出锁存值
[9]	R/W	0	DR9	PIN9 数据读写：读返回 PAD 当前电平，写设置输出锁存值
[8]	R/W	0	DR8	PIN8 数据读写：读返回 PAD 当前电平，写设置输出锁存值
[7]	R/W	0	DR7	PIN7 数据读写：读返回 PAD 当前电平，写设置输出锁存值
[6]	R/W	0	DR6	PIN6 数据读写：读返回 PAD 当前电平，写设置输出锁存值

[5]	R/W	0	DR5	PIN5 数据读写：读返回 PAD 当前电平，写设置输出锁存值
[4]	R/W	0	DR4	PIN4 数据读写：读返回 PAD 当前电平，写设置输出锁存值
[3]	R/W	0	DR3	PIN3 数据读写：读返回 PAD 当前电平，写设置输出锁存值
[2]	R/W	0	DR2	PIN2 数据读写：读返回 PAD 当前电平，写设置输出锁存值
[1]	R/W	0	DR1	PIN1 数据读写：读返回 PAD 当前电平，写设置输出锁存值
[0]	R/W	0	DR0	PIN0 数据读写：读返回 PAD 当前电平，写设置输出锁存值

14.3.2 Px_DRS 数据置位寄存器(x=A,B,C)

地址:0x4800_0008/0x4800_0408/0x4800_0808

复位值: PA 0x0; PB 0x0; PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	R/W	0	DRS15	PIN15 对应位写 1，将 GPIO 输出电平置高
[14]	R/W	0	DRS14	PIN14 对应位写 1，将 GPIO 输出电平置高
[13]	R/W	0	DRS13	PIN13 对应位写 1，将 GPIO 输出电平置高
[12]	R/W	0	DRS12	PIN12 对应位写 1，将 GPIO 输出电平置高
[11]	R/W	0	DRS11	PIN11 对应位写 1，将 GPIO 输出电平置高
[10]	R/W	0	DRS10	PIN10 对应位写 1，将 GPIO 输出电平置高
[9]	R/W	0	DRS9	PIN9 对应位写 1，将 GPIO 输出电平置高
[8]	R/W	0	DRS8	PIN8 对应位写 1，将 GPIO 输出电平置高
[7]	R/W	0	DRS7	PIN7 对应位写 1，将 GPIO 输出电平置高
[6]	R/W	0	DRS6	PIN6 对应位写 1，将 GPIO 输出电平置高
[5]	R/W	0	DRS5	PIN5 对应位写 1，将 GPIO 输出电平置高
[4]	R/W	0	DRS4	PIN4 对应位写 1，将 GPIO 输出电平置高
[3]	R/W	0	DRS3	PIN3 对应位写 1，将 GPIO 输出电平置高
[2]	R/W	0	DRS2	PIN2 对应位写 1，将 GPIO 输出电平置高
[1]	R/W	0	DRS1	PIN1 对应位写 1，将 GPIO 输出电平置高
[0]	R/W	0	DRS0	PIN0 对应位写 1，将 GPIO 输出电平置高

14.3.3 Px_DRC 数据清除寄存器(x=A,B,C)

地址:0x4800_000C/0x4800_040C/0x4800_080C

复位值: PA 0x0; PB 0x0; PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	W	0	DRC15	PIN15 对应位写 1，将 GPIO 输出电平置低
[14]	W	0	DRC14	PIN14 对应位写 1，将 GPIO 输出电平置低
[13]	W	0	DRC13	PIN13 对应位写 1，将 GPIO 输出电平置低
[12]	W	0	DRC12	PIN12 对应位写 1，将 GPIO 输出电平置低
[11]	W	0	DRC11	PIN11 对应位写 1，将 GPIO 输出电平置低
[10]	W	0	DRC10	PIN10 对应位写 1，将 GPIO 输出电平置低
[9]	W	0	DRC9	PIN9 对应位写 1，将 GPIO 输出电平置低
[8]	W	0	DRC8	PIN8 对应位写 1，将 GPIO 输出电平置低
[7]	W	0	DRC7	PIN7 对应位写 1，将 GPIO 输出电平置低
[6]	W	0	DRC6	PIN6 对应位写 1，将 GPIO 输出电平置低
[5]	W	0	DRC5	PIN5 对应位写 1，将 GPIO 输出电平置低
[4]	W	0	DRC4	PIN4 对应位写 1，将 GPIO 输出电平置低
[3]	W	0	DRC3	PIN3 对应位写 1，将 GPIO 输出电平置低
[2]	W	0	DRC2	PIN2 对应位写 1，将 GPIO 输出电平置低
[1]	W	0	DRC1	PIN1 对应位写 1，将 GPIO 输出电平置低
[0]	W	0	DRC0	PIN0 对应位写 1，将 GPIO 输出电平置低

14.3.4 Px_DRT 数据翻转寄存器(x=A,B,C)

地址:0x4800_0010/0x4800_0410/0x4800_0810

复位值: PA 0x0; PB 0x0; PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	R/W	0	DRT15	PIN15 对应位写 1，将 GPIO 输出电平翻转（高↔低）

[14]	R/W	0	DRT14	PIN14 对应位写 1，将 GPIO 输出电平翻转（高↔低）
[13]	R/W	0	DRT13	PIN13 对应位写 1，将 GPIO 输出电平翻转（高↔低）
[12]	R/W	0	DRT12	PIN12 对应位写 1，将 GPIO 输出电平翻转（高↔低）
[11]	R/W	0	DRT11	PIN11 对应位写 1，将 GPIO 输出电平翻转（高↔低）
[10]	R/W	0	DRT10	PIN10 对应位写 1，将 GPIO 输出电平翻转（高↔低）
[9]	R/W	0	DRT9	PIN9 对应位写 1，将 GPIO 输出电平翻转（高↔低）
[8]	R/W	0	DRT8	PIN8 对应位写 1，将 GPIO 输出电平翻转（高↔低）
[7]	R/W	0	DRT7	PIN7 对应位写 1，将 GPIO 输出电平翻转（高↔低）
[6]	R/W	0	DRT6	PIN6 对应位写 1，将 GPIO 输出电平翻转（高↔低）
[5]	R/W	0	DRT5	PIN5 对应位写 1，将 GPIO 输出电平翻转（高↔低）
[4]	R/W	0	DRT4	PIN4 对应位写 1，将 GPIO 输出电平翻转（高↔低）
[3]	R/W	0	DRT3	PIN3 对应位写 1，将 GPIO 输出电平翻转（高↔低）
[2]	R/W	0	DRT2	PIN2 对应位写 1，将 GPIO 输出电平翻转（高↔低）
[1]	R/W	0	DRT1	PIN1 对应位写 1，将 GPIO 输出电平翻转（高↔低）
[0]	R/W	0	DRT0	PIN0 对应位写 1，将 GPIO 输出电平翻转（高↔低）

14.3.5 Px_OESR 输出使能置位寄存器(x=A,B,C)

地址:0x4800_0014/0x4800_0414/0x4800_0814

复位值: PA 0x0; PB 0x0; PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	R/W	0	OESR15	PIN15 对应位写 1，使 OE 信号有效，开启输出使能
[14]	R/W	0	OESR14	PIN14 对应位写 1，使 OE 信号有效，开启输出使能
[13]	R/W	0	OESR13	PIN13 对应位写 1，使 OE 信号有效，开启输出使能
[12]	R/W	0	OESR12	PIN12 对应位写 1，使 OE 信号有效，开启输出使能
[11]	R/W	0	OESR11	PIN11 对应位写 1，使 OE 信号有效，开启输出使能
[10]	R/W	0	OESR10	PIN10 对应位写 1，使 OE 信号有效，开启输出使能
[9]	R/W	0	OESR9	PIN9 对应位写 1，使 OE 信号有效，开启输出使能

[8]	R/W	0	OESR8	PIN8 对应位写 1，使 OE 信号有效，开启输出使能
[7]	R/W	0	OESR7	PIN7 对应位写 1，使 OE 信号有效，开启输出使能
[6]	R/W	0	OESR6	PIN6 对应位写 1，使 OE 信号有效，开启输出使能
[5]	R/W	0	OESR5	PIN5 对应位写 1，使 OE 信号有效，开启输出使能
[4]	R/W	0	OESR4	PIN4 对应位写 1，使 OE 信号有效，开启输出使能
[3]	R/W	0	OESR3	PIN3 对应位写 1，使 OE 信号有效，开启输出使能
[2]	R/W	0	OESR2	PIN2 对应位写 1，使 OE 信号有效，开启输出使能
[1]	R/W	0	OESR1	PIN1 对应位写 1，使 OE 信号有效，开启输出使能
[0]	R/W	0	OESR0	PIN0 对应位写 1，使 OE 信号有效，开启输出使能

14.3.6 Px_OECR 输出使能清除寄存器(x=A,B,C)

地址:0x4800_0018/0x4800_0418/0x4800_0818

复位值: PA 0x0; PB 0x0; PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	W	0	OECR15	PIN15 对应位写 1，使 OE 信号无效，关闭输出使能
[14]	W	0	OECR14	PIN14 对应位写 1，使 OE 信号无效，关闭输出使能
[13]	W	0	OECR13	PIN13 对应位写 1，使 OE 信号无效，关闭输出使能
[12]	W	0	OECR12	PIN12 对应位写 1，使 OE 信号无效，关闭输出使能
[11]	W	0	OECR11	PIN11 对应位写 1，使 OE 信号无效，关闭输出使能
[10]	W	0	OECR10	PIN10 对应位写 1，使 OE 信号无效，关闭输出使能
[9]	W	0	OECR9	PIN9 对应位写 1，使 OE 信号无效，关闭输出使能
[8]	W	0	OECR8	PIN8 对应位写 1，使 OE 信号无效，关闭输出使能
[7]	W	0	OECR7	PIN7 对应位写 1，使 OE 信号无效，关闭输出使能
[6]	W	0	OECR6	PIN6 对应位写 1，使 OE 信号无效，关闭输出使能
[5]	W	0	OECR5	PIN5 对应位写 1，使 OE 信号无效，关闭输出使能
[4]	W	0	OECR4	PIN4 对应位写 1，使 OE 信号无效，关闭输出使能
[3]	W	0	OECR3	PIN3 对应位写 1，使 OE 信号无效，关闭输出使能

[2]	W	0	OECR2	PIN2 对应位写 1，使 OE 信号无效，关闭输出使能
[1]	W	0	OECR1	PIN1 对应位写 1，使 OE 信号无效，关闭输出使能
[0]	W	0	OECR0	PIN0 对应位写 1，使 OE 信号无效，关闭输出使能

14.3.7 Px_ITESR 中断使能置位寄存器(x=A,B,C)

地址:0x4800_001C/0x4800_041C/0x4800_081C

复位值: PA 0x0; PB 0x0; PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	R/W	0	ITESR15	PIN15 对应位写 1，开启 GPIO 外部中断
[14]	R/W	0	ITESR14	PIN14 对应位写 1，开启 GPIO 外部中断
[13]	R/W	0	ITESR13	PIN13 对应位写 1，开启 GPIO 外部中断
[12]	R/W	0	ITESR12	PIN12 对应位写 1，开启 GPIO 外部中断
[11]	R/W	0	ITESR11	PIN11 对应位写 1，开启 GPIO 外部中断
[10]	R/W	0	ITESR10	PIN10 对应位写 1，开启 GPIO 外部中断
[9]	R/W	0	ITESR9	PIN9 对应位写 1，开启 GPIO 外部中断
[8]	R/W	0	ITESR8	PIN8 对应位写 1，开启 GPIO 外部中断
[7]	R/W	0	ITESR7	PIN7 对应位写 1，开启 GPIO 外部中断
[6]	R/W	0	ITESR6	PIN6 对应位写 1，开启 GPIO 外部中断
[5]	R/W	0	ITESR5	PIN5 对应位写 1，开启 GPIO 外部中断
[4]	R/W	0	ITESR4	PIN4 对应位写 1，开启 GPIO 外部中断
[3]	R/W	0	ITESR3	PIN3 对应位写 1，开启 GPIO 外部中断
[2]	R/W	0	ITESR2	PIN2 对应位写 1，开启 GPIO 外部中断
[1]	R/W	0	ITESR1	PIN1 对应位写 1，开启 GPIO 外部中断
[0]	R/W	0	ITESR0	PIN0 对应位写 1，开启 GPIO 外部中断

14.3.8 Px_ITECR 中断使能清除寄存器(x=A,B,C)

地址:0x4800_0020/0x4800_0420/0x4800_0820

复位值：PA 0x0；PB 0x0；PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	W	0	ITECR15	PIN15 对应位写 1，关闭 GPIO 外部中断
[14]	W	0	ITECR14	PIN14 对应位写 1，关闭 GPIO 外部中断
[13]	W	0	ITECR13	PIN13 对应位写 1，关闭 GPIO 外部中断
[12]	W	0	ITECR12	PIN12 对应位写 1，关闭 GPIO 外部中断
[11]	W	0	ITECR11	PIN11 对应位写 1，关闭 GPIO 外部中断
[10]	W	0	ITECR10	PIN10 对应位写 1，关闭 GPIO 外部中断
[9]	W	0	ITECR9	PIN9 对应位写 1，关闭 GPIO 外部中断
[8]	W	0	ITECR8	PIN8 对应位写 1，关闭 GPIO 外部中断
[7]	W	0	ITECR7	PIN7 对应位写 1，关闭 GPIO 外部中断
[6]	W	0	ITECR6	PIN6 对应位写 1，关闭 GPIO 外部中断
[5]	W	0	ITECR5	PIN5 对应位写 1，关闭 GPIO 外部中断
[4]	W	0	ITECR4	PIN4 对应位写 1，关闭 GPIO 外部中断
[3]	W	0	ITECR3	PIN3 对应位写 1，关闭 GPIO 外部中断
[2]	W	0	ITECR2	PIN2 对应位写 1，关闭 GPIO 外部中断
[1]	W	0	ITECR1	PIN1 对应位写 1，关闭 GPIO 外部中断
[0]	W	0	ITECR0	PIN0 对应位写 1，关闭 GPIO 外部中断

14.3.9 Px_ITTR1　　中断类型寄存器 1(x=A,B,C)

地址:0x4800_0024/0x4800_0424/0x4800_0824

复位值：PA 0x0；PB 0x0；PC 0x0

位域	类型	复位	名称	描述
[31]	--	--	--	--
[30:28]	R/W	0	ITTR7	PIN7 选择中断触发类型
[27]	--	--	--	--
[26:24]	R/W	0	ITTR6	PIN6 选择中断触发类型

[23]	--	--	--	--
[22:20]	R/W	0	ITTR5	PIN5 选择中断触发类型
[19]	--	--	--	--
[18:16]	R/W	0	ITTR4	PIN4 选择中断触发类型
[15]	--	--	--	--
[14:12]	R/W	0	ITTR3	PIN3 选择中断触发类型
[11]	--	--	--	--
[10:8]	R/W	0	ITTR2	PIN2 选择中断触发类型
[7]	--	--	--	--
[6:4]	R/W	0	ITTR1	PIN1 选择中断触发类型
[3]	--	--	--	--
[2:0]	R/W	0	ITTR0	PIN0 选择中断触发类型 该位由软件置位和清除 000:低电平触发 001:高电平触发 010:下降沿触发 011:上升沿触发 100:任意沿触发

14.3.10 Px_ITTR2 中断类型寄存器 2(x=A,B,C)

地址:0x4800_0028/0x4800_0428/0x4800_0828

复位值: PA 0x0; PB 0x0; PC 0x0

位域	类型	复位	名称	描述
[31]	--	--	--	--
[30:28]	R/W	0	ITTR15	PIN15 选择中断触发类型
[27]	--	--	--	--
[26:24]	R/W	0	ITTR14	PIN14 选择中断触发类型
[23]	--	--	--	--

[22:20]	R/W	0	ITTR13	PIN13 选择中断触发类型
[19]	--	--	--	--
[18:16]	R/W	0	ITTR12	PIN12 选择中断触发类型
[15]	--	--	--	--
[14:12]	R/W	0	ITTR11	PIN11 选择中断触发类型
[11]	--	--	--	--
[10:8]	R/W	0	ITTR10	PIN10 选择中断触发类型
[7]	--	--	--	--
[6:4]	R/W	0	ITTR9	PIN9 选择中断触发类型
[3]	--	--	--	--
[2:0]	R/W	0	ITTR8	PIN8 选择中断触发类型

14.3.11 Px_ITTCR 中断触发类型清除寄存器(x=A,B,C)

地址:0x4800_0034/0x4800_0434/0x4800_0834

复位值: PA 0x0; PB 0x0; PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	W	0	ITTCR15	PIN15 对应位写 1，清零中断触发类型
[14]	W	0	ITTCR14	PIN14 对应位写 1，清零中断触发类型
[13]	W	0	ITTCR13	PIN13 对应位写 1，清零中断触发类型
[12]	W	0	ITTCR12	PIN12 对应位写 1，清零中断触发类型
[11]	W	0	ITTCR11	PIN11 对应位写 1，清零中断触发类型
[10]	W	0	ITTCR10	PIN10 对应位写 1，清零中断触发类型
[9]	W	0	ITTCR9	PIN9 对应位写 1，清零中断触发类型
[8]	W	0	ITTCR8	PIN8 对应位写 1，清零中断触发类型
[7]	W	0	ITTCR7	PIN7 对应位写 1，清零中断触发类型
[6]	W	0	ITTCR6	PIN6 对应位写 1，清零中断触发类型
[5]	W	0	ITTCR5	PIN5 对应位写 1，清零中断触发类型

[4]	W	0	ITTCR4	PIN4 对应位写 1，清零中断触发类型
[3]	W	0	ITTCR3	PIN3 对应位写 1，清零中断触发类型
[2]	W	0	ITTCR2	PIN2 对应位写 1，清零中断触发类型
[1]	W	0	ITTCR1	PIN1 对应位写 1，清零中断触发类型
[0]	W	0	ITTCR0	PIN0 对应位写 1，清零中断触发类型

14.3.12 Px_ITSR 中断状态寄存器(x=A,B,C)

地址:0x4800_0038/0x4800_0438/0x4800_0838

复位值: PA 0x0; PB 0x0; PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	RW1C	0	ITSR15	PIN15 中断标志位，读返回状态标志，写 1 可清除标志
[14]	RW1C	0	ITSR14	PIN14 中断标志位，读返回状态标志，写 1 可清除标志
[13]	RW1C	0	ITSR13	PIN13 中断标志位，读返回状态标志，写 1 可清除标志
[12]	RW1C	0	ITSR12	PIN12 中断标志位，读返回状态标志，写 1 可清除标志
[11]	RW1C	0	ITSR11	PIN11 中断标志位，读返回状态标志，写 1 可清除标志
[10]	RW1C	0	ITSR10	PIN10 中断标志位，读返回状态标志，写 1 可清除标志
[9]	RW1C	0	ITSR9	PIN9 中断标志位，读返回状态标志，写 1 可清除标志
[8]	RW1C	0	ITSR8	PIN8 中断标志位，读返回状态标志，写 1 可清除标志
[7]	RW1C	0	ITSR7	PIN7 中断标志位，读返回状态标志，写 1 可清除标志
[6]	RW1C	0	ITSR6	PIN6 中断标志位，读返回状态标志，写 1 可清除标志
[5]	RW1C	0	ITSR5	PIN5 中断标志位，读返回状态标志，写 1 可清除标志
[4]	RW1C	0	ITSR4	PIN4 中断标志位，读返回状态标志，写 1 可清除标志
[3]	RW1C	0	ITSR3	PIN3 中断标志位，读返回状态标志，写 1 可清除标志
[2]	RW1C	0	ITSR2	PIN2 中断标志位，读返回状态标志，写 1 可清除标志
[1]	RW1C	0	ITSR1	PIN1 中断标志位，读返回状态标志，写 1 可清除标志
[0]	RW1C	0	ITSR0	PIN0 中断标志位，读返回状态标志，写 1 可清除标志

14.3.13 Px_AFSR1　复用功能置位寄存器 1(x=A,B,C)

地址:0x4800_003C/0x4800_043C/0x4800_083C

复位值: PA 0x0; PB 0x0000_0011; PC 0x0

位域	类型	复位	名称	描述
[31:28]	R/W	0	AFSR7	PIN7 选择复用功能
[27:24]	R/W	0	AFSR6	PIN6 选择复用功能
[23:20]	R/W	0	AFSR5	PIN5 选择复用功能
[19:16]	R/W	0	AFSR4	PIN4 选择复用功能
[15:12]	R/W	0	AFSR3	PIN3 选择复用功能
[11:8]	R/W	0	AFSR2	PIN2 选择复用功能
[7:4]	R/W	0	AFSR1	PIN1 选择复用功能
[3:0]	R/W	0	AFSR0	PIN0 选择复用功能 该位由软件置位和清除 0000：选择 AF0 功能 0001：选择 AF1 功能 0010：选择 AF2 功能 0011：选择 AF3 功能 0100：选择 AF4 功能 0101：选择 AF5 功能 0110：选择 AF6 功能 0111：选择 AF7 功能 1000：选择 AF8 功能 1001：选择 AF9 功能 1010：选择 AF10 功能 1011：选择 AF11 功能 1100：选择 AF12 功能 1101：选择 AF13 功能 1110：选择 AF14 功能

				1111：选择 AF15 功能

14.3.14 Px_AFSR2　复用功能置位寄存器 2(x=A,B,C)

地址:0x4800_0040/0x4800_0440/0x4800_0840

复位值: PA 0x1000_0000; PB 0x0; PC 0x0

位域	类型	复位	名称	描述
[31:28]	R/W	0001	AFSR15	PIN15 选择复用功能
[27:24]	R/W	0	AFSR14	PIN14 选择复用功能
[23:20]	R/W	0	AFSR13	PIN13 选择复用功能
[19:16]	R/W	0	AFSR12	PIN12 选择复用功能
[15:12]	R/W	0	AFSR11	PIN11 选择复用功能
[11:8]	R/W	0	AFSR10	PIN10 选择复用功能
[7:4]	R/W	0	AFSR9	PIN9 选择复用功能
[3:0]	R/W	0	AFSR8	PIN8 选择复用功能

14.3.15 Px_AFCR　复用功能清除寄存器(x=A,B,C)

地址:0x4800_004C/0x4800_044C/0x4800_084C

复位值: PA 0x0; PB 0x0; PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	W	0	AFCR15	PIN15 对应位写 1，关闭复用功能，切为普通 IO 模式
[14]	W	0	AFCR14	PIN14 对应位写 1，关闭复用功能，切为普通 IO 模式
[13]	W	0	AFCR13	PIN13 对应位写 1，关闭复用功能，切为普通 IO 模式
[12]	W	0	AFCR12	PIN12 对应位写 1，关闭复用功能，切为普通 IO 模式
[11]	W	0	AFCR11	PIN11 对应位写 1，关闭复用功能，切为普通 IO 模式
[10]	W	0	AFCR10	PIN10 对应位写 1，关闭复用功能，切为普通 IO 模式
[9]	W	0	AFCR9	PIN9 对应位写 1，关闭复用功能，切为普通 IO 模式

[8]	W	0	AFCR8	PIN8 对应位写 1，关闭复用功能，切为普通 IO 模式
[7]	W	0	AFCR7	PIN7 对应位写 1，关闭复用功能，切为普通 IO 模式
[6]	W	0	AFCR6	PIN6 对应位写 1，关闭复用功能，切为普通 IO 模式
[5]	W	0	AFCR5	PIN5 对应位写 1，关闭复用功能，切为普通 IO 模式
[4]	W	0	AFCR4	PIN4 对应位写 1，关闭复用功能，切为普通 IO 模式
[3]	W	0	AFCR3	PIN3 对应位写 1，关闭复用功能，切为普通 IO 模式
[2]	W	0	AFCR2	PIN2 对应位写 1，关闭复用功能，切为普通 IO 模式
[1]	W	0	AFCR1	PIN1 对应位写 1，关闭复用功能，切为普通 IO 模式
[0]	W	0	AFCR0	PIN0 对应位写 1，关闭复用功能，切为普通 IO 模式

14.3.16 Px_ANASR 模拟功能置位寄存器(x=A,B,C)

地址:0x4800_0050/0x4800_0450/0x4800_0850

复位值: PA 0x0; PB 0x0; PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	R/W	0	ANASR15	PIN15 对应位写 1，开启 GPIO 模拟模式
[14]	R/W	0	ANASR14	PIN14 对应位写 1，开启 GPIO 模拟模式
[13]	R/W	0	ANASR13	PIN13 对应位写 1，开启 GPIO 模拟模式
[12]	R/W	0	ANASR12	PIN12 对应位写 1，开启 GPIO 模拟模式
[11]	R/W	0	ANASR11	PIN11 对应位写 1，开启 GPIO 模拟模式
[10]	R/W	0	ANASR10	PIN10 对应位写 1，开启 GPIO 模拟模式
[9]	R/W	0	ANASR9	PIN9 对应位写 1，开启 GPIO 模拟模式
[8]	R/W	0	ANASR8	PIN8 对应位写 1，开启 GPIO 模拟模式
[7]	R/W	0	ANASR7	PIN7 对应位写 1，开启 GPIO 模拟模式
[6]	R/W	0	ANASR6	PIN6 对应位写 1，开启 GPIO 模拟模式
[5]	R/W	0	ANASR5	PIN5 对应位写 1，开启 GPIO 模拟模式
[4]	R/W	0	ANASR4	PIN4 对应位写 1，开启 GPIO 模拟模式
[3]	R/W	0	ANASR3	PIN3 对应位写 1，开启 GPIO 模拟模式

[2]	R/W	0	ANASR2	PIN2 对应位写 1，开启 GPIO 模拟模式
[1]	R/W	0	ANASR1	PIN1 对应位写 1，开启 GPIO 模拟模式
[0]	R/W	0	ANASR0	PIN0 对应位写 1，开启 GPIO 模拟模式

14.3.17 Px_ANACR 模拟功能清除寄存器(x=A,B,C)

地址:0x4800_0054/0x4800_0454/0x4800_0854

复位值: PA 0x0; PB 0x0; PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	W	0	ANACR15	PIN15 对应位写 1，关闭 GPIO 模拟模式
[14]	W	0	ANACR14	PIN14 对应位写 1，关闭 GPIO 模拟模式
[13]	W	0	ANACR13	PIN13 对应位写 1，关闭 GPIO 模拟模式
[12]	W	0	ANACR12	PIN12 对应位写 1，关闭 GPIO 模拟模式
[11]	W	0	ANACR11	PIN11 对应位写 1，关闭 GPIO 模拟模式
[10]	W	0	ANACR10	PIN10 对应位写 1，关闭 GPIO 模拟模式
[9]	W	0	ANACR9	PIN9 对应位写 1，关闭 GPIO 模拟模式
[8]	W	0	ANACR8	PIN8 对应位写 1，关闭 GPIO 模拟模式
[7]	W	0	ANACR7	PIN7 对应位写 1，关闭 GPIO 模拟模式
[6]	W	0	ANACR6	PIN6 对应位写 1，关闭 GPIO 模拟模式
[5]	W	0	ANACR5	PIN5 对应位写 1，关闭 GPIO 模拟模式
[4]	W	0	ANACR4	PIN4 对应位写 1，关闭 GPIO 模拟模式
[3]	W	0	ANACR3	PIN3 对应位写 1，关闭 GPIO 模拟模式
[2]	W	0	ANACR2	PIN2 对应位写 1，关闭 GPIO 模拟模式
[1]	W	0	ANACR1	PIN1 对应位写 1，关闭 GPIO 模拟模式
[0]	W	0	ANACR0	PIN0 对应位写 1，关闭 GPIO 模拟模式

14.3.18 Px_PUSR 上拉使能置位寄存器(x=A,B,C)

地址:0x4800_0058/0x4800_0458/0x4800_0858

复位值：PA 0x00008000；PB 0x0000_0003；PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	R/W	0	PUSR15	PIN15 对应位写 1，开启内部上拉电阻
[14]	R/W	0	PUSR14	PIN14 对应位写 1，开启内部上拉电阻
[13]	R/W	0	PUSR13	PIN13 对应位写 1，开启内部上拉电阻
[12]	R/W	0	PUSR12	PIN12 对应位写 1，开启内部上拉电阻
[11]	R/W	0	PUSR11	PIN11 对应位写 1，开启内部上拉电阻
[10]	R/W	0	PUSR10	PIN10 对应位写 1，开启内部上拉电阻
[9]	R/W	0	PUSR9	PIN9 对应位写 1，开启内部上拉电阻
[8]	R/W	0	PUSR8	PIN8 对应位写 1，开启内部上拉电阻
[7]	R/W	0	PUSR7	PIN7 对应位写 1，开启内部上拉电阻
[6]	R/W	0	PUSR6	PIN6 对应位写 1，开启内部上拉电阻
[5]	R/W	0	PUSR5	PIN5 对应位写 1，开启内部上拉电阻
[4]	R/W	0	PUSR4	PIN4 对应位写 1，开启内部上拉电阻
[3]	R/W	0	PUSR3	PIN3 对应位写 1，开启内部上拉电阻
[2]	R/W	0	PUSR2	PIN2 对应位写 1，开启内部上拉电阻
[1]	R/W	0	PUSR1	PIN1 对应位写 1，开启内部上拉电阻
[0]	R/W	0	PUSR0	PIN0 对应位写 1，开启内部上拉电阻

14.3.19 Px_PUCR 上拉使能清除寄存器(x=A,B,C)

地址:0x4800_005C/0x4800_045C/0x4800_085C

复位值：PA 0x0；PB 0x0；PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	W	0	PUCR15	PIN15 对应位写 1，关闭内部上拉电阻
[14]	W	0	PUCR14	PIN14 对应位写 1，关闭内部上拉电阻
[13]	W	0	PUCR13	PIN13 对应位写 1，关闭内部上拉电阻

[12]	W	0	PUCR12	PIN12 对应位写 1，关闭内部上拉电阻
[11]	W	0	PUCR11	PIN11 对应位写 1，关闭内部上拉电阻
[10]	W	0	PUCR10	PIN10 对应位写 1，关闭内部上拉电阻
[9]	W	0	PUCR9	PIN9 对应位写 1，关闭内部上拉电阻
[8]	W	0	PUCR8	PIN8 对应位写 1，关闭内部上拉电阻
[7]	W	0	PUCR7	PIN7 对应位写 1，关闭内部上拉电阻
[6]	W	0	PUCR6	PIN6 对应位写 1，关闭内部上拉电阻
[5]	W	0	PUCR5	PIN5 对应位写 1，关闭内部上拉电阻
[4]	W	0	PUCR4	PIN4 对应位写 1，关闭内部上拉电阻
[3]	W	0	PUCR3	PIN3 对应位写 1，关闭内部上拉电阻
[2]	W	0	PUCR2	PIN2 对应位写 1，关闭内部上拉电阻
[1]	W	0	PUCR1	PIN1 对应位写 1，关闭内部上拉电阻
[0]	W	0	PUCR0	PIN0 对应位写 1，关闭内部上拉电阻

14.3.20 Px_PDSR 下拉使能置位寄存器(x=A,B,C)

地址:0x4800_0060/0x4800_0460/0x4800_0860

复位值: PA 0x0; PB 0x0000_0010; PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	R/W	0	PDSR15	PIN15 对应位写 1，开启内部下拉电阻
[14]	R/W	0	PDSR14	PIN14 对应位写 1，开启内部下拉电阻
[13]	R/W	0	PDSR13	PIN13 对应位写 1，开启内部下拉电阻
[12]	R/W	0	PDSR12	PIN12 对应位写 1，开启内部下拉电阻
[11]	R/W	0	PDSR11	PIN11 对应位写 1，开启内部下拉电阻
[10]	R/W	0	PDSR10	PIN10 对应位写 1，开启内部下拉电阻
[9]	R/W	0	PDSR9	PIN9 对应位写 1，开启内部下拉电阻
[8]	R/W	0	PDSR8	PIN8 对应位写 1，开启内部下拉电阻
[7]	R/W	0	PDSR7	PIN7 对应位写 1，开启内部下拉电阻

[6]	R/W	0	PDSR6	PIN6 对应位写 1，开启内部下拉电阻
[5]	R/W	0	PDSR5	PIN5 对应位写 1，开启内部下拉电阻
[4]	R/W	0	PDSR4	PIN4 对应位写 1，开启内部下拉电阻
[3]	R/W	0	PDSR3	PIN3 对应位写 1，开启内部下拉电阻
[2]	R/W	0	PDSR2	PIN2 对应位写 1，开启内部下拉电阻
[1]	R/W	0	PDSR1	PIN1 对应位写 1，开启内部下拉电阻
[0]	R/W	0	PDSR0	PIN0 对应位写 1，开启内部下拉电阻

14.3.21 Px_PDCR 下拉使能清除寄存器(x=A,B,C)

地址:0x4800_0064/0x4800_0464/0x4800_0864

复位值: PA 0x0; PB 0x0; PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	W	0	PDCR15	PIN15 对应位写 1，关闭内部下拉电阻
[14]	W	0	PDCR14	PIN14 对应位写 1，关闭内部下拉电阻
[13]	W	0	PDCR13	PIN13 对应位写 1，关闭内部下拉电阻
[12]	W	0	PDCR12	PIN12 对应位写 1，关闭内部下拉电阻
[11]	W	0	PDCR11	PIN11 对应位写 1，关闭内部下拉电阻
[10]	W	0	PDCR10	PIN10 对应位写 1，关闭内部下拉电阻
[9]	W	0	PDCR9	PIN9 对应位写 1，关闭内部下拉电阻
[8]	W	0	PDCR8	PIN8 对应位写 1，关闭内部下拉电阻
[7]	W	0	PDCR7	PIN7 对应位写 1，关闭内部下拉电阻
[6]	W	0	PDCR6	PIN6 对应位写 1，关闭内部下拉电阻
[5]	W	0	PDCR5	PIN5 对应位写 1，关闭内部下拉电阻
[4]	W	0	PDCR4	PIN4 对应位写 1，关闭内部下拉电阻
[3]	W	0	PDCR3	PIN3 对应位写 1，关闭内部下拉电阻
[2]	W	0	PDCR2	PIN2 对应位写 1，关闭内部下拉电阻
[1]	W	0	PDCR1	PIN1 对应位写 1，关闭内部下拉电阻

[0]	W	0	PDCR0	PIN0 对应位写 1，关闭内部下拉电阻

14.3.22 Px_DRVSR1　驱动强度置位寄存器 1(x=A,B,C)

地址:0x4800_0068/0x4800_0468/0x4800_0868

复位值: PA 0x0; PB 0x0; PC 0x0

位域	类型	复位	名称	描述
[31]	--	--	--	--
[30:28]	R/W	0	DRVSR7	PIN7 配置输出驱动电流
[27]	--	--	--	--
[26:24]	R/W	0	DRVSR6	PIN6 配置输出驱动电流
[23]	--	--	--	--
[22:20]	R/W	0	DRVSR5	PIN5 配置输出驱动电流
[19]	--	--	--	--
[18:16]	R/W	0	DRVSR4	PIN4 配置输出驱动电流
[15]	--	--	--	--
[14:12]	R/W	0	DRVSR3	PIN3 配置输出驱动电流
[11]	--	--	--	--
[10:8]	R/W	0	DRVSR2	PIN2 配置输出驱动电流
[7]	--	--	--	--
[6:4]	R/W	0	DRVSR1	PIN1 配置输出驱动电流
[3]	--	--	--	--
[2:0]	R/W	0	DRVSR0	PIN0 配置输出驱动电流 该位由软件置位和清除 000:4mA 001:8mA 010:16mA 011:24mA

14.3.23 Px_DRVSR2 驱动强度置位寄存器 2(x=A,B,C)

地址:0x4800_006C/0x4800_046C/0x4800_086C

复位值: PA 0x0; PB 0x0; PC 0x0

位域	类型	复位	名称	描述
[31]	--	--	--	--
[30:28]	R/W	0	DRVSR15	PIN15 配置输出驱动电流
[27]	--	--	--	--
[26:24]	R/W	0	DRVSR14	PIN14 配置输出驱动电流
[23]	--	--	--	--
[22:20]	R/W	0	DRVSR13	PIN13 配置输出驱动电流
[19]	--	--	--	--
[18:16]	R/W	0	DRVSR12	PIN12 配置输出驱动电流
[15]	--	--	--	--
[14:12]	R/W	0	DRVSR11	PIN11 配置输出驱动电流
[11]	--	--	--	--
[10:8]	R/W	0	DRVSR10	PIN10 配置输出驱动电流
[7]	--	--	--	--
[6:4]	R/W	0	DRVSR9	PIN9 配置输出驱动电流
[3]	--	--	--	--
[2:0]	R/W	0	DRVSR8	PIN8 配置输出驱动电流 该位由软件置位和清除 000:4mA 001:8mA 010:16mA 011:24mA

14.3.24 Px_DRVCR 驱动强度清除寄存器(x=A,B,C)

地址:0x4800_0078/0x4800_0478/0x4800_0878

复位值: PA 0x0; PB 0x0; PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	W	0	DRVCR15	PIN15 对应位写 1，清除驱动强度配置
[14]	W	0	DRVCR14	PIN14 对应位写 1，清除驱动强度配置
[13]	W	0	DRVCR13	PIN13 对应位写 1，清除驱动强度配置
[12]	W	0	DRVCR12	PIN12 对应位写 1，清除驱动强度配置
[11]	W	0	DRVCR11	PIN11 对应位写 1，清除驱动强度配置
[10]	W	0	DRVCR10	PIN10 对应位写 1，清除驱动强度配置
[9]	W	0	DRVCR9	PIN9 对应位写 1，清除驱动强度配置
[8]	W	0	DRVCR8	PIN8 对应位写 1，清除驱动强度配置
[7]	W	0	DRVCR7	PIN7 对应位写 1，清除驱动强度配置
[6]	W	0	DRVCR6	PIN6 对应位写 1，清除驱动强度配置
[5]	W	0	DRVCR5	PIN5 对应位写 1，清除驱动强度配置
[4]	W	0	DRVCR4	PIN4 对应位写 1，清除驱动强度配置
[3]	W	0	DRVCR3	PIN3 对应位写 1，清除驱动强度配置
[2]	W	0	DRVCR2	PIN2 对应位写 1，清除驱动强度配置
[1]	W	0	DRVCR1	PIN1 对应位写 1，清除驱动强度配置
[0]	W	0	DRVCR0	PIN0 对应位写 1，清除驱动强度配置

14.3.25 Px_STSR 施密特触发置位寄存器(x=A,B,C)

地址:0x4800_0084/0x4800_0484/0x4800_0884

复位值: PA 0x0; PB 0x0; PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	R/W	0	STSR15	PIN15 对应位写 1，开启输入施密特触发
[14]	R/W	0	STSR14	PIN14 对应位写 1，开启输入施密特触发
[13]	R/W	0	STSR13	PIN13 对应位写 1，开启输入施密特触发

[12]	R/W	0	STSR12	PIN12 对应位写 1，开启输入施密特触发
[11]	R/W	0	STSR11	PIN11 对应位写 1，开启输入施密特触发
[10]	R/W	0	STSR10	PIN10 对应位写 1，开启输入施密特触发
[9]	R/W	0	STSR9	PIN9 对应位写 1，开启输入施密特触发
[8]	R/W	0	STSR8	PIN8 对应位写 1，开启输入施密特触发
[7]	R/W	0	STSR7	PIN7 对应位写 1，开启输入施密特触发
[6]	R/W	0	STSR6	PIN6 对应位写 1，开启输入施密特触发
[5]	R/W	0	STSR5	PIN5 对应位写 1，开启输入施密特触发
[4]	R/W	0	STSR4	PIN4 对应位写 1，开启输入施密特触发
[3]	R/W	0	STSR3	PIN3 对应位写 1，开启输入施密特触发
[2]	R/W	0	STSR2	PIN2 对应位写 1，开启输入施密特触发
[1]	R/W	0	STSR1	PIN1 对应位写 1，开启输入施密特触发
[0]	R/W	0	STSR0	PIN0 对应位写 1，开启输入施密特触发

14.3.26 Px_STCR 施密特触发清除寄存器(x=A,B,C)

地址:0x4800_0088/0x4800_0488/0x4800_0888

复位值: PA 0x0; PB 0x0; PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	W	0	STCR15	PIN15 对应位写 1，关闭输入施密特触发
[14]	W	0	STCR14	PIN14 对应位写 1，关闭输入施密特触发
[13]	W	0	STCR13	PIN13 对应位写 1，关闭输入施密特触发
[12]	W	0	STCR12	PIN12 对应位写 1，关闭输入施密特触发
[11]	W	0	STCR11	PIN11 对应位写 1，关闭输入施密特触发
[10]	W	0	STCR10	PIN10 对应位写 1，关闭输入施密特触发
[9]	W	0	STCR9	PIN9 对应位写 1，关闭输入施密特触发
[8]	W	0	STCR8	PIN8 对应位写 1，关闭输入施密特触发
[7]	W	0	STCR7	PIN7 对应位写 1，关闭输入施密特触发

[6]	W	0	STCR6	PIN6 对应位写 1，关闭输入施密特触发
[5]	W	0	STCR5	PIN5 对应位写 1，关闭输入施密特触发
[4]	W	0	STCR4	PIN4 对应位写 1，关闭输入施密特触发
[3]	W	0	STCR3	PIN3 对应位写 1，关闭输入施密特触发
[2]	W	0	STCR2	PIN2 对应位写 1，关闭输入施密特触发
[1]	W	0	STCR1	PIN1 对应位写 1，关闭输入施密特触发
[0]	W	0	STCR0	PIN0 对应位写 1，关闭输入施密特触发

14.3.27 Px_IESR 输入使能置位寄存器(x=A,B,C)

地址:0x4800_008C/0x4800_048C/0x4800_088C

复位值: PA 0x0; PB 0x0000_0003; PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	R/W	0	IESR15	PIN15 对应位写 1，开启 GPIO 输入路径
[14]	R/W	0	IESR14	PIN14 对应位写 1，开启 GPIO 输入路径
[13]	R/W	0	IESR13	PIN13 对应位写 1，开启 GPIO 输入路径
[12]	R/W	0	IESR12	PIN12 对应位写 1，开启 GPIO 输入路径
[11]	R/W	0	IESR11	PIN11 对应位写 1，开启 GPIO 输入路径
[10]	R/W	0	IESR10	PIN10 对应位写 1，开启 GPIO 输入路径
[9]	R/W	0	IESR9	PIN9 对应位写 1，开启 GPIO 输入路径
[8]	R/W	0	IESR8	PIN8 对应位写 1，开启 GPIO 输入路径
[7]	R/W	0	IESR7	PIN7 对应位写 1，开启 GPIO 输入路径
[6]	R/W	0	IESR6	PIN6 对应位写 1，开启 GPIO 输入路径
[5]	R/W	0	IESR5	PIN5 对应位写 1，开启 GPIO 输入路径
[4]	R/W	0	IESR4	PIN4 对应位写 1，开启 GPIO 输入路径
[3]	R/W	0	IESR3	PIN3 对应位写 1，开启 GPIO 输入路径
[2]	R/W	0	IESR2	PIN2 对应位写 1，开启 GPIO 输入路径
[1]	R/W	0	IESR1	PIN1 对应位写 1，开启 GPIO 输入路径

[0]	R/W	0	IESR0	PIN0 对应位写 1，开启 GPIO 输入路径

14.3.28 Px_IECR 输入使能清除寄存器(x=A,B,C)

地址:0x4800_0090/0x4800_0490/0x4800_0890

复位值: PA 0x0; PB 0x0; PC 0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	W	0	IECR15	PIN15 对应位写 1，关闭 GPIO 输入路径
[14]	W	0	IECR14	PIN14 对应位写 1，关闭 GPIO 输入路径
[13]	W	0	IECR13	PIN13 对应位写 1，关闭 GPIO 输入路径
[12]	W	0	IECR12	PIN12 对应位写 1，关闭 GPIO 输入路径
[11]	W	0	IECR11	PIN11 对应位写 1，关闭 GPIO 输入路径
[10]	W	0	IECR10	PIN10 对应位写 1，关闭 GPIO 输入路径
[9]	W	0	IECR9	PIN9 对应位写 1，关闭 GPIO 输入路径
[8]	W	0	IECR8	PIN8 对应位写 1，关闭 GPIO 输入路径
[7]	W	0	IECR7	PIN7 对应位写 1，关闭 GPIO 输入路径
[6]	W	0	IECR6	PIN6 对应位写 1，关闭 GPIO 输入路径
[5]	W	0	IECR5	PIN5 对应位写 1，关闭 GPIO 输入路径
[4]	W	0	IECR4	PIN4 对应位写 1，关闭 GPIO 输入路径
[3]	W	0	IECR3	PIN3 对应位写 1，关闭 GPIO 输入路径
[2]	W	0	IECR2	PIN2 对应位写 1，关闭 GPIO 输入路径
[1]	W	0	IECR1	PIN1 对应位写 1，关闭 GPIO 输入路径
[0]	W	0	IECR0	PIN0 对应位写 1，关闭 GPIO 输入路径

15 UART

15.1 概述

15.1.1 简介

UART 是一种全双工异步的串行数据交换接口，波特率及数据格式可配置并支持 DMA 功能传输数据。还是实现了其他类型的串行数据交换模式，如 LIN。数据帧支持从 LSB 或者 MSB 开始传输。数据位的极性和 TX/RX 引脚都可以灵活配置。

15.1.2 功能框图

如图 15-1 所示。

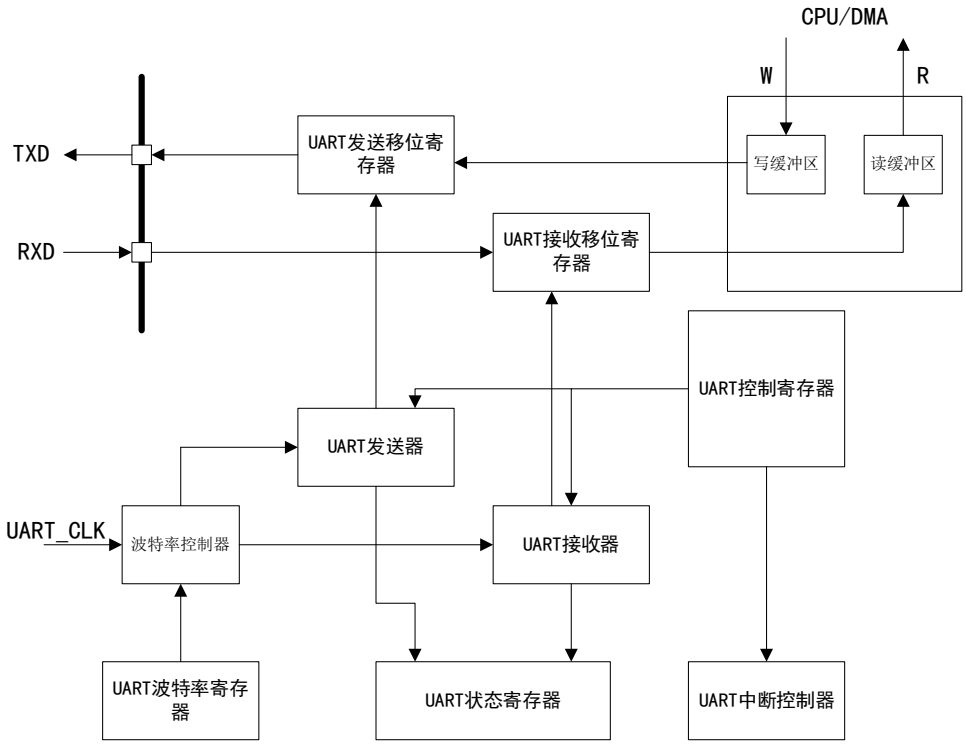


图 15-1 UART 模块功能框图

15.1.3 功能特点

- NRZ 标准格式
- 全双工异步通信
- 深度 16 级的收发 FIFO
- 收发 FIFO 水位线设置
- 可编程的波特率产生器
- 可编程的串口特性：
  - 数据位（8/9 或 10 位）
  - 奇偶校验的配置及生成和检测
  - 1 或 2 位停止位的生成和检测
  - MSB 及 LSB 的传输
- 可配置数据极性
- 可配置 DMA 访问数据缓冲区
- LIN BREAK 帧的产生和检测
- 多种状态标志：
  - 接收数据噪声标志位
  - 接收数据校验错误标志位
  - 接收数据帧格式错误标志位
  - 接收 FIFO 到达水位线标志位
  - 接收 FIFO 下溢标志位
  - 接收 FIFO 上溢标志位
  - 接收 FIFO 空状态位
  - 接收 FIFO 满状态位
  - 发送 FIFO 到达水位线标志位
  - 发送 FIFO 下溢标志位
  - 发送 FIFO 上溢标志位
  - 发送 FIFO 空状态位
  - 发送 FIFO 满状态位

15.2 功能描述

15.2.1 UART 帧格式

UART 数据帧开始于起始位，结束于停止位。UARTx_CR 寄存器中的 MODE 位可以设置数据长度。将 UARTx_CR 寄存器中的 PE 置为 1，最后一个数据位可以作校验位。若 MODE 位配置为 2'b00，第七位为校验位。若 MODE 位配置为 2'b01,第八位为校验位，UARTx_CR 寄存器中的 PT 位，用于选择校验位的类型（奇/偶校验）。

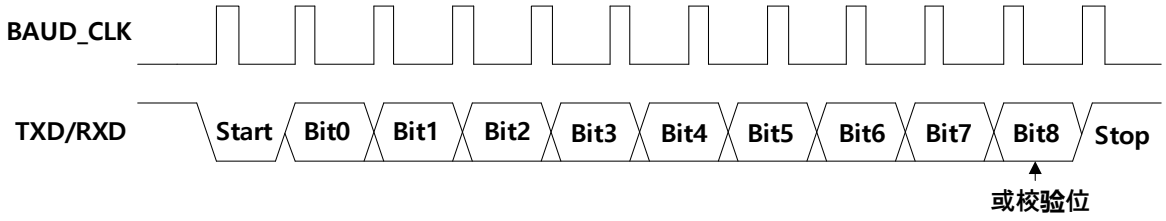


图 15-2 UART 字符帧（8 位数据位和一位停止位）

在发送和接收中，停止位可以在 UARTx_CR 寄存器中的 SBNS 位中配置。

15.2.2 波特率配置

UART 输入时钟为系统主时钟，波特率通过两级分频实现。

可以通过配置 UARTx_BBR 寄存器的 OSR 和 BR 来确定波特率。UART_CLK 时钟经过 BR 设置的分频得到过采样时钟，设置 OSR 定义一个数据位包含多少个过采样周期，得到最终的波特率时钟。

$$\text{BaudRate} = \frac{UART_CLK}{BR\ x\ (OSR+1)}$$

15.2.3 UART 发送器

在发送数据前应先通过 UARTx_CR 寄存器，配置好数据字长，停止位长度，设定好波特率。再操作 UARTx_CR 寄存器的发送使能位 TXEN 和 TXFE，通过 UARTx_DR 寄存器写入数据，向 TX FIFO 填充数据。当 TX FIFO 不为空时，发送器会通过 TX 引脚发送数据帧，TX 引脚的极性可以通过 UARTx_CR 寄存器中 TXPOL 位来配置。

15.2.4 UART 接收器

在接收数据前应先通过 UARTx_CR 寄存器，配置好数据字长，停止位长度，设定好波特率。再

操作 UARTx_CR 寄存器的接收使能位 RXEN 和 RXFE，接收器会通过 RX 引脚发送数据帧，RX 引脚的极性可以通过 UARTx_CR 寄存器中 RXPOL 位来配置。

接收器在使能后，若检测到了一个有效的起始脉冲，会开始接收码流。在接收过程中会检测噪声错误，奇偶校验错误，帧错误。可以通过 UARTx_SR/IR 查看 RX FIFO 接收状态，软件可以读取 UARTx_DR 寄存器读取 RX FIFO 中接收到的数据。

噪声检测：接收器通过三个采样点来确定一个位的值，如果有其中一个采样点与其他采样点不同，就会产生噪声错误。

奇偶校验检测：接收器在接收一个数据帧时，计算预期的奇偶校验值，并与其收到的奇偶校验位对比，如果不相等则会产生奇偶校验错误。

帧错误检测：如果在停止位传输过程中 RX 引脚为 0，将产生帧错误。

15.2.5 DMA 功能

为减轻 CPU 负担，可以采用 DMA 直接访问 TX FIFO 和 RX FIFO 协助处理数据。首先要通过 UARTx_WATER 设定 TX/RX FIFO 的水位线。再置位 UARTx_CR 中的 TXDMAE 和 RXDMAE。

当 RXSIZE 的当前数据量大于设定水位线时，发送 DMA 请求，触发 DMA 搬运，处理 RXFIFO 数据。

当 TXSIZE 的当前数据量小于设定水位线时，发送 DMA 请求，可以触发 DMA 搬运，向 TXFIFO 继续填充待发送的数据。

15.2.6 LIN 功能

UARTx_CR 寄存器的 SENBRK 位使能可以开启 LIN BREAK 帧的发送使能，LENBRK 位可以控制发送帧的字符长度，DETBRK 位可以开启 LIN BREAK 帧的接收检测。

LIN BREAK 帧长度设置由 LENBRK 位，SBNS 位以及 UARTx_DR 寄存器共同决定：

表 15-1

LENBRK	DATA[9:8]	SBNS	LIN BRAK 长度
1	00	0	12
		1	13
	01	0	13
		1	14
	10	0	14
		1	15
	11	0	15
		1	16
0	00	0	9
		1	10
	01	0	11
		1	12
	10	0	11
		1	11
	11	0	10
		1	12

LIN BREAK 检测开启时，会监测 RX 引脚上的低电平 bit 数，当发生连续 11 个低电平时，UA RTx_IR 的 LBKDF 标志位会置位。

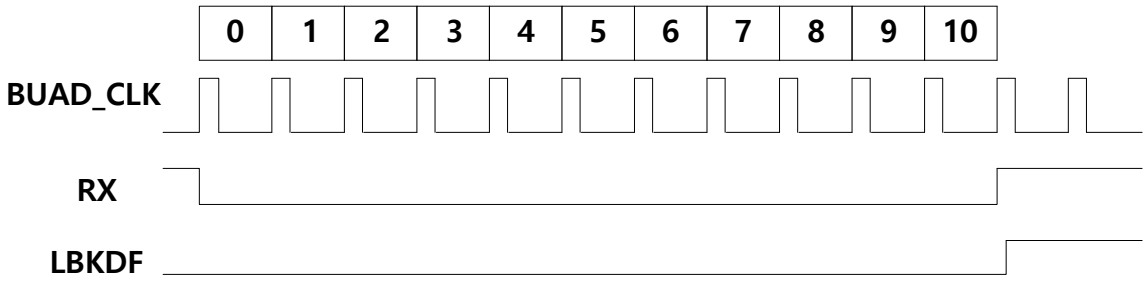


图 15-3 LIN BREAK 帧检测

15.3 UART 中断

UART 中断事件和标志位如下表 15-2 所示：

中断事件	事件标志	中断使能控制位
LIN 帧头检测中断	LBKDF	LBKDIE
接收噪声错误	RNSF	RNSIE
接收奇偶校验错误	RPEF	RPEIE
接收帧格式错误	RFEF	RFEIE
接收 FIFO 到达水位	RXDF	RXDFFIE
接收 FIFO 满	RXFULL	RXFFFIE
接收 FIFO 空	RXEMPTY	RXFFEIE
接收 FIFO 下溢	RXUDF	RXUDIE
接收 FIFO 上溢	RXOVF	RXOVIE
发送 FIFO 到达水位	TXDF	TXDFFIE
发送 FIFO 满	TXFULL	TXFFFIE
发送 FIFO 空	TXEMPTY	TXFFEIE
发送 FIFO 下溢	TXUDF	TXUDIE
发送 FIFO 上溢	TXOVF	TXOVIE

15.4 寄存器列表

UART0 在芯片中的基地址是 0x4000_4000

表 15-3 UART0 寄存器地址分配

名称	偏移地址	描述
UART0_SR	0x04	UART0 状态寄存器
UART0_IR	0x08	UART0 中断标志寄存器
UART0_IER	0x0C	UART0 中断使能寄存器
UART0_BRR	0x10	UART0 波特率设置寄存器
UART0_DR	0x14	UART0 数据寄存器

UART0_CR	0x18	UART0 控制寄存器
UART0_WATER	0x2C	UART0 水位线寄存器

UART1 在芯片中的基地址是 0x4000_4400

表 15-4 UART1 寄存器地址分配

名称	偏移地址	描述
UART1_SR	0x04	UART1 状态寄存器
UART1_IR	0x08	UART1 中断标志寄存器
UART1_IER	0x0C	UART1 中断使能寄存器
UART1_BRR	0x10	UART1 波特率设置寄存器
UART1_DR	0x14	UART1 数据寄存器
UART1_CR	0x18	UART1 控制寄存器
UART1_WATER	0x2C	UART1 水位线寄存器

15.5 UART0 寄存器描述

15.5.1 UART0_SR UART0 状态寄存器

地址:0x4000_4004

复位值:0x0005_1010

位域	类型	复位	名称	描述
[31:22]	--	--	--	--
[21]	RO	0	RXDWF	RX FIFO 到达水位线状态 1:RXFIFO 数据量到达设定水位线 0: RXFIFO 数据量未到达设定水位线
[20]	RO	0	TXDWF	TX FIFO 到达水位线状态 1:TXFIFO 数据量到达设定水位线 0: TXFIFO 数据量未到达设定水位线
[19]	RO	0	RXFULL	RX FIFO 满状态 1：RX FIFO 接收数据量满

				0：RX FIFO 接收数据量未满
[18]	RO	1	RXEMPTY	RX FIFO 空状态 1：RX FIFO 接收数据量为空 0：RX FIFO 接收数据量非空
[17]	RO	0	TXFULL	TX FIFO 满状态 1：TX FIFO 接收数据量满 0：TX FIFO 接收数据量未满
[16]	RO	1	TXEMPTY	TX FIFO 空状态 1：TX FIFO 接收数据量为空 0：TX FIFO 接收数据量非空
[15:13]	--	--	--	--
[12:8]	RO	5'h10	RXFIFOSIZE	RX FIFO 尺寸 5'h10：16 级深度 FIFO
[7:5]	--	--	--	--
[4:0]	RO	5'h10	TXFIFOSIZE	TX FIFO 尺寸 5'h10：16 级深度 FIFO

15.5.2 UART0_IR UART0 中断标志寄存器

地址:0x4000_4008

复位值:0x0

位域	类型	复位	名称	描述
[31]	RW1C	0	LBKDF	LIN 帧头检测标志位 1：检测到 LIN 帧头 0：未检测到 LIN 帧头
[30:19]	--	--	--	--
[18]	RW1C	0	RNSF	接收数据噪声标志位 1：检测到噪声错误 0：未检测到噪声错误

[17]	RW1C	0	RPEF	接收校验错误标志位 1：检测到校验错误 0：未检测到校验错误
[16]	RW1C	0	RFEF	接收帧错误标志位 1：检测到帧错误 0：未检测到帧错误
[15:13]	--	--	--	--
[12]	RW1C	0	RRXDF	接收 FIFO 到达水位线标志位 1：接收 FIFO 到达水位线 0：接收 FIFO 未到达水位线
[11:10]	--	--	--	--
[9]	RW1C	0	RXUDF	接收 FIFO 下溢标志位 1：接收 FIFO 下溢 0：接收 FIFO 未下溢 注：RXFIFO 最后一个数据读取完成，之后的数据读取可忽略
[8]	RW1C	0	RXOVF	接收 FIFO 上溢标志位 1：接收 FIFO 上溢 0：接收 FIFO 未上溢 注：RXFIFO FULL 之后写入数据，当前接收的数据会被忽略
[7:5]	--	--	--	--
[4]	RW1C	0	TXDF	发送 FIFO 到达水位线标志位 1：发送 FIFO 到达水位线 0：发送 FIFO 未到达水位线
[3:2]	--	--	--	--
[1]	RW1C	0	TXUDF	发送 FIFO 下溢标志位 1：发送 FIFO 下溢

				0：发送 FIFO 未下溢 注：TXFIFO 最后一个数据读取完成，并发送
[0]	RW1C	0	TXOVF	发送 FIFO 上溢标志位 1：发送 FIFO 上溢 0：发送 FIFO 未上溢 注：TXFIFO FULL 之后写入数据，当前发送的数据会被忽略

15.5.3 UART0_IER UART0 中断使能寄存器

地址:0x4000_400C

复位值:0x0

位域	类型	复位	名称	描述
[31]	R/W	0	LBKDIE	LIN 帧头检测中断使能 1：LIN 帧头检测标志位立起时，会进入中断 0：LIN 帧头检测中断禁止
[30:19]	--	--	--	--
[18]	R/W	0	RNSIE	接收数据噪声中断使能 1：接收数据噪声中断标志位立起时，会进入中断 0：接收噪声中断禁止
[17]	R/W	0	RPEIE	接收数据校验错误中断使能 1：接收数据校验错误中断标志位立起时，会进入中断 0：接收数据校验中断禁止
[16]	R/W	0	RFEIE	接收数据帧格式错误中断使能 1：接收数据帧格式错误标志位立起时，会进入中断 0：接收数据帧格式错误中断禁止
[15:13]	--	--	--	--
[12]	R/W	0	RXDFFIE	接收 FIFO 到达水位线中断使能 1：接收 FIFO 数据量到达水位线时，会进入中断

				0：接收 FIFO 到达水位线中断禁止
[11]	R/W	0	RXFFFIE	接收 FIFO 满中断使能 1：接收 FIFO 数据量满时，会进入中断 0：接收 FIFO 满中断禁止
[10]	R/W	0	RXFFEIE	接收 FIFO 空中断使能 1：接收 FIFO 数据空时，会进入中断 0：接收 FIFO 空中断禁止
[9]	R/W	0	RXUDIE	接收 FIFO 下溢中断使能 1：接收 FIFO 下溢中断标志位立起时，会进入中断 0：接收 FIFO 下溢中断禁止
[8]	R/W	0	RXOVIE	接收 FIFO 上溢中断使能 1：接收 FIFO 上溢中断标志位立起时，会进入中断 0：接收 FIFO 上溢中断禁止
[7:5]	--	--	--	--
[4]	R/W	0	TXDFFIE	发送 FIFO 到达水位线中断使能 1：发送 FIFO 数据量到达水位线时，会进入中断 0：发送 FIFO 到达水位线中断禁止
[3]	R/W	0	TXFFFIE	发送 FIFO 满中断使能 1：发送 FIFO 数据量满时，会进入中断 0：发送 FIFO 满中断禁止
[2]	R/W	0	TXFFEIE	发送 FIFO 空中断使能 1：发送 FIFO 数据空时，会进入中断 0：发送 FIFO 空中断禁止
[1]	R/W	0	TXUDIE	发送 FIFO 下溢中断使能 1：发送 FIFO 下溢中断标志位立起时，会进入中断 0：发送 FIFO 下溢中断禁止
[0]	R/W	0	TXOVIE	发送 FIFO 上溢中断使能 1：发送 FIFO 上溢中断标志位立起时，会进入中断

				0：发送 FIFO 上溢中断禁止

15.5.4 UART0_BRR　UART0 波特率设置寄存器

地址:0x4000_4010

复位值:0x000F_0004

位域	类型	复位	名称	描述
[31:21]	--	--	--	--
[20:16]	R/W	5'hF	OSR	设置过采样率，决定每个数据位包含多少个过采样时钟周期，从而将高速时钟转换为真正的位时钟。
[15:13]	--	--	--	--
[12:0]	R/W	13'h4	BR	设置过采样时钟的分频比，决定了内部高速时钟的频率。

最终 UART 的波特率由 OSR 和 BR 两者共同决定

$$\text{BaudRate} = \frac{UART_CLK}{BR\ x\ (OSR+1)}$$

15.5.5 UART0_DR　UART0 数据寄存器

地址:0x4000_4014

复位值:0x0

位域	类型	复位	名称	描述
[31:13]	--	--	--	--
[12]	R	0	RNSF	接收数据噪声标志位 1：检测到噪声错误 0：未检测到噪声错误
[11]	R	0	RPEF	接收校验错误标志位 1：检测到校验错误 0：未检测到校验错误
[10]	R	0	RFEF	接收帧错误标志位 1：检测到帧错误

				0：未检测到帧错误
[9:0]	R/W	0	DATA	写入：往 TX FIFO 填充数据 读取：从 RX FIFO 读出数据

15.5.6 UART0_CR　UART0 控制寄存器

地址:0x4000_4018

复位值:0x0

位域	类型	复位	名称	描述
[31]	R/W	0	LPM	回绕模式控制使能 1：使能 0：禁止
[30:27]	--	--	--	--
[26]	R/W	0	DETBRK	LIN BREAK 帧接收检测使能 1：使能 0：禁止
[25]	R/W	0	LENBRK	发送 LIN BREAK 帧字符长度 1:13~15 0:9~12 具体长度设置查看 LIN 功能章节
[24]	R/W	0	SENBRK	LIN BREAK 帧字符发送使能 1：使能 0：禁止
[23:22]	--	--	--	--
[21]	R/W	0	MSB	传输数据高位优先使能 1：使能 0：禁止
[20]	R/W	0	SBNS	停止位设置 1：2 位停止位

				0：1 位停止位
[19:18]	R/W	0	MODE	数据帧模式设置 00:8 位数据/7 位数据+1 位校验 01:9 位数据/8 位数据+1 位校验 10/11:10 位数据/9 位数据+1 位校验
[17]	R/W	0	PE	奇偶校验使能 1：使能 0：禁止
[16]	R/W	0	PT	校验类型设置 1：奇校验 0：偶校验
[15]	--	--	--	--
[14]	R/W	0	RXFLUSH	接收 FIFO 清除使能 1：使能 0：禁止
[13]	R/W	0	RXFE	接收 FIFO 使能 1：使能 0：禁止
[12]	R/W	0	RXDMAE	接收 DMA 使能 1：使能 0：禁止
[11:10]	--	--	--	--
[9]	R/W	0	RXPOL	接收 RXD 极性设置 1：极性取反 0：极性正常
[8]	R/W	0	RXEN	RX 接收使能 1：使能 0：禁止

[7]	--	--	--	--
[6]	R/W	0	TXFLUSH	发送 FIFO 清除使能 1：使能 0：禁止
[5]	R/W	0	TXFE	发送 FIFO 使能 1：使能 0：禁止
[4]	R/W	0	TXDMAE	发送 DMA 使能 1：使能 0：禁止
[3:2]	--	--	--	--
[1]	R/W	0	TXPOL	发送 TXD 极性设置 1：极性取反 0：极性正常
[0]	R/W	0	TXEN	TX 发送使能 1：使能 0：禁止

15.5.7 UART0_WATER　UART0 水位线设置寄存器

地址:0x4000_402C

复位值:0x0010_0010

位域	类型	复位	名称	描述
[31:29]	--	--	--	--
[28:24]	RO	0	RXSIZE	RX FIFO 当前数据量
[23:21]	--	--	--	--
[20:16]	RW	5'h10	RXWATER	RX 水位线设定值
[15:13]	--	--	--	--
[12:8]	RO	0	TXSIZE	TX FIFO 当前数据量

[7:5]	--	--	--	--
[4:0]	RW	5'h10	TXWATER	TX 水位线设定值

15.6 UART1 寄存器描述

15.6.1 UART1_SR　UART1 状态寄存器

地址:0x4000_4404

复位值:0x0005_1010

位域	类型	复位	名称	描述
[31:22]	--	--	--	--
[21]	RO	0	RXDWF	RX FIFO 到达水位线状态 1:RXFIFO 数据量到达设定水位线 0: RXFIFO 数据量未到达设定水位线
[20]	RO	0	TXDWF	TX FIFO 到达水位线状态 1:TXFIFO 数据量到达设定水位线 0: TXFIFO 数据量未到达设定水位线
[19]	RO	0	RXFULL	RX FIFO 满状态 1：RX FIFO 接收数据量满 0：RX FIFO 接收数据量未满
[18]	RO	1	RXEMPTY	RX FIFO 空状态 1：RX FIFO 接收数据量为空 0：RX FIFO 接收数据量非空
[17]	RO	0	TXFULL	TX FIFO 满状态 1：TX FIFO 接收数据量满 0：TX FIFO 接收数据量未满
[16]	RO	1	TXEMPTY	TX FIFO 空状态 1：TX FIFO 接收数据量为空 0：TX FIFO 接收数据量非空
[15:13]	--	--	--	--

[12:8]	RO	5'h10	RXFIFOSIZE	RX FIFO 尺寸 5'h10：16 级深度 FIFO
[7:5]	--	--	--	--
[4:0]	RO	5'h10	TXFIFOSIZE	TX FIFO 尺寸 5'h10：16 级深度 FIFO

15.6.2 UART1_IR　UART1 中断标志寄存器

地址:0x4000_4408

复位值:0x0

位域	类型	复位	名称	描述
[31]	RW1C	0	LBKDF	LIN 帧头检测标志位 1：检测到 LIN 帧头 0：未检测到 LIN 帧头
[30:19]	--	--	--	--
[18]	RW1C	0	RNSF	接收数据噪声标志位 1：检测到噪声错误 0：未检测到噪声错误
[17]	RW1C	0	RPEF	接收校验错误标志位 1：检测到校验错误 0：未检测到校验错误
[16]	RW1C	0	RFEF	接收帧错误标志位 1：检测到帧错误 0：未检测到帧错误
[15:13]	--	--	--	--
[12]	RW1C	0	RRXDF	接收 FIFO 到达水位线标志位 1：接收 FIFO 到达水位线 0：接收 FIFO 未到达水位线
[11:10]	--	--	--	--

[9]	RW1C	0	RXUDF	接收 FIFO 下溢标志位 1：接收 FIFO 下溢 0：接收 FIFO 未下溢 注：RXFIFO 最后一个数据读取完成，之后的数据读取可忽略
[8]	RW1C	0	RXOVF	接收 FIFO 上溢标志位 1：接收 FIFO 上溢 0：接收 FIFO 未上溢 注：RXFIFO FULL 之后写入数据，当前接收的数据会被忽略
[7:5]	--	--	--	--
[4]	RW1C	0	TXDF	发送 FIFO 到达水位线标志位 1：发送 FIFO 到达水位线 0：发送 FIFO 未到达水位线
[3:2]	--	--	--	--
[1]	RW1C	0	TXUDF	发送 FIFO 下溢标志位 1：发送 FIFO 下溢 0：发送 FIFO 未下溢 注：TXFIFO 最后一个数据读取完成，并发送
[0]	RW1C	0	TXOVF	发送 FIFO 上溢标志位 1：发送 FIFO 上溢 0：发送 FIFO 未上溢 注：TXFIFO FULL 之后写入数据，当前发送的数据会被忽略

15.6.3 UART1_IER UART1 中断使能寄存器

地址:0x4000_440C

复位值:0x0

位域	类型	复位	名称	描述
[31]	R/W	0	LBKDIE	LIN 帧头检测中断使能 1：LIN 帧头检测标志位立起时，会进入中断 0：LIN 帧头检测中断禁止
[30:19]	--	--	--	--
[18]	R/W	0	RNSIE	接收数据噪声中断使能 1：接收数据噪声中断标志位立起时，会进入中断 0：接收噪声中断禁止
[17]	R/W	0	RPEIE	接收数据校验错误中断使能 1：接收数据校验错误中断标志位立起时，会进入中断 0：接收数据校验中断禁止
[16]	R/W	0	RFEIE	接收数据帧格式错误中断使能 1：接收数据帧格式错误标志位立起时，会进入中断 0：接收数据帧格式错误中断禁止
[15:13]	--	--	--	--
[12]	R/W	0	RXDFFIE	接收 FIFO 到达水位线中断使能 1：接收 FIFO 数据量到达水位线时，会进入中断 0：接收 FIFO 到达水位线中断禁止
[11]	R/W	0	RXFFFIE	接收 FIFO 满中断使能 1：接收 FIFO 数据量满时，会进入中断 0：接收 FIFO 满中断禁止
[10]	R/W	0	RXFFEIE	接收 FIFO 空中断使能 1：接收 FIFO 数据空时，会进入中断 0：接收 FIFO 空中断禁止
[9]	R/W	0	RXUDIE	接收 FIFO 下溢中断使能 1：接收 FIFO 下溢中断标志位立起时，会进入中断 0：接收 FIFO 下溢中断禁止
[8]	R/W	0	RXOVIE	接收 FIFO 上溢中断使能

				1：接收 FIFO 上溢中断标志位立起时，会进入中断 0：接收 FIFO 上溢中断禁止
[7:5]	--	--	--	--
[4]	R/W	0	TXDFFIE	发送 FIFO 到达水位线中断使能 1：发送 FIFO 数据量到达水位线时，会进入中断 0：发送 FIFO 到达水位线中断禁止
[3]	R/W	0	TXFFFIE	发送 FIFO 满中断使能 1：发送 FIFO 数据量满时，会进入中断 0：发送 FIFO 满中断禁止
[2]	R/W	0	TXFFEIE	发送 FIFO 空中断使能 1：发送 FIFO 数据空时，会进入中断 0：发送 FIFO 空中断禁止
[1]	R/W	0	TXUDIE	发送 FIFO 下溢中断使能 1：发送 FIFO 下溢中断标志位立起时，会进入中断 0：发送 FIFO 下溢中断禁止
[0]	R/W	0	TXOVIE	发送 FIFO 上溢中断使能 1：发送 FIFO 上溢中断标志位立起时，会进入中断 0：发送 FIFO 上溢中断禁止

15.6.4 UART1_BRR　UART1 波特率设置寄存器

地址:0x4000_4410

复位值:0x000F_0004

位域	类型	复位	名称	描述
[31:21]	--	--	--	--
[20:16]	R/W	5'hF	OSR	设置过采样率，决定每个数据位包含多少个过采样时钟周期，从而将高速时钟转换为真正的位时钟。
[15:13]	--	--	--	--
[12:0]	R/W	13'h4	BR	设置过采样时钟的分频比，决定了内部高速时钟的频

				率。

最终 UART 的波特率由 OSR 和 BR 两者共同决定

$$\text{BaudRate} = \frac{UART_CLK}{BR\ x\ (OSR+1)}$$

15.6.5 UART1_DR UART1 数据寄存器

地址:0x4000_4414

复位值:0x0

位域	类型	复位	名称	描述
[31:14]	--	--	--	--
[12]	R	0	RNSF	接收数据噪声标志位 1：检测到噪声错误 0：未检测到噪声错误
[11]	R	0	RPEF	接收校验错误标志位 1：检测到校验错误 0：未检测到校验错误
[10]	R	0	RFEF	接收帧错误标志位 1：检测到帧错误 0：未检测到帧错误
[9:0]	R/W	0	DATA	写入：往 TX FIFO 填充数据 读取：从 RX FIFO 读出数据

15.6.6 UART1_CR UART1 控制寄存器

地址:0x4000_4418

复位值:0x0

位域	类型	复位	名称	描述
[31]	R/W	0	LPM	回绕模式控制使能 1：使能 0：禁止

[30:27]	--	--	--	--
[26]	R/W	0	DETBRK	LIN BREAK 帧接收检测使能 1：使能 0：禁止
[25]	R/W	0	LENBRK	发送 LIN BREAK 帧字符长度 1:13~15 0:9~12
[24]	R/W	0	SENBRK	LIN BREAK 帧字符发送使能 1：使能 0：禁止
[23:22]	--	--	--	--
[21]	R/W	0	MSB	传输数据高位优先使能 1：使能 0：禁止
[20]	R/W	0	SBNS	停止位设置 1：2 位停止位 0：1 位停止位
[19:18]	R/W	0	MODE	数据帧模式设置 00:8 位数据/7 位数据+1 位校验 01:9 位数据/8 位数据+1 位校验 10/11:10 位数据/9 位数据+1 位校验
[17]	R/W	0	PE	奇偶校验使能 1：使能 0：禁止
[16]	R/W	0	PT	校验类型设置 1：奇校验 0：偶校验
[15]	--	--	--	--

[14]	R/W	0	RXFLUSH	接收 FIFO 清除使能 1：使能 0：禁止
[13]	R/W	0	RXFE	接收 FIFO 使能 1：使能 0：禁止
[12]	R/W	0	RXDMAE	接收 DMA 使能 1：使能 0：禁止
[11:10]	--	--	--	--
[9]	R/W	0	RXPOL	接收 RXD 极性设置 1：极性取反 0：极性正常
[8]	R/W	0	RXEN	RX 接收使能 1：使能 0：禁止
[7]	--	--	--	--
[6]	R/W	0	TXFLUSH	发送 FIFO 清除使能 1：使能 0：禁止
[5]	R/W	0	TXFE	发送 FIFO 使能 1：使能 0：禁止
[4]	R/W	0	TXDMAE	发送 DMA 使能 1：使能 0：禁止
[3:2]	--	--	--	--
[1]	R/W	0	TXPOL	发送 TXD 极性设置

				1：极性取反 0：极性正常
[0]	R/W	0	TXEN	TX 发送使能 1：使能 0：禁止

15.6.7 UART1_WATER UART1 水位线设置寄存器

地址:0x4000_442C

复位值:0x0010_0010

位域	类型	复位	名称	描述
[31:29]	--	--	--	--
[28:24]	RO	0	RXSIZE	RX FIFO 当前数据量
[23:21]	--	--	--	--
[20:16]	RW	5'h10	RXWATER	RX 水位线设定值
[15:13]	--	--	--	--
[12:8]	RO	0	TXSIZE	TX FIFO 当前数据量
[7:5]	--	--	--	--
[4:0]	RW	5'h10	TXWATER	TX 水位线设定值

16 SPI

16.1 概述

16.1.1 简介

SPI 串行外设接口提供了基于 SPI 协议的数据发送和接收功能，该模块支持主机模式与从机模式两种工作形态，可适配各类 SPI Flash、标准 SPI 外设。

控制器内置独立收发 FIFO、DMA 硬件握手、完整中断管理；支持可编程地址宽度、传输位序、时钟极性 / 相位、SCLK 分频与接口时序，地址可配置为 1~4 字节，原生支持 SPI ROM 启动场景。

16.1.2 功能框图

如图 16-1-2 所示，内部由寄存器组、AHB 内存读从口、EILM 内存读从口、控制逻辑、SPI 仲裁器、SPI 接口六大核心单元组成。外部对接 AHB/APB 系统总线，对外引出标准 SPI 信号（SCLK、CS、MOSI、MISO）引脚。

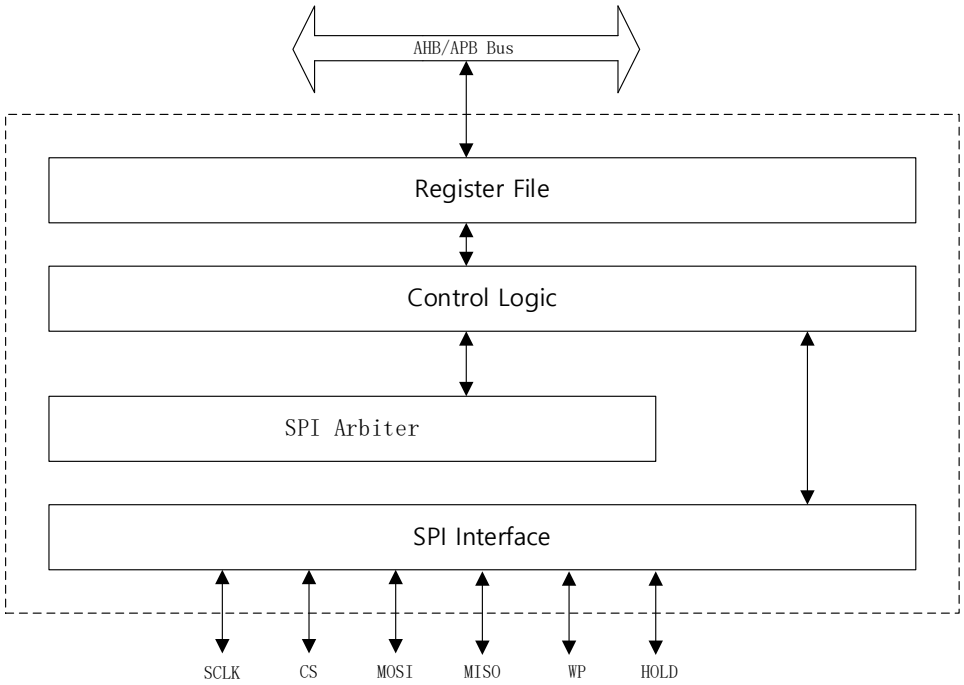


图 16-1 SPI 模块功能框图

16.1.3 功能特点

- 支持 SPI 主机、从机双工作模式
- 传输位序可配置 (MSB first) / (LSB first)
- 可编程 SPI 时钟极性 (CPOL)、时钟相位 (CPHA)，兼容全系列 SPI 工作模式
- 深度为 8 的发生和接收 FIFO
- 独立 TX/RX DMA 硬件握手，减轻 CPU 数据搬运负担
- SPI 时钟分频、CS 建立 / 保持时序可编程
- 多级中断体系：从机命令中断、传输完成中断、FIFO 水位 / 溢出 / 下溢中断，全部可独立屏蔽

16.2 功能描述

16.2.1 主机模式

通过配置命令、地址、数据寄存器组建 SPI 传输帧，支持 PIO 轮询、DMA 两种数据交互；可单独开启 / 关闭命令段、地址段、哑元段，配合 FIFO 中断实现硬件流控。

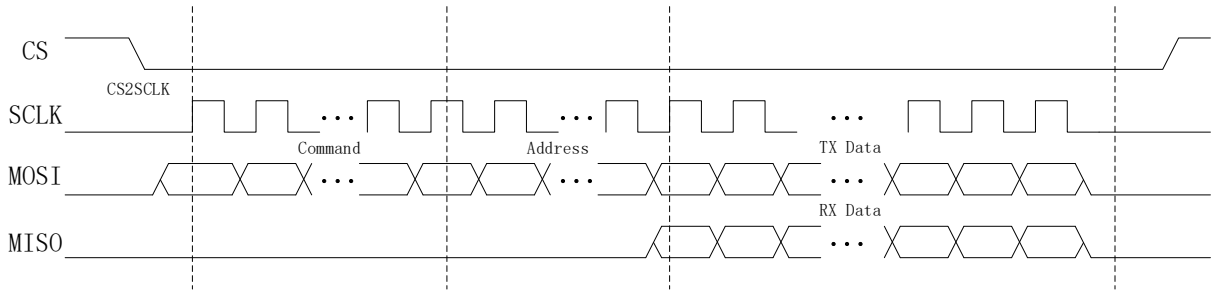


图 16-2 SPI 传输格式

16.2.2 从机模式

模块被动响应外部 SPI 主机指令。从机帧固定分为 8bit 命令段 + 8bit 哑元段 + 数据段；支持单 IO 标准指令（状态读取、数据读写）与任意 8bit 自定义指令。

模块内置从机状态寄存器、数据计数寄存器，可上报 FIFO 溢出 / 下溢、用户自定义状态；从机指令、异常事件可触发中断，实时上报总线交互状态。

16.2.3 时钟与时序配置

16.2.3.1 SCLK 时钟分频

SCLK 由外部 spi_clock 分频生成，规则：

SCLK 周期 = SPI 源时钟周期 × (SCLK_DIV + 1) × 2

当 SCLK_DIV = 0xFF 时不分频；

16.2.3.2 接口时序参数

时序基于 spi_clock 周期计算：

CS2SCLK：CS 边沿到 SCLK 首个边沿的间隔，实际时长 = (SPI 时钟 / 2) × (CS2SCLK+1)；

CSHT：CS 高电平最小保持时长，实际时长 = (SPI 时钟 / 2) × (CSHT+1) × 2；

16.2.3.3 从模式时钟约束

从机模式下，外部主机输出的 SCLK 频率不得超过 SPI 源时钟的 1/4；且 CS 到 SCLK 的间隔需大于 2 个 SPI 时钟周期。

16.2.4 DMA 功能

支持 TX/RX 独立 DMA 硬件握手：

TX FIFO 数据低于水位 → 发起 DMA 请求，DMA 应答后填充数据；

RX FIFO 数据高于水位 → 发起 DMA 请求，DMA 应答后读取数据；

DMA 总开关在 SPI 控制寄存器中独立使能。

16.2.5 SPI 中断

采用中断使能 + 中断标志架构，标志位为写 1 清零 (W1C)，所有中断可单独屏蔽：

表 16-1

中断事件	事件标志	中断使能控制位
从机命令中断	SLVCMDINT	SLVCMDEN
传输完成中断	ENDINT	ENDINTEN

TXFIFO 水位中断	TXFIFOINT	TXFIFOINTEN
RXFIFO 水位中断	RXFIFOINT	RXFIFOINTEN
TXFIFO 下溢中断	TXFIFOUDR	TXFIFOUDREN
RXFIFO 下溢中断	RXFIFOOVR	RXFIFOOVREN

16.3 寄存器列表

SPI0 在芯片中的基地址是 0x4000_3800

表 16-2 SPI0 寄存器地址分配

名称	偏移地址	描述
SPI0_IDREV	0x00	SPI0 版本寄存器
SPI0_TFMT	0x10	SPI0 传输格式寄存器
SPI0_TCTRL	0x20	SPI0 传输控制寄存器
SPI0_CMD	0x24	SPI0 命令寄存器
SPI0_ADDR	0x28	SPI0 地址寄存器
SPI0_DATA	0x2C	SPI0 数据寄存器
SPI0_CTRL	0x30	SPI0 控制寄存器
SPI0_STATUS	0x34	SPI0 状态寄存器
SPI0_IE	0x38	SPI0 中断使能寄存器
SPI0_IR	0x3C	SPI0 中断状态寄存器
SPI0_TIMING	0x40	SPI0 接口时序寄存器
SPI0_MEMCTRL	0x50	SPI0 内存访问控制寄存器
SPI0_SLVST	0x60	SPI0 从机状态寄存器
SPI0_SLVDATACNT	0x64	SPI0 从机数据计数寄存器

SPI1 在芯片中的基地址是 0x4001_3800

表 16-3 SPI1 寄存器地址分配

名称	偏移地址	描述
SPI1_IDREV	0x00	SPI1 版本寄存器
SPI1_TFMT	0x10	SPI1 传输格式寄存器
SPI1_TCTRL	0x20	SPI1 传输控制寄存器

SPI1_CMD	0x24	SPI1 命令寄存器
SPI1_ADDR	0x28	SPI1 地址寄存器
SPI1_DATA	0x2C	SPI1 数据寄存器
SPI1_CTRL	0x30	SPI1 控制寄存器
SPI1_STATUS	0x34	SPI1 状态寄存器
SPI1_IE	0x38	SPI1 中断使能寄存器
SPI1_IR	0x3C	SPI1 中断状态寄存器
SPI1_TIMING	0x40	SPI1 接口时序寄存器
SPI1_MEMCTRL	0x50	SPI1 内存访问控制寄存器
SPI1_SLVST	0x60	SPI1 从机状态寄存器
SPI1_SLVDATACNT	0x64	SPI1 从机数据计数寄存器

16.4 SPI0 寄存器描述

16.4.1 SPI0_IDREV SPI0 版本寄存器

地址:0x4000_3800

复位值:0x0200_2044

位域	类型	复位	名称	描述
[31:12]	RO	20'h2002	ID	模块设备 ID
[11:4]	RO	8'h4	MAJOR	主版本号
[3:0]	RO	4'h4	MINOR	次版本号

16.4.2 SPI0_TFMT SPI0 传输格式寄存器

地址:0x4000_3810

复位值: 0x0002_0780

位域	类型	复位	名称	描述
[31:18]	--	--	--	--
[17:16]	R/W	2'h2	ADDRLEN	地址字节数 00：1B

				01：2B 10：3B 11：4B
[15:13]	--	--	--	--
[12:8]	R/W	5'h7	DATALEN	数据位宽：DATALEN+1
[7]	R/W	1	DATAMERGE	数据合并使能，仅 DATALEN=0x7 时生效
[6:5]	--	--	--	--
[4]	R/W	0	MOSIDIR	单模式 MOSI 双向配置
[3]	R/W	0	LSB	0：MSB 优先 1：LSB 优先
[2]	R/W	0	SLVMODE	SPI 主/从模式选择 0x0：主模式 0x1：从模式
[1]	R/W	0	CPOL	SPI 时钟极性 0x0：空闲状态下串行时钟为低电平 0x1：空闲状态下串行时钟为高电平
[0]	R/W	0	CPHA	SPI 时钟相位 0x0：在串行时钟奇数边沿采样数据 0x1：在串行时钟偶数边沿采样数据

16.4.3 SPI0_TCTRL SPI0 传输控制寄存器

地址:0x4000_3820

复位值:0x0

位域	类型	复位	名称	描述
[31]	--	--	--	--
[30]	R/W	0	CMDEN	SPI 命令阶段使能 0x0：关闭命令阶段

				0x1：开启命令阶段 （仅主模式下有效）
[29]	R/W	0	ADDREN	SPI 地址阶段使能 0x0：关闭地址阶段 0x1：开启地址阶段 （仅主模式生效）
[28]	--	--	--	--
[27:24]	R/W	0	TRANSMODE	传输模式 传输序列可设置为： 0x0：同时读写 0x1：仅写入 0x2：仅读取 0x3：先写入，后读取 0x4：先读取，后写入 0x5：写入、空操作、读取 0x6：读取、空操作、写入 0x7：无数据（主模式下必须开启命令使能或地址使能） 0x8：空操作、写入 0x9：空操作、读取 0xa~0xf：保留位
[23:22]	--	--	--	--
[21]	R/W	0	TOKENEN	在 SPI 读取传输的地址阶段后追加一个单字节特殊令牌。该特殊令牌的数值需在令牌值配置项中进行设置。 0x0：禁用单字节特殊令牌 0x1：启用单字节特殊令牌 （仅主机模式生效）

[20:12]	R/W	0	WRTRANCNT	写入数据传输计数 写入传输计数寄存器（WrTranCnt）表示从数据寄存器发送至 SPI 总线的数据单元数量。 实际传输数量为（写入传输计数值 + 1）。 仅当传输模式（TransMode）取值为 0、1、3、4、5、6 或 8 时，写入传输计数寄存器才会生效。 单个数据单元的位宽由传输格式寄存器的数据长度字段（DataLen）定义。 当传输模式为 0 时，写入传输计数值必须与读取传输计数值（RdTranCnt）保持一致。
[11]	R/W	0	TOKENVALUE	SPI 读取传输中，地址阶段之后单字节特殊令牌的数值。 0x0：令牌数值 = 0x00 0x1：令牌数值 = 0x69 （仅主模式生效）
[10:9]	R/W	0	DUMMYCNT	无效数据计数。实际无效计数为（DummyCnt +1）。 SPI 接口上的无效周期数为（DummyCnt+1）*（（数据长度+1）/SPI IO 位宽）。 在无效数据阶段，数据引脚将置于高阻态。 DummyCnt 仅用于具有无效数据阶段的传输模式 5、6、8 和 9。
[8:0]	R/W	0	RDTRANCNT	读取数据的传输计数 RdTranCnt 表示需从 SPI 总线接收并存入数据寄存器的数据单元数量。实际接收的数据单元数量为（RdTranCnt+1）。 RdTranCnt 仅在传输模式为 0、2、3、4、5、6 或 9 时生效。 单个数据单元的位宽由传输格式寄存器的数据长度字

				段定义。 当传输模式为 0 时，写传输计数值必须与读传输计数值保持一致。

16.4.4 SPI0_CMD SPI0 命令寄存器

地址:0x4000_3824

复位值:0x0

位域	类型	复位	名称	描述
[31:8]	--	--	--	--
[7：0]	R/W	0	CMD	对该寄存器执行写操作会触发 SPI 传输。即便未启用命令阶段，也必须向此寄存器写入一个无效值来启动 SPI 传输。当 SPI 控制器配置为从机模式时，上一次接收的 SPI 传输数据中的命令字段会被存入本 SPI 命令寄存器。

16.4.5 SPI0_ADDR SPI0 地址寄存器

地址:0x4000_3828

复位值:0x0

位域	类型	复位	名称	描述
[31:0]	R/W	0	ADDR	SPI 地址 （仅主模式）

16.4.6 SPI0_DATA SPI0 数据寄存器

地址:0x4000_382C

复位值:0x0

位域	类型	复位	名称	描述
[31:0]	R/W	0	DATA	FIFO 数据端口 写操作：数据压入 TX FIFO；DataMerge 开启时，自动拆分多字节；TX FIFO 满时，写数据被丢弃；

				读操作：数据从 RX FIFO 弹出；DataMerge 开启时，多字节自动拼接；RX FIFO 空时，读取无效数据； 传输优先级：FIFO 数据 > 寄存器直写； 剩余数据：TX FIFO 未发完的数据会保留至下一次传输。

16.4.7 SPI0_CTRL SPI0 控制寄存器

地址:0x4000_3830

复位值:0x0

位域	类型	复位	名称	描述
[31:21]	--	--	--	--
[20:16]	R/W	0	TXTHRES	TX FIFO 水位阈值（5bit） 当 TX FIFO 有效数据数 ≤ 该值时，触发 TX FIFO 中断 / DMA 请求。
[15:13]	--	--	--	--
[12:8]	RW	0	RXTHRES	RX FIFO 水位阈值（5bit） 当 RX FIFO 有效数据数 ≥ 该值时，触发 RX FIFO 中断 / DMA 请求。
[7:5]	--	--	--	--
[4]	R/W	0	TXDMAE	TX DMA 使能 0 = 关闭 TX DMA；1 = 开启 TX DMA 握手。
[3]	R/W	--	RXDMAE	RX DMA 使能 0 = 关闭 RX DMA；1 = 开启 RX DMA 握手。
[2]	R/W	0	TXFIFORST	TX FIFO 软件复位 写 1 触发复位，硬件自动清 0； 复位行为：清空 TX FIFO、TXNUM 置 0、TXEMPTY 置 1；

				传输中禁止复位。
[1]	R/W	0	RXFIFORST	RX FIFO 软件复位 写 1 触发复位，硬件自动清 0； 复位行为：清空 RX FIFO、RXNUM 置 0、RXEMPTY 置 1； 传输中禁止复位。
[0]	R/W	0	SPIRST	SPI 全局软复位 写 1 触发，硬件自动清 0； 复位范围：所有寄存器、FIFO、状态机、中断全部恢复默认； 复位期间所有总线交互暂停。

16.4.8 SPI0_STATUS SPI0 状态寄存器

地址:0x4000_3834

复位值:0x0

位域	类型	复位	名称	描述
[31:24]	--	--	--	--
[23]	RO	0	TXFULL	TX FIFO 满标志 1 = TX FIFO 已存满（当前工程最大 8 字），禁止继续写入； 0 = TX FIFO 未满。
[22]	RO	0	TXEMPTY	TX FIFO 空标志 1 = TX FIFO 无数据；0 = 存在待发数据。
[21]	--	--	--	--
[20:16]	RO	0	TXNUM	TX FIFO 当前有效数据个数，范围 0~8。
[15]	RO	0	RXFULL	RX FIFO 满标志 1 = RX FIFO 存满，新接收数据丢弃； 0 = RX FIFO 未满。

[14]	RO	0	RXEMPTY	RX FIFO 空标志 1 = RX FIFO 无数据；0 = 存在已接收数据。
[13]	--	--	--	--
[12:8]	RO	0	RXNUM	RX FIFO 当前有效数据个数，范围 0~8。
[7:1]	--	--	--	--
[0]	RO	0	SPIACTIVE	寄存器传输忙标志（核心互斥位） 1 = 寄存器触发的 SPI 传输正在进行，禁止配置寄存器 / 内存映射； 0 = 传输空闲，可正常配置。

16.4.9 SPI0_IE SPI0 中断使能寄存器

地址:0x4000_3838

复位值:0x0

位域	类型	复位	名称	描述
[31:6]	--	--	--	--
[5]	R/W	0	SLVCMDEN	从机命令中断使能 1 = 接收到外部主机命令时触发中断（仅从机）。
[4]	R/W	0	ENDINTEN	传输完成中断使能 1 = 整组 SPI 传输结束后触发中断（主 / 从通用）。
[3]	R/W	0	TXFIFOINTEN	TX FIFO 水位中断使能 1 = TX FIFO 数据低于阈值时触发。
[2]	R/W	0	RXFIFOINTEN	RX FIFO 水位中断使能 1 = RX FIFO 数据高于阈值时触发。
[1]	R/W	0	TXFIFOUDREN	TX FIFO 下溢中断使能 仅从机模式：TX FIFO 空但主机仍请求发送，触发。
[0]	R/W	0	RXFIFOOVREN	RX FIFO 上溢中断使能 仅从机模式：RX FIFO 满但主机继续发数据，触

				发。

16.4.10 SPI0_IR SPI0 中断状态寄存器

地址:0x4000_383C

复位值:0x0

位域	类型	复位	名称	描述
[31:6]	--	--	--	--
[5]	RW1C	0	SLVCMDINT	从机命令中断标志 硬件置位：收到有效主机命令； 软件操作：写 1 清零。
[4]	RW1C	0	ENDINT	传输完成中断标志 硬件置位：一帧传输全部结束； 软件操作：写 1 清零。
[3]	RW1C	0	TXFIFOINT	发送先进先出队列阈值中断。 当发送先进先出队列阈值中断触发时，该位会被置位。 软件操作：写 1 清零。
[2]	RW1C	0	RXFIFOINT	发送先进先出队列阈值中断。 当发送先进先出队列阈值中断触发时，该位会被置位。 软件操作：写 1 清零。
[1]	RW1C	0	TXFIFOUDR	发送先进先出缓冲区下溢中断。 当发生发送先进先出缓冲区下溢中断时，该位会被置位。 （仅从机模式有效） 软件操作：写 1 清零。
[0]	RW1C	0	RXFIFOOVR	接收先进先出缓冲区溢出中断。 当接收先进先出缓冲区发生溢出中断时，该位会被

				置位。 （仅从机模式生效） 软件操作：写 1 清零。

16.4.11 SPI0_TIMING SPI0 接口时序寄存器

地址:0x4000_3840

复位值:0x0

位域	类型	复位	名称	描述
[31:14]	--	--	--	--
[13:12]	R/W	0	CS2SCLK	SPI 片选信号边沿与串行时钟信号边沿之间的最小时间间隔。实际时长为 (SCLK period/2)x(CS2SCLK+1)
[11:8]	R/W	0	CSHT	片选信号（SPI CS）保持高电平的最短时长。 实际时长为 (SCLK period/2)x(CSHT+1)
[7:0]	R/W	0	SCLK_DIV	时钟源与 SPI 接口串行时钟（SCLK）之间的时钟频率比值 SCLK period = ((SCLK_DIV+1)x2)x(Period of the SPI clocks) SCLK_DIV 的数值 0xff 为特殊值，表示串行时钟频率需与 SPI 源时钟频率保持一致。

16.4.12 SPI0_MEMCTRL SPI0 内存访问控制寄存器

地址:0x4000_3850

复位值:0x0

位域	类型	复位	名称	描述
[31:9]	--	--	--	--
[8]	RO	0	MEMCTRL	当对该寄存器（地址 0x50）或 SPI 接口时序寄存器

			CHG	（地址 0x40）执行写入操作时，该位会被置位；当新的配置生效后，该位会自动清零。
[7:4]	--	--	--	--
[3:0]	RW	0	MEMRDCMD	在 AHB/EILM 总线上执行内存映射读取操作时，选择 SPI 命令格式。

16.4.13 SPI0_SLVST SPI0 从机状态寄存器

地址:0x4000_3860

复位值:0x0

位域	类型	复位	名称	描述
[31:19]	--	--	--	--
[18]	RW1C	0	UNDERRUN	最后一笔传输出现数据下溢
[17]	RW1C	0	OVERRUN	最后一笔传输出现数据上溢
[16]	R/W	0	READY	将该位置 1，表示 SPI 已准备好进行数据传输。当除从机状态读取指令之外的 SPI 传输结束后，该位将被清零。
[15:0]	R/W	0	USR_STATUS	用户自定义状态标志

16.4.14 SPI0_SLVDATACNT SPI0 从机数据计数寄存器

地址:0x4000_3864

复位值:0x0

位域	类型	复位	名称	描述
[31:25]	--	--	--	--
[24:16]	RO	0	WCNT	从机发送数据量计数
[15:9]	--	--	--	--
[8:0]	RO	0	RCNT	从机接收数据量计数

16.5 SPI1 寄存器描述

16.5.1 SPI1_IDREV　　SPI1 版本寄存器

地址:0x4001_3800

复位值:0x0200_2044

位域	类型	复位	名称	描述
[31:12]	RO	20'h2002	ID	模块设备 ID
[11:4]	RO	8'h4	MAJOR	主版本号
[3:0]	RO	4'h4	MINOR	次版本号

16.5.2 SPI1_TFMT　　SPI1 传输格式寄存器

地址:0x4001_3810

复位值:0x0002_0780

位域	类型	复位	名称	描述
[31:18]	--	--	--	--
[17:16]	R/W	2'h2	ADDRLEN	地址字节数 00：1B 01：2B 10：3B 11：4B
[15:13]	--	--	--	--
[12:8]	R/W	5'h7	DATALEN	数据位宽：DATALEN+1
[7]	R/W	1	DATAMERGE	数据合并使能，仅 DATALEN=0x7 时生效
[6:5]	--	--	--	--
[4]	R/W	0	MOSIDIR	单模式 MOSI 双向配置

[3]	R/W	0	LSB	0：MSB 优先 1：LSB 优先
[2]	R/W	0	SLVMODE	SPI 主/从模式选择 0x0：主模式 0x1：从模式
[1]	R/W	0	CPOL	SPI 时钟极性 0x0：空闲状态下串行时钟为低电平 0x1：空闲状态下串行时钟为高电平
[0]	R/W	0	CPHA	SPI 时钟相位 0x0：在串行时钟奇数边沿采样数据 0x1：在串行时钟偶数边沿采样数据

16.5.3 SPI1_TCTRL SPI1 传输控制寄存器

地址:0x4001_3820

复位值:0x0

位域	类型	复位	名称	描述
[31]	--	--	--	--
[30]	R/W	0	CMDEN	SPI 命令阶段使能 0x0：关闭命令阶段 0x1：开启命令阶段 （仅主模式下有效）
[29]	R/W	0	ADDREN	SPI 地址阶段使能 0x0：关闭地址阶段 0x1：开启地址阶段 （仅主模式生效）
[28]	--	--	--	--
[27:24]	R/W	0	TRANSMODE	传输模式 传输序列可设置为：

				0x0：同时读写 0x1：仅写入 0x2：仅读取 0x3：先写入，后读取 0x4：先读取，后写入 0x5：写入、空操作、读取 0x6：读取、空操作、写入 0x7：无数据（主模式下必须开启命令使能或地址使能） 0x8：空操作、写入 0x9：空操作、读取 0xa~0xf：保留位
[23:22]	--	--	--	--
[21]	R/W	0	TOKENEN	在 SPI 读取传输的地址阶段后追加一个单字节特殊令牌。该特殊令牌的数值需在令牌值配置项中进行设置。 0x0：禁用单字节特殊令牌 0x1：启用单字节特殊令牌 （仅主机模式生效）
[20:12]	R/W	0	WRTRANCNT	写入数据传输计数 写入传输计数寄存器（WrTranCnt）表示从数据寄存器发送至 SPI 总线的数据单元数量。 实际传输数量为（写入传输计数值 ＋1）。 仅当传输模式（TransMode）取值为 0、1、3、4、5、6 或 8 时，写入传输计数寄存器才会生效。 单个数据单元的位宽由传输格式寄存器的数据长度字段（DataLen）定义。 当传输模式为 0 时，写入传输计数值必须与读取传输

				计数值（RdTranCnt）保持一致。
[11]	R/W	0	TOKENVALUE	SPI 读取传输中，地址阶段之后单字节特殊令牌的数值。 0x0：令牌数值 = 0x00 0x1：令牌数值 = 0x69 （仅主模式生效）
[10:9]	R/W	0	DUMMYCNT	无效数据计数。实际无效计数为（DummyCnt +1）。 SPI 接口上的无效周期数为（DummyCnt+1）*（（数据长度+1）/SPI IO 位宽）。 在无效数据阶段，数据引脚将置于高阻态。 DummyCnt 仅用于具有无效数据阶段的传输模式 5、6、8 和 9。
[8:0]	R/W	0	RDTRANCNT	读取数据的传输计数 RdTranCnt 表示需从 SPI 总线接收并存入数据寄存器的数据单元数量。实际接收的数据单元数量为（RdTranCnt+1）。 RdTranCnt 仅在传输模式为 0、2、3、4、5、6 或 9 时生效。 单个数据单元的位宽由传输格式寄存器的数据长度字段定义。 当传输模式为 0 时，写传输计数值必须与读传输计数值保持一致。

16.5.4 SPI1_CMD SPI1 命令寄存器

地址:0x4001_3824

复位值:0x0

位域	类型	复位	名称	描述
[31:8]	--	--	--	--

[7：0]	R/W	0	CMD	对该寄存器执行写操作会触发 SPI 传输。即便未启用命令阶段，也必须向此寄存器写入一个无效值来启动 SPI 传输。当 SPI 控制器配置为从机模式时，上一次接收的 SPI 传输数据中的命令字段会被存入本 SPI 命令寄存器。

16.5.5 SPI1_ADDR SPI1 地址寄存器

地址:0x4001_3828

复位值:0x0

位域	类型	复位	名称	描述
[31:0]	R/W	0	ADDR	SPI 地址 （仅主模式）

16.5.6 SPI1_DATA SPI1 数据寄存器

地址:0x4001_382C

复位值:0x0

位域	类型	复位	名称	描述
[31:0]	R/W	0	DATA	FIFO 数据端口 写操作：数据压入 TX FIFO；DataMerge 开启时，自动拆分多字节；TX FIFO 满时，写数据被丢弃； 读操作：数据从 RX FIFO 弹出；DataMerge 开启时，多字节自动拼接；RX FIFO 空时，读取无效数据； 传输优先级：FIFO 数据 > 寄存器直写； 剩余数据：TX FIFO 未发完的数据会保留至下一次传输。

16.5.7 SPI1_CTRL SPI1 控制寄存器

地址:0x4001_3830

复位值:0x0

位域	类型	复位	名称	描述
[31:21]	--	--	--	--
[20:16]	R/W	0	TXTHRES	TX FIFO 水位阈值（5bit） 当 TX FIFO 有效数据数 ≤ 该值时，触发 TX FIFO 中断 / DMA 请求。
[15:13]	--	--	--	--
[12:8]	RW	0	RXTHRES	RX FIFO 水位阈值（5bit） 当 RX FIFO 有效数据数 ≥ 该值时，触发 RX FIFO 中断 / DMA 请求。
[7:5]	--	--	--	--
[4]	R/W	0	TXDMAE	TX DMA 使能 0 = 关闭 TX DMA；1 = 开启 TX DMA 握手。
[3]	R/W	--	RXDMAE	RX DMA 使能 0 = 关闭 RX DMA；1 = 开启 RX DMA 握手。
[2]	R/W	0	TXFIFORST	TX FIFO 软件复位 写 1 触发复位，硬件自动清 0； 复位行为：清空 TX FIFO、TXNUM 置 0、TXEMPTY 置 1； 传输中禁止复位。
[1]	R/W	0	RXFIFORST	RX FIFO 软件复位 写 1 触发复位，硬件自动清 0； 复位行为：清空 RX FIFO、RXNUM 置 0、RXEMPTY 置 1； 传输中禁止复位。
[0]	R/W	0	SPIRST	SPI 全局软复位 写 1 触发，硬件自动清 0； 复位范围：所有寄存器、FIFO、状态机、中断全部

				恢复默认； 复位期间所有总线交互暂停。

16.5.8 SPI1_STATUS　SPI1 状态寄存器

地址:0x4001_3834

复位值:0x0

位域	类型	复位	名称	描述
[31:24]	--	--	--	--
[23]	RO	0	TXFULL	TX FIFO 满标志 1 = TX FIFO 已存满（当前工程最大 8 字），禁止继续写入； 0 = TX FIFO 未满。
[22]	RO	0	TXEMPTY	TX FIFO 空标志 1 = TX FIFO 无数据；0 = 存在待发数据。
[21]	--	--	--	--
[20:16]	RO	0	TXNUM	TX FIFO 当前有效数据个数，范围 0~8。
[15]	RO	0	RXFULL	RX FIFO 满标志 1 = RX FIFO 存满，新接收数据丢弃； 0 = RX FIFO 未满。
[14]	RO	0	RXEMPTY	RX FIFO 空标志 1 = RX FIFO 无数据；0 = 存在已接收数据。
[13]	--	--	--	--
[12:8]	RO	0	RXNUM	RX FIFO 当前有效数据个数，范围 0~8。
[7:1]	--	--	--	--
[0]	RO	0	SPIACTIVE	寄存器传输忙标志（核心互斥位） 1 = 寄存器触发的 SPI 传输正在进行，禁止配置寄存器 / 内存映射； 0 = 传输空闲，可正常配置。

16.5.9 SPI1_IE SPI1 中断使能寄存器

地址:0x4001_3838

复位值:0x0

位域	类型	复位	名称	描述
[31:6]	--	--	--	--
[5]	R/W	0	SLVCMDEN	从机命令中断使能 1 = 接收到外部主机命令时触发中断（仅从机）。
[4]	R/W	0	ENDINTEN	传输完成中断使能 1 = 整组 SPI 传输结束后触发中断（主 / 从通用）。
[3]	R/W	0	TXFIFOINTEN	TX FIFO 水位中断使能 1 = TX FIFO 数据低于阈值时触发。
[2]	R/W	0	RXFIFOINTEN	RX FIFO 水位中断使能 1 = RX FIFO 数据高于阈值时触发。
[1]	R/W	0	TXFIFOUDREN	TX FIFO 下溢中断使能 仅从机模式：TX FIFO 空但主机仍请求发送，触发。
[0]	R/W	0	RXFIFOOVREN	RX FIFO 上溢中断使能 仅从机模式：RX FIFO 满但主机继续发数据，触发。

16.5.10 SPI1_IR SPI1 中断状态寄存器

地址:0x4001_383C

复位值:0x0

位域	类型	复位	名称	描述
[31:6]	--	--	--	--
[5]	RW1C	0	SLVCMDINT	从机命令中断标志 硬件置位：收到有效主机命令； 软件操作：写 1 清零。

[4]	RW1C	0	ENDINT	传输完成中断标志 硬件置位：一帧传输全部结束； 软件操作：写 1 清零。
[3]	RW1C	0	TXFIFOINT	发送先进先出队列阈值中断。 当发送先进先出队列阈值中断触发时，该位会被置位。 软件操作：写 1 清零。
[2]	RW1C	0	RXFIFOINT	发送先进先出队列阈值中断。 当发送先进先出队列阈值中断触发时，该位会被置位。 软件操作：写 1 清零。
[1]	RW1C	0	TXFIFOUDR	发送先进先出缓冲区下溢中断。 当发生发送先进先出缓冲区下溢中断时，该位会被置位。 （仅从机模式有效） 软件操作：写 1 清零。
[0]	RW1C	0	RXFIFOOVR	接收先进先出缓冲区溢出中断。 当接收先进先出缓冲区发生溢出中断时，该位会被置位。 （仅从机模式生效） 软件操作：写 1 清零。

16.5.11 SPI1_TIMING SPI1 接口时序寄存器

地址:0x4001_3840

复位值:0x0

位域	类型	复位	名称	描述
[31:14]	--	--	--	--
[13:12]	R/W	0	CS2SCLK	SPI 片选信号边沿与串行时钟信号边沿之间的最小时

				间间隔。实际时长为 (SCLK period/2)x(CS2CLK+1)
[11:8]	R/W	0	CSHT	片选信号（SPI CS）保持高电平的最短时长。 实际时长为 (SCLK period/2)x(CSHT+1)
[7:0]	R/W	0	SCLK_DIV	时钟源与 SPI 接口串行时钟（SCLK）之间的时钟频率比值 SCLK period = ((SCLK_DIV+1)x2)x(Period of the SPI clocks) SCLK_DIV 的数值 0xff 为特殊值，表示串行时钟频率需与 SPI 源时钟频率保持一致。

16.5.12 SPI1_MEMCTRL SPI1 内存访问控制寄存器

地址:0x4001_3850

复位值:0x0

位域	类型	复位	名称	描述
[31:9]	--	--	--	--
[8]	RO	0	MEMCTRL CHG	当对该寄存器（地址 0x50）或 SPI 接口时序寄存器（地址 0x40）执行写入操作时，该位会被置位；当新的配置生效后，该位会自动清零。
[7:4]	--	--	--	--
[3:0]	RW	0	MEMRDC MD	在 AHB/EILM 总线上执行内存映射读取操作时，选择 SPI 命令格式。

16.5.13 SPI1_SLVST SPI1 从机状态寄存器

地址:0x4001_3860

复位值:0x0

位域	类型	复位	名称	描述

[31:19]	--	--	--	--
[18]	RW1C	0	UNDERRUN	最后一笔传输出现数据下溢
[17]	RW1C	0	OVERRUN	最后一笔传输出现数据上溢
[16]	R/W	0	READY	将该位置 1，表示 SPI 已准备好进行数据传输。 当除从机状态读取指令之外的 SPI 传输结束后，该位将被清零。
[15:0]	R/W	0	USR_STATUS	用户自定义状态标志

16.5.14 SPI1_SLVDATACNT SPI1 从机数据计数寄存器

地址:0x4001_3864

复位值:0x0

位域	类型	复位	名称	描述
[31:25]	--	--	--	--
[24:16]	RO	0	WCNT	从机发送数据量计数
[15:9]	--	--	--	--
[8:0]	RO	0	RCNT	从机接收数据量计数

16.6 应用指南

16.6.1 功能说明

主机与从机模式

SPI0/SPI1 为两个相同的 SPI 控制器，实例号为 SPI_INSTANCE0/SPI_INSTANCE1，主/从模式由 TFMT.SLVMODE 决定（0=主机，1=从机）。SDK 通过两套配置结构体分别描述：主机使用 SPI_MasterHandleTypeDef（baseConfig/transConfig/fifoDmaConfig/timingConfig 等），从机使用 SPI_SlaveHandleTypeDef（baseConfig/fifoDmaConfig/coreConfig 等）。

SDK 接口：

void spi_hal_master_init(spi_instance_e spi_instance, SPI_MasterHandleTypeDef *spi_hal_c

onfig); /*主机初始化：复位 SPI 及 FIFO、配置 TFMT/TCTRL/FIFO 水位/SCLK 分频，并使能传输结束中断*/

void spi_hal_slave_init(spi_instance_e spi_instance, SPI_SlaveHandleTypeDef *spi_hal_config); /*从机初始化：复位 SPI 及 FIFO、配置 TFMT/FIFO 水位，并使能从机命令/TX FIFO 阈值/RX FIFO 阈值中断*/

注意：从机模式下必须先置位 TFMT.SLVMODE，再配置其余传输格式，SDK 已在 spi_hal_slave_init()内部保证该顺序；从机的 CPOL/CPHA/位序/数据位宽必须与外部主机保持一致。

传输格式（CPOL/CPHA/位序/数据位宽）

通过 baseConfig 配置：CPOL（空闲时 SCLK 电平）、CPHA（采样边沿）、IsLsbFirst（0=MSB 优先，1=LSB 优先）、DataLenBit（数据位宽-1，7 表示 8bit）、AddrLenByte（地址字节数-1，0~3 对应 1~4 字节）。CPOL/CPHA 的四种组合对应 SPI Mode0~3：

Mode0：CPOL=0, CPHA=0 Mode1：CPOL=0, CPHA=1 Mode2：CPOL=1, CPHA=0 Mode3：CPOL=1, CPHA=1

注意：CPHA=0（Mode0）时，从机驱动 MISO 的第一个比特需要经过 SCLK 时钟域到 spi_clock 时钟域的同步延迟，若主机在 CS 有效后过早输出 SCLK，第一个字节可能出错。SDK demo 的处理方法是在发送缓冲区最前面固定放 1 个无效（dummy）字节，真正的数据从第 2 个字节开始，对端收到数据后丢弃第 1 个字节。

SCLK 速率配置

主机的 SCLK 速率通过 timingConfig.SCLK_SPEED（单位 Hz）设置，驱动按下式计算 TIMING.SCLK_DIV：

SCLK_DIV = (SPI 源时钟 / SCLK_SPEED) / 2 - 1 SCLK = SPI 源时钟 / ((SCLK_DIV + 1) × 2)

SPI 源时钟由 spi_hal_master_init()通过 rcc_freq_get(MODULE_SPI0/MODULE_SPI1, &clk)自动获取。SCLK_DIV 为 8bit，SCLK_SPEED 需在 SPI 源时钟的可分频范围内；从机模式下无需设置 SCLK，但外部主机的 SCLK 频率不得超过 SPI 源时钟的 1/4。

主机传输模式

主机一次传输由 TCTRL.TRANSMODE 决定数据阶段的形式，枚举为 SPI_TransMode_Ty

pedef：读写同步（SPI_TRANS_MODE_WR_SYNC）、仅写（WR_ONLY）、仅读（RD_ONLY）、写后读（WR_THEN_RD）、读后写（RD_THEN_WR）、带哑元周期的各类模式（WR_DUMMY_RD 等）以及无数据（NO_DATA）。此外可通过 transConfig 独立使能命令阶段（IsCmdEn/CmdCode）和地址阶段（IsAddrEn/TransAddr），用于访问 SPI Flash 等带命令/地址的外设。

主机传输由写 CMD 寄存器触发；SDK 的 HAL 接口在配置好 TCTRL 和数据缓冲区后写 CMD 并使能 FIFO 阈值中断，随后由 SPI 中断服务函数完成 FIFO 的填充和读取。

SDK 接口：

void spi_hal_master_send_recv_data(spi_instance_e spi_instance, transmit_data *send_data_buffer, transmit_data *recv_data_buffer); /*读写同步，收发同时进行*/

void spi_hal_master_send_data(spi_instance_e spi_instance, SPI_MasterTransConfig_Typedef transConfig, transmit_data *send_data_buffer); /*仅发送，可带命令/地址阶段*/

void spi_hal_master_recv_data(spi_instance_e spi_instance, SPI_MasterTransConfig_Typedef transConfig, transmit_data *recv_data_buffer); /*仅接收，可带命令/地址阶段*/

void spi_hal_master_change_cmd(spi_instance_e spi_instance, const uint8_t en, const uint8_t cmd); /*运行时修改命令阶段*/

void spi_hal_master_change_addr(spi_instance_e spi_instance, const uint8_t en, const uint8_t addr_len, const uint32_t addr); /*运行时修改地址阶段*/

void spi_hal_transmode_change(spi_instance_e spi_instance, uint8_t mode, uint8_t dummy_cnt); /*修改传输模式及哑元周期数*/

其中 transmit_data 结构体包含 data（缓冲区指针）和 data_len（字节数，uint8_t 类型，单次最大 255 字节）。以上接口均先等待上一次传输结束（SPIACTIVE=0）再启动新传输。

从机数据收发

从机被动响应外部主机的时钟。SDK 在 SPI 中断服务函数中处理从机收发：TX FIFO 低于水位时持续向 TX FIFO 填充待发送数据直至满，RX FIFO 高于水位时持续读取 RX FIFO 直至空，因此在高速率下可降低 TX FIFO 下溢/RX FIFO 溢出的风险。

SDK 接口：

void spi_hal_slave_send_data(spi_instance_e spi_instance, transmit_data *send_data_buffe

r); /*设置从机待发送数据（先清空 TX FIFO，再由中断重新预填充）*/

void spi_hal_slave_send_many_data(spi_instance_e spi_instance, uint8_t data_len, uint32_t data); /*批量填充测试数据（data 与 data+1 交替）*/

void spi_hal_slave_recv_buf_set(uint8_t *data_buf); /*设置从机接收缓冲区，不调用则使用内部 DATA_LENGTH(256)字节缓冲区*/

void spi_hal_slave_reset(spi_instance_e spi_instance); /*复位从机收发指针并清空 TX FIFO*/

注意：接收缓冲区不得为 NULL，且长度不得小于单次传输的字节数，否则中断服务函数会越界写入；slave_recv_buf_set()设置的缓冲区指针由 SPI0/SPI1 的从机中断共用，两个 SPI 同时作为从机时接收数据会混在一起。

FIFO 与中断

SPI 内置深度为 8 的 TX/RX FIFO，通过 fifoDmaConfig.TxThres/RxThres 设置水位：TX FIFO 数据量低于 TxThres、RX FIFO 数据量高于 RxThres 时产生对应阈值中断（INT_TX_FIFO_TH/INT_RX_FIFO_TH）。可用中断源（驱动宏定义）：

INT_RX_FIFO_OR（RX FIFO 溢出） INT_TX_FIFO_UR（TX FIFO 下溢） INT_RX_FIFO_TH（RX FIFO 阈值） INT_TX_FIFO_TH（TX FIFO 阈值） INT_TRANS_END（传输结束） INT_SLV_CMD（从机命令）

SDK 接口：

void spi_drv_interrupt_en(SPI_TypeDef *spi_ptr, uint8_t sel_int, bool en); /*使能/禁止中断源，sel_int 可按位或*/

void spi_drv_interrupt_flag_clean(SPI_TypeDef *spi_ptr, uint8_t sel_int); /*清除中断标志（写 1 清零）*/

HAL 层已提供 SPI0_IRQHandler()/SPI1_IRQHandler()，使用中断收发时须在 NVIC 中使能对应的 SPI0_IRQn/SPI1_IRQn。

DMA 收发

SPI 通过 CTRL.TXDMAE/RXDMAE 独立使能 TX/RX DMA 请求，DMA 通道在内存与 SPI 数据寄存器（DATA，偏移 0x2C）之间搬运数据，触发源为 DMA_Trigger_SPI0_TX/SPI0_RX/SPI1_TX/SPI1_RX。使用 DMA 时需要注意：

1. 必须配置 TCTRL.WRTRANCNT/RDTRANCNT（实际个数=值+1）和 TRANSMODE，告知 SPI 控制器本次传输的数据个数，否则不会触发 DMA 请求；spi_hal_master_init()/spi_hal_slave_init()不配置这些字段。

2. 主机模式下 DMA 与 TCTRL 配置完成后，还需手动写 CMD 寄存器触发传输；从机被动响应主机时钟，无需此步骤。

3. 从机模式应先启动 RX DMA，再等待主机发起传输，避免首批数据丢失。

4. SPI0/SPI1 的收发共用 DMA 通道时只能二选一使用，通道分配可在 demo 中通过 SPI_DMA_TX_CH/SPI_DMA_RX_CH 调整。

命令行测试接口

internal/cmdline/spi_cmdline.c 提供串口命令行测试接口，用于在不修改代码的情况下验证 SPI 收发：

spi_cmd_test_init <instance> <master|slave> <mode0~3> <sclk_hz> /*初始化 SPI 实例并选择工作模式(Mode0~3)和 SCLK 速率*/

spi_cmd_transmit_data <instance> <send_only|recv_only|slave_send|master_many_send|slave_many_send> <len> [data...] /*单向发送/接收*/

spi_cmd_test_write_and_recv_data <instance> <send_recv|many_send_recv> <len> [data...] /*读写同步收发并打印接收数据*/

cmd_spi_function1_test / cmd_spi_function2_test /*SPI 功能自检*/

spi_cmd_slave_trig_init / spi_cmd_trans_data_phase /*从机触发初始化、传输数据阶段测试*

16.7 快速开始

引脚配置

SPI 功能引脚均为复用功能 5（GPIO_ALT_FUNC_5），使用前需先使能 GPIO 时钟和 SPI 时钟：

SPI0：SCLK=PTB9，MISO=PTB10，MOSI=PTB11，CSN=PTB8

SPI1：CSN=PTB4，SCLK=PTB5，MISO=PTB6，MOSI=PTB7

```
gpio_settings_config_t setting_cfg;
setting_cfg.gpio_port = GPIO_B;
```

setting_cfg.gpio_pin = GPIO_PIN_9; /*SPI0_SCLK, PTB9*/

setting_cfg.gpio_alt = GPIO_ALT_FUNC_5;

setting_cfg.gpio_dir = GPIO_DIR_OUTPUT;

setting_cfg.pull_cfg = GPIO_PULL_DISABLE;

setting_cfg.gpio_strength = GPIO_STRENGTH_MODE_0;

setting_cfg.gpio_interrupt_cfg_info.int_handle = NULL;

gpio_init(&setting_cfg);

setting_cfg.gpio_pin = GPIO_PIN_8; gpio_init(&setting_cfg); /*SPI0_CSN, PTB8*/

setting_cfg.gpio_pin = GPIO_PIN_10; setting_cfg.gpio_dir = GPIO_DIR_INPUT; gpio_init(&setting_cfg); /*SPI0_MISO, PTB10*/

setting_cfg.gpio_pin = GPIO_PIN_11; setting_cfg.gpio_dir = GPIO_DIR_OUTPUT; gpio_init(&setting_cfg); /*SPI0_MOSI, PTB11*/

**demo 中不区分主/从模式，引脚方向统一按上述方式配置（SCLK/CSN/MOSI 为输出，MISO 为输入），引脚复用功能由 SPI 控制器根据主/从模式接管方向。

主机模式（中断方式）

流程：使能时钟 → 配置引脚 → 使能 NVIC 中断 → 配置并初始化主机 → 准备发送缓冲区 → 启动读写同步传输

```
flash_clk_enable();
gpio_clk_enable();
uart_clk_enable();
spi_clk_enable();
RCC_Init();
spi0_pins_init();                                   /*配置 SPI0 引脚*/
NVIC_SetPriority(SPI0_IRQn, 0);
NVIC_ClearPendingIRQ(SPI0_IRQn);
NVIC_EnableIRQ(SPI0_IRQn);
SPI_MasterHandleTypeDef spim_config = {
    .baseConfig    = { .CPOL = 0, .CPHA = 0, .IsLsbFirst = 0, .AddrLenByte = 0, .D
```

```
ataLenBit = 7 },  /*Mode0, MSB 优先, 8bit*/
        .transConfig  = { .IsCmdEn = 0, .IsAddrEn = 0 },
                /*纯数据传输，不带命令/地址*/
        .fifoDmaConfig= { .TxThres = 5UL, .RxThres = 1UL },
        .timingConfig = { .SCLK_SPEED = 1000000 }
                /*SCLK = 1MHz*/
    };
    uint8_t send_data_buf[128] = {0}, recv_data_buf[128] = {0};
    transmit_data send_data = { .data = send_data_buf, .data_len = 96 };
    transmit_data recv_data = { .data = recv_data_buf, .data_len = 96 };
    send_data_buf[0] = 0x00;                    /*dummy 字节，规避 CPHA=0
第 1 比特问题*/
    for (i = 0; i < 95; i++) { send_data_buf[1 + i] = 0x20 + i; }
    spi_hal_master_init(SPI_INSTANCE0, &spim_config);
    delay_1us(1000);
    spi_hal_master_send_recv_data(SPI_INSTANCE0, &send_data, &recv_data);  /*收发同时
进行，由中断完成数据搬运*/
    delay_1us(1000);
    INFO("RECV Data: ");
    for (i = 0; i < 128; i++) { INFO("%02X ", recv_data_buf[i]); }  /*recv_data_buf[0]为
dummy 字节，需丢弃*/
```

**数据收发由 SPI 中断服务函数完成，调用返回时传输不一定已结束，需等待足够时间或检查传输状态后再读取接收缓冲区。

从机模式（中断方式）

流程：使能时钟 → 配置引脚 → 使能 NVIC 中断 → 初始化从机 → 设置发送数据和接收缓冲区 → 等待外部主机发起传输

```
SPI_SlaveHandleTypeDef spis_config = {
    .baseConfig    = { .CPOL = 0, .CPHA = 0, .IsLsbFirst = 0, .DataLenBit = 7, .Is
```

```
MosiBiDirEn = 0 },  /*必须与主机一致*/
        .fifoDmaConfig = { .TxThres = 4, .RxThres = 1 }
    };
    spi_hal_slave_init(SPI_INSTANCE0, &spis_config);
    spi_hal_slave_send_data(SPI_INSTANCE0, &send_data);        /*预置从机待发送数据*/
    spi_hal_slave_recv_buf_set(recv_data_buf);                 /*指定接收缓冲区*/
    delay_1us(100000);                                         /*等待外部主机发起一次传输*/
    INFO("RECV Data: ");
    for (i = 0; i < 128; i++) { INFO("%02X ", recv_data_buf[i]); }
```

**主机/从机角色由 demo 中的 spi_mode 变量选择；SPI0/SPI1 由 spi_sel 变量选择，修改后重新编译即可。

DMA 方式收发

DMA 方式不使用 FIFO 阈值中断，而是通过 DMA 通道在内存与 SPI DATA 寄存器之间搬运数据，配置流程（以主机读写同步为例，参见 demo/spi_dma_demo）：

流程：复位 SPI 及 FIFO → 配置 TFMT → 配置 FIFO 水位 → 配置 TCTRL（传输模式、收发个数） → 配置 SCLK 分频 → 配置 TX/RX DMA 通道 → 使能 DMA 请求 → 主机写 CMD 触发传输

```
    spi_ptr->CTRL.SPIRST = 1;          while (1 == spi_ptr->CTRL.SPIRST) {}
    spi_ptr->CTRL.RXFIFORST = 1;       while (1 == spi_ptr->CTRL.RXFIFORST) {}
    spi_ptr->CTRL.TXFIFORST = 1;       while (1 == spi_ptr->CTRL.TXFIFORST) {}
    spi_ptr->TFMT.CPOL = 0;  spi_ptr->TFMT.CPHA = 0;
    spi_ptr->TFMT.SLVMODE = 0;                                 /*0=主机，1=从机(从机需最先置位)*/
    spi_ptr->TFMT.DATALEN = 7;  spi_ptr->TFMT.LSB = 0;
    spi_ptr->CTRL.TXTHRES = 2;  spi_ptr->CTRL.RXTHRES = 8;
    spi_ptr->TCTRL.TRANSMODE = SPI_TRANS_MODE_WR_SYNC;         /*读写同步*/
    spi_ptr->TCTRL.WRTRANCNT = (len - 1U) % 0x1FFU;            /*发送个数-1*/
```

spi_ptr->TCTRL.RDTRANCNT = (len - 1U) % 0x1FFU; /*接收个数-1*/

spi_ptr->TIMING.SCLK_DIV = (uint8_t)(((double)spi_clk / (double)sclk_speed) / 2.0 - 1.0);

/*TX 通道：内存 -> SPI DATA；RX 通道：SPI DATA -> 内存（均为 8bit，外设地址不自增）*/

g_spi_dma_tx_cfg.DMA_Trigger_periph = DMA_Trigger_SPI0_TX;

g_spi_dma_tx_cfg.src_addr = (uint32_t)txbuf; g_spi_dma_tx_cfg.dst_addr = (uint32_t)&spi_ptr->DATA;

g_spi_dma_tx_cfg.dir = DMA_DIR_MEM_TO_PERIPH; g_spi_dma_tx_cfg.transfer_size = len;

g_spi_dma_rx_cfg.DMA_Trigger_periph = DMA_Trigger_SPI0_RX;

g_spi_dma_rx_cfg.src_addr = (uint32_t)&spi_ptr->DATA; g_spi_dma_rx_cfg.dst_addr = (uint32_t)rxbuf;

g_spi_dma_rx_cfg.dir = DMA_DIR_PERIPH_TO_MEM; g_spi_dma_rx_cfg.transfer_size = len;

spi_dma_channel_start(&g_spi_dma_tx_cfg); /*DMA_CHANNEL_0*/

spi_dma_channel_start(&g_spi_dma_rx_cfg); /*DMA_CHANNEL_1*/

spi_ptr->CTRL.RXDMAE = 1; /*使能 SPI RX DMA 请求*/

spi_ptr->CTRL.TXDMAE = 1; /*使能 SPI TX DMA 请求*/

spi_ptr->CMD.CMD = 0x03; /*仅主机：写 CMD 触发传输；从机无需此步*/

while (HAL_DMA_GetRemainingCnt(DMA_CHANNEL_0) != 0U) {} /*等待 TX DMA 完成*/

while (HAL_DMA_GetRemainingCnt(DMA_CHANNEL_1) != 0U) {} /*等待 RX DMA 完成*/

**从机 DMA 初始化与主机基本一致，区别在于 TFMT.SLVMODE=1、无需配置 SCLK_DIV、无需写 CMD，且应先启动 RX DMA 通道。

17 I2C

17.1 概述

I2C 总线控制器，支持主机与从机两种工作模式，可实现 I2C 总线上的数据收发。

在主机模式下，模块作为 I2C 总线的主机，发起总线传输，生成起始、停止条件，控制 SCL 时钟，驱动 SDA 数据。

在从机模式下，模块 作为 I2C 总线的从机，响应主机的寻址，接收主机发送的数据，或者向主机发送数据。

I2C 模块内置了 Tx/Rx FIFO，用于缓存数据，支持突发传输，减少 CPU 的干预。同时，它还支持 DMA 接口，可与 DMA 控制器配合，实现无 CPU 干预的大数据量传输。

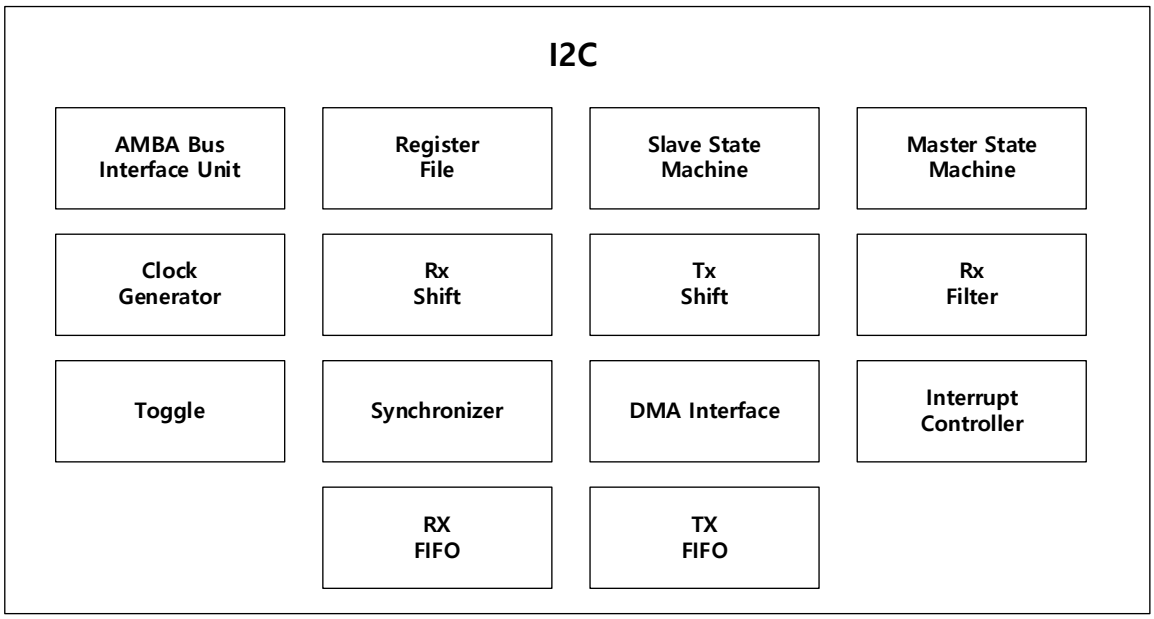


图 17-1 I2C 模块框图

17.2 特性

- 支持 7 位与 10 位地址模式
- 支持主机、从机模式
- 支持多主机仲裁
- 支持时钟同步
- 支持组合传输格式

- 支持起始字节传输
- 支持通用呼叫地址
- 内置尖峰抑制滤波器，过滤总线噪声
- 可配置的 SCL 时钟频率，最高支持 1Mbps
- 可配置的 SDA 保持时间
- 内置 Tx/Rx FIFO
- 支持 DMA 传输模式
- 支持多种中断源，可独立配置使能

17.3 I2C 专业术语

表 17-1 I2C 总线术语

术语	描述
SDA	串行数据（Serial Data），I2C 总线的数据线，双向信号
SCL	串行时钟（Serial Clock），I2C 总线的时钟线，双向信号
主机（Master）	发起总线传输、生成时钟信号的设备
从机（Slave）	响应主机寻址、接收或发送数据的设备
多主机（Multi-Master）	总线上存在多个主机，可通过仲裁决定总线控制权
仲裁（Arbitration）	总线上存在多个主机，可通过仲裁决定总线控制权
时钟同步（Clock Synchronization）	多个主机同步时钟信号的过程，确保总线时钟的一致性

表 17-2 总线传输术语

术语	描述
起始条件（START）	SCL 为高电平时，SDA 从高变低，标志着一次传输的开始
停止条件（STOP）	SCL 为高电平时，SDA 从低变高，标志着一次传输的结束
重起始条件（RESTART）	在一次传输中，不发送停止条件，直接发送新的起始条件，用于切换传输方向
应答（ACK）	接收方在接收完一个字节后，拉低 SDA，向发送方表示已接收
非应答（NACK）	接收方在接收完一个字节后，保持 SDA 为高，向发送方表

	示未接收或传输结束
字节传输	一次 8 位的数据传输，每个字节后跟随一个应答位

17.4 I2C 通讯流程

主机负责产生时钟信号并管控数据传输，从机负责与主机之间发送或接收数据。数据应答信号由接收数据的设备发出，该设备既可以是主机，也可以是从机。I2C 协议还支持 I2C 总线上挂载多个主机，并通过仲裁机制判定总线使用权。

每个从机都拥有设定的唯一地址。当主机需要与某一从机通信时，主机会先发送起始 / 重复起始信号，随后发送从机地址与读写控制位，以此确定主机是向从机发送数据，还是从从机读取数据。从机收到地址信息后，会随即发出应答脉冲。

若作为发送端的主机向作为接收端的从机写入数据，接收端每接收一个字节的数据，该数据传输过程便持续进行，直至主机发送停止信号终止此次传输。若主机作为接收端读取从机数据，则由作为发送端的从机向主机逐字节传输数据，主机收到数据后发送应答脉冲确认接收。该传输流程持续进行，直到主机接收完最后一个字节数据后发送非应答信号结束数据交互，之后主机可发送停止信号结束通信，或是发送重复起始信号后寻址其他从机。上述通信工作流程如图 17-4 所示。

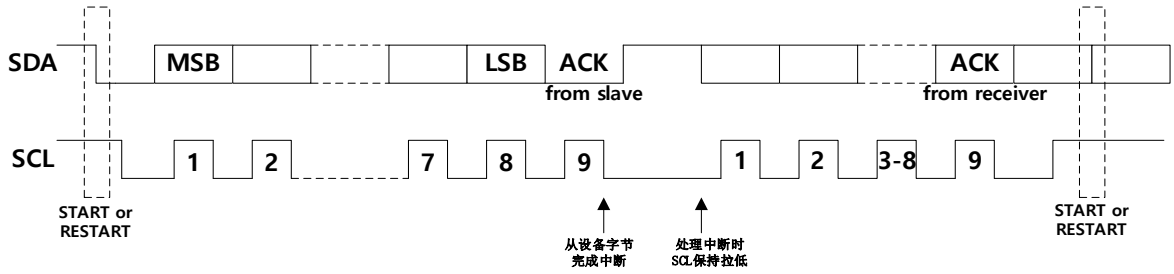


图 17-2 I2C 通信流程

17.4.1 开始与停止

在作为 I2C 主机工作时，向发送 FIFO 缓冲区写入数据会使 I2C 在总线上产生起始信号。发送 FIFO 缓冲区数据清空时，I2C 便会在总线上产生停止信号。

当 I2C 作为从设备工作时，其不会按照通信协议产生起始信号与停止信号。但若是对该 I2C 发起读取请求，它会将串行时钟线（SCL）拉低，直至获取到读取数据。这一行为会锁住 I2C 总线，直到从设备 I2C 收到读取数据，或是通过向 IC_ENABLE 寄存器的第 0 位写入

0 来禁用该 I2C 从设备。

17.4.2 组合格式

I2C 支持组合传输格式，即在一次传输中，不发送停止条件，直接发送重起始条件，切换传输方向，实现读 - 写组合传输。这种格式常用于访问 I2C 设备的寄存器，先写寄存器地址，再读寄存器数据。

17.5 I2C 协议

17.5.1 启动与停止条件

根据 I2C 协议，起始与停止条件的定义为：

- 起始条件：SCL 为高电平时，SDA 的下降沿
- 停止条件：SCL 为高电平时，SDA 的上升沿

在起始与停止条件期间，SCL 必须保持高电平，否则不会被识别为起始 / 停止条件。

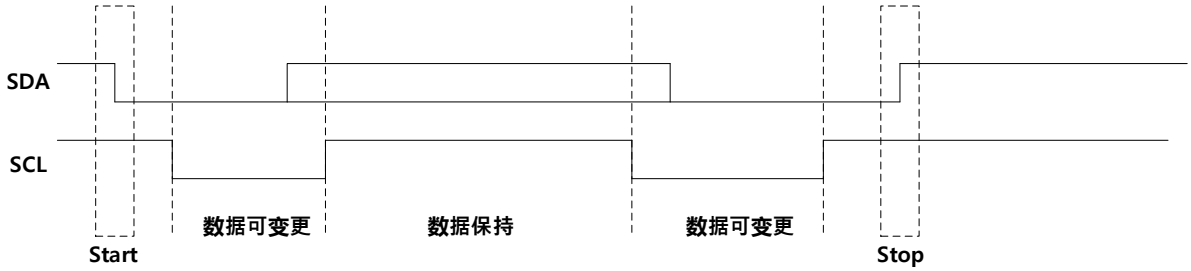


图 17-3 开始与停止状态

17.5.2 从机地址协议

I2C 支持 7 位与 10 位两种地址模式：

7 位地址模式：地址字节的高 7 位为从机地址，最低位为传输方向位（0 = 写，1 = 读）

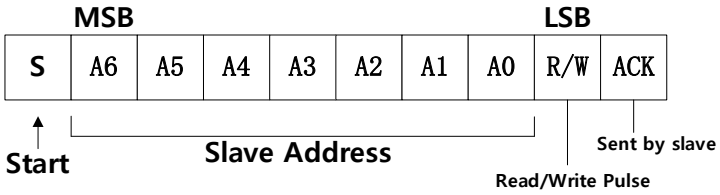


图 17-4 7BIT 地址格式

10 位地址模式：第一个地址字节的高 5 位为 11110，接下来的 2 位为 10 位地址的高 2 位，最低位为方向位；第二个字节为 10 位地址的低 8 位

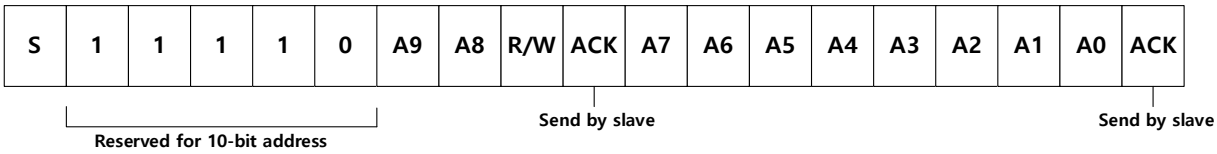


图 17-5 10BIT 地址格式

在主机模式下，I2C 会自动生成地址字节，发送给从机。在从机模式下，I2C 会检测地址字节，判断是否是寻址自己。

17.5.3 收发协议

主设备可发起与总线之间的数据收发工作，既可以作为主发送器，也可作为主接收器。从设备响应主设备发出的指令，对应完成总线数据的发送或接收工作，分别充当从发送器与从接收器。

17.5.3.1 主机发送-从机接收

所有数据均以字节格式传输，单次数据传输的字节数量无限制。当主机发送地址与读写位，或是主机向从机传输一字节数据后，接收端从机必须回复应答信号（ACK）。若接收端从机未发出应答脉冲，主机将发送停止信号终止数据传输，此时从机需将数据线 SDA 置为高电平，以便主机完成传输终止操作。

若发送端主机按照图 17-5-3-1 所示方式发送数据，那么接收端从机每接收完一字节数据后，都需向发送端主机回复一次应答脉冲。

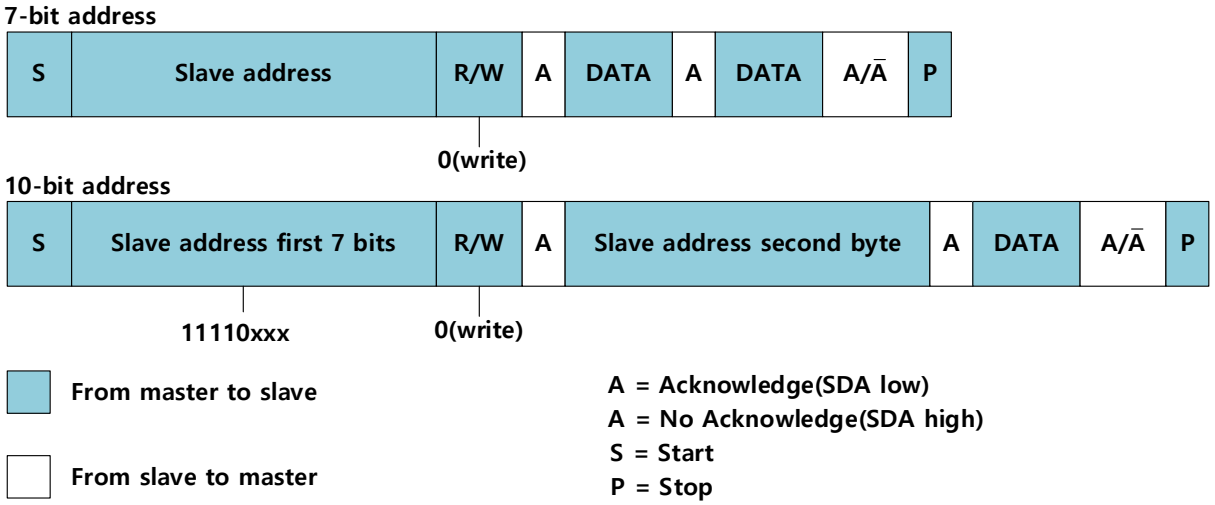


图 17-6 主机发从机收

17.5.3.2 主机接收-从机发送

若主机按照图 17-5-3-2 所示方式接收数据，那么主机在接收完一个字节的数据后，会向发送端从机发送应答脉冲，最后一个字节除外。主机接收端以此方式告知发送端从机当前已是最后一字节数据。发送端从机检测到非应答信号后，释放数据线 SDA，便于主机发送停止信号。

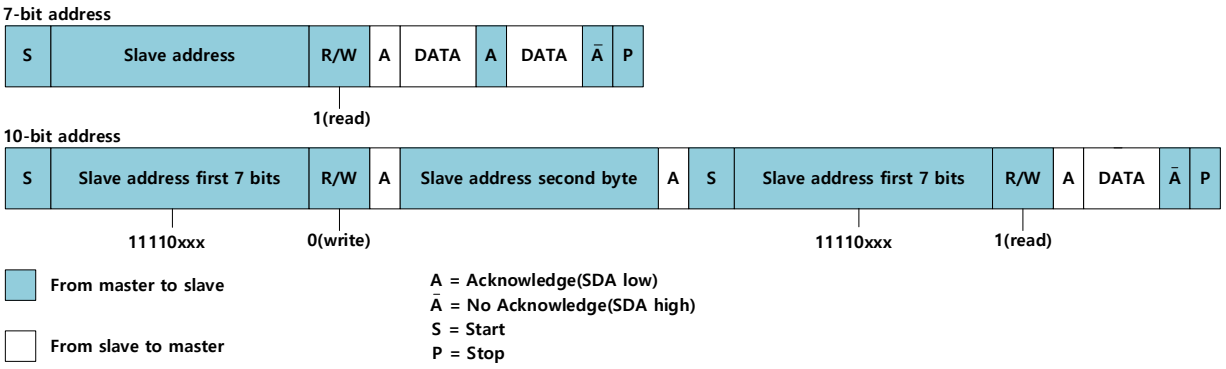


图 17-7 从机发主机收

17.5.4 起始字节传输协议

若主机按照图 17-5-4 所示方式接收数据，那么主机在接收完一个字节的数据后，会向发送端从机发送应答脉冲，最后一个字节除外。主机接收端以此方式告知发送端从机当前已是最后一字节数据。发送端从机检测到非应答信号后，释放数据线 SDA，便于主机发送停止信号。

该协议先传输七个 0，再传输一个 1，如图 8 所 示。这一设计可让轮询总线的处理器以

低采样速率对地址阶段进行采样，直至检测到 0 信号。单片机一旦检测到 0 信号，便会从低采样速率切换至主机标准采样速率。

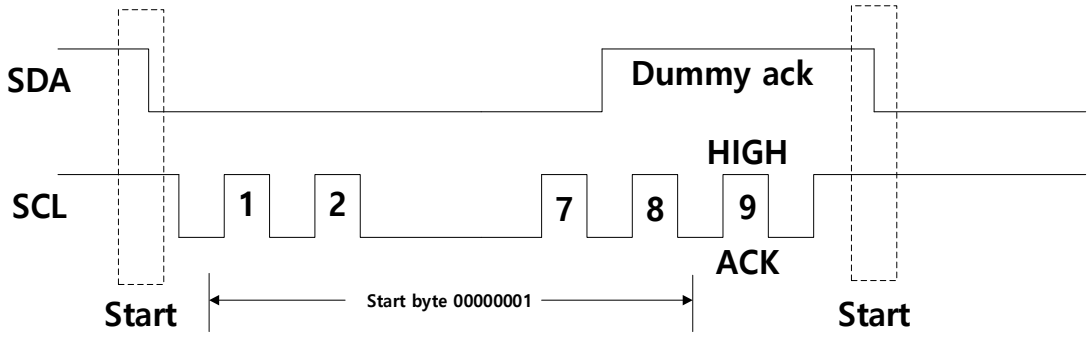


图 17-8 起始字节传输

起始字节的过程如下：

1. 主设备产生一个 START 条件。
2. 主设备发送 START 字节（0000 0001）。
3. 主设备发送 ACK 时钟脉冲。（仅为符合总线上使用的字节处理格式而存在）
4. 没有从设备将 ACK 信号设为 0。
5. 主设备产生一个 RESTART（R）条件。

硬件接收器不会响应 START 字节，因为它是一个保留地址，并在生成 RESTART 条件后重置。

17.5.5 TX FIFO

只要 TX FIFO 数据清空，在总线上就会产生停止信号。若重启信号生成功能已开启，当 TX FIFO 指令里的数据传输方向在读操作与写操作之间切换时，会生成重启信号；若未开启重启信号功能，此种情况下会先产生停止信号，再生成起始信号。

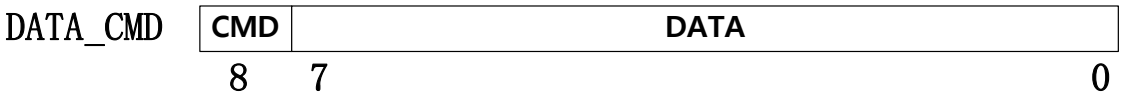


图 17-9 DATA_CMD 寄存器

DATA – 可读/可写字段；从从设备读取的数据从此字段读取；要发送到从设备的数据写入此字段。

CMD – 仅写字段；此位决定要执行的传输是读取 (CMD=1) 还是写入 (CMD=0)

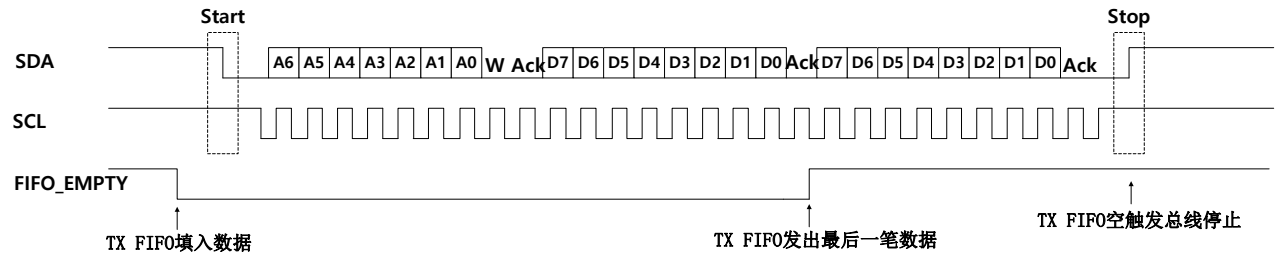


图 17-10 主机发送 Tx FIFO

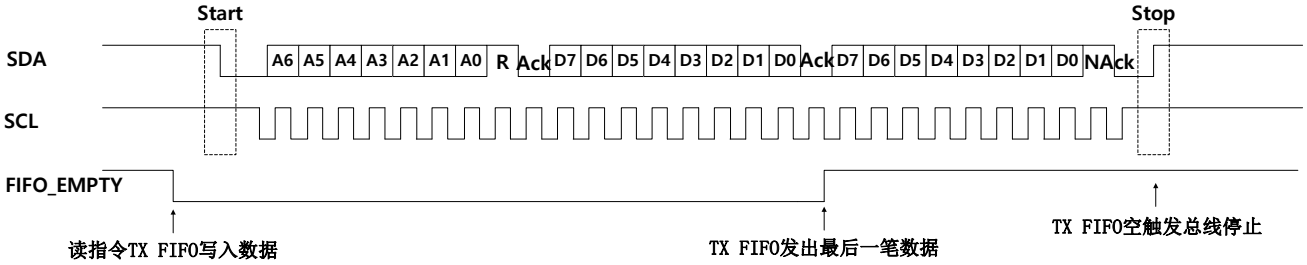


图 17-11 主机接收 Tx FIFO

17.5.6 多主机仲裁

I2C 总线协议支持同一总线上挂载多个主机。若同一条 I2C 总线上存在两个主机，当二者同时发送起始信号、尝试抢占总线控制权时，便会启动总线仲裁机制。一旦某个主机（例如微控制器）获取到总线控制权，在该主机发送停止信号并使总线进入空闲状态前，其他主机均无法夺取总线控制权。

仲裁在 SDA 数据线处于高电平、SCL 时钟线为高电平时进行。发送高电平信号而另一主机发送低电平信号的主机会仲裁失败，并关闭自身的数据输出级。仲裁失败的主机可持续输出时钟信号，直至字节传输结束。若两台主机均对同一从设备发起寻址，仲裁流程还会延续至数据传输阶段。

一旦检测到已向另一主设备仲裁失败，I2C 便会停止输出串行时钟信号（ic_clk_oe）。

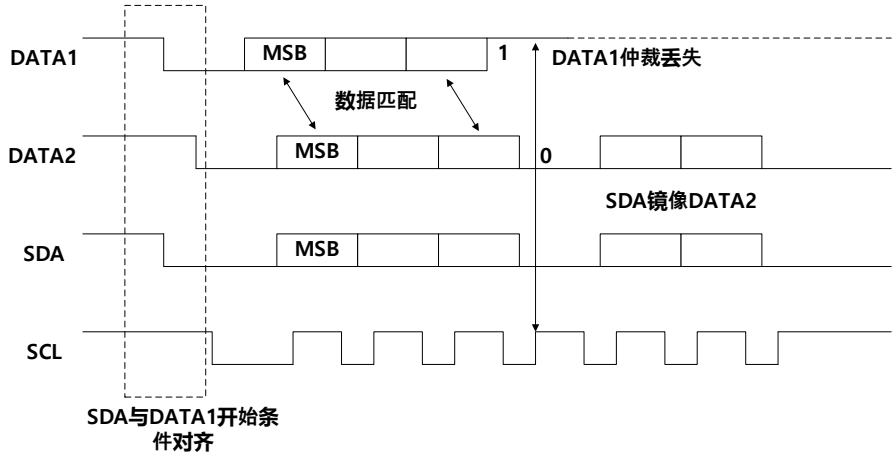


图 17-12 多主机仲裁

总线的控制权由竞争主机发送的地址、主机代码以及数据决定，因此总线上不存在中央主机，也没有优先级次序。

以下场景之间不允许进行总线仲裁：

- 重复起始条件与数据位
- 停止条件与数据位
- 重复起始条件与停止条件

从设备不参与总线仲裁流程

17.5.7 时钟同步

当两个或更多主设备尝试同时在总线上传输信息时，它们必须进行仲裁并同步 SCL 时钟。所有主设备都生成自己的时钟以传输信息。数据仅在 SCL 时钟的高电平期间有效。时钟同步通过对 SCL 信号的有线与（wired-AND）连接来实现。当主设备将 SCL 时钟切换到 0 时，主设备开始计数 SCL 时钟的低电平时间，并在下一个时钟周期开始时将 SCL 时钟信号切换到 1。然而，如果另一主设备将 SCL 线保持在 0，则该主设备进入高电平等待状态，直到 SCL 时钟线切换到 1。

随后所有主机依次输出高电平时长，高电平时长最短的主机将串行时钟线拉至低电平。接着各主机输出低电平时长，低电平时长最长的主机会迫使其余主机进入高电平等待状态，由此生成同步的串行时钟线时钟信号，具体时序如图 13 所示。此外，从设备可将串行时钟线拉低，以此放缓 I2C 总线的通信时序。

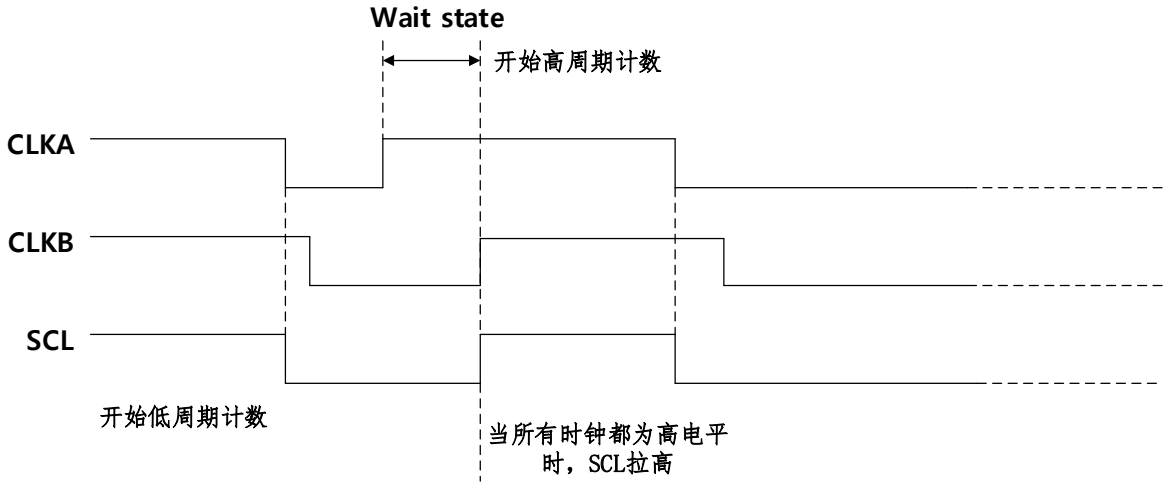


图 17-13 时钟同步

17.5.8 从模式操作

17.5.8.1 初始化配置

若将 I2C 配置为从设备，请执行以下步骤：

- 向 IC_ENABLE 寄存器的 0 位写入数值 0，禁用 DW_apb_i2c 外设
- 向 IC_SAR 寄存器（第 0 至 9 位）写入数据以设置从设备地址，该地址为 I2C 设备的响应地址。
- 向 IC_CON 寄存器写入数据，指定所支持的寻址类型（通过配置第 3 位选择 7 位寻址或 10 位寻址）。向第 6 位（IC_SLAVE_DISABLE）写入 0、向第 0 位（MASTER_MODE）写入 0，即可将 I2C 配置为仅从机模式并使其启用。

将数值 1 写入集成电路使能寄存器的第 0 位，以此开启 I2C 外设。

17.5.8.2 单字节从机发送

当总线上另一台 I2C 主设备对 I2C 发起寻址并请求数据时，I2C 将作为发送从设备，随后执行以下步骤：

- 另一路 I2C 主设备发起 I2C 传输，使用的地址与 I2C 从属地址寄存器（IC_SAR）中的从设备地址相匹配
- I2C 确认已发送的地址并识别传输方向，以此表明其工作模式为从机发送器

- I2C 触发 RD_REQ 中断（IC_RAW_INTR_STAT 寄存器的第 5 位）并保持 SCL 线为低电平。它处于等待状态，直到软件响应。如果由于 IC_INTR_MASK[5] 寄存器（M_RD_REQ 位域）被设置为 0 而屏蔽了 RD_REQ 中断，则建议使用硬件和/或软件定时程序来指示 CPU 定期读取 IC_RAW_INTR_STAT 寄存器。
  - 读取表明 IC_RAW_INTR_STAT[5]（R_RD_REQ 位字段）被设置为 1 必须被视为 RD_REQ 中断被触发的等效情况。
  - 软件必须随后执行操作以完成 I2C 传输。
  - 使用的时间间隔应约为 I2C 能处理的最快 SCL 时钟周期的 10 倍。例如，对于 400 kb/s，时间间隔为 25 微秒
- 如果在接收读取请求之前，Tx FIFO 中还有任何数据剩余，则 I2C 会触发一个 TX_ABRT 中断（IC_RAW_INTR_STAT 寄存器的第 6 位）以清除 TX FIFO 中的旧数据。如果 TX_ABRT 中断已被屏蔽，因为 IC_INTR_MASK[6] 寄存器（M_TX_ABRT 位字段）被设置为 0，则建议重新使用前一步中描述的时序例程，或类似的例程，来读取 IC_RAW_INTR_STAT 寄存器。
  - 读取结果如果显示第 6 位（R_TX_ABRT）被置为 1，必须视为 TX_ABRT 中断被触发。
  - 软件不需要进一步操作。
  - 所使用的时间间隔应类似于上一步中对 IC_RAW_INTR_STAT[5]寄存器描述的时间间隔。
- 软件将要写入的数据写入 IC_DATA_CMD 寄存器（通过在第 8 位写入“0”）。
- 软件必须在继续之前清除 IC_RAW_INTR_STAT 寄存器的 RD_REQ 和 TX_ABRT 中断（分别是第 5 位和第 6 位）。如果 RD_REQ 和/或 TX_ABRT 中断已被屏蔽，那么在 R_RD_REQ 或 R_TX_ABRT 位被读取为 1 时，IC_RAW_INTR_STAT 寄存器的清除操作已经被执行。
- I2C 释放 SCL 并传输字节。
- 主设备可以通过发出重启（RESTART）条件来占用 I2C 总线，或者通过发出停止（STOP）条件来释放总线。

17.5.8.3 单字节从机接收

当总线上有另一个 I2C 主控设备寻址 I2C 并发送数据时，I2C 会作为从设备接收器，执行以下步骤：

- 另一个 I2C 主控设备发起一个 I2C 传输，其地址与 I2C 在 IC_SAR 寄存器中的从设备地址匹配。
- I2C 对发送的地址进行应答，并识别传输方向，以指示 I2C 正在作为从设备接收器工作。
- I2C 接收传输的字节并将其放入接收缓冲区。
- I2C 会触发 RX_FULL 中断（IC_RAW_INTR_STAT[2] 寄存器）。如果 RX_FULL 中断已被屏蔽，因为设置了 IC_INTR_MASK[2] 寄存器为 0 或将 IC_TX_TL 设置为大于 0 的值，那么建议实现一个定时例程（在第 50 页的“单字节从机-发送器操作”中描述）以周期性读取 IC_STATUS 寄存器。IC_STATUS 寄存器的读取，如果第 3 位（RFNE）为 1，则软件必须将其视为 RX_FULL 中断被触发的等效情况。
- 软件可以从 IC_DATA_CMD 寄存器（第 7:0 位）读取字节。
- 另一个主设备可以通过发出 RESTART 条件来保持 I2C 总线，或通过发出 STOP 条件来释放总线。

17.5.8.4 从机批量传输操作

在标准的 I2C 协议中，所有事务都是单字节事务，程序员通过将一个字节写入从设备的 TX FIFO 来响应远程主机的读请求。当从设备（从机发送器）收到来自远程主机（主机接收器）的读请求（RD_REQ）时，至少应在从机发送器的 TX FIFO 中放入一个条目。I2C 设计用于在 TX FIFO 中处理更多数据，以便随后的读请求可以直接使用这些数据，而无需触发中断来获取更多数据。最终，这消除了如果 TX FIFO 中只允许放置一个条目时，每次获取数据所触发的中断可能导致的显著延迟的可能性。

这种模式仅在 I2C 作为从机发送器时发生。如果远程主机确认从机发送器发送的数据，并且从机的 TX FIFO 中没有数据，I2C 会将 I2C SCL 线路保持低电平，同时触发读取请求中断（RD_REQ），并等待数据写入 TX FIFO 后再发送到远程主机。

如果 RD_REQ 中断被屏蔽，由于 IC_INTR_STAT 寄存器的第 5 位 (M_RD_REQ) 被

设置为 0，则建议使用定时程序来周期性地读取 IC_RAW_INTR_STAT 寄存器。对于返回第 5 位 (R_RD_REQ) 为 1 的 IC_RAW_INTR_STAT 读取，应当视为本节中所述的 RD_REQ 中断的等效处理。这种定时程序类似于第 50 页“单字节从机发送操作”中描述的内容。

当发生读取请求时，将触发 RD_REQ 中断，并且像其他中断一样，在退出中断服务处理程序（ISR）时必须清除它。ISR 允许您向 Tx FIFO 写入 1 字节或多字节。在将这些字节传输给主机的过程中，如果主机确认了最后一个字节，那么从机必须再次触发 RD_REQ，因为主机正在请求更多数据。

如果程序员事先知道远程主控制器正在请求 n 字节的数据包，那么当另一个主控制器寻址 I2C 并请求数据时，Tx FIFO 可以写入 n 字节的数据，远程主控制器将其作为连续的数据流接收。例如，只要远程主控制器确认所发送的数据并且 Tx FIFO 中有可用数据，I2C 从设备会继续向远程主控制器发送数据。无需将 SCL 线路保持为低电平或再次发出 RD_REQ。

如果远程主机要从 I2C 接收 n 字节，但程序员向 Tx FIFO 写入的字节数大于 n，那么当从机发送完请求的 n 字节后，它会清空 Tx FIFO 并忽略任何多余的字节。

在此示例中，I2C 生成传输中止（TX_ABRT）事件以指示清除 Tx FIFO。在期望 ACK/NACK 的时候，如果收到 NACK，则远程主设备已经获取了它想要的所有数据。此时，在从设备的状态机中会触发一个标志，以清除 Tx FIFO 中剩余的数据。该标志会传输到处理器总线时钟域（FIFO 所在的时钟域），并在此时清除 Tx FIFO 的内容。

17.5.9 主模式操作

17.5.9.1 初始化配置

- 通过向 IC_ENABLE 寄存器的第 0 位写入 0 来禁用 I2C。
- 向 IC_CON 寄存器写入值以设置从操作支持的最大速度模式（位 2:1），并指定当设备为从设备时 I2C 是否以 7 位/10 位地址模式开始传输（位 3）。
- 向 IC_TAR 寄存器写入要寻址的 I2C 设备地址。同时它还指示 I2C 是否将执行通用调用或 START BYTE 命令。I2C 主控发起传输的期望速度，以及 7 位或 10 位地址模式，由 IC_10BITADDR_MASTER 位字段（位 12）控制。
- 通过在 IC_ENABLE 寄存器的第 0 位写入 1 来启用 I2C。

- 现在将传输方向和要发送的数据写入 IC_DATA_CMD 寄存器。如果在启用 I2C 之前写入 IC_DATA_CMD 寄存器，数据和命令将丢失，因为在 I2C 未启用时缓冲区会被清空。

17.5.9.2 动态 IC_TAR 或 IC_10BITADDR_MASTER 更新

I2C 支持动态更新 IC_TAR 寄存器的 IC_TAR（位 9:0）和 IC_10BITADDR_MASTER（位 12）位字段。只要软件确保在 Tx FIFO 中没有使用现有 TAR 地址的其他命令，就可以动态写入 IC_TAR 寄存器。如果软件未能确保这一点，则 IC_TAR 应该仅在满足以下条件时重新编程：

17.5.9.3 I2C 未启用 (IC_ENABLE[0]=0)；

I2C 已启用 (IC_ENABLE[0]=1)；并且当前未参与任何主设备（发送、接收）操作 (IC_STATUS[5]=0)；并且 I2C 被启用以在主模式下操作 (IC_CON[0]=1)；并且发送 FIFO 中没有数据条目 (IC_STATUS[2]=1)；

17.5.9.4 主机发送和接收

I2C 支持在读写之间动态切换。要传输数据，请将要写入的数据写入 I2C 接收/发送数据缓冲区和命令寄存器（IC_DATA_CMD）的低字节。CMD 位 [8] 在 I2C 写操作中应写入 0。随后，可以通过向 IC_DATA_CMD 寄存器的低字节写入“无关值”来发出读命令，并将 CMD 位写入 1。只要传输 FIFO 中存在命令，I2C 主机将继续发起传输。如果传输 FIFO 变为空，则将 SCL 保持为低电平，直到下一个命令写入发送 FIFO。

17.5.10 I2C 使能关闭

您可以通过 IC_ENABLE 寄存器禁用 I2C：

- 禁用后，IP 核会停止所有传输，复位内部状态
- SDA 与 SCL 线会进入高阻状态，释放总线
- 所有 FIFO 中的数据会被清空
- 所有中断标志会被复位

17.5.11 I2C 传输中止

IC_ENABLE 寄存器的 ABORT 控制位允许软件在完成从 Tx FIFO 发出的传输命令之前放弃 I2C 总线。响应 ABORT 请求时，控制器会在 I2C 总线上发送 STOP 条件，然后刷新 Tx FIFO。仅在主机模式下允许中止传输。

17.5.12 尖峰抑制滤波器

I2C 内置了尖峰抑制滤波器，用于过滤总线上的噪声尖峰：

- 滤波器可以过滤宽度小于一定阈值的尖峰，避免误检测
- 滤波器的阈值可配置，默认是 2 个 IC_CLK 周期
- 尖峰抑制滤波器对 SDA 与 SCL 线都有效，提升总线的抗干扰能力

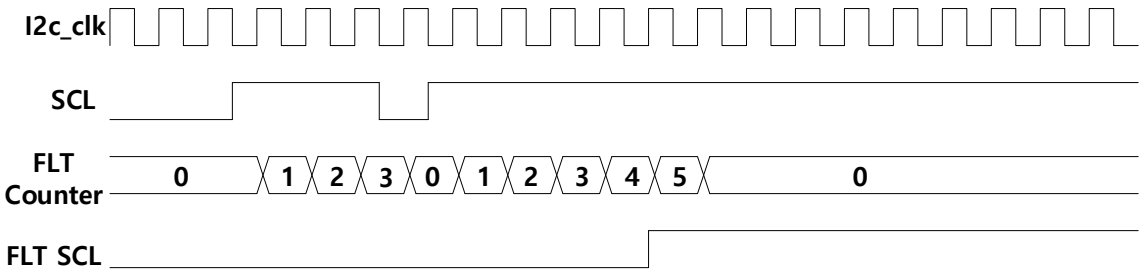


图 17-14 时钟滤波

I2C 总线规范根据工作模式规定了不同的最大脉冲长度——SS 和 FS 为 50 ns；

- 寄存器 IC_FS_SPKLEN 保存 SS 和 FS 模式的最大脉冲长度

17.5.13 快速模式增强操作

在快速模式增强下，I2C 允许将快速模式操作扩展以支持高达 1000 Kb/s 的速度。要启用快速模式增强操作，请在启动任何数据传输之前执行以下步骤：

- 将 I2C 主设备或从设备的最大速度模式配置为快速模式或高速模式 (IC_MAX_SPEED_MODE >= 2)。
- 将 ic_clk 频率设置为大于或等于 32 MHz。
- 将 IC_CON 寄存器 [2:1] 编程为 2’b10，以支持快速模式或快速模式增强。
- 编程 IC_FS_SCL_LCNT 和 IC_FS_SCL_HCNT 寄存器，以满足快速模式增强 SCL。
- 编程 IC_FS_SPKLEN 寄存器，以抑制最大 50ns 的尖峰。

- 编程 IC_SDA_SETUP 寄存器，以满足最小数据建立时间 (tSU; DAT)。

17.5.14 IC_CLK 频率配置

当 I2C 被配置为主设备时，必须在进行任何 I2C 总线传输之前设置 *CNT 寄存器，以确保正确的 I/O 时序。*CNT 寄存器有：

（1）Standard mode (0 to 100 Kb/s)

- IC_SS_SCL_HCNT
- IC_SS_SCL_LCNT

（2）Fast mode (≤ 400 Kb/s)

- IC_FS_SCL_HCNT
- IC_FS_SCL_LCNT

其中，HCNT 的配置值必须大于 SPKLEN + 5，LCNT 的配置值必须大于 SPKLEN+7。

17.5.15 SCL 计算公式

I2C 作为 Master 时，产生的 SCL 信号的高电平时间和低电平时间，也受 SCL 线的上升时间和下降时间影响，如下图 15 所示。

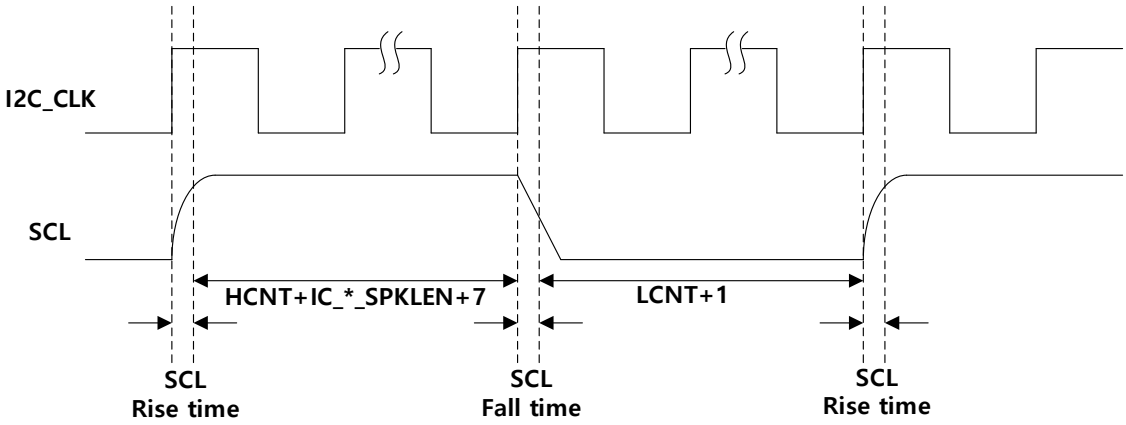


图 17-15 SCL 时钟高低电平示意图

SCL_High_time = [(HCNT + IC_*_SPKLEN + 7) * i2c_clk] + SCL_Fall_time

SCL_Low_time = [(LCNT + 1) * i2c_clk] - SCL_Fall_time + SCL_Rise_time

i2c 的内部逻辑中，在产生 SCL 高电平时，由于计数器计数以及逻辑处理等原因，高电平长度是 HCNT+SPKLEN+7；在产生 SCL 低电平时，由于计数器计数的原因，低电平长度是 LCNT+1。

i2c_clk 频率以 100MHz 为例，当时钟频率确定后，不考虑外部电路的影响，假定 I2C 速

率 100Kbps，假设 FS_ SPKLEN=5，计算出当前的 HCNT 和 LCNT 值：

i2c_clk 周期为 10ns

SCL 周期 10000ns，占空比设定 50%，高低电平各 5000ns，则：

带入高电平公式有：(SS_SCL_HCNT+5+7)*10=5000，得到 SS_SCL_HCNT=488；

带入低电平公式有：(SS_SCL_LCNT+1)*10=5000，得到 SS_SCL_LCNT=499；

17.6 DMA 功能

17.6.1 DMA 使能

要在 I2C 上启用 DMA 控制器，必须配置 DMA 控制寄存器：IC_DMA_CR，这样才能把握手信号发送到 DMA 模块 。将 1 写入 IC_DMA_CR 寄存器的 TDMAE 位字段可使能发送数据；将 1 写入 IC_DMA_CR 寄存器的 RDMAE 位字段可使能接收数据。

17.6.2 DMA 水位线设置

（1）发送数据水线设置

配置 IC_DMA_TDLR 寄存器，当 I2C TX_FIFO 中剩余的数据量达到或者低于这个水线的时候，就会发送 dma_tx_req 信号到 DMA 模块 ，DMA 模块就会搬运 brust_size 个数据到到 I2C 的 TX_FIFO 中，依次循环，直到搬运完设定的数据量。

（2）接收数据水线设置

配置 IC_DMA_RDLR 寄存器，当 RX_FIFO 中接收的数据量达到或者超过这个水线的时候，就会发送 dma_rx_req 信号到 DMA 模块，DMA 模块就会到 I2C 的 RX_FIFO 中搬走 brust_size 个数据，依次循环，直到搬运完设定的数据量。

17.7 寄存器列表

地址	寄存器	描述	备注
0x4000_5400	I2C0_CON	I2C0 控制寄存器	
0x4000_5404	I2C0_TAR	I2C0 目标地址寄存器	
0x4000_5408	I2C0_SAR	I2C0 从机地址寄存器	
0x4000_540C	I2C0_HS_MADDR	I2C0 高速模式主机地址寄存	

		器	
0x4000_5410	I2C0_DATA_CMD	I2C0 数据命令寄存器	
0x4000_5414	I2C0_SS_SCL_HCNT	I2C0 标准模式 SCL 高计数寄存器	
0x4000_5418	I2C0_SS_SCL_LCNT	I2C0 标准模式 SCL 低计数寄存器	
0x4000_541C	I2C0_FS_SCL_HCNT	I2C0 快速模式 SCL 高计数寄存器	
0x4000_5420	I2C0_FS_SCL_LCNT	I2C0 快速模式 SCL 低计数寄存器	
0x4000_542C	I2C0_INTR_STAT	I2C0 中断状态寄存器	
0x4000_5430	I2C0_INTR_MASK	I2C0 中断屏蔽寄存器	
0x4000_5434	I2C0_RAW_INTR_STAT	I2C0 中断原始状态寄存器	
0x4000_5438	I2C0_RX_TL	I2C0 接收 FIFO 阈值寄存器	
0x4000_543C	I2C0_TX_TL	I2C0 发送 FIFO 阈值寄存器	
0x4000_5440	I2C0_CLR_INTR	I2C0 组合与独立中断清除寄存器	
0x4000_5444	I2C0_CLR_RX_UNDER	I2C0 接收下溢清除寄存器	
0x4000_5448	I2C0_CLR_RX_OVER	I2C0 接收上溢清除寄存器	
0x4000_544C	I2C0_CLR_TX_OVER	I2C0 发送上溢清除寄存器	
0x4000_5450	I2C0_CLR_RD_REQ	I2C0 读请求清除寄存器	
0x4000_5454	I2C0_CLR_TX_ABRT	I2C0 发送停止清除寄存器	
0x4000_5458	I2C0_CLR_RX_DONE	I2C0 接收完成清除寄存器	
0x4000_545C	I2C0_CLR_ACTIVITY	I2C0 传输状态清除寄存器	
0x4000_5460	I2C0_CLR_STOP_DET	I2C0 停止监测清除寄存器	
0x4000_5464	I2C0_CLR_START_DET	I2C0 开始监测清除寄存器	
0x4000_5468	I2C0_CLR_GEN_CALL	I2C0 广播清除寄存器	
0x4000_546C	I2C0_ENABLE	I2C0 使能寄存器	

0x4000_5470	I2C0_STATUS	I2C0 状态寄存器	
0x4000_5474	I2C0_TXFLR	I2C0TXFIFO 级数寄存器	
0x4000_5478	I2C0_RXFLR	I2C0RXFIFO 级数寄存器	
0x4000_547C	I2C0_SDA_HOLD	I2C0SDA 保持时间寄存器	
0x4000_5480	I2C0_TX_ABRT_SOURCE	I2C0 发送停止状态寄存器	
0x4000_5484	I2C0_SLV_DATA_NACK_ONLY	I2C0 从机无应答寄存器	
0x4000_5488	I2C0_DMA_CR	I2C0DMA 控制寄存器	
0x4000_548C	I2C0_DMA_TDLR	I2C0DMA 发送水位寄存器	
0x4000_5490	I2C0_DMA_RDLR	I2C0DMA 接收水位寄存器	
0x4000_5494	I2C0_SDA_SETUP	I2C0SDA 建立时间寄存器	
0x4000_5498	I2C0_ACK_GENERAL_CALL	I2C0 广播应答寄存器	
0x4000_549C	I2C0_ENABLE_STATUS	I2C0 使能状态寄存器	
0x4000_54A0	I2C0_FS_SPKLEN	I2C0 快速模式尖峰抑制长度寄存器	
0x4000_54A4	I2C0_HS_SPKLEN	I2C0 高速模式尖峰抑制长度寄存器	
0x4000_54A8	I2C0_CLR_RESTART_DET	I2C0 清除重启监测寄存器	
0x4000_54F4	I2C0_COMP_PARAM_1	I2C0 组件参数寄存器 1	
0x4000_54F8	I2C0_COMP_VERSION	I2C0 组件版本寄存器	
0x4000_54FC	I2C0_COMP_TYPE	I2C0 组件类型寄存器	

地址	寄存器	描述	备注
0x4000_5800	I2C1_CON	I2C1 控制寄存器	
0x4000_5804	I2C1_TAR	I2C1 目标地址寄存器	
0x4000_5808	I2C1_SAR	I2C1 从机地址寄存器	
0x4000_5810	I2C1_DATA_CMD	I2C1 数据命令寄存器	
0x4000_5814	I2C1_SS_SCL_HCNT	I2C1 标准模式 SCL 高计数寄存器	

0x4000_5818	I2C1_SS_SCL_LCNT	I2C1 标准模式 SCL 低计数寄存器	
0x4000_581C	I2C1_FS_SCL_HCNT	I2C1 快速模式 SCL 高计数寄存器	
0x4000_5820	I2C1_FS_SCL_LCNT	I2C1 快速模式 SCL 低计数寄存器	
0x4000_582C	I2C1_INTR_STAT	I2C1 中断状态寄存器	
0x4000_5830	I2C1_INTR_MASK	I2C1 中断屏蔽寄存器	
0x4000_5834	I2C1_RAW_INTR_STAT	I2C1 中断原始状态寄存器	
0x4000_5838	I2C1_RX_TL	I2C1 接收 FIFO 阈值寄存器	
0x4000_583C	I2C1_TX_TL	I2C1 发送 FIFO 阈值寄存器	
0x4000_5840	I2C1_CLR_INTR	I2C1 组合与独立中断清除寄存器	
0x4000_5844	I2C1_CLR_RX_UNDER	I2C1 接收下溢清除寄存器	
0x4000_5848	I2C1_CLR_RX_OVER	I2C1 接收上溢清除寄存器	
0x4000_584C	I2C1_CLR_TX_OVER	I2C1 发送上溢清除寄存器	
0x4000_5850	I2C1_CLR_RD_REQ	I2C1 读请求清除寄存器	
0x4000_5854	I2C1_CLR_TX_ABRT	I2C1 发送停止清除寄存器	
0x4000_5858	I2C1_CLR_RX_DONE	I2C1 接收完成清除寄存器	
0x4000_585C	I2C1_CLR_ACTIVITY	I2C1 传输状态清除寄存器	
0x4000_5860	I2C1_CLR_STOP_DET	I2C1 停止监测清除寄存器	
0x4000_5864	I2C1_CLR_START_DET	I2C1 开始监测清除寄存器	
0x4000_5868	I2C1_CLR_GEN_CALL	I2C1 广播清除寄存器	
0x4000_586C	I2C1_ENABLE	I2C1 使能寄存器	
0x4000_5870	I2C1_STATUS	I2C1 状态寄存器	
0x4000_5874	I2C1_TXFLR	I2C1TXFIFO 级数寄存器	
0x4000_5878	I2C1_RXFLR	I2C1RXFIFO 级数寄存器	
0x4000_587C	I2C1_SDA_HOLD	I2C1SDA 保持时间寄存器	

0x4000_5880	I2C1_TX_ABRT_SOURCE	I2C1 发送停止状态寄存器	
0x4000_5884	I2C1_SLV_DATA_NACK_ONLY	I2C1 从机无应答寄存器	
0x4000_5888	I2C1_DMA_CR	I2C1DMA 控制寄存器	
0x4000_588C	I2C1_DMA_TDLR	I2C1DMA 发送水位寄存器	
0x4000_5890	I2C1_DMA_RDLR	I2C1DMA 接收水位寄存器	
0x4000_5894	I2C1_SDA_SETUP	I2C1SDA 建立时间寄存器	
0x4000_5898	I2C1_ACK_GENERAL_CALL	I2C1 广播应答寄存器	
0x4000_589C	I2C1_ENABLE_STATUS	I2C1 使能状态寄存器	
0x4000_58A0	I2C1_FS_SPKLEN	I2C1 快速模式尖峰抑制长度寄存器	
0x4000_58A8	I2C1_CLR_RESTART_DET	I2C1 清除重启监测寄存器	
0x4000_58F4	I2C1_COMP_PARAM_1	I2C1 组件参数寄存器 1	
0x4000_58F8	I2C1_COMP_VERSION	I2C1 组件版本寄存器	
0x4000_58FC	I2C1_COMP_TYPE	I2C1 组件类型寄存器	

17.8 寄存器描述

17.8.1 I2Cx_CON（x=0,1）　　I2C 控制寄存器

位域	类型	复位	名称	描述
[31:9]	--	--	--	--
[8]	R/W	0	TX_EMPTY_CTRL	该位用于控制发送缓冲区空中断（TX_EMPTY）的触发产生，具体规则参照原始中断状态寄存器（I2C_RAW_INTR_STAT）中的相关说明。
[7]	R/W	0	STOP_DET_IFADDRESSED	1‘b1：仅当被寻址时，触发停止检测中断。 1’b0：无论是否被寻址，均触发停止检测中断。 依赖条件：该寄存器位数值仅在从机模式下生效（主机模式配置位 = 1 位二进制数 0） 在广播地址通讯场景中，若停止检测寻址使能位配

				置为 1 位二进制数 1，即便该从机响应广播地址并发送应答信号，也不会触发停止检测中断。 仅当传输的地址与从机地址相匹配时，才会产生停止检测中断。
[6]	R/W	1	IC_SLAVE_DISABLE	该位用于控制 I2C 从设备是否禁用。也就是说，一旦复位信号生效，该位就会被配置赋值。用户可选择让设备复位后启用或禁用从设备，如此一来软件便无需额外配置从设备。默认状态下，即便处于复位状态，从设备也始终保持启用。若需在复位后禁用从设备，请将该位置 1。 当该位置 1（从设备已禁用）时，I2C 仅作为主设备工作，不会执行任何需要从设备参与的操作。 0：启用从设备 1：禁用从设备 注意：若软件将该位写为 0，则第 0 位也必须同步写为 0。
[5]	R/W	1	IC_RESTART_EN	确定作为主设备时是否可发送重复起始条件。部分老旧从设备不支持处理重复起始条件，但多项 I2C 相关操作会用到该条件。 0：关闭 1：开启 当关闭重复起始功能后，i2c 主设备将无法执行以下操作： ■ 发送起始字节

				■ 执行所有高速模式相关操作 ■ 在组合格式模式下切换传输方向 ■ 执行 10 位地址的读取操作 若用停止条件加后续起始条件替代重复起始条件，拆分后的操作会被拆解为多次 I2C 传输。一旦尝试执行上述受限操作，IC_RAW_INTR_STAT 寄存器的第 6 位（发送中止位）将会被置位。
[4]	RO	0	IC_10BITADDR_MASTER_rd_only	该位的功能由 IC_TAR 寄存器的第 12 位管控，自身为只读副本。 0：7 位寻址 1：10 位寻址
[3]	R/W	0	IC_10BITADDR_SLAVE	该位从机模式下用于控制 I2C 采用 7 位地址还是 10 位地址进行应答： 0：7 位寻址。I2C 忽略所有 10 位寻址相关传输；使用 7 位寻址时，仅比对 IC_SAR 寄存器的低 7 位。 1：10 位寻址。I2C 仅对与 IC_SAR 寄存器完整 10 位内容匹配的 10 位寻址传输做出应答。
[2:1]	R/W	2’b10	SPEED	用于控制 I2C 的工作速率；该配置仅在 I2C 工作于主机模式时生效。硬件可防止软件写入非法配置值。本寄存器仅可配置 1～2 范围内的数值；若写入超出该范围的数据，硬件会自动将寄存器改写为 2。 ■ 1：标准模式（0～100 Kb/s） ■ 2：快速模式（≤400 Kb/s）或增强快速模式（≤1000 Kb/s）
[0]	R/W	1	MASTER_M	该位用于控制 I2C 主机是否启用。

			ODE	0：主机禁用 1：主机启用 注：软件需保证若该位写入数值‘1’，则第 6 位同样需要写入数值‘1’。

17.8.2 I2Cx_TAR（x=0,1） I2C 目标地址寄存器

位域	类型	复位	名称	描述
[31:13]	--	--	--	--
[12]	R/W	0	IC_10BITADDR_MASTER	该位用于控制 I2C 作为主机发起数据传输时，采用 7 位地址模式还是 10 位地址模式。 ■ 0：7 位寻址 ■ 1：10 位寻址
[11]	R/W	0	SPECIAL	该位用于标识软件执行通用呼叫指令还是起始字节指令。 ■ 0：忽略第 10 位通用呼叫/起始位，正常使用目标地址寄存器 ■ 1：依据通用呼叫/起始位的配置执行专用 I2C 指令
[10]	R/W	0	GC_OR_START	若第 11 位（SPECIAL 位）置 1，则该位用于标识 I2C 需要执行通用呼叫指令还是起始字节指令。 ■ 0：通用呼叫地址——发送通用呼叫地址后，仅可执行写操作。若尝试发起读指令，会将 IC_RAW_INTR_STAT 寄存器的第 6 位（TX_ABRT）置位。I2C 将持续保持通用呼叫模式，直至清除 SPECIAL 位（第 11 位）。 ■ 1：起始字节
[9:0]	R/W	10’h	IC_TAR	该地址为所有主机传输的目标地址。发送通用呼叫

		55		时，控制器会忽略这些位。若要生成起始字节，中央处理器仅需对这些位执行一次写入操作。 若目标地址寄存器（IC_TAR）与源地址寄存器（IC_SAR）配置数值一致，器件可进入环回模式，但主机与从机共用先进先出缓存区，因此无法实现全双工环回。器件仅支持单工单向环回模式，不支持双工模式。主机无法向自身发送数据，仅可向从设备发起数据传输。

17.8.3 I2Cx_SAR（x=0,1） I2C 从机地址寄存器

位域	类型	复位	名称	描述
[31:10]	--	--	--	--
[9:0]	R/W	10’h 55	IC_SAR	从机地址寄存器（IC_SAR）在 I2C 作为从设备工作时存放从机地址。采用 7 位寻址模式时，仅使用 IC_SAR[6:0]位。 仅当 I2C 外设接口处于关闭状态（即 IC_ENABLE[0]置 0）时，才可以对该寄存器执行写操作，其余时段的写入操作无效。 注：寄存器默认配置值不能选用预留地址段，即 0x00~0x07 以及 0x78~0x7F。若将 IC_SAR 或目标地址寄存器（IC_TAR）配置为预留地址，无法保证器件正常工作。

17.8.4 I2Cx_DATA_CMD（x=0,1） I2C 数据命令寄存器

位域	类型	复位	名称	描述
[31:9]	--	--	--	--
[8]	W	0	CMD	该位用于控制执行读操作还是写操作。当 I2C 作为从设备时，该位无法控制数据传输方向，仅在器件作为主设备时管控传输方向。

				■ 1 = 读操作 ■ 0 = 写操作 向发送先进先出缓冲区（TX FIFO）写入指令时，依靠该位区分写指令与读指令。从机接收模式下无需配置本寄存器，因此该位取值无关紧要；从机发送模式下，该位写 0 代表需要发送 IC_DATA_CMD 寄存器内的数据。 配置本位时需留意：发送通用呼叫指令后若尝试发起读操作，会触发 TX_ABRT 中断（对应 IC_RAW_INTR_STAT 寄存器第 6 位），除非 IC_TAR 寄存器的第 11 位（SPECIAL）已置 0。 收到 RD_REQ 中断后若向本位写入 1，将会产生 TX_ABRT 中断。
[7:0]	R/W	0	DAT	该寄存器存放需要在 I2C 总线上发送或接收的数据。若对该寄存器执行写操作后想要发起读取，I2C 会忽略位 7 至位 0（DAT）；但读取此寄存器时，上述位会返回 I2C 接口接收到的数据数值。

17.8.5 I2Cx_SS_SCL_HCNT（x=0,1） I2C 标准模式 SCL 高计数寄存器

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	16'h28	IC_SS_SCL_HCNT	在发起任意 I2C 总线传输前，必须配置该寄存器，以此保证输入输出时序合规。该寄存器用于配置标准速率下 SCL 时钟高电平的计数。详细说明可参考 IC_CLK 频率配置章节。 仅当 I2C 外设接口关闭（即 IC_ENABLE[0]置 0）时，才可对本寄存器执行写操作，其余时段的写入操作无效。

				寄存器有效最小值为 6；硬件会拦截小于该数值的写入操作，若尝试写入更小数值，寄存器将被强制赋值为 6。 注：该寄存器配置值不可超过 65525。I2C 内部采用 16 位计数器，当计数器计数达到 IC_SS_SCL_HCNT + 10 时，会标记 I2C 总线处于空闲状态，限值由此而来。

17.8.6 I2Cx_SS_SCL_LCNT（x=0,1） I2C 标准模式 SCL 低计数寄存器

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	16’h2f	IC_SS_SCL_LCNT	必须在发起任意 I2C 总线传输前配置该寄存器，以此保障输入输出时序合规。本寄存器用于配置标准速率下 SCL 时钟的低电平计数值。详细说明请参阅 IC_CLK 频率配置章节。 仅当 I2C 外设接口关闭（即 IC_ENABLE[0]置 0）时，才可对该寄存器执行写操作，其余时段的写入操作无效。 寄存器有效最小值为 8；硬件会拦截小于该数值的写入请求，若尝试写入更小数值，硬件将自动写入数值 8。

17.8.7 I2Cx_FS_SCL_HCNT（x=0,1） I2C 快速模式 SCL 高计数寄存器

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	16’h06	IC_FS_SCL_LCNT	在发起任意一次 I2C 总线传输前，必须配置本寄存器，以此保障输入输出时序合规。本寄存器用于配置快速模式或增强快速模式下 SCL 时钟的高电平计数值；详细说明可参阅 IC_CLK 频率配置章节。

				仅当 I2C 外设接口处于关闭状态（即 IC_ENABLE[0]置 0）时，才可对本寄存器执行写操作，其余时段的写入操作均无效。 寄存器有效最小值为 6，硬件会拦截小于该数值的写入操作，若尝试写入更小数值，硬件会自动将寄存器配置为 6。

17.8.8 I2Cx_FS_SCL_LCNT（x=0,1） I2C 快速模式 SCL 低计数寄存器

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	16'h0d	IC_FS_SCL_LCNT	在发起任意一次 I2C 总线传输前，必须配置该寄存器，以此保障输入输出时序合规。本寄存器用于配置快速模式或增强快速模式下 SCL 时钟的低电平计数值；在高速模式中，该寄存器用于发送主机码、起始字节或通用呼叫帧。更多详情可参考第 59 页的《IC_CLK 时钟频率配置》章节。 仅当 I2C 外设接口处于关闭状态（即 IC_ENABLE[0]位配置为 0）时，方可对本寄存器执行写操作；其余时段的写入操作均无效。 寄存器有效配置最小值为 8，硬件会拦截小于该数值的写入操作，若尝试写入更小数值，硬件会自动将计数值置为 8。

17.8.9 I2Cx_INTR_STAT（x=0,1） I2C 中断状态寄存器

位域	类型	复位	名称	描述
[31:14]	--	--	--	--
[13]	RO	0	R_MST_ON_HOLD	有关这些位的详细说明请参见 IC_RAW_INTR_STAT。

[12]	RO	0	R_RESTART_DET	
[11]	RO	0	R_GEN_CALL	
[10]	RO	0	R_START_DET	
[9]	RO	0	R_STOP_DET	
[8]	RO	0	R_ACTIVITY	
[7]	RO	0	R_RX_DONE	
[6]	RO	0	R_TX_ABRT	
[5]	RO	0	R_RD_REQ	
[4]	RO	0	R_TX_EMPTY	
[3]	RO	0	R_TX_OVER	
[2]	RO	0	R_RX_FULL	
[1]	RO	0	R_RX_OVER	
[0]	RO	0	R_RX_UNDER	

17.8.10 I2Cx_INTR_MASK（x=0,1） I2C 中断屏蔽寄存器

位域	类型	复位	名称	描述
[31:14]	--	--	--	--
[13]	RO	0	M_MST_ON_HOLD	该位用于屏蔽 IC_INTR_STAT 寄存器中的 R_MST_ON_HOLD 中断位。 M_MST_ON_HOLD 位仅可读。
[12]	R/W	0	M_RESTART_DET	该位用于屏蔽 IC_INTR_STAT 寄存器中的 R_RESTART_DET 中断状态位。
[11]	R/W	1	M_GEN_CALL	这些位用于屏蔽 IC_INTR_STAT 寄存器中与之对应的中断状态位。
[10]	R/W	0	M_START_DET	
[9]	R/W	0	M_STOP_DE	

			T	
[8]	R/W	0	M_ACTIVITY	
[7]	R/W	1	M_RX_DONE	
[6]	R/W	1	M_TX_ABRT	
[5]	R/W	1	M_RD_REQ	
[4]	R/W	1	M_TX_EMPTY	
[3]	R/W	1	M_TX_OVER	
[2]	R/W	1	M_RX_FULL	
[1]	R/W	1	M_RX_OVER	
[0]	R/W	1	M_RX_UNDER	

17.8.11 I2Cx_RAW_INTR_STAT（x=0,1） I2C 中断原始状态寄存器

位域	类型	复位	名称	描述
[31:14]	--	--	--	--
[13]	RO	0	MST_ON_HOLD	--
[12]	RO	0	RESTART_DET	该位用于指明：当 I2C 工作在从机模式且本从机为被寻址从机时，I2C 接口是否出现了重启条件。 注：但按照 I2C 协议规范，在高速模式或起始字节传输过程中，重启信号会出现在地址字段之前。此种场景下，发出重启信号时本从机并非被寻址从机，因此 I2C 不会产生重启检测中断。

[11]	RO	0	GEN_CALL	仅当收到通用呼叫地址且该地址应答成功时置位。该位将保持置位状态，直到关闭 DW_apb_i2c 外设，或是 CPU 读取 IC_CLR_GEN_CALL 寄存器的第 0 位后才会清零。I2C 会将接收到的数据存入接收缓冲区
[10]	RO	0	START_DET	用于指示 I2C 接口上是否产生起始或重启信号，与 I2C 工作在从机模式还是主机模式无关。
[9]	RO	0	STOP_DET	STOP_DET 中断状态的表现取决于 IC_CON 寄存器内 STOP_DET_IFADDRESSED 位的配置选择 ■ 当 STOP_DET_IFADDRESSED =0 时 无论 I2C 工作在从机模式还是主机模式，该标识位均用于指示 I2C 接口上是否产生了停止（STOP）信号。从机模式下，无论本机从机是否被寻址，都会触发 STOP_DET 中断。 ■ 当 STOP_DET_IFADDRESSED = 1 时 主机模式（MASTER_MODE = 1’b1）：用于标识 I2C 接口出现停止（STOP）信号。 从机模式（MASTER_MODE = 1'b0）：仅当从机被寻址时，才会产生 STOP_DET 中断。 备注：当 STOP_DET_IFADDRESSED=1'b1 时，即便是从机对广播地址应答并发出确认位（ACK），接收广播地址期间也不会产生 STOP_DET 中断；只有传输地址与从机地址（SAR）匹配时，才会触发 STOP_DET 中断。
[8]	RO	0	ACTIVITY	该位用于捕获 I2C 的工作状态，置位后将保持

				该状态直至被清零。共有四种清零方式： ■ 关闭 I2C 外设 ■ 读取 IC_CLR_ACTIVITY 寄存器 ■ 读取 IC_CLR_INTR 寄存器 ■ 系统复位 该位一旦置位，除非采用上述四种方式之一进行清零，否则会持续保持置位状态。即便 I2C 模块进入空闲状态，该位也维持置位直至被清除，以此标识总线上曾产生过数据交互。
[7]	RO	0	RX_DONE	当 I2C 作为从机发送器工作时，若主机未对已发送字节作出应答，则该位置 1。该情况出现在传输的最后一个字节，代表本次传输结束。
[6]	RO	0	TX_ABRT	IC_TX_ABRT_SOURCE 寄存器用于指明发送中止发生的原因。注：只要由 IC_TX_ABRT_SOURCE 寄存器所记录的任意事件引发发送中止，I2C 控制器就会清空/复位发送先进先出缓存（TX_FIFO）与接收先进先出缓存（RX_FIFO）。在读取 IC_CLR_TX_ABRT 寄存器前，发送先进先出缓存会持续处于清空状态；完成该寄存器读取操作后，发送先进先出缓存才可通过高级外设总线（APB）接口接收新的数据字节。 当 I2C 工作在从机模式，且有其他 I2C 主机尝试从该器件读取数据时，该位会被置 1。I2C 会将 I2C 总线时钟（SCL）拉低，使总线保持等待状态，直至该中断得到处理；该状态代表从机被外部主机寻址，主机请求从机向外传输数据。处理器需要响应该中断，并将所需数据写

				入
[5]	RO	0	RD_REQ	IC_DATA_CMD 寄存器。处理器读取完 IC_CLR_RD_REQ 寄存器后，该位被置 0。发送缓冲区空（TX_EMPTY）中断状态的表现会随相关条件发生变化
[4]	RO	0	TX_EMPTY	TX_EMPTY 中断状态的表现由 IC_CON 寄存器内 TX_EMPTY_CTRL 位的配置决定。 ■ 当 TX_EMPTY_CTRL = 0 时： 发送缓冲器的数据量小于等于 IC_TX_TL 寄存器配置的阈值时，该位置 1。 ■ 当 TX_EMPTY_CTRL = 1 时： 发送缓冲器的数据量小于等于 IC_TX_TL 寄存器配置的阈值，且最新弹出指令对应的内部移位寄存器已完成地址/数据发送时，该位置 1 缓冲器存储量超出阈值后，硬件会自动将该位清零。若 IC_ENABLE[0]置 0，发送 FIFO 缓冲区会被清空并保持复位状态，此时发送 FIFO 等效为空；若主机或从机状态机仍在运行，该位会被置 1。待状态机全部停止工作后，在 IC_ENABLE=0 的条件下，该位清零。
[3]	RO	0	TX_OVER	若发送缓冲区数据填满至 IC_TX_BUFFER_DEPTH 设定深度，且处理器通过向 IC_DATA_CMD 寄存器写入数据来发起新的 I2C 指令时，该位置位。外设模块禁用后，该位电平将保持不变，直至主机或从机状态机进入空闲状态；在 IC_ENABLE=0 的条件下，该位清零。

[2]	RO	0	RX_FULL	当接收缓冲区的数据量达到或超过寄存器 IC_RX_TL 中的 RX_TL 阈值时，该位置 1；当缓冲区数据量低于阈值时，硬件自动将该位清零。若外设模块被禁用（IC_ENABLE[0]=0），接收先进先出缓冲区（RX FIFO）会被清空并保持复位状态，因此 RX FIFO 不会出现满溢情况。在 IC_ENABLE=0 的条件下，该位清零。
[1]	RO	0	RX_OVER	当接收缓冲区数据填满至寄存器 IC_RX_BUFFER_DEPTH 设定深度后，又从外部 I2C 设备接收到额外一字节数据时，该标志位会被置位。I2C 控制器会对该额外字节做出应答，但 FIFO 已满之后再接收到的所有数据字节都会丢失。若模块处于禁用状态（IC_ENABLE[0]=0），该位电平将维持不变，直至主机或从机状态机进入空闲状态；在 IC_ENABLE=0 的条件下，该位清零。
[0]	RO	0	RX_UNDER	若处理器通过读取 IC_DATA_CMD 寄存器，在接收缓冲区为空时仍尝试执行读操作，则该位置 1。若外设模块已被禁用（IC_ENABLE[0]=0），该位电平将维持不变，直至主机或从机状态机进入空闲状态；在 IC_ENABLE=0 的条件下，该位清零。

17.8.12 I2Cx_RX_TL（x=0,1）　I2C 接收 FIFO 阈值寄存器

位域	类型	复位	名称	描述
[31:8]	--	--	--	--
[7:0]	R/W	0	RX_TL	接收 FIFO 阈值电平 该参数用于控制触发 RX_FULL 中断

				（IC_RAW_INTR_STAT 寄存器第 2 位）所需的条目数量（达到或超出该数值即触发）。取值有效范围为 0~255，同时受硬件限制，配置值不能超过缓冲区深度；若尝试配置超出缓冲区深度的数值，硬件会自动将实际配置值设为缓冲区最大深度。 配置数值为 0 时，阈值对应 1 个条目；配置数值为 255 时，阈值对应 256 个条目。

17.8.13 I2Cx_TX_TL（x=0,1） I2C 发送 FIFO 阈值寄存器

位域	类型	复位	名称	描述
[31:8]	--	--	--	--
[7:0]	R/W	0	TX_TL	发送 FIFO 阈值电平 用于控制触发发送空中断（原始中断状态寄存器第 4 位）的存储条目数量上限（含该数值）。有效取值范围为 0~255，同时参数不得配置为超出缓冲区深度的数值；若强行写入超限数值，寄存器将自动配置为缓冲区最大深度值。 参数设为 0 代表阈值为 0 个条目，设为 255 代表阈值为 255 个条目。

17.8.14 I2Cx_CLR_INTR（x=0,1） I2C 组合与独立中断清除寄存器

位域	类型	复位	名称	描述
[31:1]	--	--	--	--
[0]	RO	0	CLR_INTR	读取该寄存器可清除组合中断、所有独立中断以及 IC_TX_ABRT_SOURCE 寄存器。该位无法清除硬件可清零中断，仅能清除软件可清零中断。关于 IC_TX_ABRT_SOURCE 寄存器的清零特例，请查阅该寄存器的第 9 位。

17.8.15 I2Cx_CLR_RX_UNDER（x=0,1） I2C 接收下溢清除寄存器

位域	类型	复位	名称	描述
[31:1]	--	--	--	--
[0]	RO	0	CLR_RX_UNDER	读取该寄存器可清除 IC_RAW_INTR_STAT 寄存器的接收数据下溢中断（第 0 位）。

17.8.16 I2Cx_CLR_RX_OVER（x=0,1） I2C 接收上溢清除寄存器

位域	类型	复位	名称	描述
[31:1]	--	--	--	--
[0]	RO	0	CLR_RX_OVER	读取该寄存器可清除 IC_RAW_INTR_STAT 寄存器的接收溢出中断（第 1 位）。

17.8.17 I2Cx_CLR_TX_OVER（x=0,1） I2C 发送上溢清除寄存器

位域	类型	复位	名称	描述
[31:1]	--	--	--	--
[0]	RO	0	CLR_TX_OVER	读取该寄存器，用以清除 IC_RAW_INTR_STAT 寄存器的发送溢出中断（第 3 位）。

17.8.18 I2Cx_CLR_RD_REQ（x=0,1） I2C 读请求清除寄存器

位域	类型	复位	名称	描述
[31:1]	--	--	--	--
[0]	RO	0	CLR_RD_REQ	读取该寄存器，用以清除 IC_RAW_INTR_STAT 寄存器的读请求中断（第 5 位）。

17.8.19 I2Cx_CLR_TX_ABRT（x=0,1） I2C 发送停止清除寄存器

位域	类型	复位	名称	描述
[31:1]	--	--	--	--
[0]	RO	0	CLR_TX_ABRT	读取该寄存器可清除 IC_RAW_INTR_STAT 寄存

				器的发送中止中断（第 6 位）以及 IC_TX_ABRT_SOURCE 寄存器的相关标记。 该操作还会解除发送 FIFO 的清空/复位状态，从而支持向发送 FIFO 继续写入数据。 如需了解 IC_TX_ABRT_SOURCE 寄存器清除操作的例外规则，请查阅该寄存器第 9 位说明。

17.8.20 I2Cx_CLR_RX_DONE（x=0,1） I2C 接收完成清除寄存器

位域	类型	复位	名称	描述
[31:1]	--	--	--	--
[0]	RO	0	CLR_RX_DONE	读取该寄存器可清零 IC_RAW_INTR_STAT 寄存器的接收完成中断（第 7 位）。

17.8.21 I2Cx_CLR_ACTIVITY（x=0,1） I2C 接收完成清除寄存器

位域	类型	复位	名称	描述
[31:1]	--	--	--	--
[0]	RO	0	CLR_ACTIVITY	若 I2C 总线已停止工作，读取该寄存器会清除活动中断。若 I2C 外设仍在总线中处于工作状态，则活动中断标志位保持置 1。当外设被禁用且总线无后续通讯动作时，该标志位由硬件自动清零。读取此寄存器可获取 IC_RAW_INTR_STAT 寄存器（第 8 位）的活动中断状态。

17.8.22 I2Cx_CLR_STOP_DET（x=0,1） I2C 停止检测清除寄存器

位域	类型	复位	名称	描述
[31:1]	--	--	--	--
[0]	RO	0	CLR_STOP_DET	读取该寄存器可清除 IC_RAW_INTR_STAT 寄存器的停机检测中断（第 9 位）。

17.8.23 I2Cx_CLR_START_DET（x=0,1） I2C 开始检测清除寄存器

位域	类型	复位	名称	描述
[31:1]	--	--	--	--
[0]	RO	0	CLR_START_DET	读取该寄存器可清除 IC_RAW_INTR_STAT 寄存器的 START_DET 中断（第 10 位）。

17.8.24 I2Cx_CLR_GEN_CALL（x=0,1） I2C 广播清除寄存器

位域	类型	复位	名称	描述
[31:1]	--	--	--	--
[0]	RO	0	CLR_GEN_CALL	读取该寄存器可清除 IC_RAW_INTR_STAT 寄存器中的通用呼叫中断（第 11 位）。

17.8.25 I2Cx_ENABLE（x=0,1） I2C 使能寄存器

位域	类型	复位	名称	描述
[31:2]	--	--	--	--
[1]	R/W	0	ABORT	该位置 1 时，控制器发起传输中止操作。 ■ 0：未发起中止或中止操作已完成 ■ 1：中止操作正在执行 软件可通过置位该位，在主机模式下中止 I2C 传输。仅当使能位（ENABLE）已置 1 时，软件才能够对该位进行置位；否则控制器会忽略所有针对中止位的写入操作。该中止位被置 1 后，无法由软件手动清零。收到中止指令后，控制器在完成当前传输后生成停止信号、清空发送先进先出缓冲区，并在中止流程结束后触发发送中止中断。中止操作完成后，中止位由硬件自动清零。
[0]	R/W	0	ENABLE	用于控制 I2C 模块的使能状态。 ■ 0：关闭 I2C（发送、接收先进先出缓存保持清空

				状态） ■ 1：开启 I2C I2C 关闭后会执行下述动作： ■ 清空发送先进先出缓存与接收先进先出缓存。 ■ IC_INTR_STAT 寄存器内的状态位持续有效，直至 I2C 进入空闲状态。 若模块正处于发送数据阶段，会在当前数据传输完成后停止发送，并清空发送缓冲区数据；若模块正在接收数据，则在当前字节接收结束后终止本次传输，且不再对本次传输做出应答。

17.8.26 I2Cx_STATUS（x=0,1） I2C 状态寄存器

位域	类型	复位	名称	描述
[31:7]	--	--	--	--
[6]	RO	0	SLV_ACTIVITY	从机有限状态机运行状态位。当从机有限状态机（FSM）未处于空闲状态时，该位置 1。 ■ 0：从机有限状态机处于空闲状态，I2C 的从机模块未工作 ■ 1：从机有限状态机非空闲状态，I2C 的从机模块处于工作状态
[5]	RO	0	MST_ACTIVITY	主状态机运行状态位。当主机有限状态机（FSM）未处于空闲状态时，该位置 1。 ■ 0：主机状态机处于空闲状态，I2C 的主机模块未工作 ■ 1：主机状态机非空闲状态，I2C 的主机模块处于工作状态 注：IC_STATUS 寄存器第 0 位，即运行状态位，由从机活动位与主机活动位做逻辑或运算得到。

[4]	RO	0	RFF	接收先进先出缓存已满。接收先进先出缓存存满时，该位置 1；接收先进先出缓存存在一处及以上空余存储空间时，该位清零。 ■ 0：接收先进先出缓存未满 ■ 1：接收先进先出缓存已满
[3]	RO	0	RFNE	接收先进先出缓冲区非空标志位。当接收先进先出缓冲区存有一条及以上数据时，该位置 1；接收先进先出缓冲区清空后，该位清零。 ■ 0：接收先进先出缓冲区为空 ■ 1：接收先进先出缓冲区存有数据
[2]	RO	0	TFE	发送先进先出缓冲区全空标志位。发送先进先出缓冲区完全为空时，该位置 1；缓冲区存有一条及以上有效数据时，该位清零。该位字段不会触发中断请求。 ■ 0：发送先进先出缓冲区非空 ■ 1：发送先进先出缓冲区为空
[1]	RO	0	TFNF	发送先进先出缓冲区非满标志。当发送先进先出缓冲区存在一处及以上空余存储空间时该位置 1，缓冲区存满时该位清零。 ■ 0：发送先进先出缓冲区已满 ■ 1：发送先进先出缓冲区未满
[0]	RO	0	ACTIVITY	I2C 运行状态。

17.8.27 I2Cx_TXFLR（x=0,1） I2C TXFIFO 级数寄存器

位域	类型	复位	名称	描述
[31:4]	--	--	--	--
[3:0]	RO	0	TXFLR	发送 FIFO 电平寄存器：存放发送 FIFO 内有效数据的条目数量。

17.8.28 I2Cx_RXFLR（x=0,1） I2C RXFIFO 级数寄存器

位域	类型	复位	名称	描述
[31:4]	--	--	--	--
[3:0]	RO	0	RXFLR	接收 FIFO 电平寄存器：存放接收 FIFO 内有效数据的条目数量。

17.8.29 I2Cx_SDA_HOLD（x=0,1） I2C SDA 保持时间寄存器

位域	类型	复位	名称	描述
[31:24]	--	--	--	--
[23:16]	R/W	1	IC_SDA_RX_HOLD	当 I2C 作为接收器时，以 ic_clk 时钟周期为单位设置所需的 SDA 保持时间。
[15:0]	R/W	1	IC_SDA_TX_HOLD	当 I2C 作为发送器时，以 ic_clk 时钟周期为单位配置所需的 SDA 保持时间。

17.8.30 I2Cx_TX_ABRT_SOURCE（x=0,1） I2C 发送停止状态寄存器

位域	类型	复位	名称	描述
[31:24]	RO	0	TX_FLUSH_CNT	该字段保存上一次发送中止事件发生前的发送 FIFO 电平值。I2C 总线被禁用时，此字段会被清零。
[23:17]	--	--	--	--
[16]	RO	0	ABRT_USER_ABRT	该位仅在主机模式下有效。主机已检测到传输中止事件（IC_ENABLE[1]）。
[15]	RO	0	ABRT_SLVRD_INTX	1：当处理器端响应从机模式的数据请求（数据需发送至远端主机），且用户在 IC_DATA_CMD 寄存器的 CMD 位（第 8 位）写入 1 时。
[14]	RO	0	ABRT_SLV_ARBLOST	1：从设备在向远端主设备传输数据时丢失总线控制权，同时 IC_TX_ABRT_SOURCE 寄存器第

				12 位被置 1。 注：即便从设备并不持有总线所有权，总线通信仍可能出现异常，该机制属于故障安全校验。举例：在数据传输过程中，SCL 时钟电平由低跳变至高的瞬间，若数据总线上的电平与待发送数据不一致，则 I2C 控制器将失去总线控制权。
[13]	RO	0	ABRT_SLVFLUSH_TXFIFO	1：从设备收到读取指令，且发送先进先出缓存中存有部分数据，因此从设备触发发送中止（TX_ABRT）中断，清空发送先进先出缓存内的原有数据。
[12]	RO	0	ARB_LOST	1：主机仲裁失败；若 IC_TX_ABRT_SOURCE[14]位同样置 1，则从机发送器仲裁失败。
[11]	RO	0	ABRT_MASTER_DIS	1：用户在主模式已禁用的情况下尝试发起主控操作
[10]	RO	0	ABRT_10B_RD_NORSTRT	1：重启功能被禁用（IC_RESTART_EN 位（IC_CON[5]）= 0）且主机在 10 位寻址模式下发送读取指令。
[9]	RO	0	ABRT_SBYTE_NORSTRT	想要清零第 9 位，必须先消除 ABRT_SBYTE_NORSTRT 异常的触发源头；需启用重启功能（配置 IC_CON[5]=1）、清零 SPECIAL 位（IC_TAR[11]）或清零 GC_OR_START 位（IC_TAR[10]）。在 ABRT_SBYTE_NORSTRT 异常根源处理完毕后，即可按照该寄存器内其余位的清零方式对本位进行清零。若未排除异常源头就直接尝试清零第 9 位，该位仅会维持一个时钟周期的清零状态，随后会再次置位。

				1：重启功能处于禁用状态（IC_RESTART_EN 位，即 IC_CON[5]=0），且用户正在尝试发送起始字节。
[8]	--	--	--	--
[7]	RO	0	ABRT_SBYTE_ACKDET	1：主机已发送起始字节，且该起始字节收到了应答（此为错误行为）。
[6]	--	--	--	--
[5]	RO	0	ABRT_GCALL_READ	1：处于主机模式的 I2C 发送了一条通用呼叫指令，但用户配置通用呼叫的后续字节为总线读取操作（IC_DATA_CMD 寄存器第 9 位置 1）。
[4]	RO	0	ABRT_GCALL_NOACK	1：处于主机模式的 I2C 发送了一次通用呼叫，但总线上没有从设备响应该通用呼叫。
[3]	RO	0	ABRT_TXDATA_NOACK	1：该位仅在主机模式下有效。主机已收到对应地址的应答，但在发送地址后续的数据字节时，未收到从机的应答信号。
[2]	RO	0	ABRT_10ADDR2_NOACK	1：主机处于 10 位地址模式，且从机均未应答该 10 位地址的第二个地址字节。
[1]	RO	0	ABRT_10ADDR1_NOACK	1：主机处于 10 位地址模式，且首个 10 位地址字节未得到任何从机的应答。
[0]	RO	0	ABRT_7B_ADDR_NOACK	1：主机处于 7 位寻址模式，发送的地址未得到任何从机应答。

17.8.31 I2Cx_SLV_DATA_NACK_ONLY（x=0,1） I2C 从机无应答寄存器

位域	类型	复位	名称	描述
[31:1]	--	--	--	--
[0]	R/W	0	NACK	产生非应答信号（NACK）。仅当 DW_apb_i2c 作为从机接收器时，才会触发该非应答信号生成操作。

				若此寄存器配置为 1，则器件仅在收到一个数据字节后输出非应答信号；由此终止数据传输，且已接收的数据不会存入接收缓冲区。 若该寄存器配置为 0，器件将按照常规判定规则生成非应答信号（NACK）或应答信号（ACK）。 ■ 1 = 接收数据字节后产生非应答信号 ■ 0 = 按常规逻辑产生非应答/应答信号

17.8.32 I2Cx_DMA_CR（x=0,1） I2C DMA 控制寄存器

位域	类型	复位	名称	描述
[31:2]	--	--	--	--
[1]	R/W	0	TDMAE	发送 DMA 使能位。该位用于开启或关闭发送 FIFO 的 DMA 通道。 ■ 0 = 关闭发送 DMA 功能 ■ 1 = 开启发送 DMA 功能
[0]	R/W	0	RDMAE	接收 DMA 使能位。该位用于开启或关闭发送 FIFO 的 DMA 通道。 ■ 0 = 关闭接收 DMA 功能 ■ 1 = 开启接收 DMA 功能

17.8.33 I2Cx_DMA_TDLR（x=0,1） I2C DMA 发送水位寄存器

位域	类型	复位	名称	描述
[31:3]	--	--	--	--
[2:0]	R/W	0	DMATDL	发送数据阈值。该位字段用于控制发送逻辑发起 DMA 请求的触发阈值，该数值与水位阈值一致；即当发送 FIFO 内有效数据条目数量小于等于本字段设定值，且 TDMAE 配置为 1 时，将会产生 dma_tx_req 请求信号。

17.8.34 I2Cx_DMA_RDLR（x=0,1） I2C DMA 接收水位寄存器

位域	类型	复位	名称	描述
[31:3]	--	--	--	--
[2:0]	R/W	0	DMARDL	接收数据阈值。该位字段用于控制发送逻辑发起 DMA 请求的触发阈值，该数值与水位阈值一致；即当接收 FIFO 内有效数据条目数量小于等于本字段设定值，且 RDMAE 配置为 1 时，将会产生 dma_rx_req 请求信号。

17.8.35 I2Cx_SDA_SETUP（x=0,1） I2C SDA 建立时间寄存器

位域	类型	复位	名称	描述
[31:8]	--	--	--	--
[7:0]	R/W	8'h64	SDA_SETUP	SDA 建立时间。若所需延时为 1000ns，在 ic_clk 频率为 10MHz 的条件下，建议将 IC_SDA_SETUP 配置为 11。IC_SDA_SETUP 的配置数值最小不得低于 2。

17.8.36 I2Cx_ACK_GENERAL_CALL（x=0,1） I2C 广播应答寄存器

位域	类型	复位	名称	描述
[31:1]	--	--	--	--
[0]	R/W	1	ACK_GEN_CALL	ACK 通用呼叫。当设置为 1 时，I2C 在接收到通用呼叫时会以 ACK 进行响应。当设置为 0 时，I2C 不会生成通用呼叫中断。

17.8.37 I2Cx_ENABLE_STATUS（x=0,1） I2C 使能状态寄存器

位域	类型	复位	名称	描述
[31:3]	--	--	--	--
[2]	RO	0	SLV_RX_DA	从机接收数据丢失。该位用于标识：因

			TA_LOST	IC_ENABLE[0]位由 1 置 0，导致从机接收传输终止，且 I2C 总线传输过程中至少已收到一个数据字节。 若该位读数值为 1，代表 I2C 控制器当时正处理地址匹配的 I2C 传输、已进入传输的数据阶段，后续传输被中止，即便控制器已对收到的数据字节回复非应答（NACK）。 注：若远端 I2C 主机在 I2C 控制器来得及发送 NACK 应答前，通过停止（STOP）信号终止传输，同时 IC_ENABLE[0]被置 0，该位同样会被置 1。 若该位读数值为 0，代表 I2C 控制器在退出从机接收的数据传输阶段后才被关闭。 注：当 IC_EN（第 0 位）读取为 0 时，处理器可安全读取该状态位。
[1]	RO	0	SLV_DISABLED_WHILE_BUSY	从忙态禁用从机（收发状态）该位用于标识：因将 IC_ENABLE 寄存器第 0 位由 1 置 0，导致正在执行或待启动的从机操作被中止。满足下述任一条件时，若 CPU 向 IC_ENABLE 寄存器第 0 位写入 0，该位将被置 1： (a) I2C 正接收外部主机发来的从机发送模式地址字节； (b) I2C 正接收外部主机发来的从机接收模式地址与数据字节。 读取该位为 1 时：判定在 I2C 传输过程中任意阶段主动发出非应答（NACK），无论 I2C 地址是否与 I2C 配置的从机地址（IC_SAR 寄存器）匹配，也无论传输在 IC_ENABLE 第 0 位清零前已完成但配

				置尚未生效。 注：若在 I2C 来不及发送非应答（NACK）前，外部 I2C 主机就通过停止（STOP）信号终止传输，且 IC_ENABLE 寄存器第 0 位已被置 0，该位同样会被置 1。 读取该位为 0 时：判定 I2C 是在总线存在主机通讯动作或 I2C 总线空闲状态下被关闭。 注：当 IC_EN（第 0 位）读取为 0 时，CPU 可安全读取本状态位。
[0]	RO	0	IC_EN	ic_en 状态位。该位始终反映输出端口 ic_en 上的驱动电平。 ■ 读取为 1 时，I2C 处于使能工作状态。 ■ 读取为 0 时，I2C 完全停止工作。 注：CPU 可在任意时刻安全读取该位。当该位读取值为 0 时，CPU 能够安全读取 SLV_RX_DATA_LOST（第 2 位）与 SLV_DISABLED_WHILE_BUSY（第 1 位）。

17.8.38 I2Cx_FS_SPKLEN（x=0,1） I2C 快速模式尖峰抑制长度寄存器

位域	类型	复位	名称	描述
[31:8]	--	--	--	--
[7:0]	R/W	8'h4	IC_FS_SPKLEN	必须在发起任意 I2C 总线传输操作前配置该寄存器，以此保障设备稳定运行。该寄存器用于设定 SCL 或 SDA 信号线中毛刺脉冲的最大过滤时长，计时单位为内部时钟（ic_clk）周期，超出该时长的毛刺会由毛刺抑制电路滤除； 仅当 I2C 接口处于关闭状态（即 IC_ENABLE[0]位配置为 0）时，才可对本寄存器执行写操作，其余

				时段的写入操作均无效。 寄存器有效最小值为 1，硬件会拦截小于该数值的写入指令，若尝试写入更小数值，寄存器最终仍会被置为 1。

17.8.39 I2Cx_CLR_RESTART_DET（x=0,1） I2C 清除重启监测寄存器

位域	类型	复位	名称	描述
[31:1]	--	--	--	--
[0]	R	0	CLR_RESTART_DET	读取该寄存器，用以清零 IC_RAW_INTR_STAT 寄存器中的重启检测中断（第 12 位）。

17.8.40 I2Cx_COMP_PARAM_1（x=0,1） I2C 组件参数寄存器 1

位域	类型	复位	名称	描述
[31:24]	--	--	--	--
[23:16]	R	0	TX_BUFFER_DEPTH	该寄存器的数值由内核参数 IC_TX_BUFFER_DEPTH 确定。 ■ 0x00 = 保留 ■ 0x01 = 2 ■ 0x02 = 3 …… ■ 0xFF = 256
[15:8]	R	0	RX_BUFFER_DEPTH	该寄存器的数值由内核参数 IC_RX_BUFFER_DEPTH 确定。 ■ 0x00 = 保留 ■ 0x01 = 2 ■ 0x02 = 3 … ■ 0xFF = 256

[7]	R	0	ADD_ENCODED_PARAMS	若读取到该位为 1，软件可读取各项编码参数；若该位为 0，则无论寄存器其余比特位配置何种编码参数，整个寄存器的数值均为 0。 ■ 0：否 ■ 1：是
[6]	R	0	HAS_DMA	该寄存器的数值由内核参数 IC_HAS_DMA 确定。该位为 1，顶层输入输出端口将包含 DMA 握手接口信号。 ■ 0：否 ■ 1：是
[5]	R	0	INTR_IO	该寄存器的数值由内核参数 IC_INTR_IO 确定。若该位为 0，每个中断源拥有独立输出；若为 1，所有中断源组合为单一输出。 ■ 0：单独模式 ■ 1：组合模式
[4]	R	0	HC_COUNT_VALUES	该寄存器的数值由内核参数 IC_HC_COUNT_VALUES 确定。该位为 1，CNT 寄存器将被设为只读模式。该位为 0，CNT 寄存器支持写入操作。 ■ 0：否 ■ 1：是
[3:2]	R	0	MAX_SPEED_MODE	该寄存器的数值由内核参数 IC_MAX_SPEED_MODE 确定。所支持的最大速率 I2C 工作模式。 ■ 0x0 = 保留项 ■ 0x1 = 标准模式 ■ 0x2 = 快速模式 ■ 0x3 = 高性能模式

[1:0]	R	0	APB_DATA_WIDTH	该寄存器的数值由内核参数 APB_DATA_WIDTH 确定。 ■ 0x0 = 8 位 ■ 0x1 = 16 位 ■ 0x2 = 32 位 ■ 0x3 = 保留位

17.8.41 I2Cx_COMP_VERSION（x=0,1） I2C 组件版本寄存器

位域	类型	复位	名称	描述
[31:0]	--	--	--	--

17.8.42 I2Cx_COMP_TYPE（x=0,1） I2C 组件类型寄存器

位域	类型	复位	名称	描述
[31:0]	--	--	--	--

17.9 应用指南

17.9.1 功能说明

1. 工作模式与速率

I2C 模块支持主机和从机两种工作模式，同一时刻只能工作在其中一种模式（由 IC_CON.MASTER_MODE 决定）。速率模式由 i2c_mode_e 选择：I2C_STANDARD_MODE（标准模式，100kbit/s）、I2C_FAST_MODE（快速模式，400kbit/s）。主机模式下，SCL 的实际频率由 i2c_master_config_t.i2c_speed（单位 Hz）和 I2C 功能时钟共同决定，初始化时 SDK 根据二者计算 SCL 高/低电平计数（IC_SS_SCL_HCNT/LCNT、IC_FS_SCL_HCNT/LCNT）。

2. 引脚配置

I2C 使用 SDA、SCL 两根信号线，需通过 GPIO 复用功能引出。参考 demo\i2c_demo：PA3 复用为 I2C_SDA，PA4 复用为 I2C_SCL，复用功能均为 GPIO_ALT_FUNC_4。I2C 总线为开漏结构，SDA 和 SCL 需外接上拉电阻。

SDK 接口：

void gpio_init(gpio_settings_config_t *setting_cfg); /*配置引脚复用功能，SDA/SCL 均使用 GPIO_ALT_FUNC_4*/

3. I2C 时钟

使用 I2C 前必须先使能 I2C 外设时钟（I2C0 和 I2C1 共用同一个使能接口），时钟配置详见“时钟复位”章节。

SDK 接口：

void i2c_clk_enable(void); /*使能 I2C0/I2C1 外设时钟*/

void i2c_clk_disable(void); /*关闭 I2C0/I2C1 外设时钟*/

4. 主机初始化

主机初始化通过 i2c_master_init()完成：先关闭 I2C 并等待 IC_ENABLE_STATUS.IC_EN 清零，再根据 i2c_speed 和 I2C 功能时钟计算 SCL 计数值，设置 IC_CON.SPEED、主机模式、使能 RESTART（IC_RESTART_EN=1）、屏蔽全部中断后使能 I2C，最后使能 RX_FULL、TX_ABRT、TX_OVER、RX_OVER、STOP_DET、START_DET、ACTIVITY 中断并开启 NVIC。

i2c_master_config_t 主要成员：

master_mode：速率模式，I2C_STANDARD_MODE / I2C_FAST_MODE / I2C_FASTPLUS_MODE / I2C_HIGHSPEED_MODE

i2c_speed：SCL 波特率（Hz），例如 100000

master_interrupt_priority：I2C 中断优先级

SDK 接口：

status_t i2c_master_init(uint32_t i2c_instance, const i2c_master_config_t *i2c_master_config); /*i2c_instance：I2C_INSTANCE0 / I2C_INSTANCE1，成功返回 STATUS_SUCCESS*/

void i2cm_deinit(uint32_t i2c_instance); /*关闭主机、清空软件缓冲区并关闭 NVIC 中断*/

5. 主机发送

主机发送采用中断方式，函数调用后立即返回（非阻塞）。i2c_master_msg_t 描述一次传输：addr_10bit 选择 7 位/10 位从机地址，slave_addr 为从机地址（不含读写位），send_data/send_data_len 为发送缓冲区及长度。i2cm_send_data()依次完成：初始化软件缓冲区；关闭 I2

C，把从机地址写入 IC_TAR 并配置 7 位/10 位地址模式，重新使能 I2C；使能 TX_EMPTY 中断。随后在中断服务函数中，每次 TX_EMPTY 中断向 IC_DATA_CMD 写入 1 字节（CMD 位为写），直至 send_data_len 字节发完并关闭 TX_EMPTY 中断。

SDK 接口：

void i2cm_send_data(uint32_t i2c_instance, i2c_master_msg_t *i2c_master_msg);

**i2cm_send_data()不会自动发送 slave_reg_addr，如需先写寄存器地址，请把寄存器地址放在 send_data 的首字节。

6. 主机接收

i2cm_receive_data()同样为中断方式（非阻塞）。函数先配置 IC_TAR，再把 slave_reg_addr（低 8 位）作为写命令发送给从机，用于指定读取的寄存器地址；随后使能 TX_EMPTY 和 RX_FULL 中断。中断服务函数中每次 TX_EMPTY 向 IC_DATA_CMD 写入一个读命令（CMD 位为读），共 rec_data_len 个；每次 RX_FULL 中断从 IC_DATA_CMD 读取 1 字节存入 rec_data 缓冲区。由于已使能 RESTART，写寄存器地址与读数据之间由硬件产生 RESTART 条件，构成“写-读”组合传输。

SDK 接口：

void i2cm_receive_data(uint32_t i2c_instance, i2c_master_msg_t *i2c_master_msg);

**发送和接收均为非阻塞，函数返回不代表总线传输结束，使用接收缓冲区前需要等待传输完成。

7. 从机初始化与收发

从机通过 i2c_slave_init()初始化：关闭 I2C，把从机地址写入 IC_SAR（7 位模式取低 7 位，10 位模式取低 10 位），配置 IC_CON 为纯从机模式（MASTER_MODE=0、IC_SLAVE_DISABLE=0、RESTART 使能、STOP_DET_IFADDRESSED=1），使能 I2C，并使能 RX_FULL、RX_OVER、TX_OVER、RD_REQ、TX_ABRT、STOP_DET、START_DET、RESTART_DET、GEN_CALL 中断。当前实现仅使用 i2c_slave_config_t 中的 slave_addr0 和 addr_10bit 作为从机地址。

从机收发由中断服务函数处理：

接收（主机写）：RX_FULL 中断中把 RX FIFO 的数据依次存入 rec_data 缓冲区，缓冲区写满后多余的数据被丢弃。

发送（主机读）：主机读请求触发 RD_REQ 中断，SCL 被拉低，中断中向 IC_DATA_CMD 写入 send_data 的下一个字节并清除 RD_REQ 释放 SCL；send_data 发完后，主机继续读取时发送 0xFF。

STOP 或 RESTART：软件缓冲区位置复位到起始位置，下一次传输重新从缓冲区开头收发。

SDK 接口：

status_t i2c_slave_init(uint32_t i2c_instance, const i2c_slave_config_t *i2c_slave_config, i2c_slave_msg_t *i2c_slave_msg);

void i2cs_deinit(uint32_t i2c_instance); /*关闭从机、清空软件缓冲区并关闭 NVIC 中断*/

8. 中断服务函数

I2C0_IRQHandler()和 I2C1_IRQHandler()已在 i2c_hal.c 中实现，内部根据 IC_CON.MASTER_MODE 分发到主机或从机处理流程，应用层无需再实现。应用工程的 isr.c 中不要重复定义同名函数（demo\i2c_demo\isr.c 中的实现已通过#if 0 屏蔽）。使用 I2C 前需要调用__enable_irq()使能全局中断。主机中断和从机中断的优先级分别由 master_interrupt_priority 和 slave_interrupt_priority 配置。

9. 其他接口与限制

void i2cm_end_transfer(uint32_t i2c_instance, bool stop_cmd, bool reset_fifo); /*stop_cmd 为 true 时向 TX FIFO 写入 STOP 命令*/

**以下接口当前版本保留了函数原型但暂未实现实际功能：i2cm_dmatch_init()（数据匹配）、i2cs_filter_cfg()（从机滤波配置）、i2cm_tx_address_get()/i2cm_rx_address_get()/i2cs_tx_address_get()/i2cs_rx_address_get()（均返回 0）。DMA 方式和这些接口配合使用前请先确认芯片版本支持情况。

17.10 快速开始

主机最小例程（参考 demo\i2c_demo）：使用 I2C0 作为主机，通过 PA3（SDA）、PA4（SCL）与从机地址为 0x50 的设备通信，先发送一段数据，再读回数据并打印。

流程：使能外设时钟 → 配置系统时钟 → 配置 SDA/SCL 引脚 → 初始化 I2C 主机 →

发送数据 → 接收数据 → 使能全局中断

```
static void i2c_gpio_init(void)
{
    gpio_settings_config_t setting_cfg;

    /* I2C_SDA, PTA3 */
    setting_cfg.gpio_port = GPIO_A;
    setting_cfg.gpio_pin = GPIO_PIN_3;
    setting_cfg.gpio_alt = GPIO_ALT_FUNC_4;
    setting_cfg.gpio_dir = GPIO_DIR_OUTPUT;
    setting_cfg.pull_cfg = GPIO_PULL_DISABLE;
    setting_cfg.gpio_strength = GPIO_STRENGTH_MODE_0;
    setting_cfg.gpio_interrupt_cfg_info.int_handle = NULL;
    gpio_init(&setting_cfg);

    /* I2C_SCL, PTA4 */
    setting_cfg.gpio_pin = GPIO_PIN_4;
    gpio_init(&setting_cfg);
}

i2c_master_config_t i2c_master_config = {
    .master_mode = I2C_STANDARD_MODE,          /*标准模式*/
    .i2c_speed = 100000U,                      /*SCL = 100kHz*/
    .master_interrupt_priority = 1U
};

uint8_t i2c_write_data[32] = {0};
uint8_t i2c_read_data[32] = {0};
```

```
i2c_master_msg_t i2c_master_msg = {
    .addr_10bit = false,                    /*7 位从机地址*/
    .slave_addr = 0x50U,
    .slave_reg_addr = 0x00U,                /*接收时先写入的寄存器地址*/
    .send_data = i2c_write_data,
    .rec_data = i2c_read_data,
    .send_data_len = 32U,
    .rec_data_len = 32U,
};

int main(void)
{
    flash_clk_enable();
    gpio_clk_enable();
    uart_clk_enable();
    i2c_clk_enable();                       /*使能 I2C 外设时钟*/
    RCC_Init();                             /*根据 sys_config.h 配置系统时钟*/

    i2c_gpio_init();
    i2c_master_init(I2C_INSTANCE0, &i2c_master_config);

    /*发送数据*/
    strcpy((char *)i2c_write_data, "I2C Master Test Data");
    i2c_master_msg.send_data_len = strlen((char *)i2c_write_data);
    i2cm_send_data(I2C_INSTANCE0, &i2c_master_msg);
    delay_1us(1000);                        /*等待发送完成，实际应用请按数据长
度和 SCL 速率留足时间*/

    /*接收数据*/
```

```
    i2c_master_msg.rec_data_len = 32U;
    i2cm_receive_data(I2C_INSTANCE0, &i2c_master_msg);
    delay_1us(500);                                    /*等待接收完成*/
    INFO("I2C Master Recv Data: %s\n", i2c_read_data);


    __enable_irq();                                    /*使能全局中断*/
    while (1)
    {
    }
}
```

**I2C 的发送和接收全部在中断中完成，必须使能 NVIC 和全局中断；I2C0_IRQHandler ()已在 i2c_hal.c 中提供，应用工程的 isr.c 中不要重复定义。

从机最小例程（可选）

使用 I2C1 作为从机，从机地址 0x50，接收主机写入的数据，并在主机读时返回预先准备的数据。引脚配置与时钟使能方式与主机相同。

```
uint8_t slave_tx_buf[32] = "I2C Slave Data";
uint8_t slave_rx_buf[32] = {0};

i2c_slave_msg_t i2c_slave_msg = {
    .send_data = slave_tx_buf,
    .send_data_len = sizeof(slave_tx_buf),
    .rec_data = slave_rx_buf,
    .rec_data_len = sizeof(slave_rx_buf),
};

i2c_slave_config_t i2c_slave_config = {
    .slave_mode = I2C_STANDARD_MODE,
    .addr_10bit = false,                               /*7 位地址*/
```

```
        .slave_addr0  =  0x50U,                    /*从机地址*/
        .slave_interrupt_priority  =  1U
    };

    i2c_clk_enable();
    i2c_gpio_init();
    i2c_slave_init(I2C_INSTANCE1,  &i2c_slave_config,  &i2c_slave_msg);
    __enable_irq();
```

主机写入的数据存放在 slave_rx_buf 中；主机读取时依次返回 slave_tx_buf 的内容，发完后返回 0xFF。每次 STOP 或 RESTART 后收发位置回到缓冲区起始。

18 CAN

18.1 概述

18.1.1 简介

CAN 控制器是一款支持 CAN 2.0B 和 CAN FD 协议的全双工串行通信控制器，适用于多主总线系统。它支持高可靠性数据传输，具备错误检测与处理机制，适用于车载、工业控制等实时通信场景。控制器支持可编程波特率、多种工作模式、硬件时间戳、灵活的中断配置。

CAN 总线控制器可以处理总线上的数据收发，在本产品中，CAN 控制器具有 4 组筛选器。筛 选器用于为应用程序选择要接收的消息。

CAN 控制器中，发送缓冲器（Tx Buffer）包含两组缓冲器：高优先级的主发送缓冲器（Primary Transmit Buffer，以下简称 PTB）和低优先级的辅发送缓冲器（Secondary Transmit Buffer，以下简称 STB）。PTB 包含一个 Slot，STB 包含两个 Slot，由硬件调度决定发送的顺序。 接收缓冲器（Receive Buffer，以下简称 Rx Buffer）获取总线数据，Rx Buffer 包含十个 Slot。Slot 为存储 CAN 的单位（13-byte）

18.1.2 功能框图

如图 18-1-2 所示，本接口采用同步串行设计，实现 MCU 同外部设备之间的 CAN 传输。支持轮询和中断方式获得 传输状态信息。

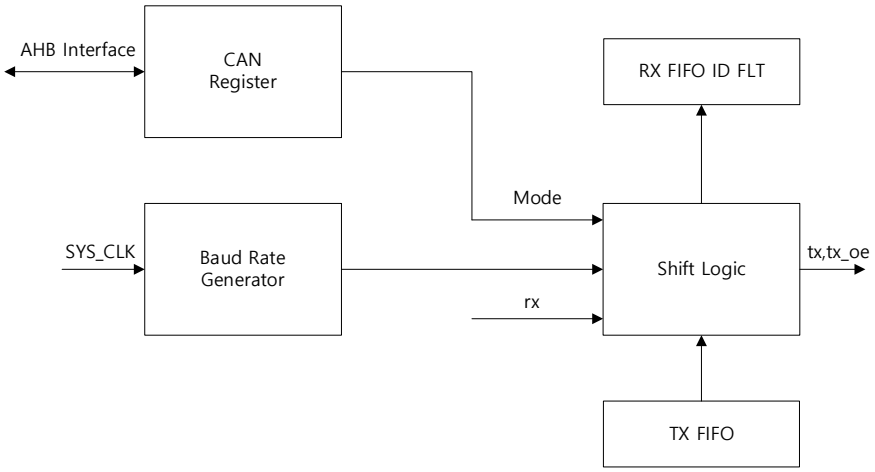


图 18-1 CAN 模块功能框图

CAN 接口同外界通讯只有 tx 和 rx 两根信号线。tx 发送有效数据时候，tx_oe 有效。tx 不发送 有效数据的时候，tx_oe 无效。

rx：数据信号。接收来自外界的 CAN 数据。

tx：数据信号。发送 CAN 数据到 CAN 总线。

tx_oe：数据使能信号。当 tx 输出时，tx_oe 有效；当 tx 无数据输出时，tx_oe 无效。

18.2 功能描述

18.2.1 工作模式

CAN 模块主要包含两个工作模式：正常工作模式和复位模式。

复位模式，时序参数、ID 配置、错误统计值等均在此模式下设定好。CAN_STCMDCR. RESET 为 1，是复位模式。硬件复位结束后，CAN 模块处于复位模式下。

正常工作模式，CAN_STCMDCR. RESET 为 0。此时，可以正常地响应 CAN 总线请求。

在上述两个模式下，扩展了监听模式（Listen Only）和自测试模式（Self Test）。前者，类似 一个搜集器，只接收 CAN 总线上的数据，不发送任何数据。后者，是内部自测试，发送的数据同 时被自己接收，检测内部功能是否正确。

18.2.2 传输

根据 SFF（Standard Frame Format），或 EFF（Extended Frame Format），搬移 11 个字节或 者 13 个字节；完成后需要通过中断或者轮询的方式判断传输是否完成。 MCU 传输，推荐软件配置流程如下：

初始化 GPIO 模块，将 CAN 复用的 GPIO 配置完毕。

初始化 CAN 接口，控制寄存器配置完毕。

通过 CAN_TBUF 寄存器，写入 PTB/STB

触发 CAN 接口进入发送流程

18.2.3 中断处理

CAN 接口包含中断事件比较多，在 CAN_INTLMT 和 CAN_ERRINT 寄存器描述中有相

应说明。根据实际使用情况，开启对应的中断事件使能开关。

CAN 控制器提供了以下七种中断：

- 接收中断

接收中断 CAN_INTLMT.RIE，当 CAN Rx Buffer 接收到有效帧，CAN_INTLMT.RIF 置位。接收中断进一 步细化，还有三个中断源头 CAN_INTLMT.ROIE、CAN_INTLMT.RFIE 和 CAN_INTLMT.RAFIE。分别是 CAN Rx Buffer 接收溢出中断，CAN Rx Buffer 接收满中断，和 CAN Rx Buffer 接收即将满中断。所以，产生 一次接收中断，可能有几个标志位会置位。 只有先使能中断，才会置位标志位。

- 发送中断

CAN Tx Buffer 有两个源头，一个是 PTB，一个是 STB。对应的中断也是两个，CAN_INTLMT.TPIE 和 CAN_INTLMT.TSIE。另外，因本 CAN 设备支持发送终止传输，所以，有一个终止传输状态位（无中 断使能标志）CAN_INTLMT.AIF。

- 错误中断

CAN 在传输过程中，发生总线错误事件，包括接收错误和发送错误。一般，产生错误，对应 的错误计数器会递增，此时，根据计数器的值，将会产生不同类型的错误中断。

错误中断 CAN_INTLMT.EIE。若 CAN_ERRABR.RECNT/CAN_TECNT 错误计数器的值从小于 CAN_ INTLMT.EWL 寄存器规定的值到超过 CAN_INTLMT.EWL 寄存器规定的值，或者反之；从非 BusOff 状态进入 BusOff 状态，或者反之，均会触发本中断，中断标志位是 CAN_INTLMT.EIF。

被动错误中断 CAN_INTLMT.EPIE。若 CAN_RECNT/CAN_TECNT 错误计数器的值超过规定值 （>127），本 CAN 设备从主动错误状态进入被动错误状态，或者从被动错误状态进入主动错误状 态（此时错误计数器的值<127），均会触发本中断，中断标志位是 CAN_ERRINT.EPIF。

只有先使能中断，才会置位标志位。

- 数据溢出中断

当 CAN Rx Buffer 接收满，数据没有被 MCU 及时取走，若再接收到有效帧，此时，触发接收溢 出中断 CAN_INTLMT.ROIE，中断标志位是 CAN_INTLMT.ROIF。

- 仲裁丢失中断

仲裁丢失中断 CAN_INTLMT.ALIE。CAN 总线支持仲裁机制，本 CAN 设备在总线竞争

过程中， 丢失了总线权利，将触发仲裁丢失中断。中断标志位是 CAN_INTLMT.ALIF。

只有先使能中断，才会置位标志位。

- 总线错误中断

总线错误中断 CAN_INTLMT.BEIE。CAN 在传输过程中，发生总线错误事件，将触发总线错误中断。中断标志位是 CAN_INTLMT.BEIF。总线错误中断和错误中断的区别是，错误中断是总线错误 积累到一定量后，才会触发。

只有先使能中断，才会置位标志位。

18.2.4 波特率设置

CAN 的波特率设定，主要依靠 CAN_LSEQ 寄存器完成。CAN_LSEQ 主要是配置 TQ 参数（TQ 的计算见 CAN_LSEQ 寄存器说明），采样点配置以及容差范围信息。

CAN_LSEQ.S_PRESC 设置传输基本时间单元参数 TQ：TQ = sysclk * (CAN_LSEQ.S_PRESC + 1)，最快系统时钟 PLL 为 96MHz，对应的 Tclk 为 10.4ns，TQ 最大值为 2.6624us。

CAN_LSEQ 设置波特率；同时可调节 BIT 信息中各个部分的宽度（S_SEQ1 和 S_SEQ2），找到理想采样点。

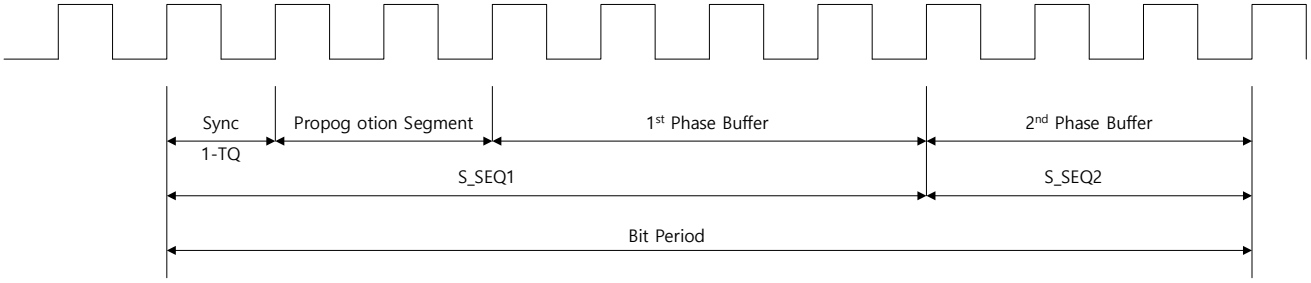


图 18-2 CAN 模块 Bit 周期图

上图可知，S_SEQ1 包含三个部分：Sync，Propagation Segment 和 1st Phase Buffer。Sync 为此 BIT 的同步，固定 1 个 TQ。Propagation Segment 为传输延迟，1st Phase Buffer 为第一段相位缓 冲。在具体 CAN 网络中，需要注意 Propagation Segment 这个参数，它决定了一个数据最少需要 多长实际才能传递过来。同时，Propagation Segment 也直接影响了 SEQ1 的配置。本设计将上述 三者融合到一个参数：S_SEQ1。S_SEQ2 仅包含 2nd Phase Buffer 一个部分，其为第二相位缓冲，补 偿边沿（Sync）阶段的误差，通过重新同步拉长或缩短。S_SEQ1 与 S_SEQ2 的长度决定了 CAN 数据 的采样点，这种方式允许一定范围内的数

据传输延迟和晶体的误差。其中，S_SEQ1 用来调整数据 传输延迟时间造成的误差，而 S_SEQ2 则用来调整不同点节点晶体频率的误差。

SEQ1 段时间计算公式：S_SEQ1 = TQ*(CAN_LSEQ.SEQ1 + 2)

SEQ2 段时间计算公式：S_SEQ2 = TQ*(CAN_LSEQ.SEQ2 + 1)

波特率计算公式为：CAN Baudrate = 1/(S_SEQ1 + S_SEQ2)

CAN 通讯属于异步通讯，需要通过不断的重新同步才能保证收发节点的采样准确，所以，SJW （同步跳转宽度）决定了接收节点是否能有比较好的兼容性。CAN_LSEQ.S_SJW 是 SJW 位，其为重 同步跳转寄存器。在一定波特率下通讯的各个设备，允许多大的通讯时间误差存在。

总线下界容差 < 总线波特率 < 总线上限容差

容差计算公式为：TQ * (SJW + 1)

配置 CAN_LSEQ 寄存器，一般可以参考如下几条：

- S_SEQ1 比 S_SEQ2 要略大点，此时采样点会略晚于中间点，保证信号的稳定。也可以根据 实际情况调整，让采样点在此前后挪动。S_SEQ2 一般>=2（即 S_SEQ2>=1）。BOSCH 规定 S_SEQ1+S_SEQ2>=8。一般芯片时钟快，但是波特率比较低的时候，可以通过组合 CAN_LSEQ.S_PRESC，CAN_LSEQ.S_SEQ1 和 CAN_LSEQ.S_SEQ2 实现。
- SJW 的值不能大于 S_SEQ2，一般是后者的一半比较合理。
- 同一个 CAN 网络，波特率配置尽可能接近或一致，是比较好的选择。因各个芯片频率不同，CAN_LSEQ.S_PRESC 是不同的。波特率配置尽可能一致，是 CAN_LSEQ.S_PRESC 分频 后产生的 TQ 尽可能一致。TQ 一致了，S_SEQ1 和 S_SEQ2 以及 SJW 就基本可以一致了。

波特率索引值如下：

表 18-1 波特率索引值

CAN 波特率	CAN_LSEQ.S_PRESC[7:0]	CAN_LSEQ.S_SJW[6:0]	CAN_LSEQ.S_SEQ2[6:0]	CAN_LSEQ.S_SEQ1[7:0]	Sample Point
1Mbps	0x05	0x02	0x05	0x08	63%
800Kbps	0x0B	0x01	0x03	0x04	60%
666Kbps	0x0B	0x01	0x03	0x06	67%
500Kbps	0x0B	0x02	0x05	0x08	63%

400Kbps	0x0B	0x02	0x06	0x0B	65%
250Kbps	0x17	0x02	0x05	0x08	63%
200Kbps	0x17	0x02	0x06	0x0B	65%
125Kbps	0x2F	0x02	0x05	0x08	63%
100Kbps	0x2F	0x02	0x06	0x0B	65%
80Kbps	0x35	0x02	0x06	0x0B	65%
50Kbps	0x5F	0x02	0x06	0x0B	65%
40Kbps	0x77	0x02	0x06	0x0B	65%
25Kbps	0xBF	0x02	0x06	0x0B	65%
20Kbps	0xEF	0x02	0x06	0x0B	65%
10Kbps	0xEF	0x06	0x0D	0x18	65%
05Kbps	0xEF	0x0D	0x1B	0x32	65%

18.2.5 ID 过滤器

CAN 总线，可以挂在很多设备。通过 ID 号，不同设备可以知道当前总线上发送的帧，是否需要被自己接收，或者发送出去的帧，是否有响应。本 CAN 设备，共计 4 组 ID 过滤器（ACF0/1/2/3）。只有当 CAN 设备处于复位模式时，MCU 才可以配置 ID 过滤器。CAN 帧的 ID 号 有两种长度--11 位和 29 位。前者对应的 SFF（Standard Frame Format），后者对应 EFF（Extended Frame Format）。本 CAN 设备的 ID 滤波的长度均为 29 位。

如果当前接收到的帧，通过了其中一个 ID 过滤器，则它将被 CAN 设备接受。如果接受，此帧 的数据将存储到 Rx Buffer 中，如果使能了 RIE，同时会置位 RIF。如果帧，不被接受，则不会置位 RIF 且 Rx Buffer 指针也不增加。未被接受的帧，将被丢弃，并被下一帧覆盖。任何存储的有效帧， 都不会被任何未接受的帧覆盖。

通过 CAN_ACR 和 CAN_AMR 决定当前 CAN 设备可接收的 ID 范围。CAN_ACR 列出了一个特定 的 ID，CAN_AMR 为对应的 MASK 寄存器，标识 CAN_ACR 中对应的位数据同接收到的 ID 对应的位 的数据是否需要匹配。CAN_AMR 某一位是 0，表示这一位需要同 CAN_ACR 匹配；CAN_AMR 某一位是 1，表示这一位不需要同 CAN_ACR 匹配。极端情况就是 CAN_ACR 上每一位都要匹配，或者每一位都不要匹配。

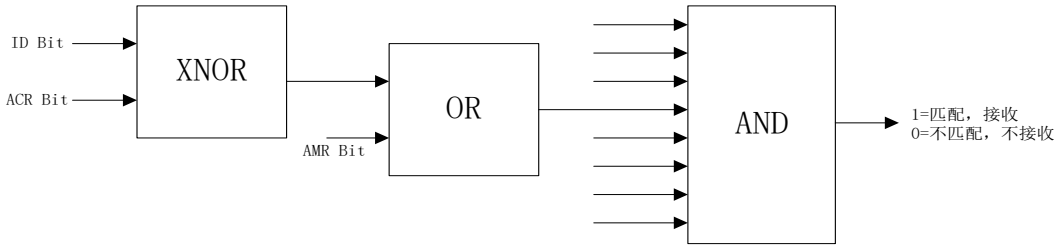


图 18-3 CAN 模块 ID 滤波逻辑关系

本 CAN 设备，通过 CAN_ACFTIM 和 CAN_ACF 两个寄存器实现对 CAN_ACR0/1/2/3 和 CAN_AMR0/1/2/3 的配置。CAN_ACRx 和 CAN_AMRx 为一组，共计四组 ID 过 滤器（ACF0/1/2/3）。CAN_ACFTIM 为选择寄存器，选择上述哪组过滤器中将被 MCU 选中；CAN_ACFEN 为使能寄存器，选择上述哪组过滤器将被使能/关闭；CAN_ACFTIM 选中 ACRx/AMRx，下一步通过访问 CAN_ACF，实现对 ACRx/AMRx 的读写。默认情况下，ACF0 使能，同时 CAN_ACR0 为全 0，CAN_AMR0 为全 1。

本 CAN 设备，CAN_ACRx 和 CAN_AMRx 均是按照最大 29-bit 设计，若是标准帧，ID 只有 11 bit，此时可以屏蔽 CAN_AMRx 对应高位（写 1）。

- 标准帧：CAN_ACRx[10:00]写入需要过滤的 ID 值；CAN_AMRx[28:11]写入全 1，CAN_AMRx[10:00]写入对应值。
- 扩展帧：CAN_ACRx[28:00]写入需要过滤的 ID 值；CAN_AMRx[28:00]写入对应值。

18.2.6 CAN 帧的接收

接收到的有效且被接受的帧，均存储到 Rx Buffer 中。一帧数据，存放到一个 Slot 中，本 CAN 设备，Rx Buffer 提供 10 个 Slot。Rx Buffer 按照 FIFO 模式存储和读出，即先存入 Rx Buffer 的帧， 先被 MCU 读取。

- 如果使能了 CAN_INTLMT.RIE，则每次接收到的有效且被接受的帧，CAN_INTLMT.RIF 将置位为 1。
- 如果使能了 CAN_INTLMT.RAFIE，则接收到的有效且被接受的帧的数量达到 CAN_INTLMT.AFWL 规 定的值，CAN_INTLMT.RAFIF 将置位为 1。

如果使能了 CAN_INTLMT.RFIE，则接收到的有效且被接受的帧占满 Rx Buffer，CAN_INTLMT.RFIF 将置位为 1。

通过 CAN_RBUF 寄存器来读取最早存入 Rx Buffer Slot 且没有被取走的帧。MCU 读取完毕后，将 CAN_RCTRL.RREL 设置为 1（硬件自动回零），释放已经读取的 Rx Buffer S

lot。CAN_RBUF 寄存 器自动指向下一个 Rx Buffer Slot。

哪怕 Rx Buffer 已满，本 CAN 设备仍然可以继续接收帧。一旦接收到的帧，是本 CAN 设备需 要的（通过 ID 滤波器），根据 CAN_STCMDCR.ROM 的配置，决定如何处理。CAN_STCMDCR.ROM 为 1，此新接受的帧，被抛弃；CAN_STCMDCR.ROM 为 0，最早进入 Rx Buffer 的帧，被抛弃。

CAN 帧的结构，分成 ID 部分和数据部分。第一个字节包含了帧分类等信息，确定是 SFF（标准）帧，还是 EFF（扩展）帧；确定是远程帧，还是数据帧；以及，确定数据长度。SFF 的 ID 长 度为 2 个字节，EFF 的 ID 长度为 4 个字节。数据长度为最大 8 个字节。

表 18-2 CAN 帧的结构表

SFF								
	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
CAN_RBUF0[31:24]	/							
CAN_RBUF0[23:16]	/							
CAN_RBUF0[15:08]	/					ID[10:08]		
CAN_RBUF0[07:00]	ID[07:00]							
CAN_RBUF1[31:24]	/							
CAN_RBUF1[23:16]	/							
CAN_RBUF1[15:08]	KOER[2:0]			TX	/			
CAN_RBUF1[07:00]	0	RTR	/		DLC[3:0]			
CAN_RBUF2[31:24]	DATA3							
CAN_RBUF2[23:16]	DATA2							
CAN_RBUF2[15:08]	DATA1							
CAN_RBUF2[07:00]	DATA0							
CAN_RBUF3[31:24]	DATA7							
CAN_RBUF3[23:16]	DATA6							
CAN_RBUF3[15:08]	DATA5							
CAN_RBUF3[07:00]	DATA4							
EFF								

	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
CAN_RBUF0[31:24]	/			ID[28:24]				
CAN_RBUF0[23:16]	ID[23:16]							
CAN_RBUF0[15:08]	ID[15:08]							
CAN_RBUF0[07:00]	ID[07:00]							
CAN_RBUF1[31:24]	/							
CAN_RBUF1[23:16]	/							
CAN_RBUF1[15:08]	KOER[2:0]			TX	/			
CAN_RBUF1[07:00]	1	RTR	/		DLC[3:0]			
CAN_RBUF2[31:24]	DATA3							
CAN_RBUF2[23:16]	DATA2							
CAN_RBUF2[15:08]	DATA1							
CAN_RBUF2[07:00]	DATA0							
CAN_RBUF3[31:24]	DATA7							
CAN_RBUF3[23:16]	DATA6							
CAN_RBUF3[15:08]	DATA5							
CAN_RBUF3[07:00]	DATA4							

IDE（IDentifier Extension）：0：标准帧格式；1：扩展帧格式

RTR（Remote Transmission Request）：0：数据帧；1：远程帧

DLC（Data Length Code）：数据长度码。设定范围为 0~8，对应数据长度为 0 Byte~8 Byte

KOER：同 CAN_ERRABR.KOER

TX：回环模式中接收自己发送的数据时此位置 1

CAN 帧的接收步骤如下：

- 配置 ID 滤波器
- 配置 CAN_INTLMT.RFIE，CAN_INTLMT.RAFIE 和 CAN_INTLMT.AFWL
- 等待 CAN_INTLMT.RFIF 或者 CAN_INTLMT.RAFIF（中断或者轮询皆可）
- 通过 CAN_RBUF 从 Rx Buffer 中读取最早接收到的 CAN 帧；

- 将 CAN_STCMDCR.RREL 设置为 1（硬 件自动回零），释放已经读取的 Rx Buffer Slot，CAN_RBUF 寄存器自动指向下一个 Rx Buffer Slot。如此反复，直至通过 CAN_STCMDCR.RSTAT 确认 Rx Buffer 为空。

18.2.7 CAN 帧的发送

CAN 模块要发送数据，必须在正常工作模式下。在开始任何传输之前，至少有一帧被写入传 输缓冲区（Primary Transmit Buffer 或 Secondary Transmit Buffer）。CAN_TBUF 寄存器提供对 Primary Transmit Buffer（PTB）和 Secondary Transmit Buffer（STB）的访问。本 CAN 设备优先发 送 PTB 的 CAN 帧。在发送空闲情况下，CAN_STCMDCR.TPE 配置为 1，即可触发 PTB 的数据发送，此时 STB 只能等待 PTB 发送完毕才能发送。

PTB 的存储深度是一，即只能存储一帧，STB 可以存储两帧。CAN_STCMDCR.TBSEL 为 1，选中 STB，为 0，选中 PTB。CAN_STCMDCR.TBSEL 为 1，MCU 可写入 STB。因可以存入两帧数据， CAN_TBUF 写入完成当前帧后，设置 CAN_STCMDCR.TSNEXT，硬件自动跳转到下一个 STB Slot。所有 CAN_TBUF 的数据都可以按任意顺序写入。CAN_STCMDCR.TSSTAT 可知道 STB 的状态。

CAN 帧的发送步骤如下：

配置 CAN_STCMDCR.TBSEL，选择发送哪个 Tx Buffer 的数据（0：PTB；1：STB）

将发送的数据，写入 CAN_TBUF 寄存器中

如果选择的是 STB，设置 TSNEXT=1 以完成 STB Slot 的装载

CAN_STCMDCR.TPE 写 1，触发 PTB 的 CAN 帧的发送；CAN_STCMDCR.TSALL 或者 CAN_STCMDCR.TSONE 写 1，触发 STB 的 CAN 帧的发送

发送完成状态确认。CAN_INTLMT.TPIE 使能，PTB 的 CAN 帧，发送完成后，CAN_INTLMT.TPIF 置 位；CAN_STCMDCR.TSONE 单帧模式，CAN_INTLMT.TSIE 使能，STB 的 CAN 帧，发送完成后， CAN_INTLMT.TSIF 置位；CAN_STCMDCR.TSALL 多帧模式，CAN_INTLMT.TSIE 使能，STB 的所有 CAN 帧， 发送完成后，CAN_INTLMT.TSIF 置位

注意，若 STB 和 PTB 均触发的发送请求，同等条件下，PTB 先发送

本 CAN 设备发送 CAN 帧，支持 CAN 仲裁。一旦仲裁失败，可以等总线空闲重发此帧，也支 持不再重发此帧。CAN_STCMDCR. TPSS 配置 PTB 是否重发（0：重发；1：单

次），CAN_STCMDCR. TSSS 配置 STB 是否重发（0：重发；1：单次）。配置为单次发送模式后，无论最后是否成功发送 此帧，CAN_INTLMT.TPIF/CAN_INTLMT.TSIF 均会被置位，对应的 Slot 均会被清除（数据失效）。此时， 需要借助另外两个状态位来判断：

有总线错误发生时，CAN_ERRABR.KOER 更新，CAN_INTLMT.BEIF 置位（CAN_INTLMT.BEIE 使能）

仲裁失败，CAN_INTLMT.ALIF 置位（CAN_INTLMT.ALIE 使能）

CAN 帧的最大有效负载长度为 8 字节。每帧的数据长度由 DLC 定义。对于远程帧（位 RTR），

DLC 的值是无意义，因为远程帧的数据长度始终为 0 字节。

表 18-3

SFF								
	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
CAN_TBUF0[31:24]	/							
CAN_TBUF0[23:16]	/							
CAN_TBUF0[15:08]	/					ID[10:08]		
CAN_TBUF0[07:00]	ID[07:00]							
CAN_TBUF1[31:24]	/							
CAN_TBUF1[23:16]	/							
CAN_TBUF1[15:08]	KOER[2:0]			TX	/			
CAN_TBUF1[07:00]	0	RTR	/		DLC[3:0]			
CAN_TBUF2[31:24]	DATA3							
CAN_TBUF2[23:16]	DATA2							
CAN_TBUF2[15:08]	DATA1							
CAN_TBUF2[07:00]	DATA0							
CAN_TBUF3[31:24]	DATA7							
CAN_TBUF3[23:16]	DATA6							
CAN_TBUF3[15:08]	DATA5							
CAN_TBUF3[07:00]	DATA4							

EFF								
	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]
CAN_TBUF0[31:24]	/			ID[28:24]				
CAN_TBUF0[23:16]	ID[23:16]							
CAN_TBUF0[15:08]	ID[15:08]							
CAN_TBUF0[07:00]	ID[07:00]							
CAN_TBUF1[31:24]	/							
CAN_TBUF1[23:16]	/							
CAN_TBUF1[15:08]	KOER[2:0]			TX	/			
CAN_TBUF1[07:00]	1	RTR	/		DLC[3:0]			
CAN_TBUF2[31:24]	DATA3							
CAN_TBUF2[23:16]	DATA2							
CAN_TBUF2[15:08]	DATA1							
CAN_TBUF2[07:00]	DATA0							
CAN_TBUF3[31:24]	DATA7							
CAN_TBUF3[23:16]	DATA6							
CAN_TBUF3[15:08]	DATA5							
CAN_TBUF3[07:00]	DATA4							

可以通过配置 CAN_STCMDCR.TPA 或者 CAN_STCMDCR.TSA 取消已请求但还没有被执行的数据发送。 有如下几种情况，需注意：

仲裁中。节点仲裁失败，则取消数据发送；节点仲裁成功，则继续发送

数据发送中。成功发送数据且收到 ACK，对应的标志和状态正常置位，数据发送不取消；成功 发送数据但没有收到 ACK，数据发送取消，错误计数器增加。CAN_STCMDCR.TSALL 发送，正在发 送的 STB Slot 正常发送，没有开始发送的 STB Slot 被取消

18.2.8 错误管理

CAN 协议要求每个节点中都包含发送错误计数和接收错误计数。这两个错误计数的数值决定了控制器当前的错误状态（如主动错误、被动错误、离线）。控制器将数值分别存储在

CAN_RECNT 和 CAN_TECNT 中，MCU 可随时进行读取。除了错误状态之外，控制器还提供错误报 警限制的功能（CAN_LIMIT.EWL），这个功能可在 CAN 控制器进入被动错误状态之前，提醒用户，当 前发生的严重总线错误。

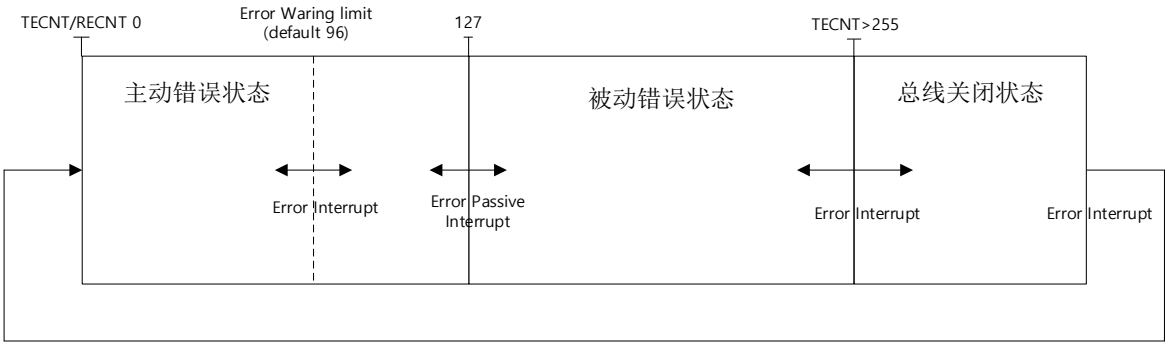


图 18-4 CAN 模块错误管理

18.2.9 错误计数

发送错误计数和接收错误计数根据下表规则递增/递减。注，一帧传输中可应用多个规则。

表 18-4

	接收和发送错误计数值的变动条件	发送错误计数值（TECNT）	接收错误计数值（RECNT）
1	接收单元检测出错误时 例外：接收单元在发送错误标志或过载标志中检测出“位错误” 时，接收错误计数值不增加	-	+1
2	接收单元在发送完错误标志后检测到的第一个位为显性电平	-	+8
3	发送单元在输出错误标志时	+8	-
4	发送单元在发送主动错误标志或过载标志时，检测出位错误	+8	-
5	接收单元在发送主动错误标志或过载标志时，检测出位错误	-	+8
6	接收单元在发送主动错误标志或过载标志时，检测出位错误	发送时+8	接收时+8

7	检测出在被动错误标志后追加的连续 8 个显性位时	发送时+8	接收时+8
8	发送单元正常发送数据结束时（返回 ACK 并且到帧结束，也 未检测出错误时）	TECNT=0 时， TECNT=0 TECNT-1	-
9	接收单元正常接收数据结束时（到 CRC 未检测出错误并且正 常返回 ACK 时）	-	RECNT=0 时， RECNT=0 1<=RECNT<=127 时， RECNT-1 RECNT>127 时， RECNT27
10	处于总线关闭状态的单元，检测到 128 次、连续 11 个隐性位	=0	=0

18.2.10 主动错误状态

当发送错误计数器（CAN_TECNT）和接收错误计数器（CAN_RECNT）的值都<=127 时，为主 动错误状态；主动错误状态下的 CAN 设备，可以正常参与 CAN 总线通讯，并在检测到错误时能发 出主动错误信号（连续 6 个显性信号）。

18.2.11 被动错误状态

当发送错误计数器（CAN_TECNT）或接收错误计数器（CAN_RECNT）的值>127 但<256 时，为被动错误状态；处于被动错误状态的 CAN 设备，检测到错误时，不能发出主动错误信号（连续 6 个显性信号）；只能发出被动错误信号（隐性信号），等待其他 CAN 设备发现错误。

18.2.12 关闭状态与关闭恢复

当发送错误计数器（CAN_TECNT）或接收错误计数器（CAN_RECNT）的值>255 时，本 CAN 设备自动进入关闭状态，从而不参与 CAN 总线的通信，直到返回到主动错误状态。可以通过 CAN_STCMDCR.BUSOFF 位确认本 CAN 设备的状态。BUSOFF 被置位的同

时，产生 CAN_INTLMT.EIF 中断。

CAN 从关闭状态恢复到主动错误状态有以下两种方法：

上电复位

接收到连续 128 次、连续 11 位的隐性位序列（恢复序列） 节点关闭状态下，CAN_TECNT 值保持不变，CAN_RECNT 用于计数恢复序列。从节点关闭状态 恢复后，CAN_TECNT 和 CAN_RECNT 被复位为 0。

18.2.13 仲裁

本 CAN 设备能够精确捕捉到仲裁失败的位置点，并反映到 CAN_ERRABR.ALC 寄存器中。 CAN_ERRABR.ALC 寄存器中保存着，最近一次仲裁失败的位置点，如果本 CAN 设备赢得仲裁，则 CAN_ERRABR.ALC 位不更新。

CAN_ALC 值定义如下：

SOF 位后，第一个 ID 数据位 CAN_ERRABR.ALC 为 0，第二个 ID 数据位 CAN_ERRABR.ALC 为 1， 依次类推。因为仲裁只发生在仲裁场内，所以 CAN_ERRABR.ALC 的最大值为 31。比如一个标准格 式远程帧和一个扩展帧仲裁，扩展帧将在 IDE 位争抢总线失败，则 CAN_ERRABR.ALC=12。

18.3 寄存器列表

CAN 在芯片中的基地址是 0x4002_2000

表 18-5 CAN 寄存器地址分配

名称	偏移地址	描述
CAN_RBUF	0x00~0x4F	CAN 接收缓冲区
CAN_TBUF	0x50~0x97	CAN 发送缓冲区
CAN_TTS	0x98~0x9F	CAN 发送时间戳
CAN_STCMDCR	0xA0	CAN 状态/命令/控制寄存器
CAN_INTLMT	0xA4	CAN 中断/阈值寄存器
CAN_LSEQ	0xA8	CAN 低速位时序寄存器
CAN_HSEQ	0xAC	CAN 快速位时序寄存器
CAN_ERRABR	0xB0	CAN 错误与仲裁寄存器

CAN_ACFTIM	0xB4	CAN 滤波器控制与时间戳寄存器
CAN_ACF	0xB8	CAN 验收码寄存器
CAN_VERTTCR	0xBC	CAN 版本信息 TTCAN 控制寄存器
CAN_TTCANMSG	0xC0	CAN TTCAN 参考报文寄存器
CAN_TTCANCFG0	0xC4	CAN TTCAN 触发配置寄存器 0
CAN_TTCANCFG1	0xC8	CAN TTCAN 触发配置寄存器 1

18.4 CAN 寄存器描述

18.4.1 CAN_RBUF CAN 接收缓冲区

地址:0x4002_2000 ~ 0x4002_204F

复位值:0x0

位域	类型	复位	名称	描述
[31:0]	RO	0	RBUF	接收缓冲区和接收时间戳

18.4.2 CAN_TBUF CAN 发送缓冲区

地址:0x4002_2050 ~ 0x4002_2097

复位值:0x0

位域	类型	复位	名称	描述
[31:0]	R/W	0	TBUF	发送缓冲区

18.4.3 CAN_TTS CAN 发送时间戳

地址:0x4002_2098 ~ 0x4002_209F

复位值:0x0

位域	类型	复位	名称	描述
[31:0]	RO	0	TTS	传输时间戳 (TTS) 用于 CiA 603 标准，记录上一帧报文的发送时间戳。当 TTSEN=1 时，新报文会覆盖该寄存器。

18.4.4 CAN_STCMDCR　CAN 状态命令控制寄存器

地址:0x4002_20A0

复位值:0x0

位域	类型	复位	名称	描述
[31]	R/W	0	SACK	自应答使能 仅外部回环 (LBME) 生效： 1：开启自应答； 0：关闭自应答；
[30]	R/W	0	ROM	接收溢出模式 RBUF 满时新报文处理规则： 0：覆盖 RBUF 最旧报文； 1：直接丢弃新报文；
[29]	RO	0	ROV	接收溢出标志 1：发生报文覆盖 / 丢失； 0：无溢出 写 RREL=1 可清除该位
[28]	R/W	0	RREL	RBUF 插槽释放 主机读完报文后置 1，指针跳转至下一帧，同时清零 ROV
[27]	R/W	0	RBALL	接收所有帧 0：仅存储合法数据帧； 1：存储含错误的数据帧（总线监听用途）
[26]	--	--	--	--
[25:24]	RO	0	RSTAT	RBUF 状态（2 位） 00：缓冲区空； 01：未达告警阈值； 10：达到告警阈值； 11：缓冲区满

[23]	R/W	0	FD_ISO	CAN FD 模式选择 0：博世非标准 CAN FD； 1：ISO 11898-1 标准 CAN FD 两类 CRC / 填充规则不兼容，同网络不可混用
[22]	R/W	0	TSNEXT	STB 插槽切换 写完一帧后置 1，标记当前插槽已满并切换至下一个空闲槽 PTB 模式下无效；STB 满时该位保持置 1
[21]	R/W	0	TSMODE	STB 工作模式 0：FIFO 模式（按写入顺序发送）； 1：ID 优先级模式 仅 STB 为空时允许切换
[20]	R/W	0	TTTBM	TTCAN 缓冲区模式 0：混合模式（保留常规收发 + 时间戳）； 1：全 TTCAN 模式（所有插槽统一编址，无优先级） 仅 TBUF 为空时允许切换
[19:18]	--	--	--	--
[17:16]	RO	0	TSSTAT	STB 状态（2 位） 00：STB 空； 01：半满及以下； 10：超过半满； 11：STB 满 TTCAN 模式下定义变更：00 = 全空，11 = 全满
[15]	R/W	0	TBSEL	发送缓冲区选择 0：选中 PTB（主缓冲区）； 1：选中 STB（次缓冲区） 读写 TBUF 期间该位必须保持稳定；TTCAN 模式

				下硬件清零
[14]	R/W	0	LOM	监听模式 0：禁用； 1：启用（纯总线监听，不发送任何显性位、无应答） 传输请求有效时无法置位
[13]	R/W	0	STBY	收发器待机模式 0：收发器正常工作； 1：收发器进入低功耗 报文发送请求有效时无法置位
[12]	R/W	0	TPE	PTB 发送使能 1：启动 PTB 报文传输； 0：无请求 仅主机可置 1，传输 / 中止完成后硬件自动清零
[11]	R/W	0	TPA	PTB 传输中止 1：中止未启动的 PTB 传输； 0：无动作 置位后清除 TPE，报文数据保留，硬件自动清零
[10]	R/W	0	TSONE	STB 单帧发送 1：发送 STB 中一帧报文； 0：无请求 FIFO 取最旧帧，优先级模式取最高 ID 帧
[9]	R/W	0	TSALL	STB 全部发送 1：连续发送 STB 所有报文（含期间新写入帧）； 0：无请求 与 TSONE 同时置位时，仅 TSALL 生效
[8]	R/W	0	TSA	STB 传输中止 1：中止 STB 未启动传输；

				0：无动作 中止后 STB 报文永久丢失，硬件自动清零
[7]	R/W	0	RESET	软件复位位 1：发起 CAN-CTRL 本地软件复位； 0：正常工作 仅该位为 1 时，位时序、滤波器等配置寄存器可写 节点进入总线关闭状态时硬件自动置 1；清除后需等待 11 个 CAN 位时间恢复总线通信
[6]	R/W	0	LBME	外部回环模式 0：禁用； 1：启用 禁止在报文传输过程中切换该位
[5]	R/W	0	LBMI	内部回环模式 0：禁用； 1：启用 断开外部总线，内部短接收发，用于芯片自测，禁止传输中切换
[4]	R/W	0	TPSS	PTB 单次传输模式 0：禁用（出错 / 仲裁丢失自动重传）； 1：启用（仅发送一次，不重传）
[3]	R/W	0	TSSS	STB 单次传输模式 0：禁用； 1：STB 开启单次传输模式
[2]	RO	0	RACTIVE	接收状态 1：内核正在接收报文； 0：无接收活动
[1]	RO	0	TACTIVE	发送状态 1：内核正在发送报文；

				0：无发送活动
[0]	R/W	0	BUSOFF	总线关闭状态 1：节点脱离 CAN 总线（TECNT>255）； 0：总线正常 主机写 1 可手动清零收发错误计数器（仅调试使用）

18.4.5 CAN_INTLMT CAN 中断阈值寄存器

地址:0x4002_20A4

复位值:0x0

位域	类型	复位	名称	描述
[31:28]	R/W	4'h1	AFWL	RBUF 半满告警阈值 根据 RBUF 总插槽数自动换算阈值： 插槽 ＜16→ 阈值 ＝AFWL；16＜ 插槽≤32→ 2×AFWL；32＜ 插槽 →4×AFWL AFWL=0 视为 1，超出插槽数则取最大值
[27:24]	R/W	4'hB	EWL	错误告警阈值 实际阈值 ＝(EWL＋1)×8，范围 8~128（步长 8） 错误计数器触及该阈值时触发 EWARN/EIF
[23]	RO	0	EWARN	错误告警标志 1：收发错误计数器达到预设阈值； 0：正常
[22]	RO	0	EPASS	错误被动状态 1：节点处于错误被动模式； 0：错误主动模式
[21]	R/W	0	EPIE	错误被动中断使能： 1＝ 开启； 0＝ 关闭

[20]	R/W	0	EPIF	错误被动切换标志： 状态变化时置位
[19]	R/W	0	ALIE	仲裁丢失中断使能： 1 = 开启； 0 = 关闭
[18]	R/W	0	ALIF	仲裁丢失标志： 总线仲裁失败时置位
[17]	R/W	0	BEIE	总线错误中断使能： 1 = 开启； 0 = 关闭
[16]	R/W	0	BEIF	总线错误标志： 检测位错误、格式错误等总线异常时置位
[15]	R/W	0	RIF	接收中断标志： 1 = 收到有效报文
[14]	R/W	0	ROIF	溢出中断标志： 1=RBUF 发生溢出
[13]	R/W	0	RFIF	RBUF 满标志： 1 = 缓冲区已满
[12]	R/W	0	RAFIF	半满告警标志： 1 = 达到预设告警阈值
[11]	R/W	0	TPIF	PTB 发送完成标志： TTCAN 模式下永久为 0
[10]	R/W	0	TSIF	STB 发送完成标志： TTCAN 下上报所有传输完成事件
[9]	R/W	0	EIF	错误中断标志： 错误计数器越限、总线状态切换时置位
[8]	R/W	0	AIF	传输中止标志： 执行 TPA/TSA 中止操作后置位；无独立使能位

[7]	R/W	1	RIE	接收中断使能： 1 = 开启； 0 = 关闭
[6]	R/W	1	ROIE	接收溢出中断使能： 1 = 开启； 0 = 关闭
[5]	R/W	1	RFIE	RBUF 满中断使能： 1 = 开启； 0 = 关闭
[4]	R/W	1	RAFIE	RBUF 半满告警中断使能： 1 = 开启； 0 = 关闭
[3]	R/W	1	TPIE	PTB 发送完成中断使能： 1 = 开启； 0 = 关闭
[2]	R/W	1	TSIE	STB 发送完成中断使能： 1 = 开启； 0 = 关闭
[1]	R/W	1	EIE	通用错误中断使能： 1 = 开启； 0 = 关闭
[0]	R/W	0	TSFF	STB 满标志 常规模式：1=STB 已满； TTCAN 模式：1 = 当前插槽已满

18.4.6 CAN_LSEQ CAN 低速位时序寄存器

地址:0x4002_20A8

复位值:0x0

位域	类型	复位	名称	描述
[31:24]	R/W	8'h1	S_PRESC	TQ 预分频率设定值（S_Prescaler），主要配置 TQ 的大小。 TQ = system clock period *(S_PRESC+1)
[23]	--	--	--	--
[22:16]	R/W	7'h2	S_SJW	低速同步跳转宽度： 最大同步偏移 = (S_SJW + 1) × TQ
[15]	--	--	--	--
[14:8]	R/W	7'h2	S_SEQ2	低速相位段 2： 采样点后时长 = (S_Seg_2 + 1) × TQ
[7:0]	R/W	8'h3	S_SEQ1	低速相位段 1： 采样点 = (S_Seg_1 + 2) × TQ

18.4.7 CAN_HSEQ CAN 高速位时序寄存器

地址:0x4002_20AC

复位值:0x0

位域	类型	复位	名称	描述
[31:24]	R/W	8'h1	F_PRESC	高速时钟分频系数： 实际分频 = 寄存器值 +1，范围 1~256
[23]	--	--	--	--
[22:16]	R/W	7'h2	F_SJW	高速同步跳转宽度： 最大同步偏移 = (F_SJW + 1) × TQ
[15]	--	--	--	--
[14:8]	R/W	7'h2	F_SEQ2	高速相位段 2： 采样点后时长 = (F_Seg_2 + 1) × TQ
[7:0]	R/W	8'h3	F_SEQ1	高速相位段 1： 采样点 = (F_Seg_1 + 2) × TQ

18.4.8 CAN_ERRABR　CAN 错误与仲裁寄存器

地址:0x4002_20B0

复位值:0x0

位域	类型	复位	名称	描述
[31:24]	RO	0	TECNT	发送错误计数； 总线关闭状态下可能溢出
[23:16]	RO	0	RECNT	接收错误计数； 遵循 CAN 标准增减规则，无溢出
[15]	R/W	0	TDCEN	TDC 使能 1：CAN FD 数据段启用发送延迟补偿； 0：禁用
[14:8]	R/W	0	SSPOFF	次级采样点偏移 单位 TQ，配合 TDC 计算 CAN FD 次级采样点位置
[7:5]	RO	0	KOER	错误类型码，记录最后一次总线错误
[4:0]	RO	0	ALC	仲裁丢失位置 记录帧内发生仲裁的位序号，仅仲裁失败时更新，最大值 31

18.4.9 CAN_ACFTIM　CAN 滤波器控制与时间戳寄存器

地址:0x4002_20B4

复位值:0x0

位域	类型	复位	名称	描述
[31:24]	R/W	0	AE15~AE8	启用接收过滤器 1 - 接收过滤器已开启 0 - 接收过滤器关闭 每一组接收过滤器（地址屏蔽码/地址编码）均可单独开启或关闭。

				关闭状态的过滤器会直接丢弃报文；只有开启的过滤器，在匹配对应的地址屏蔽码/地址编码配置时，才能接收报文。
[23:16]	R/W	8'h1	AE7~AE0	验收过滤器使能 1 - 验收过滤器开启 0 - 验收过滤器关闭 每个验收过滤器（屏蔽寄存器 AMASK / 校验码寄存器 ACODE）均可单独开启或关闭。硬件复位后，仅 0 号过滤器默认处于开启状态。 关闭的过滤器会直接丢弃报文；只有开启的过滤器，在对应的 AMASK/ACODE 配置匹配时，才会接收报文。 若需接收所有报文，需将某一路过滤器 x 配置为：AE_x=1（开启该过滤器）、AMASK_x=0xff、ACODE_x=0x00。硬件复位后，0 号过滤器默认为此配置，其余所有过滤器均为关闭状态。
[15:10]	--	--	--	--
[9]	R/W	1	TIMEPOS	时间戳位置 0 – 帧起始符 1 – 帧结束符 仅当 TIMEEN=0 时可修改 TIMEPOS，但可通过置位 TIMEEN=1 的同一写操作修改 TIMEPOS
[8]	R/W	0	TIMEEN	启用时间戳功能 0 – 关闭 1 – 开启
[7:6]	--	--	--	--
[5]	R/W	0	SELMASK	选择验收掩码 0 -ACF_x 寄存器指向验收代码

				1 -ACF_x 寄存器指向验收掩码。 ACFADR 用于选定某一特定验收滤波器。
[4]	--	--	--	--
[3:0]	R/W	0	ACFADR	滤波器地址 选择 1~16 号滤波器； 地址超出总数时自动取最大值

18.4.10 CAN_ACF CAN 验收码寄存器

地址:0x4002_20B8

复位值:0x0

位域	类型	复位	名称	描述
[31]	--	--	--	--
[30]	R/W	0	IDCMP	SELMASK=1 时有效，ID AMR（MASK）选择作用范围 1：ID 滤波器接收 MASK 按照标准帧格式或者扩展帧格式处理 0：ID 滤波器接收 MASK 兼容标准帧格式和扩展帧格式两种格式
[29]	R/W	0	IDMASK	IDCMP 位为 1 时，MASK 选择哪种帧格式 1：ID 滤波器仅接收扩展帧格式 0：ID 滤波器仅接收标准帧格式
[28:00]	R/W	0	IDCODE/IDMASK	ACFADR 选择是那一组 ID 滤波器 SELMASK=0，表示 ID 滤波器选择的是 ID ACR SELMASK=1，表示 ID 滤波器选择的是 AMR 标准帧格式时 ID ACR/AMR 的范围是 Bit10-Bit0，扩展帧格式时的范围 是 Bit28-Bit0

18.4.11 CAN_VERTTCR CAN 版本信息 TTCAN 控制寄存器

地址:0x4002_20BC

复位值:0x0

位域	类型	复位	名称	描述
[31]	R/W	1	WTIE	看门狗触发中断使能
[30]	R/W	0	WTIF	看门触发中断标志位 当周期计数达到定时看门触发阈值（TT_WTRIG）设定的限值，且看门中断使能位（WTIE）置位时，WTIF 标志位将会被置 1。
[29]	R/W	0	TEIF	触发错误中断标志位 触发配置非法时置位，写 1 清零（无使能位）
[28]	R/W	1	TTIE	时间触发中断使能 若置位 TTIE，当周期时间等于触发时间 TT_TRIG 时，将置位 TTIF。
[27]	R/W	0	TTIF	时间触发中断标志位 若 TTIE 置 1，且周期时间等于触发时间 TT_TRIG 时，TTIF 将被置位。 向 TTIF 写入 1 可将其清零，写入 0 则无任何作用。 TTIF 仅会置位一次。若未更新 TT_TRIG 数值，则在下一个基础周期中，TTIF 不会再次置位。
[26:25]	R/W	0	T_PRESC	TTCAN 定时器预分频器 00 - 1 01 - 2 10 - 4 11 - 8 TTCAN 时基为 CAN 位定时，由 S_PRES、S_SEG_1 与 S_SEG_2 参数定义。 通过 T_PRESC 可额外设定 1、2、4 或 8 倍的预分频系数。 仅当 TTEN=0 时才能修改 T_PRESC，但可通过单次

				写操作同时修改 T_PRESC 并配置 TTEN。
[24]	R/W	0	TTEN	时间触发器启用 1 - 时间触发控制器启用，定时器运行中 0 - 已禁用
[23]	R/W	0	TBE	插槽清空标记 置 1 将 TBPTR 指向的插槽标记为空，硬件自动清零
[22]	R/W	0	TBF	插槽填满标记 置 1 标记插槽为已满，硬件自动清零； TBE 优先级高于 TBF
[21:16]	R/W	0	TBPTR	缓冲区指针 0 = 指向 PTB；其余值指向 STB 插槽；指针越界则操作无效
[15:8]	RO	8'h7	VER_1	存储 IP 主版本号
[7:0]	RO	8'h6	VER_0	存储 IP 次版本号

18.4.12 CAN_TTCANMSG CAN TTCAN 参考报文寄存器

地址:0x4002_20C0

复位值:0x0

位域	类型	复位	名称	描述
[31]	R/W	0	REF_IDE	参考信息标识位
[30:29]	--	--	--	--
[28:0]	R/W	0	REF_ID	参考报文标识符。 若 REF_IDE 为 1 — REF_ID(28:0)有效（扩展标识符） 0 — REF_ID(10:0)有效（标准标识符） REF_ID 用于 TTCAN 模式下识别参考报文。该规则对时间从节点（接收）与时间主节点（发送）均适

				用。若识别到参考报文且无错误，则该帧的同步标记将置为参考标记。 REF_ID(2:0)不参与校验，对应寄存器位强制置 0。该组比特位最多可区分 8 个备选时间主节点。 CAN 控制器仅依据标识符识别参考报文，不对报文数据段进行校验。 补充说明：时间主节点发送参考报文的方式与普通数据帧一致。REF_ID 用于判定参考报文是否发送成功。

18.4.13 CAN_TTCANCFG0 CAN TTCAN 触发配置寄存器 0

地址:0x4002_20C4

复位值:0x0

位域	类型	复位	名称	描述
[31:24]	R/W	0	TT_TRIG1	触发时间 TT_TRIG (15:0) 用于定义触发事件的周期时长。对于发送类触发，对应帧的帧起始段（SOF）最早发送时刻为 TT_TRIG 数值加 1。
[23:16]	R/W	0	TT_TRIG0	
[15:12]	R/W	0	TEW	发送使能窗口 对于单次发送触发，存在一段最长为 16 个周期时标的时间区间，帧可在此区间内开始发送。 TWE+1 用于定义时标数量。 TEW=0 为有效配置，该配置会将发送使能窗口缩短至 1 个时标。
[11]	--	--	--	--

[10:8]	R/W	0	TTYPE	触发类型 000 - 即时触发：用于即时传输 001 - 时间触发：用于接收触发信号 010 - 单次发送触发：用于独立时间窗口 011 - 发送启动触发：用于合并仲裁时间窗口 100 - 发送停止触发：用于合并仲裁时间窗口 其他值 - 无操作 触发时刻由 TT_TRIG 寄存器定义。TTPTR 用于为发送触发信号选择 TB 时隙。
[7:6]	--	--	--	--
[5:0]	R/W	0	TTPTR	传输触发 TB 时隙指针 若 TTPTR 数值过大，指向一个不可用的时隙，则会置位 TEIF，且对 TT_TRIG_1 执行写访问后无法激活新触发。 若 TTPTR 指向空时隙，则到达触发时刻时将立即置位 TEIF。

对 TT_TRIG_1 执行写访问操作，会将触发配置数据传输至 CAN 时钟域（跨时钟域操作）并启用该触发器。当触发器处于启用状态时，TRIG_CFG_0、TRIG_CFG_1、TT_TRIG_0 与 TT_TRIG_1 全部寄存器将开启写锁定保护。当触发时刻到来（若 TTIE 使能，则 TTIF 置位），或检测到故障（TEIF 置位）时，写锁定状态解除。

18.4.14 CAN_TTCANCFG1 CAN TTCANCFG 触发配置寄存器 1

地址:0x4002_20C8

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:8]	R/W	8'hFF	TT_WTRIG1	看门触发时间

[7:0]	R/W	8'hFF	TT_WTRIG0	TT_WTRIG(15:0) 用于定义看门触发的周期时长。看门触发的初始值为最大周期值 0xffff。

19 ADC

19.1 概述

19.1.1 简介

MCU 内集成了 12BIT SAR-ADC 模块，采样可以来自于 24 个外部通道和模拟内部 CSA 模块，温度传感器模块和内部电源模块信号。

19.1.2 功能特点

- 1.5Msps 采样率，32MHz/96MHz 工作频率
- 支持自定义转换序列，最多可支持 16 个转换序列
- 支持转换序列分段配置，可配置 1 段转换，2 段转换，3 段转换和 4 段转换
- 支持模拟看门狗，可对任意转换序列进行上下阈值监测
- 支持软件、硬件触发
- 硬件触发源可来自 MCPWM/TIMER
- 支持过采样转换结果求平均功能
- 支持转换结果校准功能

19.1.3 模块框图

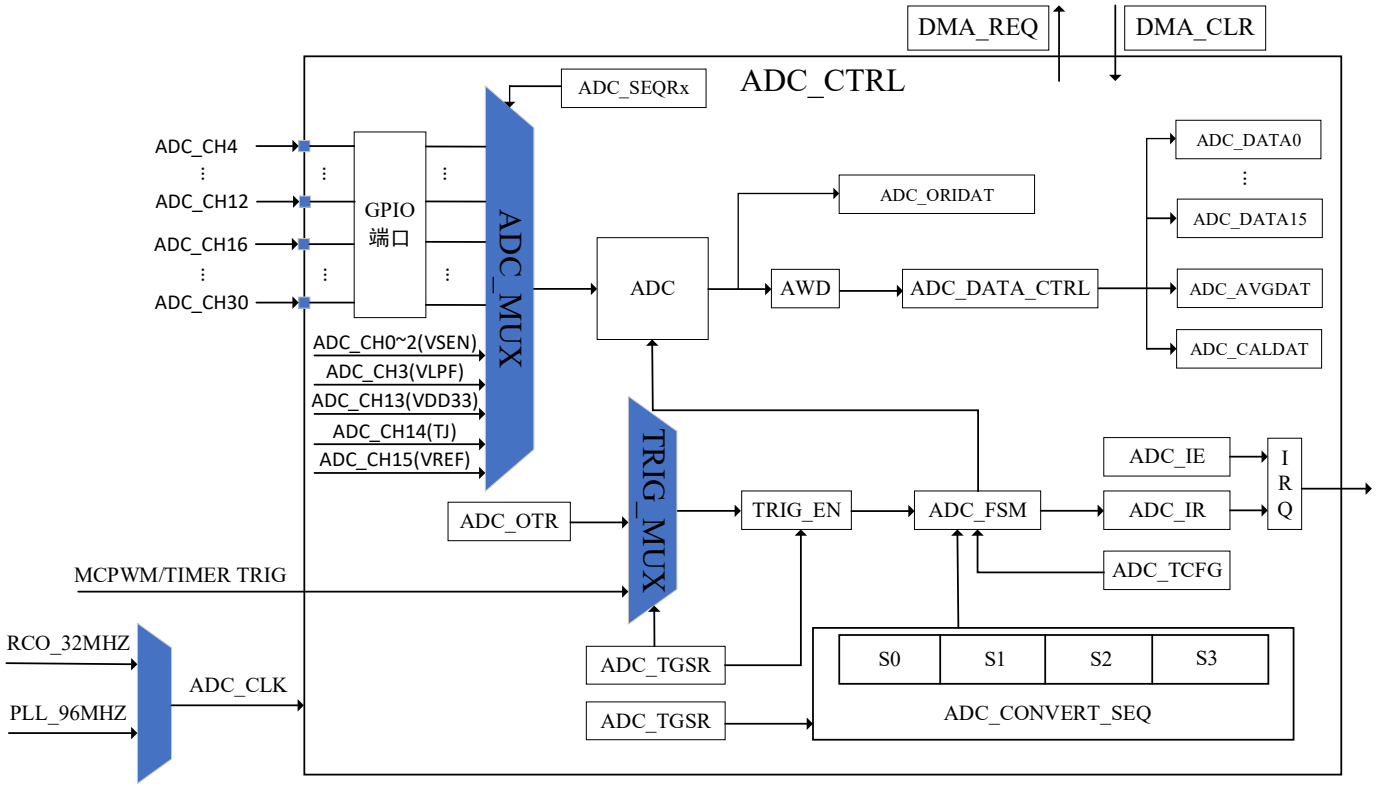


图 19-1 ADC 模块框图

19.2 功能描述

19.2.1 ADC 转换序列

ADC 可自定义转换序列的段数和每段的序列数。每个 ADC 的触发，触发的是每段的转换。序列数量通过 ADC_TGNR 配置，TNG0/1/2/3 信号分别配置 S0/1/2/3 段的转换序列数量。每段序列数量配置区间都是 0-16，最大配置 16 个转换序列。

ADC_DATA0~ADC_DATA15 对应的结果是转换序列 0~15，若配置序列总数量不满 16 个，按 ADC_DATA0 至 ADCDATA15 顺序填充。

每段转换序列都有独立的转换完成中断标志（S0/1/2/3CDIF）和一个任意段转换完成标志位(SACDIF)。

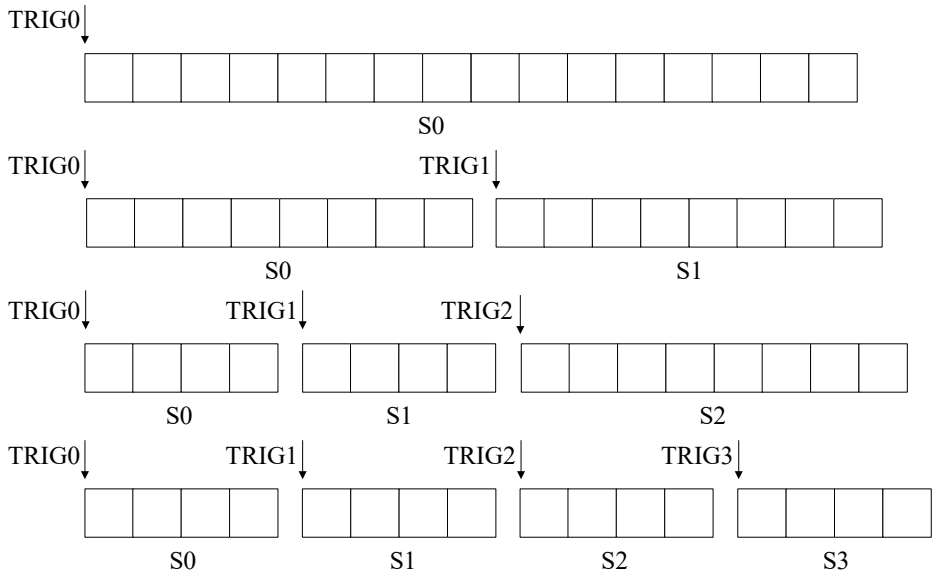


图 19-2 ADC 转换序列

19.2.2 ADC 触发方式

ADC 有软件触发和硬件触发两种方式，每段序列的触发源和触发使能可独立配置。但不同段的转换序列不可选用相同的触发源，否则会造成转换序列触发错误。

硬件触发源可选择 MCPWM 的 ADC_TRIG0/1/2/3 和 TIMER 的 TRIG(含捕获和匹配事件)。

软件触发源通过对 ADC_OTR 寄存器的低 16 位，写入 0x5AA5，可产生一次触发。

触发源选择详见 ADC_TGSR 寄存器描述。

19.2.3 ADC 通道选择

ADC 通道通过 ADC_SEQR0/1/2/3 寄存器配置，可配置每个转换序列对应的 ADC 采样通道。

表 19-1

ADC 转换序列	转换序列对应的结果寄存器	采样通道选择配置
转换序列 0	ADC_DATA0	CHSEQ0
转换序列 1	ADC_DATA1	CHSEQ1
转换序列 2	ADC_DATA2	CHSEQ2
转换序列 3	ADC_DATA3	CHSEQ3
转换序列 4	ADC_DATA4	CHSEQ4
转换序列 5	ADC_DATA5	CHSEQ5
转换序列 6	ADC_DATA6	CHSEQ6

转换序列 7	ADC_DATA7	CHSEQ7
转换序列 8	ADC_DATA8	CHSEQ8
转换序列 9	ADC_DATA9	CHSEQ9
转换序列 10	ADC_DATA10	CHSEQ10
转换序列 11	ADC_DATA11	CHSEQ11
转换序列 12	ADC_DATA12	CHSEQ12
转换序列 13	ADC_DATA13	CHSEQ13
转换序列 14	ADC_DATA14	CHSEQ14
转换序列 15	ADC_DATA15	CHSEQ15

每一个转换序列都对应一个结果寄存器，不同的转换序列可以设置相同的 CHSEQx 值，选择相同的 ADC 采样通道。

ADC 采样通道选择见下表 19-2：

表 19-2

CHSEQx	ADC_MUX
00000	CSA_VSEN
00001	CSA_VSEN
00010	CSA_VSEN
00011	CSA_VLPF
00100	PB12
00101	PB13
00110	PB14
00111	PC3
01000	PC4
01001	PC5
01010	PC6
01011	PC7
01100	PC8

01101	VDD33_VSEN
01110	TJ_REF
01111	VREF_1P20
10000	PA0
10001	PA1
10010	PA2
10011	PA3
10100	PA4
10101	PA5
10110	PA6
10111	PA7
11000	PA8
11001	PA9
11010	PA10
11011	PA11
11100	PA12
11101	PA13
11110	PA14

19.2.4 模拟看门狗

ADC 每个转换序列都配备模拟看门狗功能，可同时监测转换序列的上下阈值。ADC_AWE 可单独配置是否启用某个转换序列的阈值监测功能，通过 ADC_AWTH 寄存器可以配置监测的上下阈值。详细的上下阈值配置请查看 ADC_AWTH 寄存器描述。

如果使能了阈值监测功能，并配置了上下阈值，则在对应转换序列转换完成后，会进行阈值比较，经比较的数据是 ADC 的原始数据，不经过过采样求平均以及校准。

19.2.5 ADC 校准

该 ADC 校准模块对原始 12 位 ADC 转换数据进行增益（GAIN）和偏移（OFFSET）校准，输出 24 位校准结果。校准使能信号通过 ADC_CR 寄存器的 CALIB_EN 控制。

校准公式如下：

ADC_CALDAT = ADC_RAW*GAIN+OFFSET

GAIN：12 位增益系数，定点小数格式，范围 0~19995，step 为 0.0005

OFFSET：12 位偏移系数，有符号整数，范围 -2048 ~ +2047

最终输出 ADC_CALDAT 为 24 位

当校准发生上溢时，结果会输出满幅值 0x7FFFFF，单校准发生下溢时，结果会输出 0。

19.2.6 ADC 过采样

ADC 支持对信号过采样求平均后再写入 ADC_AVGDATA 数据寄存器，过采样范围可以通过 ADC_CR 寄存器的 AVG_MODE 配置，范围为 1~128 倍采样。详见 ADC_CR 寄存器。配置了采样倍数后还应该通过 ADC_CR 的 AVG_SEQ 配置过采样序列。

配置了过采样后，每段序列的转换时间按选择的倍数增加，在设定的采样转换次数后才产生完成中断。

19.2.7 ADC 中断

ADC 中断事件的标志和使能如下表 19-3 所示：

表 19-3

中断事件	事件标志	中断使能控制位
模拟看门狗事件	AWDIF	AWDIE
序列触发错误事件	TGTERRIF	TGTERRIE
序列数据设置错误	TGNERRIF	TGNERRIE
过采样求平均完成事件	AVGUPIF	AVGUPIE
校准完成事件	CALUPIF	CALUPIE
任意段序列转换完成	SACDIF	SACDIE
第 3 段序列转换完成	S3CDIF	S3CDIE
第 2 段序列转换完成	S2CDIF	S2CDIE
第 1 段序列转换完成	S1CDIF	S1CDIE
第 0 段序列转换完成	S0CDIF	S0CDIE

19.2.8 DMA 功能

可以设置 ADC_CR 的 DME_EN 位来使能，用来传输转换结果，每一个序列的转换完成后，都会产生一个 DMA 请求。DMA 接收到请求后可以将转换的数据从 ADC_DATAx 寄存器传输到用户指定的目的地址。

19.2.9 ADC 时序设置

可以设置 ADC_TCFG 寄存器的信号，控制 ADC 模块的时序。

如下图所示:

T0：对应 ADC_TCFG 寄存器的 INITIAL_PULSE 信号，可配置 ADC_EN 至 ADC_RDY 时间。

T1：对应 ADC_TCFG 寄存器的 START_PILSE 信号，可配置 ADC_START 触发信号的脉宽。

T2：对应 ADC_TCFG 寄存器的 SEQUENCE_PULSE 信号，可配置每段采样初始的额外延时，用于延长 ADC_MUX 的建立时间。

T3：对应 ADC_TCFG 寄存器的 SAMPLE_PILSE 信号，可配置采样完成后，到 ADC_MUX 切换的时间。

T4：对应 ADC_TCFG 寄存器的 INTERVAL_PULSE 信号，可配置采样+转换完成后，到下一个通道转换的时间。

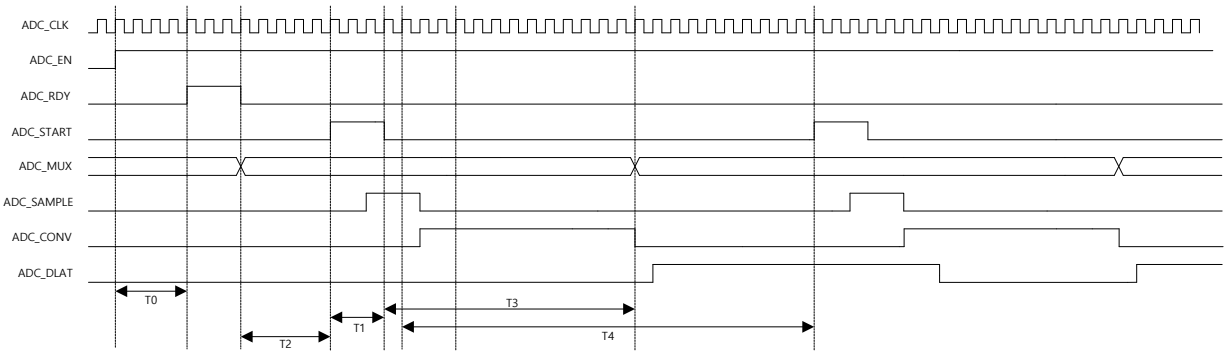


图 19-2 ADC 时序设置寄存器列表

ADC 在芯片中的基地址是 0x4001_2400

表 19-4 ADC 寄存器地址分配

名称	偏移地址	描述
ADC_DATA0	0x04	ADC 数据寄存器 0
ADC_DATA1	0x08	ADC 数据寄存器 1

ADC_DATA2	0x0C	ADC 数据寄存器 2
ADC_DATA3	0x10	ADC 数据寄存器 3
ADC_DATA4	0x14	ADC 数据寄存器 4
ADC_DATA5	0x18	ADC 数据寄存器 5
ADC_DATA6	0x1C	ADC 数据寄存器 6
ADC_DATA7	0x20	ADC 数据寄存器 7
ADC_DATA8	0x24	ADC 数据寄存器 8
ADC_DATA9	0x28	ADC 数据寄存器 9
ADC_DATA10	0x2C	ADC 数据寄存器 10
ADC_DATA11	0x30	ADC 数据寄存器 11
ADC_DATA12	0x34	ADC 数据寄存器 12
ADC_DATA13	0x38	ADC 数据寄存器 13
ADC_DATA14	0x3C	ADC 数据寄存器 14
ADC_DATA15	0x40	ADC 数据寄存器 15
ADC_AVGDATA	0x44	ADC 求平均数据寄存器
ADC_TGSR	0x48	ADC 触发源选择寄存器
ADC_TGNR	0x4C	ADC 序列数量配置寄存器
ADC_SEQR0	0x50	ADC 通道选择寄存器 0
ADC_SEQR1	0x54	ADC 通道选择寄存器 1
ADC_SEQR2	0x58	ADC 通道选择寄存器 2
ADC_SEQR3	0x5C	ADC 通道选择寄存器 3
ADC_CR	0x60	ADC 控制寄存器
ADC_OTR	0x64	ADC 单次触发寄存器
ADC_TCFG	0x68	ADC 时序配置寄存器
ADC_AWTH	0x6C	ADC 模拟看门狗阈值寄存器
ADC_AWE	0x70	ADC 模拟看门狗使能寄存器
ADC_IR	0x74	ADC 中断标志寄存器
ADC_IE	0x78	ADC 中断使能寄存器

ADC_CALIB	0x80	ADC 校准参数寄存器
ADC_CALIBDAT	0x84	ADC 校准结果寄存器
ADC_ORIDAT	0x88	ADC 原始数据寄存器

19.3 ADC 寄存器描述

19.3.1 ADC_DATA0 ADC 数据寄存器 0

地址:0x4001_2404

复位值:0x0

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11:0]	RO	0	DATA0	ADC0 序列转换结果

19.3.2 ADC_DATA1 ADC 数据寄存器 1

地址:0x4001_2408

复位值:0x0

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11:0]	RO	0	DATA1	ADC1 序列转换结果

19.3.3 ADC_DATA2 ADC 数据寄存器 2

地址:0x4001_240C

复位值:0x0

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11:0]	RO	0	DATA2	ADC2 序列转换结果

19.3.4 ADC_DATA3 ADC 数据寄存器 3

地址:0x4001_2410

复位值:0x0

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11:0]	RO	0	DATA3	ADC3 序列转换结果

19.3.5 ADC_DATA4 ADC 数据寄存器 4

地址:0x4001_2414

复位值:0x0

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11:0]	RO	0	DATA4	ADC4 序列转换结果

19.3.6 ADC_DATA5 ADC 数据寄存器 5

地址:0x4001_2418

复位值:0x0

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11:0]	RO	0	DATA5	ADC5 序列转换结果

19.3.7 ADC_DATA6 ADC 数据寄存器 6

地址:0x4001_241C

复位值:0x0

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11:0]	RO	0	DATA6	ADC6 序列转换结果

19.3.8 ADC_DATA7 ADC 数据寄存器 7

地址:0x4001_2420

复位值:0x0

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11:0]	RO	0	DATA7	ADC7 序列转换结果

19.3.9 ADC_DATA8 ADC 数据寄存器 8

地址:0x4001_2424

复位值:0x0

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11:0]	RO	0	DATA8	ADC8 序列转换结果

19.3.10 ADC_DATA9 ADC 数据寄存器 9

地址:0x4001_2428

复位值:0x0

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11:0]	RO	0	DATA9	ADC9 序列转换结果

19.3.11 ADC_DATA10 ADC 数据寄存器 10

地址:0x4001_242C

复位值:0x0

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11:0]	RO	0	DATA10	ADC10 序列转换结果

19.3.12 ADC_DATA11 ADC 数据寄存器 11

地址:0x4001_2430

复位值:0x0

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11:0]	RO	0	DATA11	ADC11 序列转换结果

19.3.13 ADC_DATA12 ADC 数据寄存器 12

地址:0x4001_2434

复位值:0x0

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11:0]	RO	0	DATA12	ADC12 序列转换结果

19.3.14 ADC_DATA13 ADC 数据寄存器 13

地址:0x4001_2438

复位值:0x0

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11:0]	RO	0	DATA13	ADC13 序列转换结果

19.3.15 ADC_DATA14 ADC 数据寄存器 14

地址:0x4001_243C

复位值:0x0

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11:0]	RO	0	DATA14	ADC14 序列转换结果

19.3.16 ADC_DATA15　ADC 数据寄存器 15

地址:0x4001_2440

复位值:0x0

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11:0]	RO	0	DATA15	ADC15 序列转换结果

19.3.17 ADC_AVGDATA　ADC 求平均结果寄存器

地址:0x4001_2444

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:12]	RO	0	AVG_SEQ_R	ADC 过采样求平均序列
[11:0]	RO	0	AVGDATA	ADC 过采样求平均计算结果

19.3.18 ADC_TGSR　ADC 触发源选择寄存器

地址:0x4001_2448

复位值:0x000F_8888

位域	类型	复位	名称	描述
[31:20]	--	--	--	--
[19:16]	R/W	4'hF	TRIG_EN	ADC 转换序列触发使能位 0001：使能第 0 段转换序列触发 0010：使能第 1 段转换序列触发 0100：使能第 2 段转换序列触发 1000：使能第 3 段转换序列触发
[15:12]	R/W	4'h8	TRIG_SEL3	第 3 段序列触发源选择 0000：选择 MCPWM_TRIG0 0001：选择 MCPWM_TRIG1

				0010：选择 MCPWM_TRIG2 0011：选择 MCPWM_TRIG3 0100：选择 TMR13_TRIG 0101：选择 TMR14_TRIG 0110：选择 TMR15_TRIG 0111：选择 TMR16_TRIG 1000：选择 ADC_SOC 触发
[11:8]	R/W	4’h8	TRIG_SEL2	第 2 段序列触发源选择 0000：选择 MCPWM_TRIG0 0001：选择 MCPWM_TRIG1 0010：选择 MCPWM_TRIG2 0011：选择 MCPWM_TRIG3 0100：选择 TMR13_TRIG 0101：选择 TMR14_TRIG 0110：选择 TMR15_TRIG 0111：选择 TMR16_TRIG 1000：选择 ADC_SOC 触发
[7:4]	R/W	4’h8	TRIG_SEL1	第 1 段序列触发源选择 0000：选择 MCPWM_TRIG0 0001：选择 MCPWM_TRIG1 0010：选择 MCPWM_TRIG2 0011：选择 MCPWM_TRIG3 0100：选择 TMR13_TRIG 0101：选择 TMR14_TRIG 0110：选择 TMR15_TRIG 0111：选择 TMR16_TRIG 1000：选择 ADC_SOC 触发
[3:0]	R/W	4’h8	TRIG_SEL0	第 0 段序列触发源选择

				0000：选择 MCPWM_TRIG0 0001：选择 MCPWM_TRIG1 0010：选择 MCPWM_TRIG2 0011：选择 MCPWM_TRIG3 0100：选择 TMR13_TRIG 0101：选择 TMR14_TRIG 0110：选择 TMR15_TRIG 0111：选择 TMR16_TRIG 1000：选择 ADC_SOC 触发

19.3.19 ADC_TGNR　IADC 序列数量配置寄存器

地址:0x4001_244C

复位值:0x0404_0404

位域	类型	复位	名称	描述
[31:29]	--	--	--	--
[28:24]	R/W	5'h4	TGN3	设置第 3 段转换序列数量 5'd0：0 个序列 5'd1：1 个序列 5'd2：2 个序列 5'd3：3 个序列 …… 5'd16：16 个序列
[23:21]	--	--	--	--
[20:16]	R/W	5'h4	TGN2	设置第 2 段转换序列数量 5'd0：0 个序列 5'd1：1 个序列 5'd2：2 个序列 5'd3：3 个序列

				…… 5’d16：16 个序列
[15:13]	--	--	--	--
[12:8]	R/W	5’h4	TGN1	设置第 1 段转换序列数量 5’d0：0 个序列 5’d1：1 个序列 5’d2：2 个序列 5’d3：3 个序列 …… 5’d16：16 个序列
[7:5]	--	--	--	--
[4:0]	R/W	5’h4	TGN0	设置第 0 段转换序列数量 5’d0：0 个序列 5’d1：1 个序列 5’d2：2 个序列 5’d3：3 个序列 …… 5’d16：16 个序列

注：TGN0+TGN1+TGN2+TGN3 设定的转换序列之和如果大于 16 个，则在 ADC_IR 寄存器的 TGNERRIF 通道数量错误标志位会立起。

19.3.20 ADC_SEQR0　ADC 通道选择寄存器 0

地址:0x4001_2450

复位值:0x0302_0100

位域	类型	复位	名称	描述
[31:29]	--	--	--	--
[28:24]	R/W	5’h3	CHSEQ3	设置序列 3 对应的 ADC 通道
[23:21]	--	--	--	--

[20:16]	R/W	5'h2	CHSEQ2	设置序列 2 对应的 ADC 通道
[15:13]	--	--	--	--
[12:8]	R/W	5'h1	CHSEQ1	设置序列 1 对应的 ADC 通道
[7:5]	--	--	--	--
[4:0]	R/W	5'h0	CHSEQ0	设置序列 0 对应的 ADC 通道

19.3.21 ADC_SEQR1 ADC 通道选择寄存器 1

地址:0x4001_2454

复位值:0x0706_0504

位域	类型	复位	名称	描述
[31:29]	--	--	--	--
[28:24]	R/W	5'h7	CHSEQ7	设置序列 7 对应的 ADC 通道
[23:21]	--	--	--	--
[20:16]	R/W	5'h6	CHSEQ6	设置序列 6 对应的 ADC 通道
[15:13]	--	--	--	--
[12:8]	R/W	5'h5	CHSEQ5	设置序列 5 对应的 ADC 通道
[7:5]	--	--	--	--
[4:0]	R/W	5'h4	CHSEQ4	设置序列 4 对应的 ADC 通道

19.3.22 ADC_SEQR2 ADC 通道选择寄存器 2

地址:0x4001_2458

复位值:0x0B0A_0908

位域	类型	复位	名称	描述
[31:29]	--	--	--	--
[28:24]	R/W	5'hB	CHSEQ11	设置序列 11 对应的 ADC 通道
[23:21]	--	--	--	--
[20:16]	R/W	5'hA	CHSEQ10	设置序列 10 对应的 ADC 通道
[15:13]	--	--	--	--

[12:8]	R/W	5'h9	CHSEQ9	设置序列 9 对应的 ADC 通道
[7:5]	--	--	--	--
[4:0]	R/W	5'h8	CHSEQ8	设置序列 8 对应的 ADC 通道

19.3.23 ADC_SEQR3 ADC 通道选择寄存器 3

地址:0x4001_245C

复位值:0x0F0E_0D0C

位域	类型	复位	名称	描述
[31:29]	--	--	--	--
[28:24]	R/W	5'hF	CHSEQ15	设置序列 15 对应的 ADC 通道
[23:21]	--	--	--	--
[20:16]	R/W	5'hE	CHSEQ14	设置序列 14 对应的 ADC 通道
[15:13]	--	--	--	--
[12:8]	R/W	5'hD	CHSEQ13	设置序列 13 对应的 ADC 通道
[7:5]	--	--	--	--
[4:0]	R/W	5'hC	CHSEQ12	设置序列 12 对应的 ADC 通道

19.3.24 ADC_CR ADC 控制寄存器

地址:0x4001_2460

复位值:0x0060_0060

位域	类型	复位	名称	描述
[31:22]	--	--	--	--
[21:20]	R/W	2'b11	VREFH	配置 ADC 量程 00：1.5V 01：2V 10：2.5V 11：3V
[19:18]	--	--	--	--

[17]	R/W	0	VREFH_BYPS	配置 ADC 量程到 VDD33 使能： 0：ADC 量程按照 VREFH 配置； 1：ADC 量程等于 VDD33 大小，电路仅保留 VDD33 到 VREFH 通路
[16]	R/W	0	VREFH_EN	控制 ADC_VREFH 参考使能: 1: 使能; 0: 关闭;
[15:12]	R/W	0	AVG_SEQ	设置求平均序列
[11]	--	--	--	--
[10:8]	R/W	0	AVG_MOD	求平均模式设置 000：不求平均 001：2 次转换结果求平均 010：4 次转换结果求平均 011：8 次转换结果求平均 100：16 次转换结果求平均 101：32 次转换结果求平均 110：64 次转换结果求平均 111：128 次转换结果求平均
[7:6]	--	--	--	--
[5:4]	R/W	2’b11	CLK	ADC 采样时钟选择: 00: 250K sps (4us) 01: 500K sps (2us) 10: 1M sps (1us) 11: 2M sps (0.5us)
[3]	--	--	--	--
[2]	R/W	0	DME_EN	DMA 功能使能位 1：使能 0：禁止

[1]	R/W	0	CALIB_EN	校准功能使能位 1：使能 0：禁止
[0]	R/W	0	ADC_EN	ADC 使能位 1：使能 0：禁止

19.3.25 ADC_OTR ADC 单次触发寄存器

地址:0x4001_2464

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	WO	0	SOC	一次写 16'h5AA5 操作，可产生一个 ADC_SOC 信号，触发一次 ADC 转换

19.3.26 ADC_TCFG ADC 时序配置寄存器

地址:0x4001_2468

复位值:0x0A40_20F4

位域	类型	复位	名称	描述
[31:24]	R/W	8'hA	SEQUENCE_PULSE	每段采样初始的额外延时，用于延长 ADC_MUX 建立时间。
[23:16]	R/W	8'h40	INITIAL_PULSE	该位用于设置 ADC 使能后，到 ADC_RDY 准备就绪的时间。
[15:8]	R/W	8'h20	INTERVAL_PULSE	用于设置采样+转换完成后，延迟启动下一通道转换时间。
[7:4]	R/W	4'hF	SAMPLE_PULSE	用于设置采样完成后，到 ADC_MUX 切换时间。
[3:0]	R/W	4'h4	START_PULSE	用于设置 ADC_START 触发信号的脉宽。

19.3.27 ADC_AWTH ADC 模拟看门狗阈值寄存器

地址:0x4001_246C

复位值:0x0

位域	类型	复位	名称	描述
[31:28]	--	--	--	--
[27:16]	R/W	0	AWHTH	模拟看门狗上阈值配置 写入的数值应与 ADC 的 12 位分辨率对齐的格式
[15:12]	--	--	--	--
[11:0]	R/W	0	AWLTH	模拟看门狗下阈值配置 写入的数值应与 ADC 的 12 位分辨率对齐的格式

若参考电压为 3.3V，12 位 ADC，电压范围 0~3.3V。

保护窗口：1.0V~2.5V

计算公式：数字值=(电压/参考电压)×4095

上限值(HTH)≈(2.5/3.3)×4095≈3102(**0xC1E**)

下限值(LTH)≈(1.0/3.3)×4095≈1241(**0x4D9**)

结果：当转换结果 ≥0xC1E(电压≥2.5V)或≤0x4D9(电压≤1.0V)时触发告警。

19.3.28 ADC_AWE ADC 模拟看门狗使能寄存器

地址:0x4001_2470

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15]	R/W	0	AWE15	转换序列 15 模拟看门狗使能位 1：使能 0：禁止
[14]	R/W	0	AWE14	转换序列 14 模拟看门狗使能位 1：使能 0：禁止

[13]	R/W	0	AWE13	转换序列 13 模拟看门狗使能位 1：使能 0：禁止
[12]	R/W	0	AWE12	转换序列 12 模拟看门狗使能位 1：使能 0：禁止
[11]	R/W	0	AWE11	转换序列 11 模拟看门狗使能位 1：使能 0：禁止
[10]	R/W	0	AWE10	转换序列 10 模拟看门狗使能位 1：使能 0：禁止
[9]	R/W	0	AWE9	转换序列 9 模拟看门狗使能位 1：使能 0：禁止
[8]	R/W	0	AWE8	转换序列 8 模拟看门狗使能位 1：使能 0：禁止
[7]	R/W	0	AWE7	转换序列 7 模拟看门狗使能位 1：使能 0：禁止
[6]	R/W	0	AWE6	转换序列 6 模拟看门狗使能位 1：使能 0：禁止
[5]	R/W	0	AWE5	转换序列 5 模拟看门狗使能位 1：使能 0：禁止
[4]	R/W	0	AWE4	转换序列 4 模拟看门狗使能位

				1：使能 0：禁止
[3]	R/W	0	AWE3	转换序列 3 模拟看门狗使能位 1：使能 0：禁止
[2]	R/W	0	AWE2	转换序列 2 模拟看门狗使能位 1：使能 0：禁止
[1]	R/W	0	AWE1	转换序列 1 模拟看门狗使能位 1：使能 0：禁止
[0]	R/W	0	AWE0	转换序列 0 模拟看门狗使能位 1：使能 0：禁止

19.3.29 ADC_IR　ADC 中断标志寄存器

地址:0x4001_2474

复位值:0x0

位域	类型	复位	名称	描述
[31]	RO	0	RDY	ADC 准备就绪标志位 1：准备就绪 0：未准备就绪
[30:21]	--	--	--	--
[20]	RW1C	0	AWDIF	模拟看门狗标志位 1：模拟看门狗检测到达阈值 0：模拟看门狗检测未到达阈值
[19:18]	--	--	--	--
[17]	RW1C	0	TGTERRIF	序列转换触发错误标志位

				1：序列转换触发错误 0：序列转换触发正常
[16]	RW1C	0	TGNERRIF	序列数量配置错误标志位 1：序列数量配置错误 0：序列数量配置正常
[15:10]	--	--	--	--
[9]	RW1C	0	CALUPIF	校准完成标志位 1：校准完成 0：校准未完成
[8]	RW1C	0	AVGUPIF	求平均完成标志位 1：求平均完成 0：求平均未完成
[7:5]	--	--	--	--
[4]	RW1C	0	SACDIF	任意段序列转换完成标志位 1：任意段转换完成 0：任意段转换未完成 注：该位写 1 也会清除 S0/1/2/3CDIF
[3]	RW1C	0	S3CDIF	第 3 段序列转换完成标志位 1：第 3 段序列转换完成 0：第 3 段序列转换未完成
[2]	RW1C	0	S2CDIF	第 2 段序列转换完成标志位 1：第 2 段序列转换完成 0：第 2 段序列转换未完成
[1]	RW1C	0	S1CDIF	第 1 段序列转换完成标志位 1：第 1 段序列转换完成 0：第 1 段序列转换未完成
[0]	RW1C	0	S0CDIF	第 0 段序列转换完成标志位 1：第 0 段序列转换完成

				0：第 0 段序列转换未完成

19.3.30 ADC_IE　ADC 中断使能寄存器

地址:0x4001_2478

复位值:0x0

位域	类型	复位	名称	描述
[31:21]	--	--	--	--
[20]	R/W	0	AWDIE	模拟看门狗中断使能位 1：模拟看门狗中断使能 0：模拟看门狗中断禁止
[19:18]	--	--	--	--
[17]	R/W	0	TGTERRIE	序列转换触发错误中断使能位 1：序列转换触发错误中断使能 0：序列转换触发错误中断禁止
[16]	R/W	0	TGNERRIE	序列数量配置错误中断使能位 1：序列数量配置错误中断使能 0：序列数量配置错误中断禁止
[15:10]	--	--	--	--
[9]	R/W	0	CALUPIE	校准完成中断使能位 1：校准完成中断使能 0：校准完成中断禁止
[8]	R/W	0	AVGUPIE	求平均完成中断使能位 1：求平均完成中断使能 0：求平均完成中断禁止
[7:5]	--	--	--	--
[4]	R/W	0	SACDIE	任意段序列转换完成中断使能位 1：任意段序列转换完成中断使能 0：任意段序列转换完成中断禁止

[3]	R/W	0	S3CDIE	第 3 段序列转换完成中断使能位 1：第 3 段序列转换完成中断使能 0：第 3 段序列转换完成中断禁止
[2]	R/W	0	S2CDIE	第 2 段序列转换完成中断使能位 1：第 2 段序列转换完成中断使能 0：第 2 段序列转换完成中断禁止
[1]	R/W	0	S1CDIE	第 1 段序列转换完成中断使能位 1：第 1 段序列转换完成中断使能 0：第 1 段序列转换完成中断禁止
[0]	R/W	0	S0CDIE	第 0 段序列转换完成中断使能位 1：第 0 段序列转换完成中断使能 0：第 0 段序列转换完成中断禁止

19.3.31 ADC_CALIB ADC 校准参数寄存器

地址:0x4001_2480

复位值:0x0800_0000

位域	类型	复位	名称	描述
[31:28]	--	--	--	--
[27:16]	R/W	12'h800	GAIN	
[15:12]	--	--	--	--
[11:0]	R/W	0	OFFSET	

19.3.32 ADC_CALIBDAT ADC 校准结果寄存器

地址:0x4001_2484

复位值:0x0

位域	类型	复位	名称	描述
[31:29]	--	--	--	--
[28:24]	RO	0	CAL_MUX	当前校准完成的 ADC 通道

[23:0]	RO	0	CAL_DATA	ADC 校准输出的结果

19.3.33 ADC_ORIDAT ADC 原始数据寄存器

地址:0x4001_2488

复位值:0x0

位域	类型	复位	名称	描述
[31:12]	--	--	--	--
[11:0]	RO	0	ORIG_DAT	当前 ADC 转换原始数据

19.4 应用指南

19.4.1 功能说明

1. 时钟

ADC 外设时钟由 AHB 时钟门控寄存器控制：

寄存器	地址	位	说明
AHBADC 时钟门控	0x4002_005C	[0]	1=使能 ADC 时钟

SDK 接口：

void ADC_Clk_Enable(void); /*置位 0x4002005C bit0*/

void ADC_Clk_Disable(void); /*清零 0x4002005C bit0 */

注意：ADC_Clk_Disable()只关闭外设时钟；ADC_EnableIrqSrc()与 NVIC_EnableIRQ()的状态不会随之复位。

ADC 的采样时钟由“ADC_CR”寄存器的 ADC_CLK_SEL[5:4]选择：

寄存器	地址	位	位名称	说明
ADC_CLK_SEL	0x40012460	[5:4]	ADC_CLK_SEL	0=250K,1=500K,2=1M,3=2M

2. 采样组

ADC 支持 1~4 个采样组并行配置。组数由寄存器 ADC_TGSR 的[19:16]决定，编码为(1<<n)-1：

组数	ADC_TGSR[19:16]	SDK 宏
1 组	0x1	SampleGrpMode_1
2 组	0x3	SampleGrpMode_2
3 组	0x7	SampleGrpMode_3
4 组	0xF	SampleGrpMode_4

每个采样组可独立配置：

触发源：ADC_TGSR 中该组对应的 4 位字段

通道数：ADC_TGNR 中该组对应的 5 位字段（0–16）

SDK 接口：

void ADC_SetSampleGrpMode(uint32_t state); /*组数*/

void ADC_SetGrpTrigSource(uint32_t SampGrp, uint32_t TriSrc); /*组触发源*/

void ADC_SetGrpChCount(uint32_t SampGrp, uint32_t ChannelNum); /*组通道数*/

3. 通道序列

每组最多 16 个 SEQ 槽位。SEQx 对应哪个模拟通道，由 SEQR0~SEQR3 配置：

寄存器				
ADC_SEQR00	SEQ0 (bit[4:0])	SEQ1 (bit[12:8])	SEQ2 (bit[20:16])	SEQ3 (bit[28:24])
ADC_SEQR01	SEQ4	SEQ5	SEQ6	SEQ7
ADC_SEQR02	SEQ8	SEQ9	SEQ10	SEQ11
ADC_SEQR03	SEQ12	SEQ13	SEQ14	SEQ15

触发一次，只转换该组前 ADC_TGNR 个 SEQ。例如 ADC_TGNR[4:0]=4 时，只转换 SEQ0~SEQ3，DATA[4…15]不刷新。

SDK 接口：

void ADC_SEQx_CFG(uint32_t Sqex, uint32_t Chx); /* Sqex: ADC_SEQ0..ADC_SEQ15, Chx: ADC_CH0..ADC_CH31 */

触发

触发源编码（ADC_TGSR 每 4 位）

值	SDK 宏	触发源
0	ADC_GrpTrig_MCPWM_TRIG0	MCPWM 触发 0
1	ADC_GrpTrig_MCPWM_TRIG1	MCPWM 触发 1
2	ADC_GrpTrig_MCPWM_TRIG2	MCPWM 触发 2
3	ADC_GrpTrig_MCPWM_TRIG3	MCPWM 触发 3
4	ADC_GrpTrig_TMR13_TRIG	TIMER13
5	ADC_GrpTrig_TMR14_TRIG	TIMER14
6	ADC_GrpTrig_TMR15_TRIG	TIMER15
7	ADC_GrpTrig_TMR16_TRIG	TIMER16
8	ADC_GrpTrig_SW_TRIG	软件触发

软件触发：向 ADC_OTR 写入 0x5AA5 即启动一次转换。

SDK 接口：

void ADC_SWTrig(void); /*ADC->OTR=0x5AA5*/

写入其他值无效。ADC_OTR 为触发命令寄存器，读回值无意义。

硬件触发：选择 MCPWM 或 TIMER 作为触发源后，对应的硬件事件会自动启动采样组转换，无需软件干预。

MCPWM 触发：由 MCPWM 的 TMR0~TMR3 寄存器设定采样时刻（相对 PWM 周期的偏移）

TIMER 触发：由 TIMER 的 CH0 比较/捕获事件产生

采样时序

时序参数在 ADC_TCFG 中配置：

字段	位	含义	范围
STPT	[3:0]	start 高电平保持时间	0–15
COVT	[7:4]	CONV 完成到 ADC_MUX 切换之间的间隔	0–15
SAPT	[15:8]	转换完成后读取结果的等待时间	≥0xC
INIT	[23:16]	ADC_EN 置位到 ADC_RDY 的等待时间	0–255
TDELY	[31:24]	每组第 1 次采样前的延时	0–255

SDK 接口：

void ADC_SetCLK_STPT(uint32_t STPT_cnt);

void ADC_SetCLK_COVT(uint32_t COVT_cnt);

void ADC_SetCLK_SAPT(uint32_t SAPT_cnt); /*<0xC 时驱动强制写 0xC */

void ADC_SetCLK_INIT(uint32_t INIT_cnt);

void ADC_SetCLK_TrigDelay(uint32_t Delay_cnt); /*写 TDELY*/

**ADC_SetCLK_SAPT()有下限保护，小于 0xC 会被抬升到 0xC。

参考电压与量程

由 ADC_CR 的三个字段共同决定：

字段	位	取值
VREF_RANGE	[21:20]	0=1.5V，1=2.0V，2=2.5V，3=3.0V
VREFH_TO_VDD33	[17]	1=VREFH 取 VDD33
VREFH_EN	[16]	1=使能 VREFH

**ADC_CRCR 无 SDK 接口函数，需按位组合后直写寄存器地址 0x4001_2460。

转换结果

转换结果保存在 DATA[0..15]，与 SEQ 一一对应。低 12 位有效：

寄存器	地址	对应
DATA[0]	0x4001_2404	SEQ0
DATA[1]	0x4001_2408	SEQ1
…		
DATA[15]	0x4001_2440	SEQ15

SDK 接口：

uint32_t ADC_GetResult(u8 Sqex); /* 返回 ADC->DATA[Sqex]，需自行&0xFFF*/

**ADC_GetResult()返回完整 32 位寄存器值，使用前必须&0xFFF 取低 12 位。

中断

位	中断源	宏	说明
0	Group0 转换完成	ADC_IRQSRC_S0CP	IR.S0CDIF
1	Group1 转换完成	ADC_IRQSRC_S0CP	IR.S1CDIF

2	Group2 转换完成	ADC_IRQSRC_S0CP	IR.S2CDIF
3	Group3 转换完成	ADC_IRQSRC_S0CP	IR.S3CDIF
16	组通道数错误	ADC_IRQSRC_TGN_ERR	IR.TGNERRIF
17	触发错误	ADC_IRQSRC_TGT_ERR	IR.TGTERRIF
20	模拟看门狗	ADC_IRQSRC_AWD`	IR.AWDIF
31	ADC 就绪（只读）	IR.RDY	

中断处理：

1)使能中断源：ADC_Enable_IrqSrc(ADC_IRQSRC_S0CP);

2)使能 NVIC：

```
NVIC_SetPriority(ADC_IRQn, 2);
NVIC_ClearPendingIRQ(ADC_IRQn);
NVIC_EnableIRQ(ADC_IRQn);
```

3) SR(中断函数处理)：

```
void ADC_IRQHandler(void)
{
if (ADC->IR.Word & ADC->IE.Word & (1UL << ADC_IRQSRC_S0CP))
{
ADC->IR.S0CDIF = 1; /*先写 1 清零 */
raw=ADC_GetResult(0) & 0xFFFU; /*再读结果*/
}
}
```

关键规则：

1.标志位写 1 清零，写 0 无效

2.必须先清标志，再读 ADC_DATA[x]，ADC_DATA[x]在下次转换时被覆盖

3.读 ADC->IR.Word & ADC->IE.Word 可一次性取得所有已使能且已置位的源

4.一次性清全部标志可写 ADC->IR.Word = 0xFFFFFFFFUL

5.若把 ISR 放在其他外设（如 TIMx_IRQHandler）中，仍需按上述顺序操作 ADC 的 IR

DMA 请求

置位 ADC_CR.DME_EN（bit2）后，每次 SEQ 转换完成都会产生一个 DMA 请求。

注意：DMA 请求源固定为 ADC_DATA[0]。若需搬移 SEQ0~SEQn 的多个结果，需配合目的地址自增，并确认 DMA 请求与 SEQ 完成的对应关系。

模拟看门狗

对指定 SEQ 的转换结果做上下限比较，越限时置 ADC_IR.AWDIF 并（若使能）产生中断。

寄存器	位	说明
AWTH	[27:16]	高阈值 HTH（0–4095）
AWTH	[11:0]	低阈值 LTH（0–4095）
AWE	[x]	SEQx 的看门狗使能，位序号=SEQ 号

SDK 接口：

void ADC_Set_AW_HTH(uint32_t HTH); /*建议 HTH>LTH*/

void ADC_Set_AW_LTH(uint32_t LTH);

void ADC_Set_AWChannel(uint32_t Sqex); /*使能 SEQx 的看门狗 */

**ADC_Set_AWChannel()只做置位，要关闭需直写 AWE 寄存器。

19.4.2 快速开始

最小可用的 ADC 软件触发采集（单通道、轮询方式）示例：

流程：开时钟→建采样组→配序列→配时序→ADC 使能→触发→清标志→读结果

时钟使能：ADC_Clk_Enable();

采样组结构：几组、每组多少通道、每组由谁触发。

ADC_SetSampleGrpMode(SampleGrpMode_1); /*组数→TGSR[19:16]*/

ADC_SetGrpChCount(ADC_Sample_Group_0, 4); /*通道数 → TGNR[4:0]*/

ADC_SetGrpTrigSource(ADC_Sample_Group_0, ADC_GrpTrig_SW_TRIG); /*触发源*/

通道数必须等于实际要转换的 SEQ 个数。配多了会转换未定义的通道，配少了后面几个 SEQ 不刷新

序列映射：

ADC_SEQx_CFG(ADC_SEQ0, ADC_CH0); /*第 1 个转换 CH0*/

ADC_SEQx_CFG(ADC_SEQ1, ADC_CH1); /*第 2 个转换 CH1*/

ADC_SEQx_CFG(ADC_SEQ2, ADC_CH2);

ADC_SEQx_CFG(ADC_SEQ3, ADC_CH3);

转换顺序固定为 SEQ0→SEQ1→…→SEQn，结果依次存入 DATA[0]→DATA[1]→…→DATA[n]。

采样时序：

ADC_SetCLK_STPT(10); /*TCFG[3:0]start 保持时间*/

ADC_SetCLK_COVT(15); /*TCFG[7:4]MUX 切换间隔*/

ADC_SetCLK_SAPT(0xC); /*TCFG[15:8]读取等待（下限 0xC）*/

ADC_SetCLK_INIT(6); /*TCFG[23:16]上电等待*/

ADC_SetCLK_TrigDelay(32); /*TCFG[31:24]首次采样延时*/

**最小配置只需 ADC_SetCLK_STPT(10)，其余保持复位值。

参数调大的场景：

INIT：上电后首次转换结果异常时增大

TDELY：硬件触发后采样点位置不对（采到 MUX 切换瞬间）时增大

SAPT：结果偏差或跳变时增大

ADC 采样时钟配置/参考电压配置/使能：

```
void ADC_ConfigDirect
(
uint8_t vref_range,          /*[21:20] 0:1.5V 1:2.0V 2:2.5V 3:3.0V*/
uint8_t vrefh_to_vdd33,      /*[17] 1=VREFH 接 VDD33 */
uint8_t vrefh_en,            /*[16] 1=使能 VREFH*/
uint8_t adc_clk_sel,         /*[5:4] 0:250K 1:500K 2:1M 3:2M*/
uint8_t adc_en               /*[0] 1=使能 ADC*/
)
{
uint32_t val = 0;
val |= ((uint32_t)(vref_range & 0x3) << 20);
val |= ((uint32_t)(vrefh_to_vdd33 & 0x1) << 17);
```

```
val |= ((uint32_t)(vrefh_en & 0x1) << 16);
val |= ((uint32_t)(adc_clk_sel & 0x3) << 4);
val |= ((uint32_t)(adc_en & 0x1) << 0);
*(volatile uint32_t *)0x40012460UL = val;
}
```

DMA 请求使能（可选）：

在 ADC_CR 组合中额外加 bit2：

```
val |= (1UL<<2); /*DME_EN=1，每次 SEQ 转换完成产生 DMA 请求*/
```

中断使能（可选）：

```
ADC_Enable_IrqSrc(ADC_IRQSRC_S0CP); /*IE[0] = 1 */
NVIC_SetPriority(ADC_IRQn, 2); /*ADC_IRQn = 17 */
NVIC_ClearPendingIRQ(ADC_IRQn);
NVIC_EnableIRQ(ADC_IRQn);
```

启动转换：

软件触发：ADC_SWTrig()，或在任意时刻直写 ADC_OTR=0x5AA5。

硬件触发：配置好 MCPWM/TIMER 后由硬件自动产生，软件不干预

```
ADC_SetGrpTrigSource(ADC_Sample_Group_0, ADC_GrpTrig_MCPWM_TRIG0);（MCPWM 触发）
ADC_SetGrpTrigSource(ADC_Sample_Group_0, ADC_GrpTrig_TMR13_TRIG);（TIMER13 触发）
```

20 CSA

20.1 概述

20.1.1 简介

CSA 模块是模拟控制模块，其中包含了 CSA 模块和温度传感器模块的控制。CSA 模块把采样电阻上的微弱差分电压放大、叠加直流偏置后，送到 ADC 内部通道由 ADC 采样；同时输出的信号还可送内部过流比较器做硬件过流保护。

详细信息描述请参考寄存器模块描述。

20.1.2 主要功能

CSA 模块主要提供以下功能：

- CSA 电流采样放大器控制；
- CSA 增益、偏置、DAC 参考、输入通道选择；
- CSA 输出滤波控制；
- CSA 过流检测、迟滞、消抖控制；
- 内部结温 TJ 检测控制；
- 外部温度 TS 检测控制，支持 GPIO5/GPIO6 外接 NTC 电阻；
- 过温预警 OTW、过温关断 OTS 阈值及消抖控制；
- 提供 CSA 过流、TJ_OTW、TJ_OTS 的实时和锁存状态指示。

20.1.3 CSA 框图

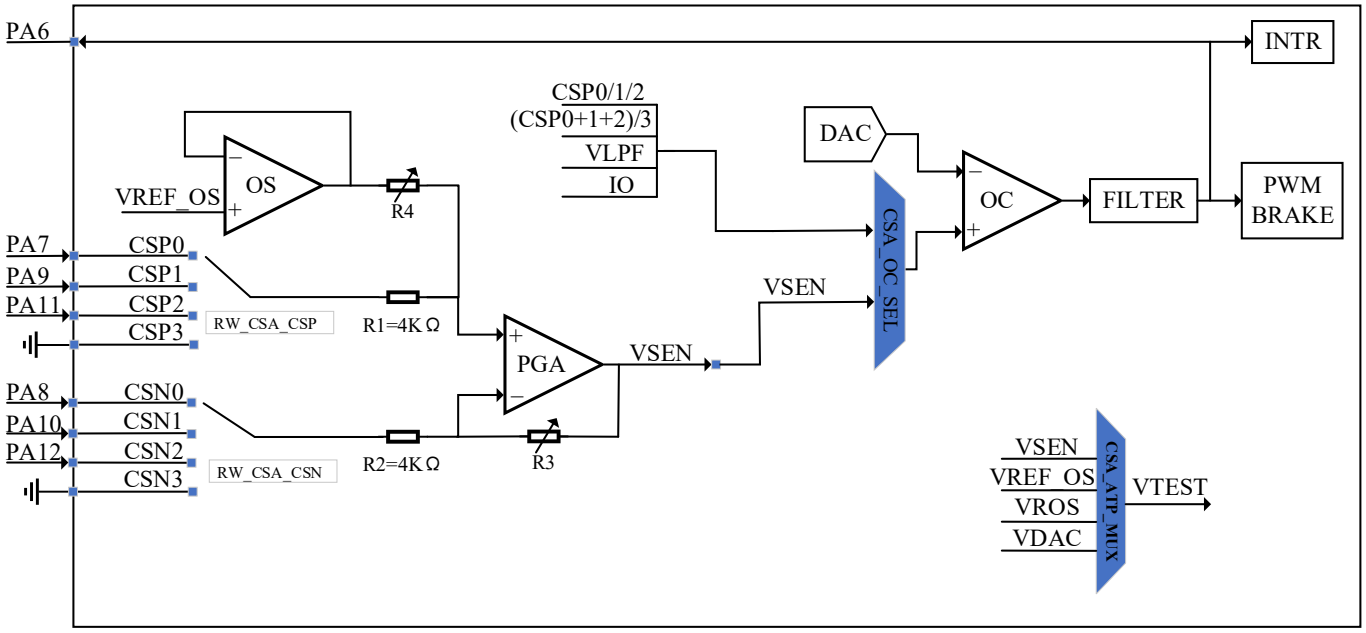


图 20-1 CSA 模块框图

20.2 寄存器列表

CSA 在芯片中的基地址是 0x4000_7800

表 20-1 CSA 寄存器地址分配

名称	偏移地址	描述
MISC_IR	0x04	MISC 中断寄存器
MISC_CSACR0	0x10	MISC CSA 控制寄存器 0
MISC_CSACR1	0x14	MISC CSA 控制寄存器 1
MISC_TEMPCR	0x18	MISC 温度传感器控制寄存器

20.3 CSA 寄存器描述

20.3.1 MISC_IR　　MISC 中断寄存器

地址:0x4000_7804

复位值:0x00

位域	类型	复位	名称	描述
[31:19]	--	--	--	--
[18]	RO		IR_TJ_OTW_NG	TJ Over Temp Warning (OTW) Protect (e.g.120C) index signal. When die temp >120C, TJ_OTW=1 (高温); When die temp <120C, TJ_OTW=0 (正常);
[17]	RO		IR_TJ_OTS_NG	TJ Over Temp Shutdown (OTS) Protect (e.g. 140C)　index signal. When die temp >140C, TJ_OTS=1 (过温); When die temp <140C, TJ_OTS=0 (正常);
[16]	RO		IR_CSA_OC_NG	Current Sense Amplifier Over-Curent　(OC) (过流报警) Index Signal. When CSA < OC_VTH, CSA_OC=0 (正常); When CSA > OC_VTH, CSA_OC=1 (过流);
[15:3]	--	--	--	--
[2]	RO		RO_TJ_OTW_LAT	TJ Over Temp Warning (OTW) Protect (e.g. 120C) index signal. When die temp >120C, TJ_OTW=1 (高温); When die temp <120C, TJ_OTW=0 (正常);
[1]	RO		RO_TJ_OTS_LAT	TJ Over Temp Shutdown (OTS) Protect (e.g.140C) index signal. When die temp >140C, TJ_OTS=1 (Latch); When die temp <140C, TJ_OTS=0;

[0]	RO		RO_CSA_OC_LAT	Current Sense Amplifier Over-Curent (OC) (过流报警) Index Signal. When CSA < OC_VTH, CSA_OC=0 (正常); When CSA > OC_VTH, CSA_OC=1 (过流);

20.3.2 MISC_CSACR0 CSA 控制寄存器 0

地址:0x4000_7810

复位值:0x0001_F802

位域	类型	复位	名称	描述
[31:24]	R/W	0	RW_CSA_DAC_OUT	DAC 输入信号
[23]	--	--	--	--
[22:20]	R/W	0	RW_CSA_DAC_REF	配置 DAC 量程： 000：1.2V 001：1.5V 010：2V 011：2.5V 100：3V 101：vdd33 110：vdd33 111：vdd33
[19:17]	--	--	--	--
[16:12]	R/W	5'h1F	RW_CSA_REF	配置 CSA 偏置电压： 00000:1.5/8V 00001:1.5/6V 00010:1.5/4V 00011:1.5/2V 00100:2/8V 00101:2/6V

				00110:2/4V 00111:2/2V 01000:2.5/8V 01001:2.5/6V 01010:2.5/4V 01011:2.5/2V 01100:3/8V 01101:3/6V 01110:3/4V 01111:3/2V 1xx00:vdd33/8V 1xx01:vdd33/6V 1xx10:vdd33/4V 1xx11:vdd33/2V
[11]	R/W	1	RW_CSA_DAC_EN	控制 DAC 使能： 0:不使能； 1:使能；
[10:8]	R/W	0	RW_CSA_GAIN	配置 CSA 增益： 000:1 001:2 010:4 011:8 100:12 101:16 110:32 111:64
[7:6]	R/W	0	RW_CSA_CSN	选择反相采样点通道： 00：GPIO8

				01：GPIO10 10：GPIO12 11：GND
[5:4]	R/W	0	RW_CSA_CSP	选择同相采样点通道： 00：GPIO7 01：GPIO9 10：GPIO11 11：GND
[3:2]	R/W	0	RW_CSA_MODE	--
[1]	R/W	1	RW_CSA_AMP_EN	控制 CSA 运放使能： 0：不使能； 1：使能；
[0]	R/W	0	RW_ANA_CSA_EN	控制 CSA 电流采样模块使能: 1:使能; 0:关闭;

20.3.3 MISC_CSACR1 CSA 控制寄存器 1

地址:0x4000_7814

复位值:0x0000_4001

位域	类型	复位	名称	描述
[31]	RO	0	IR_CSA_OC_NG	Current Sense Amplifier Over-Curent(OC) (过流报警)Index Signal. When CSA < OC_VTH, CSA_OC=0 (正常); When CSA > OC_VTH, CSA_OC=1 (过流);
[30:20]	--	--	--	--
[19:18]			RW_CSA_LPF	配置 CSA 输出滤波截止频率： 00:200Hz 01:400Hz

				10:600Hz 11:800Hz
[17]			RW_CSA_LPF_SEL	CSA 输出片外滤波使能： 0：不使能，片上电容滤波； 1：使能，片外电容滤波；
[16]			RW_CSA_LPF_GPIO	CSA LPF 外接滤波电容路径使能： 0：路径关闭，不外接电容； 1：路径开启，外接电容；
[15:14]			RW_CSA_OC_NG	配置 OC 指示信号消抖时长： 00:0.5us 01:1us 10:2us 11:4us
[13]	--	--	--	--
[12]			RW_CSA_OC_NG_EN	控制 OC 指示信号消抖使能： 0：不消抖； 1：消抖；
[11]	--	--	--	--
[10:8]			RW_CSA_OC_HYS	配置 OC 检测迟滞： 000:0 001:13mV 010:24mV 011:33mV 100:41mV 101:48mV 110:54mV 111:59mV
[7]	--	--	--	--

[6:4]			RW_CSA_OC_SEL	选择 OC 检测信号： 000：csp0 001：csp1 010：csp2 011：csp1+csp2+csp3 100：vsen 101：vtest 110：null 111：null
[3:1]	--	--	--	--
[0]	R/W	1	RW_CSA_OC_EN	控制 CSA OC 检测使能： 0：不使能； 1：使能；

20.3.4 MISC_TEMPCR　MISC 温度传感器控制寄存器

地址:0x4000_7818

复位值:0x0000_0660

位域	类型	复位	名称	描述
[31:19]	--	--	--	--
[18]	R/W	0	RW_TS_GPIO6_EN	控制 TS 电流流到 GPIO6 使能 (外部接 NTC 电阻) 0：不使能； 1：使能； 注: 用于双路 TS 时，需要分时切换 GPIO5,GPIO6;只有在 ADC 要采样 GPIO5/6 时， 才打开 TS 输出;不采样时,可以关闭 TS 电流;
[17]	R/W	0	RW_TS_GPIO5_EN	控制 TS 电流流到 GPIO5 使能 (外部接 NTC 电阻) 0：不使能； 1：使能；
[16]	R/W	0	RW_ANA_TS_EN	控制 TS 外部温度检测模块使能: 1: 使能; 0: 关闭;
[15:12]	--	--	--	--
[11:10]	R/W	1	RW_TJ_OTW_NG	配置过温警报信号消抖时长： 00:0.25ms 01:0.5ms 10:1ms 11:2ms
[9:8]	R/W	2’b10	RW_TJ_OTW	配置触发过温警报的温度： 00:120℃ 01:125℃

				10:130℃ 11:135℃
[7:6]	R/W	1	TW_TJ_OTS_NG	配置过温关断信号消抖时长： 00:0.25ms 01:0.5ms 10:1ms 11:2ms
[5:4]	R/W	2’b10	RW_TJ_OTS	配置触发过温关断的温度： 00:135℃ 01:140℃ 10:145℃ 11:150℃
[3]	--	--	--	--
[2]	R/W	0	RW_TJ_OTW_NG_EN	控制过温警报信号消抖使能： 0：不消抖； 1：消抖；
[1]	R/W	0	RW_TJ_OTS_NG_EN	控制过温关断信号消抖使能： 0：不消抖； 1：消抖；
[0]	R/W	0	RW_ANA_TJ_EN	控制 TJ 内部温度检测模块使能: 1: 使能; 0: 关闭;

20.4 应用指南

20.4.1 功能说明

1. 输入

引脚必须切为模拟模式，否则数字 I/O 会把模拟信号拉偏：

I/O 引脚切换执行步骤：清复用（AF 清零）-关输出使能-关输入使能-关上拉-关下拉-切模拟。

通道号由寄存器 RW_CSA_CSP/RW_CSA_CSN 各 2bit 选择（SDK 中由 CSA_SET_InputCH()同时写两个字段）：

字段值	CSP（同相端）	CSN（反相端）	SDK 宏
0	GPIO7	GPIO7	CSA_FIXED_INPUT_CH0`
1	GPIO7	GPIO7	CSA_FIXED_INPUT_CH0`
2	GPIO7	GPIO7	CSA_FIXED_INPUT_CH0`
3	GPIO7	GPIO7	CSA_FIXED_INPUT_CH0`

RW_CSA_MODE[3:2]决定通道切换方式（CSA_InputCH_SWMode()）：

值	SDK 宏	说明
0	CSA_FIXED_CH_MODE	固定通道，始终采 CSP/CSN 指定的那一对
1	CSA_AUTO_DIFF_MODE	自动差分，按序轮换差分输入对（三电阻方案）
2	CSA_AUTO_SINGLE_MODE	自动单端

2. 增益

RW_CSA_GAIN[10:8]（CSA_SET_PGAGAIN()）：

字段值	增益	SDK 宏
0	×1	CSA_PGA_GAIN_X1
1	×2	CSA_PGA_GAIN_X2
2	×4	CSA_PGA_GAIN_X4
3	×8	CSA_PGA_GAIN_X8
4	×12	CSA_PGA_GAIN_X12
5	×16	CSA_PGA_GAIN_X16
6	×32	CSA_PGA_GAIN_X32

7	×64	CSA_PGA_GAIN_X64

◆ **注意放大倍数越大，输出摆幅越容易被偏置电压与电源轨限幅，选档时需同时校验偏置+I_max×Rs×增益≤满量程。**

3. 偏置电压

RW_CSA_REF[16:12]共 5bit，编码为{VDD33 选择,基准选择[1:0],分压选择[1:0]}：

子字段	位	取值
VDD33 基准选择	[16]	1=基准取 VDD33（此时基准选择位无效）
基准选择	[15:14]	0=1.5V，1=2.0V，2=2.5V，3=3.0V
分压选择	[13:12]	0=/8，1=/6，2=/4，3=/2

4. 输出通路与 ADC 采样

CSA 有两路输出：

CSA_VSEN：运放直接输出，未经滤波。

CSA_VLPF 经 LPF 后的输出：

两路输出都在芯片内部连到 ADC 的模拟输入通道，无需外部连线。ADC 通道号与 CSA 输出信号的对应关系见章节 19.2.3 中表 19-2。

ADC 侧配置与结果读取见章节 19

5. 低通滤波（LPF）

片内 200/400/600/800Hz，或经 GPIOA13 外接电容（截止频率可调）

MISC_CSACR1 中 4 个位控制：

位	字段	说明
[17]	RW_CSA_LPF_SEL	0=片内电容滤波 1=片外电容滤波
[16]	RW_CSA_LPF_GPIO	LPF 外接电容通路使能
[19:18]	RW_CSA_LPF	片内截止频率： 0=200Hz 1=400Hz 2=600Hz 3=800Hz

◆ **低频截止会同时拖慢过流比较器的响应（两者共用模拟通路），电流环带宽要求高时不要把 LPF 压得太低。**

6. 内部 DAC

MISC_CSACR0 中的 DAC 用于产生过流比较阈值 OC_VTH：

位	字段	说明
[11]	RW_CSA_DAC_EN	DAC 使能（复位值=1）
[22:20]	RW_CSA_DAC_REF	量程： 0=1.2V 1=1.5V 2=2V 3=2.5V 4=3V 5=VDD33
[31:24]	RW_CSA_DAC_OUT	8bit 阈值码 0–255

阈值换算：V_OC_VTH≈RW_CSA_DAC_OUT/256×V_DAC_REF

例：DAC_REF=1.2V、DAC_OUT=171，V_OC_VTH≈0.80V。

比较器方向：CSA 输出>OC_VTH 判为过流（IR_CSA_OC_NG=1）

7. 过流比较器（OC）

MISC_CSACR1

位	字段	说明
[0]	RW_CSA_OC_EN	OC 检测使能（复位值=1）
[6:4]	RW_CSA_OC_SEL	000：csp0 001：csp1 010：csp2 011：(csp1+csp2+csp3)/3 100：vsen 101：vlpf 110：io 111：null

[10:8]	RW_CSA_OC_HYS	配置 OC 检测迟滞： 000:0 001:13mV 010:24mV 011:33mV 100:41mV 101:48mV 110:54mV 111:59mV
[12]	RW_CSA_OC_NG_EN	消抖使能
[15:14]	RW_CSA_OC_NG	配置 OC 指示信号消抖时长： 00:0.5us 01:1us 10:2us 11:4us

21 MATH

21.1 概述

21.1.1 简介

MATH 是一个计算处理单元，提供了开方、除法、三角函数运算，MATH 模块可以在不同中断程序和主程序中多次调用且计算结果互不干扰。

21.1.2 功能特点

- 硬件加速，减少 CPU 负担
- 支持以下运算模式：
  - 16 位开方运算
  - 32 位有符号除法
  - 三角函数运算

21.2 功能描述

21.2.1 开方运算

开方运算使用到 MATH_SQRTRI 寄存器作为输入，使用 MATH_SQRTRO 寄存器作为输出。

表 21-1

输入数据寄存器	输入内容	输出结果寄存器	输出内容
MATH_SQRTRI	16 无符号数	MATH_SQRTRO	16 位无符号数

◆ **注：当输入不完美开方值时，会舍去小数部分只保留整数值输出。例如输入 35，输出结果为 5。**

21.2.2 除法运算

除法运算使用到 MATH_DIVIDE 和 MATH_DIVIDS 寄存器分别作为被除数和除数的输入，MATH_QUOTI 寄存器为商输出值，MATH_REMAIN 寄存器位余数输出值。

表 21-2

输入数据寄存器	输入内容	输出结果寄存器	输出内容
MATH_DIVIDE	32 位有符号数	MATH_QUOTI	32 位有符号数
MATH_DIVIDS	32 位有符号数	MATH_REMAIN	32 位有符号数

◆ **注：该除法运算中，若除数设置为 0，则 MATH_IR 寄存器的 DIVI_ERR 位会被置位，且运算结果会输出错误值（商：0x7FFF_FFFF 余数 0x0000_0064）。且在常规带符号运算时，余数存在负数情况。**

21.2.3 三角函数运算

三角函数运算使用到 MATH_THETA 寄存器作为角度输入。

表 21-3

输入数据寄存器	输入内容	输出结果寄存器	输出内容
MATH_THETA	16 位无符号数	MATH_SINCOS（高 16 位）	16 位有符号数
		MATH_SINCOS（低 16 位）	16 位有符号数

◆ **注：MATH_THETA 的 16 位角度输入范围，对应的输入值 0-65535 映射到角度的 0°～360°，角度输入可以使用 MATH_THETA=（ANGLE*65535）/360。SIN_OUT 和 COS_OUT 实数的输出范围是-1.0～1.0，对应寄存器的 0x0～0x7FFF，16 位输出的最高位都是符号位。**

21.3 寄存器列表

MATH 在芯片中的基地址是 0x4000_6000

表 21-4MATH 寄存器地址分配

名称	偏移地址	描述
MATH_IR	0x04	算数中断标志寄存器
MATH_IE	0x08	算数中断使能寄存器
MATH_SQRTRI	0x0C	开方运算输入寄存器
MATH_SQRTRO	0x10	开方运算结果输出寄存器
MATH_DIVIDE	0x14	除法被除数输入寄存器

MATH_DIVIDS	0x18	除法除数输入寄存器
MATH_QUOTI	0x1C	商输出寄存器
MATH_REMAIN	0x20	余数输出寄存器
MATH_THETA	0x24	角度输入寄存器
MATH_SINCOS	0x28	正余弦结果寄存器

21.4 MATH 寄存器描述

21.4.1 MATH 寄存器描述

地址:0x4000_6004

复位值:0x0

位域	类型	复位	名称	描述
[31:9]	--	--	--	--
[8]	RO	0	SINCOS_RDY	三角函数运算已就绪
[7:6]	--	--	--	--
[5]	RO	0	DIVI_ERR	除法运算错误
[4]	RO	0	DIVI_RDY	除法运算已就绪
[3:1]	--	--	--	--
[0]	RO	0	SQRT_RDY	开方运算已就绪

21.4.2 MATH_SQRTRI 开方运算输入寄存器

地址:0x4000_600C

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	SQRT_IN	开方运算输入值，输入被开方数

21.4.3 MATH_SQRTRO 开方运算结果输出寄存器

地址:0x4000_6010

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	RO	0	SQRT_OUT	开方运算结果输出值

21.4.4 MATH_DIVIDE 除法被除数输入寄存器

地址:0x4000_6014

复位值:0x0

位域	类型	复位	名称	描述
[31:0]	R/W	1	DIVIDE_IN	除法运算被除数输入值

21.4.5 MATH_DIVIDS 除法除数输入寄存器

地址:0x4000_6018

复位值:0x0

位域	类型	复位	名称	描述
[31:0]	R/W	1	DIVIDS_IN	除法运算除数输入值

21.4.6 MATH_QUOTI 商输出寄存器

地址:0x4000_601C

复位值:0x0

位域	类型	复位	名称	描述
[31:0]	RO	0	QUOTI_OUT	除法运算商输出值

21.4.7 MATH_REMAIN 余数输出寄存器

地址:0x4000_6020

复位值:0x0

位域	类型	复位	名称	描述
[31:0]	RO	0	REMAIN_OUT	除法运算余数输出值

21.4.8 MATH_THETA 角度输入寄存器

地址:0x4000_6024

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	--	--	--	--
[15:0]	R/W	0	ANGLE_IN	三角函数运算角度输入值

21.4.9 MATH_SINCOS 正余弦结果寄存器

地址:0x4000_6028

复位值:0x0

位域	类型	复位	名称	描述
[31:16]	RO	16'h7FFF	COS_OUT	三角函数 cos 运算输出值
[15:0]	RO	0	SIN_OUT	三角函数 sin 运算输出值

22 FLC

22.1 概述

22.1.1 简介

FLC 模块是芯片内部 FLASH 的控制模块，提供； 片上闪存需要的所有功能。在 128K 字节空间内，CPU 执行指令需要少量等待时间。FLC 也提供了 MAIN 区，INFO 区的编程操作。

22.1.2 功能特点

- 128KB 内存用于存储指令和数据
- 可自定义的 CPU 执行指令等待时间
- 16B INFO 块（一次性编程），用于存储用户数据
- 具备安全保护模式，可以阻止对代码或数据的非法访问
- 具备编程保护状态，可防止意外写入操作

22.1.3 地址分配

- INFO 区域地址范围：0x1FFF_F800 ~ 0x1FFF_F80F
- MAIN 区域地址范围：0x0800_0000 ~ 0x0801_FFDF

22.2 功能描述

22.2.1 读操作

MAIN 和 INFO 区域可以像普通存储空间意义直接寻址访问。对 FLASH 的取指指令和取数据使用 CPU 的 SBUS 总线。

AHB 读访问 FLASH 时插入的等待周期个数关系如下：

表 22-1

NWS[1:0]	插入等待周期数	AHB 读总周期（地址→数据）
2'b00	0	1 个地址周期+0 等待+1 数据周期
2'b01	1	1 个地址周期+1 等待
2'b10	2	1 个地址周期+2 等待
2'b11	3	1 个地址周期+3 等待

22.2.2 解锁与保护机制

MAIN 区解锁：向 FLC_KEYR 寄存器连续写入 0x4567_0123 和 0xCDEF_89AB 解锁。

INFO 区解锁：向 FLC_INFOKEYR 寄存器连续写入 0x0819_2A3B 和 0x4C5D_6E7F 解锁。

保护机制：当 PROTECT 功能开启时，进入调试模式后，MAIN 区的 LOCK 无法由密钥接触 LOCK,且读接口的值会返回 0xFFFF_FFFF

对 INFO 区域地址 0x1FFF_F808 写入不同 code 后，在 boot 阶段会启动两种不同的保护机制：

PROTECT1：对 FLASH 和 SRAM 的接口进行保护，调试模式下的 FLASH 数据口和 SRAM 数据口的数据会被屏蔽。

PROTECT2：直接禁用 SWD 接口，SWDIO 管脚被强制拉高。

22.2.3 编程操作

MAIN 区和 INFO 区编程：在 MAIN 区和 INFO 区域解锁后，对 FLC_CR 寄存器的 PG 位置位后，直接对地址写入数据即可。

22.2.4 中断

表 22-2

中断事件	事件标志	中断使能控制位
编程结束事件	EOP	EOPIE
编程错误事件	PGERR	PGERRIE
读取错误事件	ECCERR	ECCERRIE

超时事件	TO	TOIE

22.3 寄存器列表

FLC 在芯片中的基地址是 0x4002_2000

表 22-3 FLC 寄存器地址分配

名称	偏移地址	描述
FLC_ACR	0x04	FLC 访问控制寄存器
FLC_KEYR	0x08	FLC 主区域解锁寄存器
FLC_INFOKEYR	0x0C	FLC INFO 区域解锁寄存器
FLC_SR	0x10	FLC 状态寄存器
FLC_IER	0x14	FLC 中断使能寄存器
FLC_CR	0x18	FLC 控制寄存器
FLC_DR	0x24	FLC 数据寄存器

22.4 FLC 寄存器描述

22.4.1 FLC_ACR FLC 访问控制寄存器

地址:0x4002_2004

复位值:0x0000_0003

位域	类型	复位	名称	描述
[31:2]	R/W	--	--	--
[1:0]	R/W	2'b11	NWS	AHB 读等待状态数 00：不增加等待状态 01：增加 1 个等待状态 10：增加 2 个等待状态 11：增加 3 个等待状态

22.4.2 FLC_KEYR FLC 主区域解锁寄存器

地址:0x4002_2008

复位值:0x0

位域	类型	复位	名称	描述
[31:0]	WO	0	LOCK	主 Flash 阵列解锁密钥寄存器。需连续写入两个特定值才能清除 LOCK 位： 1.写 0x4567_0123; 2.写 0xCDEF_89AB; 任意其他写入或中断序列都将复位解锁状态，LOCK 保持为 1。

22.4.3 FLC_INFO_KEYR FLC INFO 区域解锁寄存器

地址:0x4002_200C

复位值:0x0

位域	类型	复位	名称	描述
[31:0]	WO	0	INFOLOCK	INFO 区域阵列解锁密钥寄存器。需连续写入两个特定值才能清除 INFOLOCK 位： 1.写 0x0819_2A3B; 2.写 0x4C5D_6E7F; 任意其他写入或中断序列都将复位解锁状态，INFOLOCK 保持为 1。

22.4.4 FLC_SR FLC 状态寄存器

地址:0x4002_2010

复位值:0x0

位域	类型	复位	名称	描述
[31:20]	--	--	--	--
[19]	RW1C	0	TO	超时标志。当 safe_counter[16]溢出时（约 131072 个 HCLK 周期无响应）时置 1，指示操作超时。
[18]	RW1C	0	ECCERR	读操作时检测到 par_error 时置 1。

[17]	RW1C	0	PGERR	编程错误标志。写操作时检测到 par_error 时置 1。
[16]	RW1C	0	EOP	编程/操作完成标志。当编程完成（prog_done 上升沿）时硬件置 1。
[15:3]	--	--	--	--
[2]	RO	0	TOF	超时标志原始值，来自 Core 的 safe_counter[16]，不受写清除影响。
[1]	RO	0	ERRF	错误原始状态（同步后的 par_error）
[0]	RO	0	RDYF	就绪标志（同步后的 par_ready），指示并行接口空闲。

22.4.5 FLC_IER FLC 中断使能寄存器

地址:0x4002_2014

复位值:0x0

位域	类型	复位	名称	描述
[31:20]	--	--	--	--
[19]	R/W	0	TOIE	超时中断使能。 1：使能 0：禁止
[18]	R/W	0	ECCERRIE	读错误中断使能。 1：使能 0：禁止
[17]	R/W	0	PGERRIE	编程操作错误中断使能 1：使能 0：禁止
[16]	R/W	0	EOPIE	操作完成中断使能 1：使能 0：禁止
[15:0]	--	--	--	--

22.4.6 FLC_CR FLC 控制寄存器

地址:0x4002_2018

复位值:0x0101_0000

位域	类型	复位	名称	描述
[31:25]	--	--	--	--
[24]	R/W	1	INFOLOCK	INFO 区域锁定位。 1：锁定，禁止编程操作。 0：解锁（通过写 INFOKEYR 寄存器解锁）。 注：该位写 1 也会立刻上锁。
[23:17]	--	--	--	--
[16]	R/W	1	LOCK	主区域锁定位。 1：锁定，禁止编程操作。 0：解锁（通过写 KEYR 寄存器解锁）。 注：该位写 1 也会立刻上锁。
[15:1]	--	--	--	--
[0]	R/W	0	PG	编程使能位。 1：启动编程操作（当 LOCK，INFOLOCK（）至少有一个解锁后有效） 0：无编程操作

22.4.7 FLC_DR FLC 数据寄存器

地址:0x4002_2024

复位值:0x0

位域	类型	复位	名称	描述
[31:0]	RO	0	DOUT	数据寄存器，用于读取编程操作后的校验数据和 Flash 读数据。读取时返回并行接口读回的数据。

23 IWDT

23.1 概述

23.1.1 简介

IWDT 是一个看门狗定时器模块，工作在内部 32kHz 慢时钟域下，用于监控主程序运行，防止 MCU 出现死机的情况。内部包含一个 32bit 的递减计数器，启动后从配置值开始递减计数，在计数归 0 时挂起中断。

23.1.2 功能特点

- 32 位递减计数器
- 可编程重载值
- 独立中断和复位生成
- 含软件锁保护寄存器

23.2 功能描述

23.2.1 使用说明

要配置 IWDT 模块寄存器前，应操作 IWDT_LOCK 寄存器先写入解锁 code 解锁软件操作。

若想 IWDT 产生复位，需查看 RCC_SYSRSTE 的 IWDTE 位是否使能（默认开启）。IWDTE 使能时，且 IWDT 产生了中断，则 MCU 会复位。

如下图所示，LOAD_VALUE 是 IWDT_RLR 寄存器设定的重载值，开始递减计数后，可以通过 IWDT_CLR 寄存器复位计数值。当计数归 0 后，IWDT 的中断标志 RIS 会立起，进入中断，在中断中依然可以通过 IWDT_CLR 寄存器清除 RIS 标志退出中断并复位计数值。若在计数器归 0 进入中断后，仍不喂狗清除中断以及复位计数器。则在中断中再次计数归 0 后，会产生 IWDT_RES 复位请求，复位 MCU。

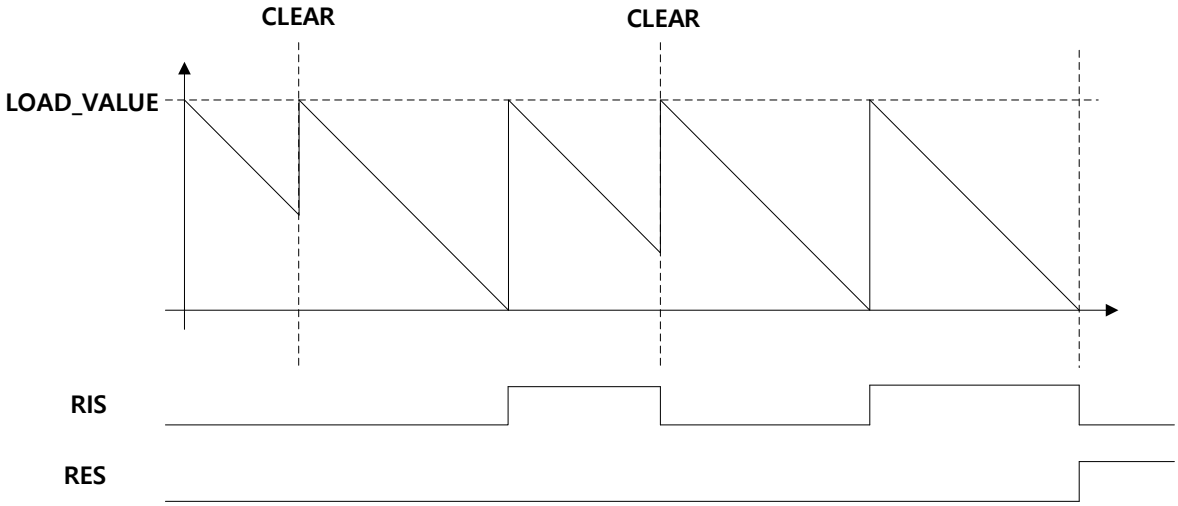


图 23-1 IWDT 时序图

23.3 寄存器列表

IWDT 在芯片中的基地址是 0x4000_3000

表 23-3 IWDT 寄存器地址分配

名称	偏移地址	描述
IWDT_RLR	0x00	IWDT 重装载寄存器
IWDT_CNT	0x04	IWDT 当前计数值寄存器
IWDT_CR	0x08	IWDT 控制寄存器
IWDT_CLR	0x0C	IWDT 中断清除寄存器
IWDT_SR	0x10	IWDT 中断状态寄存器
IWDT_LR	0x3FC	IWDT 软件锁寄存器

23.4 IWDT 寄存器描述

23.4.1 IWDT_RLR IWDT 重装载寄存器

地址:0x4000_3000

复位值:0xFFFF_FFFF

位域	类型	复位	名称	描述
[31:0]	R/W	32'hFFFF_FFFF	LOAD	看门狗重载值。写入后计数器会加载该值。

23.4.2 IWDT_CNT　IWDT 当前计数值寄存器

地址:0x4000_3004

复位值:0xFFFF_FFFF

位域	类型	复位	名称	描述
[31:0]	RO	32'hFFFF_FFFF	CNT	计数器当前计数值。若刚写 LOAD 则返回新值。

23.4.3 IWDT_CR　IWDT 控制寄存器

地址:0x4000_3008

复位值:0x0

位域	类型	复位	名称	描述
[31：1]	--	--	--	--
[0]	R/W	0	EN	IWDT 使能。 1：计数器归 0 时产生中断 0：禁止中断

23.4.4 IWDT_CLR　IWDT 中断清除寄存器

地址:0x4000_300C

复位值:0x0

位域	类型	复位	名称	描述
[31:0]	WO	0	CLEAR	任意写操作会复位 CNT 值至重装载值，并清除挂起的中断。

23.4.5 IWDT_SR　IWDT 状态寄存器

地址:0x4000_3010

复位值:0x0

位域	类型	复位	名称	描述
[31:1]	--	--	--	--

[0]	RO	0	RIS	IWDT 挂起中断 1：计数器已归 0 且尚未清除中断 0：无挂起中断

23.4.6 IWDT_LOCK IWDT 软件锁寄存器

地址:0x4000_33FC

复位值:0x1

位域	类型	复位	名称	描述
[31:0]	WO	0	KEY	向该寄存器写入 0x1ACCE551,LOCK 位变为 0，写入其他任意值，LOCK 为变为 1。
[0]	RO	1	LOCK	IWDT 软件锁标志 1：软件操作锁定 0：软件操作解锁

注：该寄存器可以写入和读取，但是写入的值不能回读，读取只能读取 bit0 的状态。

23.5 应用指南

23.5.1 功能说明

时钟与超时时间

IWDT 使用内部 32kHz 低速时钟（LSI）作为计数时钟，内部为 32 位递减计数器，计数值从 IWDT_RLR 寄存器设定的重装载值开始递减。因此，超时时间（ms）对应的 RLR 值 = ms × 32，SDK 已在 iwdg_init()/feed_iwdg()内部完成该换算，调用时直接传入毫秒数即可。

注意：32 位计数器可表示的最大超时时间约为 134217ms（0xFFFFFFFF / 32）；使用 IWDT 前必须先使能其外设时钟（wdog_clk_enable()），LSI 时钟源需处于使能状态（可调用 rcc_hal_osc_enable_all()）。

寄存器写保护

IWDT_LOCK 寄存器用于写保护。访问 IWDT 其它寄存器前需向 IWDT_LOCK 写入解锁值 0x1ACCE551，配置完成后写入任意其它值（SDK 中写 0x0）重新上锁。上锁状态下对 IWDT 其它寄存器的写操作无效。

溢出中断与复位

计数器递减到 0 时，IWDT_SR.RIS 置位并产生中断，该中断在本芯片上连接至 NMI（不可屏蔽中断）。在中断服务函数 NMI_Handler 中应向 IWDT_CLR 寄存器写入任意值以清除中断标志并重新装载计数器；若中断标志未被清除，IWDT 将触发系统复位。

IWDT 复位功能需要 RCC_SYSRSTE.IWDTRSTE 位使能（iwdg_init()内部已自动使能）。复位后可通过 RCC_SYSRSTSR 中的 IWDT 复位标志判断复位来源，详见时钟复位章节。

喂狗

在计数器溢出之前向 IWDT_CLR 寄存器写入任意值，计数器会被重新装载为 IWDT_RLR 的值，此操作称为“喂狗”。只要喂狗周期小于超时时间，芯片就不会被复位。

SDK 接口

void iwdg_init(uint32_t ms); /*初始化并启动 IWDT，ms 为超时时间，单位 ms；内部使能 NMI 中断(优先级 2)和 IWDT 复位使能*/

void feed_iwdg(uint32_t ms); /*喂狗，清除计数器并将重装载值更新为 ms，单位 ms*/

uint8_t iwdg_status(void); /*读取 SR.RIS，1=计数器已溢出到 0，0=未溢出*/

uint32_t iwdg_counts(void); /*读取 IWDT_CNT 当前计数值(单位为 32kHz 时钟周期)*/

void iwdg_interrupt_clean(void); /*向 IWDT_CLR 写入 0，清除中断标志*/

**IWDT 一旦使能（CR.EN=1）便无法通过软件关闭，如不需要看门狗功能请不要调用 iwdg_init()。调试时如需暂停 IWDT，请参考调试器相关配置。

23.6 快速开始

以下为最小可用的 IWDT 示例（超时时间 1s，参考 demo\iwdt_demo）：

流程：使能外设与时钟源 → 初始化看门狗并设定超时时间 → 在主循环中周期性喂狗 → 在 NMI 中断中处理溢出

```
flash_clk_enable();
gpio_clk_enable();
uart_clk_enable();
rcc_hal_osc_enable_all();          /*使能所有时钟源，包括 IWDT 使用的 LSI*/
wdog_clk_enable();                 /*使能 IWDT 外设时钟*/
```

```
RCC_Init();
__enable_irq();
iwdg_init(1000);                    /*初始化看门狗，超时时间为 1s*/
while (1)
{
    feed_iwdg(1000);                /*周期性喂狗，喂狗周期须小于超时时间*/
    delay_1us(24000);               /*用户业务代码，此处延时约 500ms*/
}
```

**喂狗周期必须小于 iwdg_init()设定的超时时间，否则看门狗溢出，芯片将被复位。

NMI 中断处理

IWDT 溢出时进入 NMI_Handler（见 demo\iwdt_demo\isr.c），可在其中清除中断标志并执行复位前的保护操作，如保存关键数据：

```
static IWDT_TypeDef *wodg_isr_ptr = IWDT_REG;
void NMI_Handler(void)
{
    wodg_isr_ptr->CLR.w = 160000;    /*写入任意值清除中断标志*/
    INFO("IWDT interrupt!
");
}
```

**若在 NMI_Handler 中既清除了中断标志，又调用 feed_iwdg()重新喂狗，则芯片不会复位；若不处理，则中断标志保持，IWDT 将复位芯片。

演示看门狗复位

将主循环中的 feed_iwdg()注释掉（不再喂狗），程序运行约 1s 后看门狗溢出，先进入 NMI 中断，随后芯片复位，串口将重新打印启动信息“SDK start running...”。

查询看门狗状态

```
if (iwdg_status() == 1)
{
    /*计数器已溢出到 0*/
```

```
}
uint32_t cnt = iwdg_counts();     /*读取当前计数值，除以 32 即为剩余毫秒数*/
```